Created on....25.11.2009 Last changed on/by..... 27.10.2026 ABAP7
Author.....SAP

Title.....G/L Account Line Item Display

Type.....1 Reporting
Status.....P Production
Application.....F Financial accou
Class.....
Logical database.....SD
from Application....F Financial accou
Selection screen.....908
Authorization group...
Package.....FAGL_REPORTING

Start by Variants
Deactivate editor lock
X Fixed point arithmetic active

```
*---------------------------------------------------------------------*
*                                                                     *
*  FAGL_ACCOUNT_ITEMS_GL  Lineitems General Ledger Accounts NewGL     *
*                         ____  -       -                            *
*                                                                     *
*               This report selects data to be displayed by ALV       *
*---------------------------------------------------------------------*

REPORT fagl_account_items_gl MESSAGE-ID msitem NO STANDARD PAGE HEADING.
INCLUDE fins_aging_include.
*... general data definitions:
*include rfitem_def.
INCLUDE fagl_account_items_def.
INCLUDE fin_ui_deco_include.



*... icons and symbols:
INCLUDE <list>.

TABLES: skat, skb1, bsis, admi_files.
TABLES: t001.

DATA: it_h_t001 TYPE tpit_t_vt001 WITH HEADER LINE,
      it_h_skat TYPE tpit_t_vskat WITH HEADER LINE,
      it_h_skb1 TYPE tpit_t_vskb1 WITH HEADER LINE.

* if report is called from balcance display.
DATA: gt_saknr TYPE RANGE OF ska1-saknr,
      gd_saknr LIKE LINE OF gt_saknr.
DATA:  gt_bukrs TYPE RANGE OF skb1-bukrs.                    "1530241
DATA: gt_gjahr TYPE RANGE OF bsis-gjahr,
      gd_gjahr LIKE LINE OF gt_gjahr.
DATA: gt_monat TYPE RANGE OF bsis-monat,
      gd_monat LIKE LINE OF gt_monat.
DATA: gt_yrper TYPE RANGE OF rfposx-jamon,
      gd_yrper LIKE LINE OF gt_yrper.
DATA:  gt_yrper_old TYPE RANGE OF rfposx-jamon.              "1530241
TYPES: BEGIN OF gty_acccoco,
         saknr LIKE ska1-saknr,
         bukrs LIKE skb1-bukrs,
         gvtyp LIKE ska1-gvtyp,
       END OF gty_acccoco.
DATA: gt_accoco      TYPE STANDARD TABLE OF gty_acccoco,  "PL Accounts
*      gt_accoco_bs type standard table of gty_acccoco,   "not needed
      gt_account_slt TYPE STANDARD TABLE OF gty_acccoco, "Selltlem. Acc
      gt_accoco_slt  TYPE STANDARD TABLE OF gty_acccoco,  "Sellt. PL Acc
      gd_accoco      TYPE gty_acccoco.

* new data declaration
DATA: fs_f1_icon           LIKE smp_dyntxt.
DATA: fs_f3_icon           LIKE smp_dyntxt.
DATA: fs_f4_icon           LIKE smp_dyntxt.
DATA: gv_relevant_ledger   TYPE rldnr_flex.
DATA: gv_leading_ledger    TYPE rldnr_flex.
DATA: gv_si_table          LIKE fagl_tabnames-si_table.
DATA: gv_save_si_table     LIKE fagl_tabnames-si_table.
DATA: gv_xumsav            TYPE xfeld.
DATA: gv_xerfs             TYPE xfeld.
DATA: gv_xrric             TYPE xfeld.
DATA: gv_ucomm             LIKE sy-ucomm.
```

```
 62  DATA: gv_special_account   TYPE boolean.                    "1004374
 63  CONSTANTS: gc_owner        TYPE gl_owner VALUE 'FAGL_BALANCE_DISPLAY',
 64             gc_parameter_id TYPE memoryid VALUE 'GLN_FLEX'.
 65  DATA: gb_flex              TYPE boolean.
 66  DATA: gb_xplacc            TYPE boolean.
 67  DATA: gd_max_lines         TYPE nmax_it.
 68  DATA: gd_max_pack_lines    TYPE nmax_it VALUE '50000'.      "987950
 69  DATA: gd_cursor_open       TYPE xfeld.                      "987950
 70  DATA: gd_xdels             TYPE xfeld.
 71  DATA: gd_au_blart TYPE c,                                   "2983368
 72        gd_au_gsber TYPE c.                                   "2983368
 73  *data: c_repid_FAGl_GL      like sy-repid  "is in DEF Include
 74  *                              value 'FAGL_ACCOUNT_ITEMS_GL'.
 75  DATA gt_reporting_fields   TYPE ttfieldname.
 76  DATA gs_reporting_fields   TYPE fieldname.
 77  DATA gt_dfies_reporting_fields TYPE dfies_table.
 78  DATA gt_selection_part     TYPE  gusl_t_selection.
 79  DATA gt_selection_full     TYPE  gusl_t_selection.
 80  * migration
 81  DATA: gb_xmig              TYPE boolean.
 82  DATA: gb_xmig_tpara        TYPE boolean.
 83  DATA: gb_xerfs             TYPE boolean.
 84  DATA: gb_xuncpl            TYPE boolean.
 85  DATA  gt_fagl_mig_001      TYPE  fagl_t_mig_001.
 86  DATA  gs_fagl_mig_001      TYPE  fagl_s_mig_001.
 87  DATA  gs_mig_date_2        TYPE  fagl_migdt.                "1280800
 88  * expiring currencies:
 89  DATA: gd_alw_waers TYPE waers,
 90  *     GD_ALW_FLAG          type C,
 91        gd_alw_date  TYPE sydatum.
 92  *     gd_ALW_report        type sy-repid value 'SAPDBSDF'.
 93  DATA: gt_alw_fieldlist     LIKE ddfldnam OCCURS 0 WITH HEADER LINE.
 94  * appliction log
 95  DATA : gt_log_handle TYPE bal_t_logh,
 96         gs_log_handle TYPE balloghndl,
 97         gs_log        TYPE bal_s_log,
 98         gd_xmsg       TYPE xfeld.
 99  DATA:  gd_dummy.
100  DATA:  gd_xarch       TYPE xfeld.
101  DATA:  gd_count_arch TYPE i.
102  DATA:  gd_count_noauth TYPE i.                              "1511192
103  DATA:  l_auth                LIKE pca_i_auth.               "1649564
104  DATA:  ld_del_nummax TYPE boole_d.                          "1893789
105  DATA   gd_rri_sender          TYPE rsti_ronam.              "n2075846
106  DATA lv_data_temperature TYPE data_temperature.
107  DATA ld_sknr_dummy TYPE saknr.
108
109  " Enforce outer joins between ACDOCA and BKPF                 2857175
110  DATA: gd_enforce_outer_join TYPE abap_bool.                 " 2857175
111
112  *... selection screen layout:
113
114  * button worklists on/off:
115  SELECTION-SCREEN FUNCTION KEY 1.
116  SELECTION-SCREEN FUNCTION KEY 2.
117  SELECTION-SCREEN FUNCTION KEY 3.
118  SELECTION-SCREEN FUNCTION KEY 4.
119  * selections for processing of worklists:
120  SELECTION-SCREEN BEGIN OF BLOCK glaccount WITH FRAME TITLE TEXT-w02.
121    PARAMETERS: pa_wlsak LIKE rf42b-idnts MODIF ID wkl.
122    SELECT-OPTIONS: so_wlsak FOR skb1-saknr MODIF ID wkl NO DATABASE
```

```
123    SELECTION.
124  SELECTION-SCREEN END OF BLOCK glaccount.
125  SELECTION-SCREEN BEGIN OF BLOCK company WITH FRAME TITLE TEXT-w03.
126    PARAMETERS: pa_wlbuk LIKE rf42b-idntb MODIF ID wkl.
127    SELECT-OPTIONS: so_wlbuk FOR skb1-bukrs MODIF ID wkl NO DATABASE
128    SELECTION.
129  SELECTION-SCREEN END OF BLOCK company.
130
131  * outer frame for item selection:
132  SELECTION-SCREEN BEGIN OF BLOCK items WITH FRAME TITLE TEXT-007.
133  * inner frame 1:
134    SELECTION-SCREEN BEGIN OF BLOCK status WITH FRAME TITLE TEXT-002.
135  *   open items:
136      SELECTION-SCREEN BEGIN OF LINE.
137        PARAMETERS x_opsel LIKE itemset-xopsel RADIOBUTTON GROUP rad1
138                                                          DEFAULT 'X'.
139        SELECTION-SCREEN COMMENT 3(20) TEXT-003 FOR FIELD x_opsel.
140      SELECTION-SCREEN END OF LINE.
141      PARAMETERS pa_stida LIKE rfpdo-allgstid DEFAULT sy-datlo.
142      SELECTION-SCREEN SKIP.
143  *   cleared items:
144      SELECTION-SCREEN BEGIN OF LINE.
145        PARAMETERS x_clsel LIKE itemset-xclsel RADIOBUTTON GROUP rad1.
146        SELECTION-SCREEN COMMENT 3(25) TEXT-004 FOR FIELD x_clsel.
147      SELECTION-SCREEN END OF LINE.
148      SELECT-OPTIONS so_augdt FOR bsis-augdt NO DATABASE SELECTION.
149      PARAMETERS pa_stid2 LIKE rfpdo-allgstid.
150      SELECTION-SCREEN SKIP.
151  *   all items:
152      SELECTION-SCREEN BEGIN OF LINE.
153        PARAMETERS x_aisel LIKE itemset-xaisel RADIOBUTTON GROUP rad1.
154        SELECTION-SCREEN COMMENT 3(20) TEXT-005 FOR FIELD x_aisel.
155      SELECTION-SCREEN END OF LINE.
156      SELECT-OPTIONS so_budat FOR bsis-budat NO DATABASE SELECTION.
157    SELECTION-SCREEN END OF BLOCK status.
158  * inner frame 2:
159    SELECTION-SCREEN BEGIN OF BLOCK type WITH FRAME TITLE TEXT-006.
160  *SELECTION-SCREEN BEGIN OF LINE.
161      PARAMETERS: x_norm LIKE itemset-xnorm AS CHECKBOX DEFAULT 'X'
162                                                    MODIF ID efs.
163  *SELECTION-SCREEN COMMENT 03(28) TEXT-101 for field x_norm.
164  *SELECTION-SCREEN POSITION POS_HIGH.
165      PARAMETERS: x_erfs LIKE rfpdo3-allgerfs NO-DISPLAY..
166  *SELECTION-SCREEN COMMENT 61(20) TEXT-102 for field x_erfs.
167  *SELECTION-SCREEN END OF LINE.
168      PARAMETERS: x_shbv LIKE itemset-xshbv NO-DISPLAY,
169                  x_merk LIKE itemset-xmerk AS CHECKBOX MODIF ID efs,
170                  x_park LIKE itemset-xpark AS CHECKBOX MODIF ID efs,
171                  x_apar LIKE itemset-xarit NO-DISPLAY.
172  *...parameter for ledger
173      PARAMETERS
174             rldnr TYPE fins_rldnr_and_rollup MODIF ID gls.
175  *...parameter for carry forward postings
176  *PARAMETERS x_glyec TYPE xfeld MODIF ID gly.              "Note 2669860
177
178      SELECTION-SCREEN BEGIN OF LINE.                          "Note 2669
     860
179        PARAMETERS x_glyec TYPE fagl_yec_data_sel               "Note 26
     69860
180                           AS CHECKBOX
181                           MODIF ID gly.
```

```
182
183         SELECTION-SCREEN COMMENT 3(55) gv_text FOR FIELD x_glyec  "Note 26
    69860
184           MODIF ID gly.
185       SELECTION-SCREEN END OF LINE.                                "Note 2669
    860
186   * Enforce outer join; i.e. consider records with BSTAT = C  " 2857175
187       SELECTION-SCREEN BEGIN OF LINE.                          " 2857175
188         PARAMETERS x_enfoj TYPE fagl_enfoj                     " 2857175
189   *                        AS CHECKBOX                         " 2857175
190   *                        MODIF ID gly.                       " 2857175
191                            NO-DISPLAY.                         " 2857175
192       SELECTION-SCREEN END OF LINE.                            " 2857175
193     SELECTION-SCREEN END OF BLOCK type.
194   * end of inner frames.
195   SELECTION-SCREEN END OF BLOCK items.
196   * end of outer frame.
197   * list layout frame:
198   SELECTION-SCREEN BEGIN OF BLOCK list WITH FRAME TITLE TEXT-001.
199     PARAMETERS: pa_bwber LIKE rfpdo3-allgbwbe NO-DISPLAY.
200     PARAMETERS: pa_vari TYPE slis_vari,
201                 pa_nmax LIKE itemset-nmax.
202   SELECTION-SCREEN END OF BLOCK list.
203   *... end of selection screen layout.
204
205   *... dark parameters:
206   * branch: read items posted on central office? (not used for GL)
207   *   Values: (Y)es, (N)o, ( ) prompt user
208   PARAMETERS: pa_cent TYPE c NO-DISPLAY.
209   * net due date selection:
210   SELECT-OPTIONS so_faedt FOR it_pos-faedt NO-DISPLAY.
211   * combined fiscal year/posting period
212   DATA:           gp_yrper TYPE rwcoom-fiscper.
213   SELECT-OPTIONS: so_yrper FOR gp_yrper NO-DISPLAY.
214   * internet transaction mode?
215   PARAMETERS: pa_inet TYPE c NO-DISPLAY.
216   * grid control display?
217   *   Values: (Y)es, (N)o, ( ) get parameter
218   PARAMETERS: pa_grid TYPE c NO-DISPLAY.
219
220   *... include internal select-options for derived item fields:
221   INCLUDE rfitem_sel.
222
223   *...parameter for ledger
224   *PARAMETERS
225   *      rldnr TYPE faglflext-rldnr NO-DISPLAY.
226   PARAMETERS:
227
228   *  fs_dyns TYPE rsds_type NO-DISPLAY,                        "n2199162
229     fs_num   LIKE sy-tfill   NO-DISPLAY,
230     fs_field TYPE rsdsfields_t NO-DISPLAY.
231
232   INITIALIZATION.
233     PERFORM deco_at_initialization.
234
235     IF idtimeacc_cl_switch_check=>filoc_time_acc_sfws_01( ) EQ 'X'.
236   *   Free Date In Memory
237       idtimeacc_cl_utils=>free_date_memory( ).
238     ENDIF. " Note 1731392 - 19-JUN-2012
239
240   *&---------------------------------------------------------------------*
```

```
241  *&      Form  deco_at_initialization
242  *&---------------------------------------------------------------------*
243  *       text
244  *----------------------------------------------------------------------*
245  FORM deco_at_initialization.
246
247    g_repid  = sy-repid.
248    sd_noaut = 'X'.
249    sd_nooap = 'X'.
250    gd_save_title = sy-title.
251    CLEAR gd_rri_so_set.                                    "1093945
252
253  * PROGRAM CAN ONLY BE USED IF NEW G/L IS ACTIVATED
254  * erstezen durch FB ob mindestens 1 BUKRS FlexGL aktiv hat
255  * bei AT Selection Screen dann konkret die eingegebenen
256  * Bukrs prÃ¼fen
257    CALL FUNCTION 'FAGL_CHECK_GLFLEX_ACTIVE'
258      IMPORTING
259        e_glflex_active = gb_flex.
260
261  * if not check, if new G/L is active at least in one
262  * company code
263    IF gb_flex NE 'X'.
264      CALL FUNCTION 'FAGL_BUKRS_ACTIVE_IN_CLIENT'
265        IMPORTING
266          e_glflex_active = gb_flex.
267    ENDIF.
268
269  * Moved here, so also if NewG/L is not active in migration,
270  * the FAGLL03 can be used (even in NewG/L view)
271    GET PARAMETER ID 'FAGL_MIG_FAGLL03' FIELD gb_xmig_tpara.  "1701675
272    IF gb_flex IS INITIAL AND gb_xmig_tpara IS INITIAL.       "1701675
273      IF cl_fin_ui_deco=>mv_mode = abap_false.
274        MESSAGE w866(fr) WITH 'FBL3N'.
275        LEAVE TO TRANSACTION 'FBL3N'.
276      ELSE.
277        deco_message_1 e866(fr) with 'FBL3N'.
278      ENDIF.
279    ENDIF.
280  * expiring currencies:
281    PERFORM init_expcur.
282  * deactivate user commands:
283  * perform change_status.
284    PERFORM change_status.
285  * initialize worklist button:
286    PERFORM wl_flag_and_button.
287  * set account and company code:
288    PERFORM set_acct_and_ccode.
289
290  * find relevant ledger
291  * IF sy-slset EQ space                                "796813
292  *  if rldnr is initial.
293    PERFORM find_relevant_ledger
294            CHANGING gv_relevant_ledger.
295  *  else.
296  *    gv_relevant_ledger = rldnr.
297  *  endif.
298    PERFORM find_si_table
299            USING    gv_relevant_ledger
300            CHANGING gv_si_table.
301    PERFORM get_leading_ledger
```

```
302           CHANGING gv_leading_ledger.
303
304  * Check if auth check for BLART and GSBER is needed            "2983368
305    CALL FUNCTION 'FI_ADD_AUTHORITY_CHECK'
306      EXPORTING
307        i_bukrs = space
308      IMPORTING
309        e_xblar = gd_au_blart
310        e_xgsbe = gd_au_gsber
311      EXCEPTIONS
312        OTHERS  = 1.
313    IF sy-subrc NE 0.
314    ENDIF.
315
316  * rri
317    PERFORM init_rri_data.
318
319    rldnr = gv_relevant_ledger.
320
321  * get T021S and make field administration tables:
322    PERFORM init_admin_tables.
323  * make field catalog, exclude all unnecessary fields:
324  * perform make_fieldcatalog.
325  * prepare KKBER currency lookup table:
326  * perform init_kkcurr_table.
327  * get parameters:
328    PERFORM get_general_param.
329    IF NOT gv_xrric IS INITIAL.
330      IF NOT gv_relevant_ledger IS INITIAL.
331        CLEAR x_erfs.
332        CLEAR x_merk.
333        CLEAR x_park.
334      ENDIF.
335    ENDIF.
336  ** set account and company code:
337  *  perform set_acct_and_ccode.
338  * set fields for transfer prices
339    PERFORM insert_tp_fields.
340  * check if BADIs are active
341    PERFORM check_badi_active.
342
343  * set title
344    CLEAR gd_title.
345    IF x_erfs IS INITIAL.
346      CONCATENATE gd_save_title TEXT-277 INTO gd_title
347      SEPARATED BY space.
348    ELSE.
349      CONCATENATE gd_save_title TEXT-278 INTO gd_title
350      SEPARATED BY space.
351    ENDIF.
352    SET TITLEBAR gc_title WITH gd_title.
353
354  ENDFORM.                    "deco_at_initialization
355
356  AT SELECTION-SCREEN OUTPUT.
357
358    PERFORM deco_at_sel_screen_output.
359
360  *&---------------------------------------------------------------------*
361  *&      Form  deco_at_sel_screen_output
362  *&---------------------------------------------------------------------*
```

```
363 *       text
364 *----------------------------------------------------------------------*
365 FORM deco_at_sel_screen_output.
366
367   DATA: ls_dfies TYPE dfies.                "Note 2669860
368
369 * check for "suppress dialog":
370   gd_dynp_fun = 7.
371   gd_dynp_val = 1.
372   PERFORM dynp_get_status USING    gd_dynp_fun
373                           CHANGING gd_dynp_val.
374   LOOP AT SCREEN.
375     PERFORM wl_modify_screen.
376     MODIFY SCREEN.
377   ENDLOOP.
378
379   " Begin corrections of this Note 2669860
380 * set text for gv_text
381
382   CALL FUNCTION 'DDIF_FIELDINFO_GET'
383     EXPORTING
384       tabname   = 'FAGL_YEC_DATA_SEL'
385       all_types = 'X'
386     IMPORTING
387       dfies_wa  = ls_dfies.
388
389   gv_text = ls_dfies-reptext.
390
391   " End corrections of this Note 2669860
392
393 * icon freie selection
394   PERFORM fs_set_sscrtexts_dynsel CHANGING fs_f1_icon.
395   sscrfields-functxt_01 = fs_f1_icon.
396
397 * new GL or Entry view
398   IF NOT x_erfs IS INITIAL
399   OR NOT x_merk IS INITIAL
400   OR NOT x_park IS INITIAL.
401     gv_xerfs = 'X'.
402   ELSE.
403     CLEAR gv_xerfs.
404   ENDIF.
405
406 *  if gv_xerfs is initial.
407 *    sscrfields-functxt_04 = text-278.
408 *  else.
409 *    sscrfields-functxt_04 = text-277.
410 *  endif.
411   PERFORM fs_set_sscrtexts_dynsel4 CHANGING fs_f4_icon.
412   sscrfields-functxt_04 = fs_f4_icon.
413
414   LOOP AT SCREEN.
415     PERFORM erfs_modify_screen.
416     PERFORM glyec_modify_screen.                          "1779180
417     MODIFY SCREEN.
418   ENDLOOP.
419
420 * set ledger if in Report Variant
421   IF  rldnr NE gv_relevant_ledger
422   AND gv_xerfs IS INITIAL.
423     gv_relevant_ledger = rldnr.
```

```
424      PERFORM find_si_table
425              USING    gv_relevant_ledger
426              CHANGING gv_si_table.
427    ENDIF.
428
429  * deactivate user commands:
430    PERFORM change_status.
431
432  * set title
433    CLEAR gd_title.
434    IF gv_xerfs IS INITIAL.
435      CONCATENATE gd_save_title TEXT-277 INTO gd_title
436      SEPARATED BY space.
437    ELSE.
438      CONCATENATE gd_save_title TEXT-278 INTO gd_title
439      SEPARATED BY space.
440    ENDIF.
441    SET TITLEBAR gc_title WITH gd_title.
442  * convert dynamic selection from old (migrated) variants from table name
       like FAGFLEXA_FS to ACDOCA_FS
443    IF fs_dyns IS NOT INITIAL.
444      DATA(lt_acdoca_fs_fields) = CAST cl_abap_structdescr( cl_abap_typede
     scr=>describe_by_name( 'ACDOCA_FS' ) )->components.
445      LOOP AT fs_field REFERENCE INTO DATA(lr_field) WHERE tablename CP '*
     A_FS' AND tablename <> 'ACDOCA_FS'.
446        lr_field->tablename = 'ACDOCA_FS'.
447        IF NOT line_exists( lt_acdoca_fs_fields[ name =  lr_field->fieldna
     me ] ).
448          DELETE fs_field.
449        ENDIF.
450      ENDLOOP.
451      LOOP AT fs_dyns-trange REFERENCE INTO DATA(lr_trange) WHERE tablenam
     e CP '*A_FS' AND tablename <> 'ACDOCA_FS'.
452        lr_trange->tablename = 'ACDOCA_FS'.
453        LOOP AT lr_trange->frange_t REFERENCE INTO DATA(lr_frange).
454          IF NOT line_exists( lt_acdoca_fs_fields[ name =  lr_frange->fiel
     dname ] ).
455            DELETE lr_trange->frange_t.
456          ENDIF.
457        ENDLOOP.
458        IF lr_trange->frange_t IS INITIAL.
459          DELETE fs_dyns-trange.
460        ENDIF.
461      ENDLOOP.
462
463      IF sy-subrc = 0.
464        CLEAR: fs_dyns-clauses, fs_dyns-texpr.
465        IF fs_dyns-trange IS NOT INITIAL.
466          CALL FUNCTION 'FREE_SELECTIONS_RANGE_2_WHERE'
467            EXPORTING
468              field_ranges  = fs_dyns-trange
469            IMPORTING
470              where_clauses = fs_dyns-clauses.
471          CALL FUNCTION 'FREE_SELECTIONS_RANGE_2_EX'
472            EXPORTING
473              field_ranges = fs_dyns-trange
474            IMPORTING
475              expressions  = fs_dyns-texpr.
476        ENDIF.
477      ENDIF.
478    ENDIF.
```

```
479
480 ENDFORM.                    "deco_at_sel_screen_output
481
482 AT SELECTION-SCREEN ON BLOCK glaccount.
483
484   PERFORM deco_at_sel_screen_glaccount.
485
486 *&---------------------------------------------------------------------*
487 *&      Form  deco_at_sel_screen_glaccount
488 *&---------------------------------------------------------------------*
489 *       text
490 *----------------------------------------------------------------------*
491 FORM deco_at_sel_screen_glaccount.
492
493   IF NOT gd_wl_on IS INITIAL.        " worklists on >
494     IF gd_dynp_val EQ 0.             " rebuild sd_bukrs and sd_saknr >
495 *       GL accounts:
496       REFRESH sd_saknr.
497       IF NOT pa_wlsak IS INITIAL.    " account worklist given >
498         PERFORM resolve_worklist TABLES gt_cosel
499                                  USING  pa_wlsak
500                                         'SAKNR'.
501 *         worklist overrides account select-option:
502         REFRESH so_wlsak.
503         CLEAR so_wlsak.
504         LOOP AT gt_cosel.
505           MOVE-CORRESPONDING gt_cosel TO sd_saknr.
506           APPEND sd_saknr.
507         ENDLOOP.
508       ELSE.
509         APPEND LINES OF so_wlsak TO sd_saknr.
510       ENDIF.                         " account worklist given <
511     ENDIF.                           " rebuild sd_bukrs and sd_saknr <
512   ENDIF.                             " worklists on <
513
514 ENDFORM.                    "deco_at_sel_screen_glaccount
515
516 AT SELECTION-SCREEN ON BLOCK company.
517
518   PERFORM deco_at_sel_screen_company.
519
520 *&---------------------------------------------------------------------*
521 *&      Form  deco_at_sel_screen_company
522 *&---------------------------------------------------------------------*
523 *       text
524 *----------------------------------------------------------------------*
525 FORM deco_at_sel_screen_company.
526
527   IF NOT gd_wl_on IS INITIAL.        " worklists on >
528     IF gd_dynp_val EQ 0.             " rebuild sd_bukrs and sd_saknr >
529 *       company codes:
530       REFRESH sd_bukrs.
531       IF NOT pa_wlbuk IS INITIAL.    " ccode worklist given >
532         PERFORM resolve_worklist TABLES gt_cosel
533                                  USING  pa_wlbuk
534                                         'BUKRS'.
535 *         worklist overrides company code select-option:
536         REFRESH so_wlbuk.
537         CLEAR so_wlbuk.
538         LOOP AT gt_cosel.
539           MOVE-CORRESPONDING gt_cosel TO sd_bukrs.
```

```
540           APPEND sd_bukrs.
541         ENDLOOP.
542       ELSE.
543         APPEND LINES OF so_wlbuk TO sd_bukrs.
544       ENDIF.                          " ccode worklist given <
545     ENDIF.                            " rebuild sd_bukrs and sd_saknr <
546   ENDIF.                              " worklists on <
547
548 ENDFORM.                    "deco_at_sel_screen_company
549
550 AT SELECTION-SCREEN.
551
552   PERFORM deco_at_sel_screen.
553
554 *&---------------------------------------------------------------------*
555 *&      Form  deco_at_sel_screen
556 *&---------------------------------------------------------------------*
557 *       text
558 *----------------------------------------------------------------------*
559 FORM deco_at_sel_screen.
560
561 * checks for activation per comp code
562   DATA lt_t001 TYPE TABLE OF t001.
563   DATA ls_t001 TYPE t001.
564   DATA l_yrper LIKE gp_yrper.                               "1020575
565   DATA l_gjahr LIKE bkpf-gjahr.                             "1020575
566   DATA l_poper LIKE t009b-poper.                            "1020575
567
568   IF ( x_merk IS INITIAL
569   AND x_park IS INITIAL
570   AND x_erfs IS INITIAL ).
571     IF sy-ucomm NE 'FC04'.
572       SELECT * FROM t001 INTO TABLE lt_t001 WHERE bukrs IN sd_bukrs.
573       LOOP AT lt_t001 INTO ls_t001.
574 * check at PAI, if NewGL is active for the selected company codes.
575         CALL FUNCTION 'FAGL_CHECK_GLFLEX_ACTIVE'
576           EXPORTING
577             id_bukrs        = ls_t001-bukrs
578           IMPORTING
579             e_glflex_active = gb_flex.
580         IF gb_flex NE 'X' AND gb_xmig_tpara IS INITIAL.     "1701675
581 *         MESSAGE w026(FAGL_ACCOUNT_BALANCE) WITH ls_t001-bukrs.
582           x_erfs = 'X'.
583           deco_message_1 i100 with ls_t001-bukrs.
584           EXIT.
585 *     LEAVE TO TRANSACTION 'FBL3N'.
586         ENDIF.
587       ENDLOOP.
588     ENDIF.
589   ENDIF.
590
591 * set flag for normal items
592   IF NOT x_merk IS INITIAL
593   OR NOT x_park IS INITIAL.
594     IF NOT x_norm IS INITIAL.
595       x_erfs = 'X'.
596     ENDIF.
597   ENDIF.
598 *  if x_norm is initial.
599 *    clear x_erfs.
600 *  endif.
```

```
601
602  * deactivate user commands:
603    PERFORM change_status.
604
605  * checks for migration
606    CALL FUNCTION 'FAGL_MGPLN_IN_CLIENT'
607      IMPORTING
608        e_migpln_exist = gb_xmig.
609
610    CLEAR gb_xerfs.
611    CLEAR gb_xuncpl.
612    IF ( NOT gb_xmig IS INITIAL
613    AND x_erfs IS INITIAL
614    AND x_merk IS INITIAL
615    AND x_park IS INITIAL ).
616  *    clear gb_xerfs.
617  *    clear gb_Xuncpl.
618      IF  sy-ucomm NE 'FC04'.
619        CLEAR gb_xerfs.
620        CLEAR gb_xuncpl.
621        REFRESH gt_fagl_mig_001.
622        LOOP AT lt_t001 INTO ls_t001.
623          REFRESH gt_fagl_mig_001.
624          CALL FUNCTION 'FAGL_GET_ACTIVE_MGPLN'
625            EXPORTING
626              i_bukrs             = ls_t001-bukrs
627              i_rldnr             = gv_relevant_ledger
628            IMPORTING
629  *           ET_FAGL_MIG_002     =
630              et_fagl_mig_001     = gt_fagl_mig_001
631  *           EF_ACTIVE_MGPLN     =
632            EXCEPTIONS
633              invalid_input       = 1
634              no_data_found       = 2
635              invalid_customizing = 3
636              OTHERS              = 4.
637          DELETE gt_fagl_mig_001 WHERE mg_typ = '04'.          "1294930
638          DELETE gt_fagl_mig_001 WHERE mg_typ = '05'.          "1502414
639          DELETE gt_fagl_mig_001 WHERE mg_start = ''.          "2879440
640          READ TABLE gt_fagl_mig_001                           "1294930
641                       INTO gs_fagl_mig_001 INDEX 1.           "1294930
642
643          IF sy-subrc <> 0.
644  * MESSAGE ID SY-MSGID TYPE SY-MSGTY NUMBER SY-MSGNO
645  *          WITH SY-MSGV1 SY-MSGV2 SY-MSGV3 SY-MSGV4.
646          ELSEIF sy-subrc = 0 AND NOT gb_xmig_tpara IS INITIAL.
647            deco_message_0 i125.
648  *         One message is enough                                 "1858736
649            EXIT.                                               "1858736
650          ELSE.
651            SORT gt_fagl_mig_001 BY mg_end.
652            LOOP AT gt_fagl_mig_001 INTO gs_fagl_mig_001
653              WHERE mg_end NE space.
654              EXIT.
655            ENDLOOP.
656            IF sy-subrc NE 0.
657              gb_xerfs = 'X'.
658            ELSE.
659              IF NOT x_opsel IS INITIAL.
660                IF pa_stida LT gs_fagl_mig_001-migdt.
661                  gb_xerfs = 'X'.
```

```
662               ENDIF.
663             ENDIF.
664             IF NOT x_clsel IS INITIAL.
665               IF  NOT pa_stid2 IS INITIAL
666               AND pa_stid2 LT gs_fagl_mig_001-migdt.
667                 gb_xerfs = 'X'.
668               ELSE.
669                 CLEAR gs_mig_date_2.                          "1280800
670                 gs_mig_date_2 = gs_fagl_mig_001-migdt - 1.    "1280800
671                 IF gs_mig_date_2 IN so_augdt.                 "1280800
672                   gb_xuncpl = 'X'.
673                 ELSE.
674                   LOOP AT so_augdt.
675                     IF so_augdt-low LT gs_fagl_mig_001-migdt.
676                       gb_xuncpl = 'X'.
677                     ENDIF.
678                   ENDLOOP.
679                 ENDIF.
680               ENDIF.
681             ENDIF.
682             IF NOT x_aisel IS INITIAL.
683               CLEAR gs_mig_date_2.                          "1280800
684               gs_mig_date_2 = gs_fagl_mig_001-migdt - 1.    "1280800
685               IF gs_mig_date_2 IN so_budat.                 "1280800
686                 gb_xuncpl = 'X'.
687               ELSE.
688                 LOOP AT so_budat.
689                   IF so_budat-low LT gs_fagl_mig_001-migdt.
690                     gb_xuncpl = 'X'.
691                   ENDIF.
692                 ENDLOOP.
693               ENDIF.
694             ENDIF.
695  * Special case if the FAGLL03 is called over the RRI          "1020575
696             IF sy-calld = 'X'.
697               DATA ld_xmo16 TYPE boole VALUE 'X'.            "1460960
698               IF NOT so_yrper[] IS INITIAL.
699                 CLEAR gb_xuncpl.
700                 CLEAR gs_mig_date_2.                        "1303526
701                 gs_mig_date_2 = gs_fagl_mig_001-migdt - 1.  "1303526
702                 CALL FUNCTION 'FI_PERIOD_DETERMINE'
703                   EXPORTING
704                     i_budat = gs_mig_date_2                 "1303526
705                     i_bukrs = ls_t001-bukrs
706                     i_rldnr = gv_relevant_ledger
707                     x_xmo16 = ld_xmo16                      "1460960
708                   IMPORTING
709                     e_gjahr = l_gjahr
710                     e_poper = l_poper.
711                 CONCATENATE l_gjahr l_poper INTO l_yrper.
712                 LOOP AT so_yrper.
713                   IF so_yrper-low LE l_yrper.
714                     gb_xuncpl = 'X'.
715                   ENDIF.
716                 ENDLOOP.
717               ENDIF.
718             ENDIF.
719  * Special case if the FAGLL03 is called over the RRI          "1020575
720           ENDIF.
721         ENDIF.
722       ENDLOOP.
```

```
723         IF NOT gb_xerfs IS INITIAL
724         AND ( ( x_erfs IS INITIAL AND sy-ucomm NE 'FC04' )
725         OR  ( NOT x_erfs IS INITIAL AND sy-ucomm = 'FC04' ) ).
726           x_erfs = 'X'.
727           gv_xerfs = 'X'.
728           deco_message_0 i105.
729         ENDIF.
730   *    if not gb_xuncpl is initial.
731   *      message i106.
732   *    endif.
733       ENDIF.
734     ENDIF.
735
736     CASE sy-ucomm.
737   *...freeselection
738       WHEN 'FC01'.
739         PERFORM free_selections.
740
741   *...switch worklist on or off:
742       WHEN 'FC02'.
743         PERFORM worklist_on_off.
744         PERFORM set_acct_and_ccode.
745
746   *...Ledger choice
747       WHEN 'FC03'.
748         IF  x_merk IS INITIAL
749         AND x_park IS INITIAL
750         AND x_erfs IS INITIAL.
751           DATA: BEGIN OF gt_chosen_values OCCURS 0.
752                   INCLUDE STRUCTURE sval.
753           DATA: END OF gt_chosen_values.
754           DATA lv_new_ledger TYPE rldnr_flex.
755           CLEAR lv_new_ledger.
756
757           PERFORM choose_ledger
758                       TABLES gt_chosen_values
759                       USING gv_relevant_ledger
760                       CHANGING lv_new_ledger.
761
762           IF lv_new_ledger NE gv_relevant_ledger.
763   *        REFRESH lt_modif_id.
764
765             gv_relevant_ledger = lv_new_ledger.
766             rldnr = lv_new_ledger.                          "796813
767
768   * Check free selections if still applyable for
769   * the new ledger. If not delete the NewGL lineitem
770   * Selections. Change fs_num and button for dyn. sel.
771             gv_save_si_table = gv_si_table.
772             PERFORM find_si_table
773               USING    gv_relevant_ledger
774               CHANGING gv_si_table.
775             IF gv_save_si_table NE gv_si_table.
776               CLEAR gd_xdels.
777               PERFORM del_free_selections_si_tab CHANGING gd_xdels.
778               IF NOT gd_xdels IS INITIAL.
779                 deco_message_0 i102.
780               ENDIF.
781             ELSE.
782               PERFORM check_reporting_fields.
783             ENDIF.
```

```
784           ENDIF.
785         ENDIF.
786   *...switch NewGL on or off:
787       WHEN 'FC04'.
788         PERFORM erfs_on_off.
789         IF NOT gv_xerfs IS INITIAL.
790           gv_save_si_table = gv_si_table.
791           CLEAR gd_xdels.
792           PERFORM del_free_selections_si_tab CHANGING gd_xdels.
793           IF NOT gd_xdels IS INITIAL.
794             deco_message_0 i101.
795           ENDIF.
796         ELSE.
797           CLEAR x_erfs.
798           CLEAR x_merk.
799           CLEAR x_park.
800           x_norm = 'X'.
801         ENDIF.
802   ** set title
803   *  clear gd_title.
804   *  if gv_xerfs is initial.
805   *       concatenate gd_save_title text-277 into gd_title
806   *     separated by space.
807   *   else.
808   *     concatenate gd_save_title text-278 into gd_title
809   *     separated by space.
810   *  endif.
811   *  SET TITLEBAR gc_title with gd_title.
812   *...process worklists and check selections:
813       WHEN 'ONLI' OR 'PRIN' OR 'INIT' OR space.
814         CALL FUNCTION 'BUKRS_AUTHORITY_CHECK'
815           EXPORTING
816             xdatabase = 'S'
817           TABLES
818             xbukreis  = sd_bukrs.
819         CALL FUNCTION 'BUKRS_AUTHORITY_CHECK'
820           EXPORTING
821             xdatabase = 'B'
822           TABLES
823             xbukreis  = sd_bukrs.
824   *     check input:
825         PERFORM sel_account_check.
826   *     check date (tpc)
827         PERFORM check_date TABLES sd_bukrs
828                            USING  sd_tpc.
829   **     check auth ledger
830   *      if  x_erfs is initial
831   *      and x_merk is initial
832   *      and x_park is initial.
833   *        perform check_auth_ledger.
834   *      endif.
835
836   *     message because of migration
837         IF sy-ucomm = 'ONLI'
838         OR sy-ucomm = 'PRIN'.
839           IF NOT gb_xuncpl IS INITIAL.
840             deco_message_0 i106.
841           ENDIF.
842         ENDIF.
843     ENDCASE.
844
```

```
845  * check Ledger for newGL
846    IF  NOT gv_relevant_ledger IS INITIAL
847    AND gv_xerfs IS INITIAL.
848      CALL FUNCTION 'FAGL_CHECK_IF_LEDGER_IS_GLFLEX'
849        EXPORTING
850          i_rldnr          = gv_relevant_ledger
851        EXCEPTIONS
852  *       NOT_FOUND        = 1
853          no_glflex_ledger = 2
854  *       GLFLEX_ROLLUP_LEDGER    = 3
855  *       WRONG_APPLICATION       = 4
856  *       GLFLEX_DEPENDENT_LEDGER = 5
857  *       OTHERS           = 6
858        .
859      IF sy-subrc <> 0.
860        deco_message_1 e108 with gv_relevant_ledger.
861  * MESSAGE ID SY-MSGID TYPE SY-MSGTY NUMBER SY-MSGNO
862  *         WITH SY-MSGV1 SY-MSGV2 SY-MSGV3 SY-MSGV4.
863      ENDIF.
864
865  *   check auth ledger
866      IF  x_erfs IS INITIAL
867      AND x_merk IS INITIAL
868      AND x_park IS INITIAL.
869        PERFORM check_auth_ledger.
870      ENDIF.
871    ENDIF.
872
873  * Check for extension ledger combination to the Company Code   "3789609
874    IF cl_fins_acdoc_util=>is_standard_ledger( iv_rldnr = gv_relevant_ledg
     er ) = abap_false.
875      DATA LT_BUKRS_EXT TYPE TABLE OF BUKRS.
876      DATA LT_BUKRS_ALL TYPE TABLE OF BUKRS.
877      DATA LS_BUKRS_EXT TYPE BUKRS.
878      SELECT BUKRS FROM t001 INTO TABLE LT_BUKRS_ALL
879        WHERE bukrs IN sd_bukrs
880         AND  xtemplt NE 'X'.
881      SELECT BUKRS FROM FINSC_LD_CMP INTO TABLE LT_BUKRS_EXT
882        WHERE  bukrs IN sd_bukrs
883           AND rldnr = gv_relevant_ledger.
884      LOOP AT LT_BUKRS_ALL INTO LS_BUKRS_EXT.
885        READ TABLE LT_BUKRS_EXT WITH KEY LS_BUKRS_EXT TRANSPORTING NO FIEL
     DS.
886        IF SY-SUBRC <> 0.
887          MESSAGE ID 'GU' TYPE 'E' NUMBER '167'
888            WITH gv_relevant_ledger ls_bukrs_ext.
889          EXIT.
890        ENDIF.
891      ENDLOOP.
892    ENDIF.
893  * Check for extension ledger combination to the Company Code   "3789609
894
895    DATA LT_BUKRS TYPE TABLE OF BUKRS.                          "3412695
896    DATA LS_BUKRS TYPE BUKRS.                                   "3412695
897    SELECT t~BUKRS FROM t001 as t INNER JOIN FINSC_LD_CMP       "3412695
898              AS l ON t~BUKRS = l~BUKRS  INTO TABLE lt_bukrs    "3412695
899                WHERE t~bukrs IN sd_bukrs                       "3412695
900                  AND t~xtemplt NE 'X'                          "2697244
901                  AND l~rldnr = gv_relevant_ledger              "3412695
902                  ORDER BY t~BUKRS.                             "3412695
903
```

```
904    LOOP AT lt_bukrs INTO ls_bukrs.                                  "3412695
905
906      CALL METHOD cl_fins_acdoc_util=>get_currency_info
907        EXPORTING
908          iv_bukrs  = ls_bukrs                                       "3412695
909          iv_ledger = gv_relevant_ledger
910        EXCEPTIONS
911          OTHERS    = 1.
912      IF sy-subrc <> 0.
913        MESSAGE ID sy-msgid
914              TYPE sy-msgty
915              NUMBER sy-msgno
916              WITH sy-msgv1 sy-msgv2 sy-msgv3 sy-msgv4.
917      ENDIF.
918
919    ENDLOOP.
920
921    gv_ucomm = sscrfields-ucomm.
922
923  ENDFORM.                    "deco_at_sel_screen
924
925  AT SELECTION-SCREEN ON BLOCK type.
926
927    PERFORM deco_at_sel_screen_type.
928
929  *&---------------------------------------------------------------------*
930  *&      Form  deco_at_sel_screen_type
931  *&---------------------------------------------------------------------*
932  *       text
933  *----------------------------------------------------------------------*
934  FORM deco_at_sel_screen_type.
935
936  * check item type only when action = execute:
937    IF sy-ucomm = 'ONLI'.
938      PERFORM sel_type_check.
939    ENDIF.
940
941  ENDFORM.                    "deco_at_sel_screen_type
942
943  AT SELECTION-SCREEN ON VALUE-REQUEST FOR pa_vari.
944    PERFORM alv_variant_f4 CHANGING pa_vari.
945
946  *** start of selection
947  START-OF-SELECTION.
948
949  * 2015 July 9th:
950  * logical database SDF sets the temparature for the session. Here the te
     mperature is set
951  * for context ACDOCA on callstack level
952
953    DATA lr_bukrs TYPE if_fin_selection_types=>ty_range_bukrs.
954    DATA ls_lr_bukrs LIKE LINE OF lr_bukrs.
955    DATA ls_sd_bukrs LIKE LINE OF sd_bukrs.
956
957    DATA lr_augdt TYPE if_fin_selection_types=>ty_range_augdt.
958    DATA ls_lr_augdt LIKE LINE OF lr_augdt.
959    DATA ls_so_augdt LIKE LINE OF sd_augdt.
960
961    DATA lr_budat TYPE if_fin_selection_types=>ty_range_budat.
962    DATA ls_lr_budat LIKE LINE OF lr_budat.
963    DATA ls_so_budat LIKE LINE OF sd_budat.
```

```
964
965    LOOP AT sd_bukrs INTO ls_sd_bukrs.
966      MOVE-CORRESPONDING ls_sd_bukrs TO ls_lr_bukrs.
967      INSERT ls_lr_bukrs INTO TABLE lr_bukrs.
968    ENDLOOP.
969
970    LOOP AT so_augdt INTO ls_so_augdt.
971      MOVE-CORRESPONDING ls_so_augdt TO ls_lr_augdt.
972      INSERT ls_lr_augdt INTO TABLE lr_augdt.
973    ENDLOOP.
974
975    LOOP AT so_budat INTO ls_so_budat.
976      MOVE-CORRESPONDING ls_so_budat TO ls_lr_budat.
977      INSERT ls_lr_budat INTO TABLE lr_budat.
978    ENDLOOP.
979
980  * End of change
981    PERFORM deco_at_selection_start.
982
983  GET skb1.
984
985    CALL FUNCTION 'FAGL_DETERMINE_TEMPERATURE'
986      EXPORTING
987        is_skb1             = skb1          "note 2268109
988        iv_opsel            = x_opsel
989        iv_clsel            = x_clsel
990        iv_aisel            = x_aisel
991        iv_pa_stida         = pa_stida
992        ir_so_augdt         = lr_augdt
993        ir_so_budat         = lr_budat
994        iv_pa_stid2         = pa_stid2
995        ir_bukrs            = lr_bukrs
996        iv_rldnr            = gv_relevant_ledger
997      IMPORTING
998        ev_data_temperature = lv_data_temperature.
999
1000   PERFORM deco_at_selection_get.
1001
1002 END-OF-SELECTION.
1003   PERFORM deco_at_end_of_selection.
1004   PERFORM deco_at_output_alv.
1005
1006
1007 *&---------------------------------------------------------------------*
1008 *&      Form  deco_at_selection_start
1009 *&---------------------------------------------------------------------*
1010 *       text
1011 *----------------------------------------------------------------------*
1012 FORM deco_at_selection_start.
1013
1014 * Enforce outer join between ACDOCA and BKPF                  " 2857175
1015   PERFORM enforce_outer_join.                                 " 2857175
1016
1017   IF gd_rri_so_set = 'X'.                                     "1093945
1018     IMPORT so_yrper FROM MEMORY ID 'FAGLL03_SO_YRPER'.        "1093945
1019   ENDIF.                                                      "1093945
1020   IF gd_rri_sender CS 'RFBILA'.                               "n2075846
1021     CLEAR so_yrper[].                                         "n2075846
1022   ENDIF.                                                      "n2075846
1023
1024   CLEAR: x_stop.
```

```
1025    gd_max_lines = pa_nmax.
1026    CLEAR: gd_count_arch, gd_count_noauth.                    "1511192
1027
1028  * check accounts according to the selections
1029    IF NOT x_merk IS INITIAL
1030    OR NOT x_park IS INITIAL
1031    OR NOT x_erfs IS INITIAL.
1032      CLEAR gv_relevant_ledger.
1033      CLEAR gv_si_table.
1034    ENDIF.
1035
1036  * set B0SG fields
1037    IF NOT x_erfs IS INITIAL.
1038      CLEAR x_glyec.
1039      IF x_norm <> space.
1040        b0sg-xstan = 'X'.
1041      ELSE.
1042        b0sg-xstan = ' '.
1043      ENDIF.
1044
1045      IF x_merk <> space.
1046        b0sg-xstas = 'X'.
1047      ENDIF.
1048
1049      IF x_park <> space.
1050        b0sg-xstav = 'X'.
1051      ENDIF.
1052    ENDIF.
1053
1054  * set parameters:
1055    PERFORM set_parameters.
1056
1057  * map selections to LDB logic:
1058  * perform map_sel_to_ldb.
1059
1060  * set dynamic selections of Log. Database according to the dyn.
1061  * selections of this programm for SKA1 and SKB1
1062    PERFORM set_dyn_sel_ldb.
1063
1064  * map selections to lineitem selection logic:
1065    PERFORM map_sel_to_items USING gb_xplacc
1066                            CHANGING gt_selection_part.
1067
1068    PERFORM check_selections USING gt_selection_part.         "1578577
1069
1070  * make field catalog, exclude all unnecessary fields:
1071    PERFORM make_fieldcatalog.
1072
1073  * determine company codes which use transfer prices
1074    SELECT * FROM finsv_t001a INTO TABLE it_tp_auth WHERE bukrs IN sd_bukr
      s.
1075    LOOP AT it_tp_auth.
1076      IF cl_fins_valutyp=>get_valutyp( it_tp_auth-curtp ) = '1' OR  cl_fin
      s_valutyp=>get_valutyp( it_tp_auth-curtp ) = '2'
1077        OR cl_fins_valutyp=>get_valutyp( it_tp_auth-curtp2 ) = '1' OR  cl_
      fins_valutyp=>get_valutyp( it_tp_auth-curtp2 ) = '2'.
1078      ELSE.
1079        DELETE it_tp_auth.
1080      ENDIF.
1081
1082    ENDLOOP.
```

```
1083
1084  * begin of "Transfer Pricing authority check
1085    DATA LT_BUKRS TYPE TABLE OF BUKRS.                              "3412695
1086    DATA LS_BUKRS TYPE BUKRS.                                       "3412695
1087    DATA lt_bukrs_tp_curtp TYPE cl_fins_acdoc_util=>tt_bukrs_tp_curtp.
1088
1089    SELECT t~BUKRS FROM t001 as t INNER JOIN FINSC_LD_CMP           "3412695
1090              AS l ON t~BUKRS = l~BUKRS  INTO TABLE lt_bukrs        "3412695
1091                WHERE t~bukrs IN sd_bukrs                           "3412695
1092                  AND t~xtemplt NE 'X'                              "2697244
1093                  AND l~rldnr = gv_relevant_ledger                  "3412695
1094                  ORDER BY t~BUKRS.
1095
1096    LOOP AT lt_bukrs INTO ls_bukrs.                                 "3412695
1097      CLEAR lt_bukrs_tp_curtp.
1098      CALL METHOD cl_fins_acdoc_util=>get_transfer_price_currtypes
1099        EXPORTING
1100          iv_bukrs          = ls_bukrs                            "3412695
1101          iv_rldnr          = gv_relevant_ledger
1102        IMPORTING
1103          et_bukrs_tp_curtp = lt_bukrs_tp_curtp.
1104
1105      INSERT LINES OF lt_bukrs_tp_curtp INTO TABLE gt_bukrs_tp_curtp.
1106
1107    ENDLOOP.
1108  * end of * begin of "Transfer Pricing authority check
1109
1110  * check tp authority and set 'tech'-attribute if authority is missing
1111    PERFORM make_fieldcatalog2.
1112
1113  * check selections against accounttypes
1114  * perform check_sel_for_acctype. -> see get skb1
1115
1116  * determine settllement accounts
1117    PERFORM setlmnt_accounts.
1118
1119  * MOVED above the 'exclude_pl_accounts', as it possibly changes the SD_B
     UKRS
1120  * analyze, if column text for additional currencies can be the currency
     type
1121    PERFORM analyze_curt.                                           "2658858
1122
1123  * if report called from balance display:
1124  * Exclude PL accounts now (will be included later).
1125    PERFORM exclude_pl_accounts.
1126
1127  * create appl log
1128    PERFORM create_log.
1129
1130  ENDFORM.                    "deco_at_selection_start
1131
1132  *&---------------------------------------------------------------------*
1133  *&      Form  deco_at_selection_get
1134  *&---------------------------------------------------------------------*
1135  *       text
1136  *----------------------------------------------------------------------*
1137  FORM deco_at_selection_get.
1138  * check accounts according to the selections
1139    IF  x_merk IS INITIAL
1140    AND x_park IS INITIAL
1141    AND x_erfs IS INITIAL.
```

```
1142  *   if not x_opsel is initial
1143      IF NOT x_clsel IS INITIAL.
1144        IF NOT skb1-xopvw IS INITIAL
1145        OR NOT skb1-xlgclr IS INITIAL
1146        OR NOT skb1-x_uj_clr IS INITIAL   .
1147        ELSE.
1148          IF NOT skb1-xkres IS INITIAL
1149          AND gv_relevant_ledger = gv_leading_ledger.
1150          ELSE.
1151            IF cl_fin_ui_deco=>mv_mode = abap_false.
1152              REJECT.
1153            ELSE.
1154              EXIT.
1155            ENDIF.
1156          ENDIF.
1157        ENDIF.
1158      ENDIF.
1159    ELSE.
1160      IF  skb1-xopvw IS INITIAL
1161      AND skb1-xkres IS INITIAL
1162      AND skb1-xlgclr IS INITIAL
1163      AND skb1-x_uj_clr IS INITIAL.
1164        IF cl_fin_ui_deco=>mv_mode = abap_false.
1165          REJECT.
1166        ELSE.
1167          EXIT.
1168        ENDIF.
1169      ENDIF.
1170    ENDIF.
1171
1172  * company master data:
1173    PERFORM t001_info_fill USING skb1-bukrs.
1174  * GL account text:
1175    PERFORM skat_info_fill.
1176  * company account master data:
1177    PERFORM skb1_info_fill.
1178
1179  * Selection of lineitems
1180
1181  * map selection to lineitem selection ranges
1182  * add masterdata information
1183    PERFORM map_sel_to_items_mast_dat USING
1184                                        gb_xplacc
1185                                        gt_selection_part
1186                                      CHANGING
1187                                        gt_selection_full.
1188  * select items and fill them in Output table
1189    IF NOT skb1-begru IS INITIAL.
1190      AUTHORITY-CHECK OBJECT 'F_BKPF_BES'
1191        ID 'BRGRU' FIELD skb1-begru
1192        ID 'ACTVT' FIELD '03'.
1193      IF sy-subrc NE 0.
1194        IF cl_fin_ui_deco=>mv_mode = abap_false.
1195          REJECT.
1196        ELSE.
1197          EXIT.
1198        ENDIF.
1199      ENDIF.
1200    ENDIF.
1201  *  if sy-subrc = 0.
1202  *    if gv_xerfs is initial.
```

```
1203    PERFORM check_slt_account USING skb1-saknr
1204                                    skb1-bukrs
1205                                    skb1-xkres
1206                                    gb_xplacc.
1207  *    endif.
1208    CLEAR gv_special_account.                                  "1822909
1209    PERFORM check_trtm_account.                                "1004374
1210    PERFORM check_glyec_account.                               "1779180
1211    PERFORM select_items.
1212    IF NOT x_stop IS INITIAL.
1213      STOP.
1214    ENDIF.
1215  *  endif.
1216
1217  ENDFORM.                    "deco_at_selection_get
1218
1219  *&---------------------------------------------------------------------*
1220  *&      Form  deco_at_END_OF_SELECTION
1221  *&---------------------------------------------------------------------*
1222  *       text
1223  *----------------------------------------------------------------------*
1224  FORM deco_at_end_of_selection.
1225
1226  * refresh table with selected items, no more needed
1227    REFRESH it_sel.
1228  * set variant:
1229    gs_variant-report   = g_repid.
1230    gs_variant-username = sy-uname.
1231    gs_variant-variant  = pa_vari.
1232  * if report is called from balance display:
1233    PERFORM include_pl_accounts.              "Diff. in Date selection
1234  * save all selection criteria for refresh:
1235    PERFORM save_all_selections.
1236  * determine list display mode:
1237    PERFORM display_grid_or_classic.
1238  * read special fields if necessary:
1239  * perform special_fields_init using gs_variant.
1240    PERFORM special_fields_init USING gs_variant.
1241
1242  * BADI check authority additional data customer
1243    IF  NOT gd_act_imp_auth_c IS INITIAL
1244    AND NOT gd_ref_to_auth_c  IS INITIAL.
1245      PERFORM items_check_auth_c.
1246    ENDIF.
1247
1248  * add message for arch items
1249    IF NOT gd_count_arch IS INITIAL.
1250      IF NOT gd_usear IS INITIAL.
1251        MESSAGE i112 WITH gd_count_arch INTO gd_dummy.
1252        PERFORM msg_add_to_log.
1253      ELSE.
1254        MESSAGE i113 WITH gd_count_arch INTO gd_dummy.
1255        PERFORM msg_add_to_log.
1256      ENDIF.
1257    ENDIF.
1258
1259  * message for items with no authority                          "1511192
1260    IF NOT gd_count_noauth IS INITIAL.
1261  *   deco_message_1 i126 with gd_count_noauth.                  "2604440
1262      CALL FUNCTION 'CUSTOMIZED_MESSAGE'                         "2604440
1263        EXPORTING                                                "2604440
```

```
1264          i_arbgb = 'MSITEM'                                    "2604440
1265          i_dtype = 'I'                                         "2604440
1266          i_msgnr = '126'                                       "2604440
1267          i_var01 = gd_count_noauth.                            "2604440
1268      ENDIF.
1269
1270    * export general data used in header info:                    "3446827
1271      PERFORM export_filitexts_data.                              "3446827
1272
1273    * number of selected items:
1274      IF x_stop = 'X'.
1275        deco_message_1 i023 with pa_nmax.
1276      ELSE.
1277        DESCRIBE TABLE it_pos LINES n_lines.
1278        IF n_lines > 0.
1279          deco_message_1 s024 with n_lines.
1280        ELSEIF pa_inet IS INITIAL AND gd_dynp_val IS INITIAL.
1281          PERFORM check_msg_exist USING gs_log_handle
1282                                  CHANGING gd_xmsg.
1283
1284          IF gd_xmsg IS INITIAL.
1285            deco_message_0 s033.
1286            EXIT.
1287          ELSE.
1288            MESSAGE s033 INTO gd_dummy.
1289            PERFORM msg_add_to_log.
1290            APPEND gs_log_handle TO gt_log_handle.
1291            PERFORM print_appl_log.
1292            EXIT.
1293          ENDIF.
1294        ELSE.
1295          deco_message_0 i033.
1296          PERFORM deco_leave_program.
1297        ENDIF.
1298      ENDIF.
1299
1300    * authority display or change
1301      PERFORM authority_tcode USING 'FB02' subrc.
1302      IF subrc EQ 0.
1303        x_change = 'X'.
1304      ELSE.
1305        x_change = space.
1306      ENDIF.
1307
1308    ENDFORM.                    "deco_at_END_OF_SELECTION
1309
1310    *&---------------------------------------------------------------------*
1311    *&      Form  deco_at_output_alv
1312    *&---------------------------------------------------------------------*
1313    *       text
1314    *----------------------------------------------------------------------*
1315    FORM deco_at_output_alv.
1316
1317    * adapt fieldcat
1318      DATA ls_fieldcat TYPE slis_fieldcat_alv.
1319      DATA lv_add_curr_column_text TYPE ty_add_curr_column_text.
1320      FIELD-SYMBOLS <fieldcat> TYPE slis_fieldcat_alv.
1321
1322      LOOP AT gt_add_curr_column_text INTO lv_add_curr_column_text.
1323
1324        READ TABLE gt_fieldcat WITH KEY fieldname = lv_add_curr_column_text-
```

```
      fieldname ASSIGNING <fieldcat>.  "note 2398274
1325      IF sy-subrc <> 0. "note 2398274
1326
1327        CLEAR ls_fieldcat.
1328        ls_fieldcat-fieldname       = lv_add_curr_column_text-fieldname.
1329        ls_fieldcat-seltext_l       = lv_add_curr_column_text-seltext_l.
1330        ls_fieldcat-seltext_m       = lv_add_curr_column_text-seltext_m.
1331        ls_fieldcat-seltext_s       = lv_add_curr_column_text-seltext_s.
1332        ls_fieldcat-reptext_ddic    = ls_fieldcat-seltext_l.     "3674961
1333  *      ls_fieldcat-ddictxt = 'L'.                              "3447319
1334
1335        IF lv_add_curr_column_text-fieldname = 'DMBE2' OR
1336           lv_add_curr_column_text-fieldname = 'DMBE3' OR
1337           lv_add_curr_column_text-fieldname = 'DMBE4' OR
1338           lv_add_curr_column_text-fieldname = 'DMBE5' OR
1339           lv_add_curr_column_text-fieldname = 'DMBE6' OR
1340           lv_add_curr_column_text-fieldname = 'DMBE7' OR
1341           lv_add_curr_column_text-fieldname = 'DMBE8' OR
1342           lv_add_curr_column_text-fieldname = 'DMBE9' OR
1343           lv_add_curr_column_text-fieldname = 'DMBE10'.
1344          ls_fieldcat-do_sum = 'X'.
1345        ENDIF.
1346
1347        APPEND ls_fieldcat TO gt_fieldcat.
1348
1349        CLEAR ls_fieldcat.
1350        CONCATENATE 'HWAE' lv_add_curr_column_text-fieldname+4(2) INTO ls_
      fieldcat-fieldname.
1351        CONCATENATE text-war lv_add_curr_column_text-seltext_l(10) INTO ls
      _fieldcat-seltext_l. "3416896
1352        CONCATENATE text-war lv_add_curr_column_text-seltext_m(10) INTO ls
      _fieldcat-seltext_m. "3416896
1353        CONCATENATE text-war lv_add_curr_column_text-seltext_s(10) INTO ls
      _fieldcat-seltext_s. "3416896
1354        ls_fieldcat-reptext_ddic = ls_fieldcat-seltext_l.        "3674961
1355
1356  *      ls_fieldcat-ddictxt = 'S'.                              "3447319
1357
1358        APPEND ls_fieldcat TO gt_fieldcat.
1359      ELSE. "note 2398274
1360
1361  * begin of "note 2398274
1362        CLEAR ls_fieldcat.
1363        ls_fieldcat-fieldname       = lv_add_curr_column_text-fieldname.
1364        <fieldcat>-seltext_l       = lv_add_curr_column_text-seltext_l.
1365        <fieldcat>-seltext_m       = lv_add_curr_column_text-seltext_m.
1366        <fieldcat>-seltext_s       = lv_add_curr_column_text-seltext_s.
1367        <fieldcat>-reptext_ddic    = <fieldcat>-seltext_l.       "3674961
1368  *      <fieldcat>-ddictxt = 'L'.                               "3447319
1369
1370        IF lv_add_curr_column_text-fieldname = 'DMBE2' OR
1371           lv_add_curr_column_text-fieldname = 'DMBE3' OR
1372           lv_add_curr_column_text-fieldname = 'DMBE4' OR
1373           lv_add_curr_column_text-fieldname = 'DMBE5' OR
1374           lv_add_curr_column_text-fieldname = 'DMBE6' OR
1375           lv_add_curr_column_text-fieldname = 'DMBE7' OR
1376           lv_add_curr_column_text-fieldname = 'DMBE8' OR
1377           lv_add_curr_column_text-fieldname = 'DMBE9' OR
1378           lv_add_curr_column_text-fieldname = 'DMBE10'.
1379          <fieldcat>-do_sum = 'X'.
1380        ENDIF.
```

```
1381
1382
1383        DATA lv_currkey_field TYPE fieldname.
1384        CLEAR lv_currkey_field.
1385        CONCATENATE 'HWAE' lv_add_curr_column_text-fieldname+4(2) INTO lv_
      currkey_field.
1386
1387        READ TABLE gt_fieldcat WITH KEY fieldname = lv_currkey_field ASSIG
      NING <fieldcat>.  "note 2398274
1388        ASSERT sy-subrc = 0.
1389
1390        CONCATENATE text-war lv_add_curr_column_text-seltext_l(10) INTO <f
      ieldcat>-seltext_l. "3416896
1391        CONCATENATE text-war lv_add_curr_column_text-seltext_m(10) INTO <f
      ieldcat>-seltext_m. "3416896
1392        CONCATENATE text-war lv_add_curr_column_text-seltext_s(10) INTO <f
      ieldcat>-seltext_s. "3416896
1393        <fieldcat>-reptext_ddic = <fieldcat>-seltext_l.          "3674961
1394
1395  *      <fieldcat>-ddictxt = 'S'.                               "3447319
1396  * end of "note 2398274
1397
1398      ENDIF.   "note 2398274
1399
1400    ENDLOOP.
1401
1402    IF gv_xerfs = 'X'.
1403      LOOP AT gt_fieldcat ASSIGNING <fieldcat>.
1404        IF <fieldcat>-fieldname = 'DMBE4' OR
1405           <fieldcat>-fieldname = 'HWAE4' OR
1406           <fieldcat>-fieldname = 'DMBE5' OR
1407           <fieldcat>-fieldname = 'HWAE5' OR
1408           <fieldcat>-fieldname = 'DMBE6' OR
1409           <fieldcat>-fieldname = 'HWAE6' OR
1410           <fieldcat>-fieldname = 'DMBE7' OR
1411           <fieldcat>-fieldname = 'HWAE7' OR
1412           <fieldcat>-fieldname = 'DMBE8' OR
1413           <fieldcat>-fieldname = 'HWAE8' OR
1414           <fieldcat>-fieldname = 'DMBE9' OR
1415           <fieldcat>-fieldname = 'HWAE9' OR
1416           <fieldcat>-fieldname = 'DMBE10' OR
1417           <fieldcat>-fieldname = 'DMBE10'.
1418
1419          <fieldcat>-tech = 'X'.
1420        ENDIF.
1421      ENDLOOP.
1422    ENDIF.
1423
1424  *  CLEAR ls_fieldcat.
1425  *  ls_fieldcat-fieldname       = 'DMBE3'.
1426  *  ls_fieldcat-seltext_l       = '1st freely defined Currency'."text-320
      .
1427  *  ls_fieldcat-seltext_m       = '1st free Currency'.
1428  *  ls_fieldcat-seltext_s       = '1st free Curr.'.
1429  *  ls_fieldcat-ddictxt = 'M'.
1430  *  APPEND ls_fieldcat TO gt_fieldcat.
1431  *
1432  *  CLEAR ls_fieldcat.
1433  *  ls_fieldcat-fieldname      = 'HWAE3'.
1434  *  ls_fieldcat-seltext_l       = 'curr. 1st fr. def. Curr.'(327)."text-3
      20.
```

```
1435  *  ls_fieldcat-seltext_m       = 'curr. 1st'(328).
1436  *  ls_fieldcat-seltext_s       = 'FDCur1'(329).
1437  *  ls_fieldcat-ddictxt = 'S'.
1438  *  APPEND ls_fieldcat TO gt_fieldcat.
1439  *
1440  *  CLEAR ls_fieldcat.
1441  *  ls_fieldcat-fieldname      = 'DMBE4'.
1442  *  ls_fieldcat-seltext_l       = '2nd freely defined Currency'(321)."tex
      t-320.
1443  *  ls_fieldcat-seltext_m       = '2nd free Currency'(322).
1444  *  ls_fieldcat-seltext_s       = '2nd free Curr.'(323).
1445  *  ls_fieldcat-ddictxt = 'M'.
1446  *  APPEND ls_fieldcat TO gt_fieldcat.
1447  *
1448  *   CLEAR ls_fieldcat.
1449  *  ls_fieldcat-fieldname      = 'HWAE4'.
1450  *  ls_fieldcat-seltext_l       = 'curr. 2nd fr. def. Curr.'(330)."text-3
      20.
1451  *  ls_fieldcat-seltext_m       = 'curr. 2nd'(331).
1452  *  ls_fieldcat-seltext_s       = 'FDCur2'(332).
1453  *  ls_fieldcat-ddictxt = 'S'.
1454  *  APPEND ls_fieldcat TO gt_fieldcat.
1455  *
1456  *  CLEAR ls_fieldcat.
1457  *  ls_fieldcat-fieldname      = 'DMBE5'.
1458  *  ls_fieldcat-seltext_l       = '3rd freely defined Currency'(324)."tex
      t-320.
1459  *  ls_fieldcat-seltext_m       = '3rd free Currency'(325).
1460  *  ls_fieldcat-seltext_s       = '3rd free Curr.'(326).
1461  *  ls_fieldcat-ddictxt = 'M'.
1462  *  APPEND ls_fieldcat TO gt_fieldcat.
1463  *
1464  *  CLEAR ls_fieldcat.
1465  *  ls_fieldcat-fieldname      = 'HWAE5'.
1466  *  ls_fieldcat-seltext_l       = 'curr. 3rd fr. def. Curr.'(333)."text-3
      20.
1467  *  ls_fieldcat-seltext_m       = 'curr. 3rd'(334).
1468  *  ls_fieldcat-seltext_s       = 'FDCur3'(335).
1469  *  ls_fieldcat-ddictxt = 'S'.
1470  *  APPEND ls_fieldcat TO gt_fieldcat.
1471
1472    " Begin Changes of Note 3040258
1473    " Duplicate Documents can be present when this parameter
1474    " FAGL_OUTER_JOIN = 'ENFORCE' as per this note 2857175 and
1475    " when both from Database and Archive is read
1476
1477    IF gd_enforce_outer_join = 'X' AND
1478             gd_usedb = 'X' AND gd_usear = 'X'.
1479      SORT it_pos STABLE BY bukrs belnr gjahr buzei docln rldnr xarch DESC
      ENDING.
1480      DELETE ADJACENT DUPLICATES FROM it_pos COMPARING bukrs belnr gjahr b
      uzei docln rldnr.
1481    ENDIF.
1482
1483    "End Changes of Note 3040258
1484
1485  * leave to ALV:
1486    CALL FUNCTION 'SAPGUI_PROGRESS_INDICATOR'
1487      EXPORTING
1488        text = TEXT-009.
1489
```

```
1490    CALL FUNCTION 'FAGL_ITEMS_DISPLAY'
1491      EXPORTING
1492        caller_repid  = c_repid_gl
1493        i_rldnr       = gv_relevant_ledger
1494        acctype       = c_koart_gl
1495        x_opvw        = x_gl_opvw
1496        x_change      = x_change
1497        i_u_save      = gd_alvsave
1498        is_u_variant  = gs_variant
1499        it_u_fieldcat = gt_fieldcat[]
1500        it_kontab     = it_accts[]
1501        it_slbtab     = it_comps[]
1502        it_t001       = it_h_t001[]
1503        it_skat       = it_h_skat[]
1504        it_skb1       = it_h_skb1[]
1505        x_grid        = x_grid
1506        x_inet        = pa_inet
1507        i_log_handle  = gs_log_handle
1508        i_ucomm       = gv_ucomm
1509      TABLES
1510        it_items      = it_pos.
1511
1512    FREE it_h_t001.                                          "1825079
1513    FREE it_h_skat.                                          "1825079
1514    FREE it_h_skb1.                                          "1825079
1515    FREE it_pos.                                             "1825079
1516
1517  ENDFORM.                    "deco_at_output_alv
1518
1519  *---------------------------------------------------------------------*
1520  *  FORM SEL_ACCOUNT_CHECK
1521  *---------------------------------------------------------------------*
1522  FORM sel_account_check.
1523
1524    DATA: ld_lines          LIKE sy-index,
1525          ld_single_account TYPE c,
1526          ld_single_bukrs   TYPE c.
1527
1528
1529    READ TABLE sd_saknr INDEX 1.
1530    IF sy-subrc NE 0.
1531      READ TABLE sd_bukrs INDEX 1.
1532      IF sy-subrc NE 0.
1533        deco_message_0 w019.
1534      ENDIF.
1535    ENDIF.
1536
1537    DESCRIBE TABLE sd_saknr LINES ld_lines.
1538    LOOP AT sd_saknr TRANSPORTING NO FIELDS
1539         WHERE sign NE 'I' OR
1540               option NE 'EQ'.
1541      EXIT.
1542    ENDLOOP.
1543
1544    IF sy-subrc = 0 OR ld_lines = 0.
1545      SELECT SINGLE saknr bukrs FROM skb1                    "#EC CI_GENBUFF
1546             INTO (gd_saknr_msg, gd_bukrs_msg)
1547             WHERE saknr IN sd_saknr
1548               AND bukrs IN sd_bukrs.
1549      IF sy-subrc NE 0.
1550        deco_message_0 e030(msitem).
```

```
1551      ENDIF.
1552      IF NOT x_erfs IS INITIAL
1553      OR NOT x_merk IS INITIAL
1554      OR NOT x_park IS INITIAL.
1555        SELECT SINGLE saknr FROM skb1 INTO skb1-saknr     "#EC CI_GENBUFF
1556               WHERE saknr IN sd_saknr
1557                 AND bukrs IN sd_bukrs
1558                 AND ( xkres = 'X' OR xlgclr = 'X' ).
1559        IF sy-subrc NE 0.
1560          deco_message_2 e430(f4) with gd_saknr_msg gd_bukrs_msg.
1561        ENDIF.
1562      ENDIF.
1563    ELSE.  "worklist
1564      SELECT saknr bukrs FROM skb1
1565         INTO (gd_saknr_msg, gd_bukrs_msg)
1566         FOR ALL ENTRIES IN sd_saknr
1567         WHERE saknr = sd_saknr-low
1568           AND bukrs IN sd_bukrs.
1569        EXIT.
1570      ENDSELECT.
1571      IF sy-subrc NE 0.
1572        deco_message_0 e030(msitem).
1573      ENDIF.
1574      IF NOT x_erfs IS INITIAL
1575      OR NOT x_merk IS INITIAL
1576      OR NOT x_park IS INITIAL.
1577        SELECT saknr FROM skb1 INTO skb1-saknr
1578               FOR ALL ENTRIES IN sd_saknr
1579               WHERE saknr = sd_saknr-low
1580                 AND bukrs IN sd_bukrs
1581                 AND ( xkres = 'X' OR xlgclr = 'X' ).
1582          EXIT.
1583        ENDSELECT.
1584        IF sy-subrc NE 0.
1585          deco_message_2 e430(f4) with gd_saknr_msg gd_bukrs_msg.
1586        ENDIF.
1587      ENDIF.
1588    ENDIF.
1589  *...write sd_saknr und sd_bukrs into memory
1590  *...(important later for Dispute-Management).
1591
1592    EXPORT sd_saknr TO MEMORY ID 'FILITEXTS_SAKNR'.
1593    EXPORT sd_bukrs TO MEMORY ID 'FILITEXTS_BUKRS'.
1594
1595  *...find out, if single accounts and company codes have been selected..*
1596  *...(important for header display).....................................*
1597
1598    CLEAR ld_single_account.
1599    DESCRIBE TABLE sd_saknr LINES ld_lines.
1600    IF ld_lines EQ 1.
1601      READ TABLE sd_saknr INDEX 1.
1602      IF ( sd_saknr-sign EQ 'I' AND sd_saknr-option EQ 'EQ' ) OR
1603         ( sd_saknr-sign EQ 'I' AND sd_saknr-option EQ 'BT' AND
1604           sd_saknr-low  EQ sd_saknr-high ).
1605        ld_single_account = 'X'.
1606      ENDIF.
1607    ENDIF.
1608    CLEAR ld_single_bukrs.
1609    DESCRIBE TABLE sd_bukrs LINES ld_lines.
1610    IF ld_lines EQ 1.
1611      READ TABLE sd_bukrs INDEX 1.
```

```
1612      IF ( sd_bukrs-sign EQ 'I' AND sd_bukrs-option EQ 'EQ' ) OR
1613         ( sd_bukrs-sign EQ 'I' AND sd_bukrs-option EQ 'BT' AND
1614           sd_bukrs-low  EQ sd_bukrs-high ).
1615        ld_single_bukrs = 'X'.
1616      ENDIF.
1617    ENDIF.
1618    EXPORT ld_single_bukrs ld_single_account TO MEMORY ID
1619                                          'FILITEXTS_SINGLE'.
1620
1621  ENDFORM.                    "sel_account_check
1622
1623  *---------------------------------------------------------------------*
1624  *  FORM SEL_TYPE_CHECK
1625  *---------------------------------------------------------------------*
1626  FORM sel_type_check.
1627    IF x_norm IS INITIAL AND x_merk IS INITIAL AND x_park IS INITIAL.
1628      deco_message_0 e020.
1629    ENDIF.
1630  ENDFORM.                    "sel_type_check
1631
1632  *---------------------------------------------------------------------*
1633  *      Form  POS_TABLE_FILL
1634  *---------------------------------------------------------------------*
1635  FORM pos_table_fill  CHANGING  p_stop.
1636    DATA: text_index TYPE i,
1637          okay       TYPE c VALUE space,
1638          x_noauth   TYPE c VALUE space.
1639
1640
1641    CLEAR wa_pos.
1642  * move-corresponding bsis  to wa_pos.
1643    MOVE-CORRESPONDING wa_sel  TO wa_pos.
1644  *... check item against selection flags:
1645    PERFORM check_item_ok  USING x_norm
1646                                 'X'
1647                                 x_merk
1648                                 x_park
1649                                 wa_pos
1650                           CHANGING okay.
1651    CHECK okay = 'X'.
1652  *
1653    wa_pos-koart = c_koart_gl.
1654  *  wa_pos-konto = bsis-hkont.
1655  *  wa_pos-dmshb = bsis-dmbtr.
1656  *  wa_pos-wrshb = bsis-wrbtr.
1657    wa_pos-konto = wa_sel-hkont.
1658  *  wa_pos-dmshb = wa_sel-dmbtr.
1659  *  wa_pos-wrshb = wa_sel-wrbtr.
1660
1661  * recon. account + line item display => open item management:
1662    IF ( skb1-mitkz EQ 'D' OR skb1-mitkz EQ 'K' ) AND skb1-xkres NE space.
1663  *   Possibly item from the account before it was a recon.      "3402296
1664  *   account that was archived; XPOVW should nto be set, so     "3402296
1665  *   it is displayed only as POSTED                             "3402296
1666      IF wa_sel-augbl IS INITIAL AND wa_sel-xarch = 'X'          "3402296
1667        AND wa_sel-xopvw IS INITIAL.                             "3402296
1668      ELSE.                                                      "3402296
1669        wa_sel-xopvw = 'X'.
1670      ENDIF.                                                     "3402296
1671  * Ledgergroup specific clearing => open item
1672    ELSEIF skb1-xlgclr = 'X'.
```

```
1673      wa_sel-xopvw = 'X'.
1674    ENDIF.
1675
1676  * Valuation fields not yet supported in NewGL
1677  *  wa_bsegp-bdiff = bsis-bdiff.
1678  *  wa_bsegp-bdif2 = bsis-bdif2.
1679  *  wa_bsegp-bdif3 = bsis-bdif3.
1680    wa_bsegp-bdiff = wa_sel-bdiff.
1681    wa_bsegp-bdif2 = wa_sel-bdif2.
1682    wa_bsegp-bdif3 = wa_sel-bdif3.
1683
1684    IF gv_special_account = 'X' AND ( gv_relevant_ledger NE gv_leading_led
      ger ). "3061766
1685      CLEAR wa_sel-xopvw .                                  "3061766
1686    ENDIF.                                                  "3061766
1687
1688    "  Note 2641134
1689    "  IF gv_special_account = 'X'.                         "1004374
1690    "    CLEAR wa_sel-xopvw .                               "1004374
1691    "  ENDIF.                                               "1004374
1692
1693  * derive nontrivial rfpos fields:
1694    CALL FUNCTION 'FAGL_ITEM_DERIVE_FIELDS'
1695      EXPORTING
1696        s_t001    = it_h_t001
1697        s_bsegp   = wa_bsegp
1698        key_date  = p_keydate
1699  *     xopvw     = bsis-xopvw
1700        xopvw     = wa_sel-xopvw
1701      CHANGING
1702        s_item    = wa_pos
1703      EXCEPTIONS
1704        bad_input = 1
1705        OTHERS    = 2.
1706    IF sy-subrc NE 0.
1707      deco_message_0 a022.
1708    ENDIF.
1709  * fill currency fields:
1710  * perform item_currency_fields.
1711
1712  * check authority for fields
1713    PERFORM item_check_auth CHANGING x_noauth.
1714    IF NOT x_noauth IS INITIAL.                             "1511192
1715      gd_count_noauth = gd_count_noauth + 1.                "1511192
1716    ENDIF.                                                  "1511192
1717
1718  * expiring currency convertion
1719    IF NOT gd_expcur_flag IS INITIAL.
1720      PERFORM curr_convert_item.
1721    ENDIF.
1722
1723  * check dark select-options and add item to table:
1724    IF x_noauth IS INITIAL.
1725      PERFORM item_check_append.
1726    ENDIF.
1727
1728  * check max number:
1729    DESCRIBE TABLE it_pos LINES n_lines.
1730    IF pa_nmax > 0 AND n_lines GE pa_nmax.
1731      p_stop = 'X'.
1732  *   stop.
```

```
1733    ENDIF.
1734  ENDFORM.                    "pos_table_fill
1735
1736  *&---------------------------------------------------------------------*
1737  *&      Form  T001_INFO_FILL
1738  *&---------------------------------------------------------------------*
1739  FORM t001_info_fill USING ip_bukrs TYPE skb1-bukrs.
1740
1741    READ TABLE it_h_t001 WITH KEY mandt = sy-mandt
1742                                  bukrs = ip_bukrs
1743                         BINARY SEARCH.
1744    IF sy-subrc NE 0.
1745      CLEAR: it_h_t001.
1746      SELECT SINGLE * FROM t001 INTO it_h_t001
1747          WHERE bukrs = ip_bukrs.
1748      INSERT TABLE it_h_t001.
1749
1750  *     table of companies:
1751      PERFORM fill_comp_table  USING  ip_bukrs
1752                                it_h_t001-waers
1753                                it_h_t001-kkber.
1754    ENDIF.
1755
1756    IF wa_x001-bukrs NE skb1-bukrs.
1757      CLEAR: wa_x001.
1758      CALL FUNCTION 'FI_CURRENCY_INFORMATION'
1759        EXPORTING
1760          i_bukrs                = ip_bukrs
1761        IMPORTING
1762          e_x001                 = wa_x001
1763        EXCEPTIONS
1764          currency_2_not_defined = 1
1765          currency_3_not_defined = 2
1766          OTHERS                 = 3.
1767      IF sy-subrc = 0.
1768  * ?
1769      ENDIF.
1770    ENDIF.
1771
1772  ENDFORM.                    "t001_info_fill
1773
1774  *&---------------------------------------------------------------------*
1775  *&      Form  SKAT_INFO_FILL
1776  *&---------------------------------------------------------------------*
1777  FORM skat_info_fill.
1778    DATA: langu   LIKE sy-langu.
1779    CLEAR it_h_skat.
1780    IF idtimeacc_cl_switch_check=>filoc_time_acc_sfws_01( ) EQ 'X'.
1781      DATA:
1782        lv_key_date   TYPE datum.
1783  *   Choose key date
1784      lv_key_date = idtimeacc_cl_utils=>choose_key_date(
1785          im_op_itm_flg       = x_opsel
1786          im_cl_itm_flg       = x_clsel
1787          im_al_itm_flg       = x_aisel
1788          im_op_key_dat       = pa_stida
1789          im_cl_key_dat       = pa_stid2
1790          im_rng_cl_key_dats  = so_augdt[]
1791          im_rng_all_key_dats = so_budat[]
1792      ).
1793  *   Find G/L Account Text By Two Languages
```

```
1794      CALL METHOD idtimeacc_cl_utils=>read_glacc_text_by2_langs
1795        EXPORTING
1796          im_chart_accnts = it_h_t001-ktopl
1797          im_accnt_no     = skb1-saknr
1798          im_sec_langu    = it_h_t001-spras
1799          im_date_from    = lv_key_date
1800        RECEIVING
1801          re_str_acctxt   = it_h_skat
1802        EXCEPTIONS
1803          not_maintained  = 1
1804          OTHERS          = 2.
1805      IF sy-subrc = 0.
1806        INSERT TABLE it_h_skat.
1807        EXIT.
1808      ENDIF.
1809    ENDIF. " Note 1731392
1810
1811    SELECT SINGLE * FROM skat INTO it_h_skat
1812         WHERE spras = sy-langu
1813         AND   ktopl = it_h_t001-ktopl
1814         AND   saknr = skb1-saknr.
1815    IF sy-subrc = 0.
1816      INSERT TABLE it_h_skat.
1817      EXIT.
1818    ENDIF.
1819    SELECT SINGLE * FROM skat INTO it_h_skat
1820         WHERE spras = it_h_t001-spras
1821         AND   ktopl = it_h_t001-ktopl
1822         AND   saknr = skb1-saknr.
1823    IF sy-subrc = 0.
1824      INSERT TABLE it_h_skat.
1825      EXIT.
1826    ENDIF.
1827    SELECT SINGLE * FROM skat INTO it_h_skat
1828         WHERE ktopl = it_h_t001-ktopl
1829         AND   saknr = skb1-saknr.
1830    IF sy-subrc = 0.
1831      INSERT TABLE it_h_skat.
1832      EXIT.
1833    ENDIF.
1834  ENDFORM.                    "skat_info_fill
1835
1836  *&---------------------------------------------------------------------*
1837  *&      Form  SKB1_INFO_FILL
1838  *&---------------------------------------------------------------------*
1839  FORM skb1_info_fill.
1840    DATA: sak_name   LIKE kna1-name1.
1841    CLEAR it_h_skb1.
1842    MOVE-CORRESPONDING skb1 TO it_h_skb1.
1843    INSERT TABLE it_h_skb1.
1844  * table of accounts:
1845    IF idtimeacc_cl_switch_check=>filoc_time_acc_sfws_01( ) EQ 'X'.
1846      DATA:
1847        lv_key_date   TYPE datum,
1848        lv_skat_txt50 TYPE txt50.
1849  *   Choose key date
1850      lv_key_date = idtimeacc_cl_utils=>choose_key_date(
1851          im_op_itm_flg       = x_opsel
1852          im_cl_itm_flg       = x_clsel
1853          im_al_itm_flg       = x_aisel
1854          im_op_key_dat       = pa_stida
```

```
1855          im_cl_key_dat       = pa_stid2
1856          im_rng_cl_key_dats  = so_augdt[]
1857          im_rng_all_key_dats = so_budat[]
1858       ).
1859       CALL FUNCTION 'IDCN_GLACC_TEXT_GET'
1860         EXPORTING
1861           kontenplan     = it_h_t001-ktopl
1862           sachkonto      = skb1-saknr
1863           dfrom          = lv_key_date
1864         IMPORTING
1865           txt50          = lv_skat_txt50
1866         EXCEPTIONS
1867           not_maintained = 1
1868           OTHERS         = 2.
1869       IF sy-subrc = 0.
1870         MOVE lv_skat_txt50 TO skat-txt50.
1871       ENDIF.
1872     ENDIF. " Note 1731392
1873
1874     WRITE skat-txt50 TO sak_name.
1875     PERFORM fill_acct_table  USING  c_koart_gl
1876                                     skb1-saknr
1877                                     skb1-bukrs
1878                                     space
1879                                     sak_name
1880                                     skb1-saknr.
1881   ENDFORM.                    "skb1_info_fill
1882
1883
1884   *&---------------------------------------------------------------------*
1885   *&      Form  MAP_SEL_TO_LDB
1886   *&---------------------------------------------------------------------*
1887   *form map_sel_to_ldb.
1888   *
1889   *  data: ld_stida like sy-datum.
1890   *
1891   *  clear:   sd_augdt, sd_budat, sd_opopt, sd_apopt, sd_stida.
1892   *  refresh: sd_augdt, sd_budat.
1893   *
1894   ** Determine whether parked (xstav) and/or normal items
1895   ** (xstan) must be selected by the log. database
1896   *  if x_park = 'X'.
1897   *    b0sg-xstav = 'X'.
1898   *  else.
1899   *    clear b0sg-xstav.
1900   *  endif.
1901   *  if ( x_norm = 'X' or x_shbv = 'X' or x_merk = 'X' ).
1902   *    b0sg-xstan = 'X'.
1903   *  else.
1904   *    clear b0sg-xstan.
1905   *  endif.
1906   *
1907   ** Determine relevant keydates for the selection by the log. database
1908   ** (sd_stida)
1909   *  get parameter id 'FI_STIDA' field ld_stida.
1910   *  if not ld_stida is initial and sy-subrc eq 0.
1911   *    pa_stida = ld_stida.
1912   *    set parameter id 'FI_STIDA' field '00000000'.
1913   *  endif.
1914   *  case 'X'.
1915   *    when x_opsel.
```

```
1916  *      if pa_stida is initial.
1917  *        p_keydate         = '99991231'.
1918  *        sd_stida          = '99991231'.
1919  *      else.
1920  *        p_keydate         = pa_stida.
1921  *        sd_stida          = pa_stida.
1922  *      endif.
1923  *      sd_opopt          = 'X'.
1924  *    when x_clsel.
1925  *      if pa_stid2 is initial.
1926  *        p_keydate       = '99991231'.
1927  *        sd_stida        = '00010101'.
1928  *      else.
1929  *        p_keydate       = pa_stid2.
1930  *        sd_stida        = pa_stid2.
1931  *        sd_budat-sign   = 'I'.
1932  *        sd_budat-option = 'LE'.
1933  *        sd_budat-low    = pa_stid2.
1934  *        append sd_budat.
1935  *      endif.
1936  *      sd_augdt[]        = so_augdt[].
1937  *    when x_aisel.
1938  *      p_keydate         = '99991231'.
1939  *      sd_stida          = '99991231'.
1940  *      sd_budat[]        = so_budat[].
1941  *      sd_opopt          = 'X'.
1942  *      sd_apopt          = 'X'.
1943  *  endcase.
1944  *  pa_stida_default = sd_stida.
1945  *
1946  ** For reading from archives:
1947  ** number of archived items as i-message:
1948  *  sd_iarch = 'X'.
1949  *  sd_memor = 'X'.
1950  *
1951  *endform.                    "map_sel_to_ldb
1952
1953  *&---------------------------------------------------------------------*
1954  *&      Form  map_sel_to_items
1955  *&---------------------------------------------------------------------*
1956  FORM map_sel_to_items USING
1957                          p_gb_xplacc TYPE boolean
1958                        CHANGING
1959                          p_gt_selection_part TYPE  gusl_t_selection.
1960
1961    DATA: lt_selection   TYPE  gusl_t_selection.
1962    DATA: ls_selection   TYPE  gusl_s_selection.
1963    FIELD-SYMBOLS: <wa1> TYPE any.
1964    DATA: ls_selopt      TYPE rsdsselopt.
1965    DATA: ls_selopt_t    TYPE rsds_selopt_t.
1966    DATA: ld_si_table_fs TYPE tabname.
1967    DATA: ld_bsis_fs     TYPE tabname VALUE 'BSIS_FS'.
1968    DATA: ls_trange_tab_line TYPE rsds_range.
1969    DATA: ls_frange_tab_line TYPE rsds_frange.
1970    DATA: ld_stida       TYPE budat.
1971    DATA: ld_stida_2     TYPE budat.
1972    DATA: lt_augdt   TYPE range_date OCCURS 0 WITH HEADER LINE.
1973    DATA: lt_budat   TYPE range_date OCCURS 0 WITH HEADER LINE.
1974    RANGES: lt_saknr     FOR skb1-saknr.
1975    RANGES: lt_bukrs     FOR skb1-bukrs.
1976    RANGES: lt_bstat     FOR faglflexa-bstat.
```

```
1977    RANGES: lt_rldnr     FOR faglflexa-rldnr.
1978    RANGES: lt_rrcty     FOR faglflexa-rrcty.                    "2230542
1979    RANGES: lt_xarch     FOR bsas-xarch.
1980
1981    DATA: ld_lines LIKE sy-tabix.
1982
1983  * ledger
1984    IF  x_merk IS INITIAL
1985    AND x_park IS INITIAL
1986    AND x_erfs IS INITIAL.
1987      lt_rldnr-sign = 'I'.
1988      lt_rldnr-option = 'EQ'.
1989      lt_rldnr-low    = gv_relevant_ledger.
1990      APPEND lt_rldnr.
1991
1992      REFRESH ls_selopt_t.
1993      LOOP AT lt_rldnr ASSIGNING <wa1>.
1994        MOVE-CORRESPONDING <wa1> TO ls_selopt.
1995        APPEND ls_selopt TO ls_selopt_t.
1996      ENDLOOP.
1997      ls_selection-fieldname = 'RLDNR'.
1998      ls_selection-t_range  = ls_selopt_t.
1999      APPEND ls_selection TO lt_selection.
2000    ENDIF.
2001
2002  * document status
2003    IF NOT x_norm IS INITIAL.
2004      lt_bstat-sign = 'I'.
2005      lt_bstat-option = 'EQ'.
2006      lt_bstat-low    = ' '.
2007      APPEND lt_bstat.
2008      IF NOT gv_relevant_ledger IS INITIAL.
2009        lt_bstat-sign = 'I'.
2010        lt_bstat-option = 'EQ'.
2011        lt_bstat-low    = 'L'.
2012        APPEND lt_bstat.
2013        lt_bstat-low    = if_fins_acdoc_constants=>gc_document_status-gl_o
      nly.
2014        APPEND lt_bstat.
2015        IF cl_fins_acdoc_util=>is_simulation_ledger( iv_rldnr = gv_relevan
      t_ledger ) = abap_true.
2016          lt_bstat-low    = if_fins_acdoc_constants=>gc_document_status-gl
      _simulation.
2017          APPEND lt_bstat.
2018        ENDIF.
2019  *      Posting to prediction ledger using BSTAT 'P'
2020        IF cl_fins_acdoc_util=>is_prediction_ledger( iv_rldnr = gv_relevan
      t_ledger ) = abap_true.
2021          lt_bstat-low    = if_fins_acdoc_constants=>gc_document_status-pr
      edictive.
2022          APPEND lt_bstat.
2023        ENDIF.
2024      ENDIF.
2025      IF NOT x_glyec IS INITIAL.                                  "2230542
2026        lt_bstat-sign = 'I'.
2027        lt_bstat-option = 'EQ'.
2028        lt_bstat-low    = 'J'.
2029        APPEND lt_bstat.
2030      ENDIF.
2031      IF gd_enforce_outer_join = abap_true.                       " 2857175
2032        lt_bstat-sign = 'I'.                                      " 2857175
```

```
2033        lt_bstat-option = 'EQ'.                                  " 2857175
2034        lt_bstat-low    = 'C'.                                   " 2857175
2035        APPEND lt_bstat.                                         " 2857175
2036      ENDIF.                                                     " 2857175
2037    ENDIF.
2038    IF NOT x_merk IS INITIAL.
2039      lt_bstat-sign = 'I'.
2040      lt_bstat-option = 'EQ'.
2041      lt_bstat-low    = 'S'.
2042      APPEND lt_bstat.
2043    ENDIF.
2044    IF NOT x_park IS INITIAL.
2045      lt_bstat-sign = 'I'.
2046      lt_bstat-option = 'EQ'.
2047      lt_bstat-low    = 'V'.
2048      APPEND lt_bstat.
2049      lt_bstat-sign = 'I'.
2050      lt_bstat-option = 'EQ'.
2051      lt_bstat-low    = 'W'.
2052      APPEND lt_bstat.
2053    ENDIF.
2054    REFRESH ls_selopt_t.
2055    LOOP AT lt_bstat ASSIGNING <wa1>.
2056      MOVE-CORRESPONDING <wa1> TO ls_selopt.
2057      APPEND ls_selopt TO ls_selopt_t.
2058    ENDLOOP.
2059    ls_selection-fieldname = 'BSTAT'.
2060    ls_selection-t_range  = ls_selopt_t.
2061    APPEND ls_selection TO lt_selection.
2062
2063  * New Check for the GLYEC in case of Non leading ledger       "2230542
2064    IF gv_relevant_ledger NE gv_leading_ledger
2065                               AND x_glyec IS INITIAL.
2066  *   Exclude the RRCTY = '5' postings fully!
2067      lt_rrcty-sign   = 'I'.
2068      lt_rrcty-option = 'NE'.
2069      lt_rrcty-low    = '5'.
2070      APPEND lt_rrcty.
2071      REFRESH ls_selopt_t.
2072      LOOP AT lt_rrcty ASSIGNING <wa1>.
2073        MOVE-CORRESPONDING <wa1> TO ls_selopt.
2074        APPEND ls_selopt TO ls_selopt_t.
2075      ENDLOOP.
2076      ls_selection-fieldname = 'RRCTY'.
2077      ls_selection-t_range  = ls_selopt_t.
2078      APPEND ls_selection TO lt_selection.
2079    ENDIF.
2080  * New Check for the GLYEC in case of Non leading ledger       "2230542
2081
2082  * dates
2083  * if P_gb_xplacc is initial.
2084  * normal date selection
2085    GET PARAMETER ID 'FI_STIDA' FIELD ld_stida_2.
2086    IF NOT ld_stida_2 IS INITIAL AND sy-subrc EQ 0.
2087      pa_stida = ld_stida_2.
2088      SET PARAMETER ID 'FI_STIDA' FIELD '00000000'.
2089    ENDIF.
2090    IF NOT x_opsel IS INITIAL.
2091      IF pa_stida IS INITIAL.
2092        p_keydate         = '99991231'.
2093        ld_stida          = '99991231'.
```

```
2094      ELSE.
2095        p_keydate         = pa_stida.
2096        ld_stida          = pa_stida.
2097      ENDIF.
2098      lt_budat-sign = 'I'.
2099      lt_budat-option = 'LE'.
2100      lt_budat-low    = ld_stida.
2101      APPEND lt_budat.
2102      lt_augdt-sign = 'I'.
2103      lt_augdt-option = 'GT'.
2104      lt_augdt-low    = ld_stida.
2105      APPEND lt_augdt.
2106      IF gd_usear = 'X'.                                          "1383956
2107        lt_augdt-sign = 'I'.                                      "1383956
2108        lt_augdt-option = 'EQ'.                                   "1383956
2109        lt_augdt-low    = '00000000'.                             "1383956
2110        APPEND lt_augdt.                                          "1383956
2111      ENDIF.                                                      "1383956
2112    ENDIF.
2113    IF NOT x_clsel IS INITIAL.
2114      IF pa_stid2 IS INITIAL.
2115        p_keydate       = '99991231'.
2116        ld_stida        = '00010101'.
2117        LOOP AT so_augdt ASSIGNING <wa1>.
2118          MOVE-CORRESPONDING <wa1> TO lt_augdt.
2119          APPEND lt_augdt.
2120        ENDLOOP.
2121        IF lt_augdt[] IS INITIAL.                                 "1041811
2122          lt_augdt-sign = 'I'.                                    "1008367
2123          lt_augdt-option = 'BT'.                                 "1008367
2124          lt_augdt-low    = '00010101'.                           "1008367
2125          lt_augdt-high   = '99991231'.                           "1008367
2126          APPEND lt_augdt.                                        "1008367
2127        ENDIF.                                                    "1008367
2128      ELSE.
2129        p_keydate       = pa_stid2.
2130        ld_stida        = pa_stid2.
2131        LOOP AT so_augdt ASSIGNING <wa1>.
2132          MOVE-CORRESPONDING <wa1> TO lt_augdt.
2133          APPEND lt_augdt.
2134        ENDLOOP.
2135        lt_budat-sign = 'I'.
2136        lt_budat-option = 'LE'.
2137        lt_budat-low    = ld_stida.
2138        APPEND lt_budat.
2139        lt_augdt-sign = 'I'.
2140        lt_augdt-option = 'GT'.
2141        lt_augdt-low    = ld_stida.
2142        APPEND lt_augdt.
2143        IF lt_augdt[] IS INITIAL.                                 "1041811
2144          lt_augdt-sign = 'I'.                                    "1008367
2145          lt_augdt-option = 'BT'.                                 "1008367
2146          lt_augdt-low    = '00010101'.                           "1008367
2147          lt_augdt-high   = '99991231'.                           "1008367
2148          APPEND lt_augdt.                                        "1008367
2149        ENDIF.                                                    "1008367
2150      ENDIF.
2151    ENDIF.
2152    IF NOT x_aisel IS INITIAL.
2153      p_keydate         = '99991231'.
2154      ld_stida          = '99991231'.
```

```
2155      LOOP AT so_budat ASSIGNING <wa1>.
2156        MOVE-CORRESPONDING <wa1> TO lt_budat.
2157        APPEND lt_budat.
2158      ENDLOOP.
2159  *     lt_budat-sign = 'I'.
2160  *     lt_budat-option = 'LE'.
2161  *     lt_budat-low    = ld_stida.
2162  *     append lt_budat.
2163    ENDIF.
2164
2165    pa_stida_default = ld_stida.
2166
2167  * expiring currencies:
2168    IF ld_stida IS INITIAL OR ld_stida = '00010101'
2169                           OR ld_stida = '99991231'.
2170      gd_alw_date = sy-datlo.
2171    ELSE.
2172      gd_alw_date = ld_stida.
2173    ENDIF.
2174
2175  *  CALL FUNCTION 'FAGL_ITEMS_LDB_SELECTIONS'
2176  *    EXPORTING
2177  *      RADIO_OPEN          = x_opsel
2178  *      RADIO_CLEARED       = x_clsel
2179  *      RADIO_ALL           = x_aisel
2180  *      STIDA               = ld_stida
2181  **   IMPORTING
2182  **     OPOPT               =
2183  **     APOPT               =
2184  **     STIDA_RET           =
2185  *    TABLES
2186  *      AUGDT_IN            = so_augdt
2187  *      BUDAT_IN            = so_budat
2188  *      AUGDT_RET           = lt_AUGDT_RET
2189  *      BUDAT_RET           = lt_BUDAT_RET
2190  *    EXCEPTIONS
2191  *      RADIO_ERROR         = 1
2192  *      OTHERS              = 2
2193    .
2194    IF sy-subrc <> 0.
2195  * MESSAGE ID SY-MSGID TYPE SY-MSGTY NUMBER SY-MSGNO
2196  *         WITH SY-MSGV1 SY-MSGV2 SY-MSGV3 SY-MSGV4.
2197    ENDIF.
2198
2199    IF NOT so_yrper[] IS INITIAL.
2200      IF p_gb_xplacc IS INITIAL.
2201  * date selections for accounts excl PL accounts
2202      ELSE.
2203  * date selections for PL accounts
2204  * modify SO_YRPER:
2205  ***   has to be stored specially for 'refresh' of list         "1530241
2206        gt_yrper_old = so_yrper[].                              "1530241
2207        gt_yrper = so_yrper[].
2208        DESCRIBE TABLE gt_yrper LINES ld_lines.
2209        IF ld_lines EQ 1 .
2210          READ TABLE gt_yrper INTO gd_yrper INDEX 1.
2211          IF gd_yrper-sign = 'I' AND gd_yrper-option = 'BT'.
2212            gd_yrper-low(4) = gd_yrper-high(4).
2213            MODIFY gt_yrper FROM gd_yrper INDEX 1.
2214          ENDIF.
2215        ENDIF.
```

```
2216        so_yrper[] = gt_yrper.
2217  *  loop at gt_yrper into gd_yrper.
2218  *    clear it_selscreen.
2219  *    it_selscreen-selname = 'SO_YRPER'.
2220  *    it_selscreen-kind    = 'S'.
2221  *    move-corresponding gd_yrper to it_selscreen.
2222  *    append it_selscreen.
2223  *  endloop.
2224      ENDIF.
2225    ENDIF.
2226
2227  * Called from RRI (possibly FAGLB03) and the Entry View          "1384389
2228  * was set out of a migration for example.                        "1384389
2229    IF gv_xrric = 'X' AND gv_xerfs = 'X'.
2230      IF NOT so_yrper[] IS INITIAL.
2231        DATA:   period     TYPE poper OCCURS 0 WITH HEADER LINE.
2232        DATA:   year       TYPE ryear OCCURS 0 WITH HEADER LINE.
2233        RANGES: monat      FOR bkpf-monat.
2234        RANGES: poper      FOR faglflexa-poper.
2235        RANGES: gjahr      FOR bkpf-gjahr.
2236        RANGES: ryear      FOR faglflexa-ryear.
2237        DATA:   ld_line_yp LIKE sy-tabix.
2238
2239        LOOP AT so_yrper.
2240          year = so_yrper-low(4).
2241          IF NOT year IS INITIAL.
2242            APPEND year.
2243          ENDIF.
2244          year = so_yrper-high(4).
2245          IF NOT year IS INITIAL.
2246            APPEND year.
2247          ENDIF.
2248          IF so_yrper-high+4(3) EQ '000'                          "3320865
2249             AND so_yrper-option EQ 'BT'.                         "3457542
2250  *         Migration not ended, entry view but GLYEC selected!   "3320865
2251            period = '000'.                                       "3320865
2252            APPEND period.                                        "3320865
2253          ELSE.                                                   "3320865
2254            period = so_yrper-low+4(3).
2255          IF NOT period IS INITIAL.
2256            APPEND period.
2257          ENDIF.
2258          ENDIF.                                                  "3320865
2259          IF so_yrper-low(4) = '1950'.
2260            period = '016'.
2261            APPEND period.
2262          ELSE.
2263            period = so_yrper-high+4(3).
2264            IF NOT period IS INITIAL.
2265              APPEND period.
2266            ENDIF.
2267          ENDIF.
2268        ENDLOOP.
2269
2270        SORT period.
2271        SORT year.
2272        DELETE ADJACENT DUPLICATES FROM period.
2273        DELETE ADJACENT DUPLICATES FROM year.
2274
2275        DESCRIBE TABLE period LINES ld_line_yp.
2276        IF ld_line_yp = 1.
```

```
2277          READ TABLE period INDEX 1.
2278          CLEAR monat.
2279          monat-option = 'EQ'.
2280          monat-sign   = 'I'.
2281          monat-low    = period.
2282          APPEND monat.
2283        ELSE.
2284          READ TABLE period INDEX 1.
2285          CLEAR monat.
2286          monat-option = 'BT'.
2287          monat-sign   = 'I'.
2288          monat-low    = period.
2289          READ TABLE period INDEX ld_line_yp.
2290          monat-high   = period.
2291          APPEND monat.
2292        ENDIF.
2293
2294        DESCRIBE TABLE year LINES ld_line_yp.
2295        IF ld_line_yp = 1.
2296          READ TABLE year INDEX 1.
2297          CLEAR gjahr.
2298          gjahr-option = 'EQ'.
2299          gjahr-sign   = 'I'.
2300          gjahr-low    = year.
2301          APPEND gjahr.
2302        ELSE.
2303          READ TABLE year INDEX 1.
2304          CLEAR gjahr.
2305          gjahr-option = 'BT'.
2306          gjahr-sign   = 'I'.
2307          gjahr-low    = year.
2308          READ TABLE year INDEX ld_line_yp.
2309          gjahr-high   = year.
2310          APPEND gjahr.
2311        ENDIF.
2312
2313        REFRESH ls_selopt_t.
2314        LOOP AT monat ASSIGNING <wa1>.
2315          MOVE-CORRESPONDING <wa1> TO ls_selopt.
2316          APPEND ls_selopt TO ls_selopt_t.
2317        ENDLOOP.
2318        ls_selection-fieldname = 'MONAT'.
2319        ls_selection-t_range  = ls_selopt_t.
2320        APPEND ls_selection TO lt_selection.
2321        REFRESH ls_selopt_t.
2322        LOOP AT gjahr ASSIGNING <wa1>.
2323          MOVE-CORRESPONDING <wa1> TO ls_selopt.
2324          APPEND ls_selopt TO ls_selopt_t.
2325        ENDLOOP.
2326        ls_selection-fieldname = 'GJAHR'.
2327        ls_selection-t_range  = ls_selopt_t.
2328        APPEND ls_selection TO lt_selection.
2329        REFRESH ls_selopt_t.
2330        LOOP AT poper ASSIGNING <wa1>.
2331          MOVE-CORRESPONDING <wa1> TO ls_selopt.
2332          APPEND ls_selopt TO ls_selopt_t.
2333        ENDLOOP.
2334      ENDIF.
2335    ENDIF.                                                    "1384389
2336
2337    REFRESH ls_selopt_t.
```

```
2338    LOOP AT lt_budat ASSIGNING <wa1>.
2339      MOVE-CORRESPONDING <wa1> TO ls_selopt.
2340      APPEND ls_selopt TO ls_selopt_t.
2341    ENDLOOP.
2342    ls_selection-fieldname = 'BUDAT'.
2343    ls_selection-t_range  = ls_selopt_t.
2344    APPEND ls_selection TO lt_selection.
2345
2346    REFRESH ls_selopt_t.
2347    IF pa_stid2 IS INITIAL.                                    "1041811
2348      LOOP AT lt_augdt ASSIGNING <wa1>.                        "1041811
2349        MOVE-CORRESPONDING <wa1> TO ls_selopt.                 "1041811
2350        APPEND ls_selopt TO ls_selopt_t.                       "1041811
2351      ENDLOOP.                                                 "1041811
2352      ls_selection-fieldname = 'AUGDT'.                        "1041811
2353      ls_selection-t_range  = ls_selopt_t.                     "1041811
2354      APPEND ls_selection TO lt_selection.                     "1041811
2355    ELSE.                                                      "1041811
2356      LOOP AT lt_augdt ASSIGNING <wa1>.                        "1041811
2357        MOVE-CORRESPONDING <wa1> TO ls_selopt.                 "1041811
2358        APPEND ls_selopt TO ls_selopt_t.                       "1041811
2359        ls_selection-fieldname = 'AUGDT'.                      "1041811
2360        ls_selection-t_range  = ls_selopt_t.                   "1041811
2361        APPEND ls_selection TO lt_selection.                   "1041811
2362        REFRESH ls_selopt_t.                                   "1041811
2363      ENDLOOP.                                                 "1041811
2364    ENDIF.                                                     "1041811
2365
2366  * free selections
2367    LOOP AT fs_dyns-trange INTO ls_trange_tab_line
2368      WHERE tablename = ld_bsis_fs.
2369      LOOP AT ls_trange_tab_line-frange_t INTO ls_frange_tab_line.
2370        CLEAR ls_selection.
2371        ls_selection = ls_frange_tab_line.
2372        APPEND ls_selection TO lt_selection.
2373      ENDLOOP.
2374    ENDLOOP.
2375
2376    CONCATENATE gv_si_table '_FS' INTO ld_si_table_fs.
2377    LOOP AT fs_dyns-trange INTO ls_trange_tab_line
2378      WHERE tablename = ld_si_table_fs.
2379      LOOP AT ls_trange_tab_line-frange_t INTO ls_frange_tab_line.
2380        CLEAR ls_selection.
2381        ls_selection = ls_frange_tab_line.
2382        APPEND ls_selection TO lt_selection.
2383      ENDLOOP.
2384    ENDLOOP.
2385
2386  * in S4HANA the selection is done using hot and cold store
2387  * no need to select data only from data base       "FAGLL03 4./5. HW
2388  * Therefore the following lines are commented out. "FAGLL03 4./5. HW
2389
2390  * If only data from database should be selected,
2391  * archived data should not be selected
2392  *  IF gd_usear IS INITIAL AND x_erfs IS INITIAL.             "1437369
2393  *    lt_xarch-sign = 'I'.
2394  *    lt_xarch-option = 'EQ'.
2395  *    lt_xarch-low  = ' '.
2396  *    APPEND lt_xarch.
2397  *    REFRESH ls_selopt_t.
2398  *    LOOP AT lt_xarch ASSIGNING <wa1>.
```

```
2399  *      MOVE-CORRESPONDING <wa1> TO ls_selopt.
2400  *      APPEND ls_selopt TO ls_selopt_t.
2401  *    ENDLOOP.
2402  *    ls_selection-fieldname = 'XARCH'.
2403  *    ls_selection-t_range  = ls_selopt_t.
2404  *    APPEND ls_selection TO lt_selection.
2405  *  ENDIF.                                              "FAGLL03 4./5. HW
2406
2407
2408  * modify selections according to year period selection
2409  *  if not so_yrper[] is initial.
2410  *    data:   period     type poper occurs 0 with header line.
2411  *    data:   year       type ryear occurs 0 with header line.
2412  *    Ranges: monat      for bkpf-monat.
2413  *    ranges: poper      for faglflexa-poper.
2414  *    Ranges: gjahr      for bkpf-gjahr.
2415  *    ranges: ryear      for faglflexa-ryear.
2416  *    data:   ld_line_yp like sy-tabix.
2417  *
2418  *    loop at so_yrper.
2419  *      period = so_yrper-low+4(3).
2420  *      if not period is initial.
2421  *        append period.
2422  *      endif.
2423  *      period = so_yrper-high+4(3).
2424  *      if not period is initial.
2425  *        append period.
2426  *      endif.
2427  *      year = so_yrper-low(4).
2428  *      if not year is initial.
2429  *        append year.
2430  *      endif.
2431  *      year = so_yrper-high(4).
2432  *      if not year is initial.
2433  *        append year.
2434  *      endif.
2435  *    endloop.
2436  *
2437  *    sort period.
2438  *    sort year.
2439  *    DELETE ADJACENT DUPLICATES FROM period.
2440  *    DELETE ADJACENT DUPLICATES FROM year.
2441  *
2442  *    describe table period lines ld_line_yp.
2443  *    if ld_line_yp = 1.
2444  *      read table period index 1.
2445  *      if  x_merk is initial
2446  *      and x_erfs is initial.
2447  *        clear poper.
2448  *        poper-option = 'EQ'.
2449  *        poper-sign   = 'I'.
2450  *        poper-low    = period.
2451  *        append poper.
2452  *      else.
2453  *        clear monat.
2454  *        monat-option = 'EQ'.
2455  *        monat-sign   = 'I'.
2456  *        monat-low    = period.
2457  *        append monat.
2458  *      endif.
2459  *    else.
```

```
2460  *      read table period index 1.
2461  *      if  x_merk is initial
2462  *      and x_erfs is initial.
2463  *        clear poper.
2464  *        poper-option = 'BT'.
2465  *        poper-sign   = 'I'.
2466  *        poper-low    = period.
2467  *        read table period index ld_line_yp.
2468  *        poper-high   = period.
2469  *        append poper.
2470  *      else.
2471  *        clear monat.
2472  *        monat-option = 'BT'.
2473  *        monat-sign   = 'I'.
2474  *        monat-low    = period.
2475  *        read table period index ld_line_yp.
2476  *        monat-high   = period.
2477  *        append monat.
2478  *      endif.
2479  *    endif.
2480  *
2481  *    describe table year lines ld_line_yp.
2482  *    if ld_line_yp = 1.
2483  *      read table year index 1.
2484  *      if  x_merk is initial
2485  *      and x_erfs is initial.
2486  *        clear ryear.
2487  *        ryear-option = 'EQ'.
2488  *        ryear-sign   = 'I'.
2489  *        ryear-low    = year.
2490  *        append ryear.
2491  *      else.
2492  *        clear gjahr.
2493  *        gjahr-option = 'EQ'.
2494  *        gjahr-sign   = 'I'.
2495  *        gjahr-low    = year.
2496  *        append gjahr.
2497  *      endif.
2498  *    else.
2499  *      read table year index 1.
2500  *      if  x_merk is initial
2501  *      and x_erfs is initial.
2502  *        clear ryear.
2503  *        ryear-option = 'BT'.
2504  *        ryear-sign   = 'I'.
2505  *        ryear-low    = year.
2506  *        read table year index ld_line_yp.
2507  *        ryear-high   = year.
2508  *        append ryear.
2509  *      else.
2510  *        clear gjahr.
2511  *        gjahr-option = 'BT'.
2512  *        gjahr-sign   = 'I'.
2513  *        gjahr-low    = year.
2514  *        read table year index ld_line_yp.
2515  *        gjahr-high   = year.
2516  *        append gjahr.
2517  *      endif.
2518  *    endif.
2519  *
2520  *    refresh ls_selopt_t.
```

```
2521  *    loop at monat assigning <wa1>.
2522  *      move-corresponding <wa1> to ls_selopt.
2523  *      append ls_selopt to ls_selopt_t.
2524  *    endloop.
2525  *    ls_SELECTION-fieldname = 'MONAT'.
2526  *    ls_SELECTION-T_RANGE  = Ls_selopt_t.
2527  *    append ls_SELECTION to lt_SELECTION.
2528  *    refresh ls_selopt_t.
2529  *    loop at gjahr assigning <wa1>.
2530  *      move-corresponding <wa1> to ls_selopt.
2531  *      append ls_selopt to ls_selopt_t.
2532  *    endloop.
2533  *    ls_SELECTION-fieldname = 'GJAHR'.
2534  *    ls_SELECTION-T_RANGE  = Ls_selopt_t.
2535  *    append ls_SELECTION to lt_SELECTION.
2536  *    refresh ls_selopt_t.
2537  *    loop at poper assigning <wa1>.
2538  *      move-corresponding <wa1> to ls_selopt.
2539  *      append ls_selopt to ls_selopt_t.
2540  *    endloop.
2541  *    ls_SELECTION-fieldname = 'POPER'.
2542  *    ls_SELECTION-T_RANGE  = Ls_selopt_t.
2543  *    append ls_SELECTION to lt_SELECTION.
2544  *    refresh ls_selopt_t.
2545  *    loop at RYEAR assigning <wa1>.
2546  *      move-corresponding <wa1> to ls_selopt.
2547  *      append ls_selopt to ls_selopt_t.
2548  *    endloop.
2549  *    ls_SELECTION-fieldname = 'RYEAR'.
2550  *    ls_SELECTION-T_RANGE  = Ls_selopt_t.
2551  *    append ls_SELECTION to lt_SELECTION.
2552  *  endif.
2553
2554  * archiving nothing neccesary
2555
2556    p_gt_selection_part[] = lt_selection[].
2557
2558  ENDFORM.                    "map_sel_to_items
2559
2560  *&---------------------------------------------------------------------*
2561  *&      Form  SET_PARAMETERS
2562  *&---------------------------------------------------------------------*
2563  FORM set_parameters.
2564  * set general parameters:
2565    PERFORM set_general_param.
2566  * set acc.type specific parameters:
2567    CLEAR: sd_saknr, sd_bukrs.
2568    CLEAR: so_wlsak, so_wlbuk.
2569    IF gd_wl_on IS INITIAL.
2570      READ TABLE sd_saknr INDEX 1.
2571      IF sy-subrc = 0 AND sd_saknr-sign = 'I' AND sd_saknr-option = 'EQ'
2572          AND NOT sd_saknr-low IS INITIAL.
2573        SET PARAMETER ID 'SAK' FIELD sd_saknr-low.
2574      ENDIF.
2575      READ TABLE sd_bukrs INDEX 1.
2576      IF sy-subrc = 0 AND sd_bukrs-sign = 'I' AND sd_bukrs-option = 'EQ'
2577          AND NOT sd_bukrs-low IS INITIAL.
2578        SET PARAMETER ID 'BUK' FIELD sd_bukrs-low.
2579      ENDIF.
2580      SET PARAMETER ID 'AVS' FIELD space.
2581      SET PARAMETER ID 'AVB' FIELD space.
```

```
2582    ELSE.
2583  *   worklists switched on:
2584      SET PARAMETER ID 'AVS' FIELD pa_wlsak.
2585      SET PARAMETER ID 'AVB' FIELD pa_wlbuk.
2586      READ TABLE so_wlsak INDEX 1.
2587      IF sy-subrc = 0 AND so_wlsak-sign = 'I' AND so_wlsak-option = 'EQ'
2588          AND NOT so_wlsak-low IS INITIAL.
2589        SET PARAMETER ID 'SAK' FIELD so_wlsak-low.
2590      ENDIF.
2591      READ TABLE so_wlbuk INDEX 1.
2592      IF sy-subrc = 0 AND so_wlbuk-sign = 'I' AND so_wlbuk-option = 'EQ'
2593          AND NOT so_wlbuk-low IS INITIAL.
2594        SET PARAMETER ID 'BUK' FIELD so_wlbuk-low.
2595      ENDIF.
2596    ENDIF.
2597
2598  * Set parameters for reading from archiv
2599    gd_usear = sd_usear.
2600    gd_usedb = sd_usedb.
2601    gd_memor = 'X'.
2602
2603    gd_objec   = sd_objec.
2604    gd_useas   = sd_useas.
2605    IF gd_useas IS INITIAL.
2606      gt_files[] = sd_files[].
2607    ENDIF.
2608    gd_iarch   = sd_iarch.
2609
2610  ENDFORM.                    "set_parameters
2611
2612  *&---------------------------------------------------------------------*
2613  *&      Form  SET_ACCT_AND_CCODE
2614  *&---------------------------------------------------------------------*
2615  FORM set_acct_and_ccode.
2616
2617    REFRESH: sd_saknr, sd_bukrs, so_wlsak, so_wlbuk.
2618    CLEAR:   sd_saknr, sd_bukrs, so_wlsak, so_wlbuk,
2619             pa_wlsak, pa_wlbuk.
2620
2621    IF gd_wl_on IS INITIAL.
2622      GET PARAMETER ID 'SAK' FIELD sd_saknr-low.
2623      IF sy-subrc = 0 AND NOT sd_saknr-low IS INITIAL.
2624        IF sd_saknr-sign IS INITIAL.                          "3071629
2625          sd_saknr-sign   = 'I'.                              "3071629
2626          sd_saknr-option = 'EQ'.                             "3071629
2627        ENDIF.                                                "3071629
2628        IF sd_saknr-low CA '*'.
2629          sd_saknr-sign   = 'I'.                              "2949744
2630          sd_saknr-option = 'CP'.
2631        ELSE.
2632          sd_saknr-sign   = 'I'.                              "2949744
2633          sd_saknr-option = 'EQ'.
2634        ENDIF.
2635        APPEND sd_saknr.
2636      ENDIF.
2637      GET PARAMETER ID 'BUK' FIELD sd_bukrs-low.
2638      IF sy-subrc = 0 AND NOT sd_bukrs-low IS INITIAL.
2639        IF sd_bukrs-sign IS INITIAL.                          "3071629
2640          sd_bukrs-sign   = 'I'.                              "3071629
2641          sd_bukrs-option = 'EQ'.                             "3071629
2642        ENDIF.                                                "3071629
```

```
2643        IF sd_bukrs-low CA '*'.
2644          sd_bukrs-sign   = 'I'.                              "2949744
2645          sd_bukrs-option = 'CP'.
2646        ELSE.
2647          sd_bukrs-sign   = 'I'.                              "2949744
2648          sd_bukrs-option = 'EQ'.
2649        ENDIF.
2650        APPEND sd_bukrs.
2651      ENDIF.
2652    ELSE.
2653  *   worklists switched on:
2654      GET PARAMETER ID 'AVS' FIELD pa_wlsak.
2655      GET PARAMETER ID 'AVB' FIELD pa_wlbuk.
2656      IF pa_wlsak IS INITIAL.
2657        GET PARAMETER ID 'SAK' FIELD so_wlsak-low.
2658        IF sy-subrc = 0 AND NOT so_wlsak-low IS INITIAL.
2659          so_wlsak-sign   = 'I'.
2660          so_wlsak-option = 'EQ'.
2661          APPEND so_wlsak.
2662        ENDIF.
2663      ENDIF.
2664      IF pa_wlbuk IS INITIAL.
2665        GET PARAMETER ID 'BUK' FIELD so_wlbuk-low.
2666        IF sy-subrc = 0 AND NOT so_wlbuk-low IS INITIAL.
2667          so_wlbuk-sign   = 'I'.
2668          so_wlbuk-option = 'EQ'.
2669          APPEND so_wlbuk.
2670        ENDIF.
2671      ENDIF.
2672    ENDIF.
2673
2674  ENDFORM.                                " SET_ACCT_AND_CCODE
2675
2676  *&---------------------------------------------------------------------*
2677  *&      Form  exclude_pl_acounts
2678  *&---------------------------------------------------------------------*
2679  *       text
2680  *----------------------------------------------------------------------*
2681  *  -->  p1        text
2682  *  <--  p2        text
2683  *----------------------------------------------------------------------*
2684  FORM exclude_pl_accounts .
2685
2686    DATA: ld_lines TYPE i.
2687    DATA: ls_account_slt TYPE gty_acccoco.
2688
2689    IF  sy-calld = 'X' AND
2690    NOT ( so_yrper[] IS INITIAL
2691    OR NOT gv_xumsav IS INITIAL ).
2692  *    ( sy-tcode = 'FAGLB03' or sy-tcode cp 'S+38'
2693  *      or sy-tcode eq 'SE80' ).
2694
2695  * make copy of sd_saknr
2696      gt_saknr = sd_saknr[].
2697      gt_bukrs = sd_bukrs[].                                  "1530241
2698
2699  * Exclude PL accounts.
2700  * check if worklist or not
2701
2702      DESCRIBE TABLE sd_saknr LINES ld_lines.
2703      LOOP AT sd_saknr TRANSPORTING NO FIELDS
```

```
2704           WHERE sign NE 'I' OR
2705                 option NE 'EQ'.
2706       EXIT.
2707     ENDLOOP.
2708     IF sy-subrc = 0 OR ld_lines = 0.
2709       SELECT        s~saknr t~bukrs s~gvtyp
2710                INTO TABLE gt_accoco
2711                FROM t001 AS t INNER JOIN ska1 AS s
2712                  ON t~ktopl =  s~ktopl
2713               WHERE t~bukrs IN sd_bukrs
2714                 AND s~saknr IN sd_saknr.
2715     ELSE.                                          "worklist.
2716       SELECT        s~saknr t~bukrs s~gvtyp
2717                INTO  gd_accoco
2718                FROM t001 AS t INNER JOIN ska1 AS s
2719                  ON t~ktopl =  s~ktopl
2720                 FOR ALL ENTRIES IN sd_saknr
2721               WHERE t~bukrs IN sd_bukrs
2722                 AND s~saknr = sd_saknr-low.
2723         APPEND gd_accoco TO gt_accoco.
2724       ENDSELECT.
2725     ENDIF.
2726
2727   * PL account numbers remain in GT_ACCOCO
2728   * all others are transfered into SD_SAKNR.
2729     REFRESH sd_saknr.
2730     CLEAR sd_saknr.
2731     sd_saknr-sign   = 'I'.
2732     sd_saknr-option = 'EQ'.
2733     REFRESH sd_bukrs.                                  "1573934
2734     CLEAR sd_bukrs.                                    "1573934
2735     sd_bukrs-sign = 'I'.                               "1573934
2736     sd_bukrs-option = 'EQ'.                            "1573934
2737     LOOP AT gt_accoco INTO gd_accoco WHERE gvtyp = space.
2738       DELETE gt_accoco INDEX sy-tabix.
2739       sd_saknr-low = gd_accoco-saknr.
2740       APPEND sd_saknr.
2741       sd_bukrs-low = gd_accoco-bukrs.                  "1050960
2742       APPEND sd_bukrs.                                 "1322995
2743     ENDLOOP.
2744     IF sd_saknr[] IS INITIAL.
2745       APPEND sd_saknr.
2746     ENDIF.
2747     IF sd_bukrs[] IS INITIAL.                          "1573934
2748       APPEND sd_bukrs.                                 "1573934
2749     ENDIF.                                             "1573934
2750
2751     DELETE ADJACENT DUPLICATES FROM sd_saknr           "1728409
2752                             COMPARING sign option low high.  "1728409
2753     SORT sd_bukrs.                                     "2731627
2754     DELETE ADJACENT DUPLICATES FROM sd_bukrs           "1728409
2755                             COMPARING sign option low high.  "1728409
2756
2757   * classify Settlement Accounts
2758     REFRESH gt_accoco_slt.
2759     LOOP AT gt_account_slt INTO ls_account_slt.
2760       LOOP AT gt_accoco INTO gd_accoco
2761         WHERE saknr = ls_account_slt-saknr
2762         AND   bukrs = ls_account_slt-bukrs.
2763         ls_account_slt-gvtyp = gd_accoco-gvtyp.
2764         APPEND ls_account_slt TO gt_accoco_slt.
```

```
2765        ENDLOOP.
2766        IF sy-subrc = 0.
2767          DELETE gt_account_slt.
2768        ENDIF.
2769      ENDLOOP.
2770
2771    ENDIF.
2772  ENDFORM.                    " exclude_pl_acounts
2773
2774  *&---------------------------------------------------------------------*
2775  *&      Form  include_pl_accounts
2776  *&---------------------------------------------------------------------*
2777  *       text
2778  *----------------------------------------------------------------------*
2779  FORM include_pl_accounts.
2780    DATA: sak_name   LIKE kna1-name1.
2781    DATA: ld_f_bkpf_bes_subrc LIKE sy-subrc.
2782
2783
2784    IF  sy-calld = 'X'
2785    AND NOT so_yrper[] IS INITIAL
2786    AND gv_xumsav IS INITIAL.
2787
2788      AUTHORITY-CHECK OBJECT 'F_BKPF_KOA'
2789        ID 'KOART' FIELD 'S'
2790        ID 'ACTVT' FIELD '03'.
2791      IF sy-subrc NE 0.
2792        deco_message_2 e812(fr) with TEXT-300 'S'.
2793      ENDIF.
2794
2795      gb_xplacc = 'X'.
2796  * map selections to lineitem selection logic:
2797      PERFORM map_sel_to_items USING gb_xplacc
2798                               CHANGING gt_selection_part.
2799
2800      LOOP AT gt_accoco INTO gd_accoco.
2801
2802        PERFORM t001_info_fill USING gd_accoco-bukrs.
2803
2804  * fill work area fields
2805        skb1-saknr = gd_accoco-saknr.
2806        skb1-bukrs = gd_accoco-bukrs.
2807
2808  * authority check
2809        SELECT SINGLE begru xkres xopvw mitkz xlgclr FROM skb1 "1174335
2810          INTO CORRESPONDING FIELDS OF skb1
2811          WHERE saknr = gd_accoco-saknr
2812          AND   bukrs = gd_accoco-bukrs.
2813
2814  * ONLY select the data, if the account is defined in the       "1361304
2815  * Company Code. Check was before done in the LDB_PROCESS       "1361304
2816        IF sy-subrc = 0.                                      "1361304
2817
2818          CLEAR ld_f_bkpf_bes_subrc.
2819          IF NOT skb1-begru IS INITIAL.
2820            AUTHORITY-CHECK OBJECT 'F_BKPF_BES'
2821              ID 'BRGRU' FIELD skb1-begru
2822              ID 'ACTVT' FIELD '03'.
2823            IF sy-subrc <> 0.
2824              ld_f_bkpf_bes_subrc = sy-subrc.
2825              sy-subrc = 0.
```

```
2826            ENDIF.
2827            AUTHORITY-CHECK OBJECT 'F_SKA1_BES'
2828              ID 'BRGRU' FIELD skb1-begru
2829              ID 'ACTVT' FIELD '03'.
2830            IF NOT sy-subrc IS INITIAL
2831            OR NOT ld_f_bkpf_bes_subrc IS INITIAL.
2832              CONTINUE.
2833            ENDIF.
2834          ENDIF.
2835
2836          PERFORM check_slt_account USING skb1-saknr
2837                                          skb1-bukrs
2838                                          skb1-xkres
2839                                          gb_xplacc.
2840
2841          "Determine the temperature for the aged data
2842          CALL FUNCTION 'FAGL_DETERMINE_TEMPERATURE'
2843            EXPORTING
2844              is_skb1             = skb1
2845              iv_opsel            = x_opsel
2846              iv_clsel            = x_clsel
2847              iv_aisel            = x_aisel
2848              iv_pa_stida         = pa_stida
2849              ir_so_augdt         = lr_augdt
2850              ir_so_budat         = lr_budat
2851              iv_pa_stid2         = pa_stid2
2852              ir_bukrs            = lr_bukrs
2853              iv_rldnr            = gv_relevant_ledger
2854            IMPORTING
2855              ev_data_temperature = lv_data_temperature.
2856
2857
2858  * map selection to lineitem selection ranges
2859  * add masterdata information
2860          PERFORM map_sel_to_items_mast_dat USING
2861                                             gb_xplacc
2862                                             gt_selection_part
2863                                           CHANGING
2864                                             gt_selection_full.
2865
2866  * select items and fill them in Output table
2867          CLEAR gv_special_account.                          "1822909
2868          PERFORM check_trtm_account.                        "1004374
2869          PERFORM check_glyec_account.                       "1834289
2870          PERFORM select_items.
2871          IF NOT x_stop IS INITIAL.
2872            EXIT.
2873          ENDIF.
2874  * fill account text
2875          PERFORM skat_info_fill.
2876  *     write skat-txt50 to sak_name.
2877          WRITE it_h_skat-txt50 TO sak_name.
2878          PERFORM fill_acct_table USING  'S'
2879                                         gd_accoco-saknr
2880                                         gd_accoco-bukrs
2881                                         space
2882                                         sak_name
2883                                         gd_accoco-saknr.
2884
2885        ENDIF.                                               "1361304
2886
```

```
2887      ENDLOOP.
2888
2889  * restore accounts (for refresh of list).
2890      sd_saknr[] = gt_saknr.
2891      sd_bukrs[] = gt_bukrs.                                  "1530241
2892      so_yrper[] = gt_yrper_old.                              "1530241
2893
2894    ENDIF.
2895
2896  ENDFORM.                    "include_pl_accounts
2897
2898  *&---------------------------------------------------------------------*
2899  *&      Form  sdf_selscreen
2900  *&---------------------------------------------------------------------*
2901  *       text
2902  *----------------------------------------------------------------------*
2903  FORM sdf_selscreen.
2904
2905    DATA: ld_lines TYPE sytabix.
2906
2907    REFRESH: it_selscreen.
2908  * fill selections: first single values & parameters
2909    it_selscreen-selname = 'SD_SAKNR'.
2910    it_selscreen-kind    = 'S'.
2911    it_selscreen-sign    = 'I'.
2912    it_selscreen-option  = 'EQ'.
2913    it_selscreen-low     = gd_accoco-saknr.
2914    APPEND it_selscreen.
2915    CLEAR it_selscreen.
2916    it_selscreen-selname = 'SD_BUKRS'.
2917    it_selscreen-kind    = 'S'.
2918    it_selscreen-sign    = 'I'.
2919    it_selscreen-option  = 'EQ'.
2920    it_selscreen-low     = gd_accoco-bukrs.
2921    APPEND it_selscreen.
2922    CLEAR it_selscreen.
2923    it_selscreen-selname = 'SD_OPOPT'.
2924    it_selscreen-kind    = 'P'.
2925    it_selscreen-sign    = 'I'.
2926    it_selscreen-option  = 'EQ'.
2927    it_selscreen-low     = sd_opopt.
2928    APPEND it_selscreen.
2929    CLEAR it_selscreen.
2930    it_selscreen-selname = 'SD_APOPT'.
2931    it_selscreen-kind    = 'P'.
2932    it_selscreen-sign    = 'I'.
2933    it_selscreen-option  = 'EQ'.
2934    it_selscreen-low     = sd_apopt.
2935    APPEND it_selscreen.
2936    CLEAR it_selscreen.
2937    it_selscreen-selname = 'SD_STIDA'.
2938    it_selscreen-kind    = 'P'.
2939    it_selscreen-sign    = 'I'.
2940    it_selscreen-option  = 'EQ'.
2941    it_selscreen-low     = sd_stida.
2942    APPEND it_selscreen.
2943  * now select option tables:
2944    LOOP AT sd_augdt.
2945      CLEAR it_selscreen.
2946      it_selscreen-selname = 'SD_AUGDT'.
2947      it_selscreen-kind    = 'S'.
```

```
2948      MOVE-CORRESPONDING sd_augdt TO it_selscreen.
2949      APPEND it_selscreen.
2950    ENDLOOP.
2951    LOOP AT sd_budat.
2952      CLEAR it_selscreen.
2953      it_selscreen-selname = 'SD_BUDAT'.
2954      it_selscreen-kind    = 'S'.
2955      MOVE-CORRESPONDING sd_budat TO it_selscreen.
2956      APPEND it_selscreen.
2957    ENDLOOP.
2958
2959  * modify SO_YRPER:
2960    gt_yrper = so_yrper[].
2961    DESCRIBE TABLE gt_yrper LINES ld_lines.
2962    IF ld_lines EQ 1 .
2963      READ TABLE gt_yrper INTO gd_yrper INDEX 1.
2964      IF gd_yrper-sign = 'I' AND gd_yrper-option = 'BT'.
2965        gd_yrper-low(4) = gd_yrper-high(4).
2966        MODIFY gt_yrper FROM gd_yrper INDEX 1.
2967      ENDIF.
2968    ENDIF.
2969    LOOP AT gt_yrper INTO gd_yrper.
2970      CLEAR it_selscreen.
2971      it_selscreen-selname = 'SO_YRPER'.
2972      it_selscreen-kind    = 'S'.
2973      MOVE-CORRESPONDING gd_yrper TO it_selscreen.
2974      APPEND it_selscreen.
2975    ENDLOOP.
2976    IF sy-subrc <> 0.
2977      sy-subrc = 0.
2978    ENDIF.
2979
2980  * Parameters and Select Options for Archives
2981    CLEAR it_selscreen.
2982    it_selscreen-selname = 'SD_USEDB'.
2983    it_selscreen-kind    = 'P'.
2984    it_selscreen-sign    = 'I'.
2985    it_selscreen-option  = 'EQ'.
2986    it_selscreen-low     = sd_usedb.
2987    APPEND it_selscreen.
2988    CLEAR it_selscreen.
2989    it_selscreen-selname = 'SD_USEAR'.
2990    it_selscreen-kind    = 'P'.
2991    it_selscreen-sign    = 'I'.
2992    it_selscreen-option  = 'EQ'.
2993    it_selscreen-low     = sd_usear.
2994    APPEND it_selscreen.
2995    CLEAR it_selscreen.
2996    it_selscreen-selname = 'SD_USEAS'.
2997    it_selscreen-kind    = 'P'.
2998    it_selscreen-sign    = 'I'.
2999    it_selscreen-option  = 'EQ'.
3000    it_selscreen-low     = sd_useas.
3001    APPEND it_selscreen.
3002    IF NOT sd_files[] IS INITIAL.
3003      LOOP AT sd_files.
3004        CLEAR it_selscreen.
3005        it_selscreen-selname = 'SD_FILES'.
3006        it_selscreen-kind    = 'S'.
3007        MOVE-CORRESPONDING sd_files TO it_selscreen.
3008        APPEND it_selscreen.
```

```
3009      ENDLOOP.
3010    ENDIF.
3011    CLEAR it_selscreen.
3012    it_selscreen-selname = 'SD_MEMOR'.
3013    it_selscreen-kind    = 'P'.
3014    it_selscreen-sign    = 'I'.
3015    it_selscreen-option  = 'EQ'.
3016    it_selscreen-low     = sd_memor.
3017    APPEND it_selscreen.
3018    CLEAR it_selscreen.
3019    it_selscreen-selname = 'SD_IARCH'.
3020    it_selscreen-kind    = 'P'.
3021    it_selscreen-sign    = 'I'.
3022    it_selscreen-option  = 'EQ'.
3023    it_selscreen-low     = sd_iarch.
3024    APPEND it_selscreen.
3025
3026    CLEAR it_selscreen.
3027    it_selscreen-selname = 'SD_NOOAP'.
3028    it_selscreen-kind    = 'P'.
3029    it_selscreen-sign    = 'I'.
3030    it_selscreen-option  = 'EQ'.
3031    it_selscreen-low     = sd_nooap.
3032    APPEND it_selscreen.
3033
3034  * Set parameters for reading from archiv
3035    gd_usear = sd_usear.
3036    gd_usedb = sd_usedb.
3037    gd_memor = 'X'.
3038
3039    gd_objec   = sd_objec.
3040    gd_useas   = sd_useas.
3041    IF gd_useas IS INITIAL.
3042      gt_files[] = sd_files[].
3043    ENDIF.
3044    gd_iarch   = sd_iarch.
3045
3046  ENDFORM.                                    " SDF_SELSCREEN
3047  *&---------------------------------------------------------------------*
3048  *&      Form  sdf_dynamics
3049  *&---------------------------------------------------------------------*
3050  *       text
3051  *----------------------------------------------------------------------*
3052  *form sdf_dynamics.
3053  *
3054  *  data: rt_dyn_trange  type rsds_trange.
3055  *  data: ld_rsds_expr type rsds_expr,
3056  *        ld_rsdsexpr  type rsdsexpr.
3057  *
3058  *  refresh: it_dyn_texpr.
3059  ** get report dynamic selections:
3060  *  call function 'RS_REFRESH_FROM_DYNAMICAL_SEL'
3061  *    EXPORTING
3062  *      curr_report        = g_repid
3063  *      mode_write_or_move = 'M'
3064  *    IMPORTING
3065  *      p_trange           = rt_dyn_trange
3066  *    EXCEPTIONS
3067  *      not_found          = 1
3068  *      wrong_type         = 2
3069  *      others             = 3.
```

```
3070  *  if sy-subrc ne 0.
3071  *    exit.
3072  *  endif.
3073  ** delete rt_dyn_trange where tablename ne 'BSIS'.
3074  *"740027
3075  *  call function 'FREE_SELECTIONS_RANGE_2_EX'
3076  *    EXPORTING
3077  *      field_ranges = rt_dyn_trange
3078  *    IMPORTING
3079  *      expressions  = it_dyn_texpr[].
3080  *  loop at it_dyn_texpr into ld_rsds_expr where tablename = 'BSIS'.
3081  *    loop at ld_rsds_expr-expr_tab into ld_rsdsexpr
3082  *              where fieldname = 'GJAHR' or fieldname = 'MONAT'.
3083  *      if ld_rsdsexpr-fieldname = 'GJAHR'.
3084  *        if     ld_rsdsexpr-option = 'BT'
3085  *           and ld_rsdsexpr-low ne ld_rsdsexpr-high.
3086  *          ld_rsdsexpr-option = 'EQ'.
3087  *          ld_rsdsexpr-low = ld_rsdsexpr-high.
3088  *          clear ld_rsdsexpr-high.
3089  *          modify ld_rsds_expr-expr_tab from ld_rsdsexpr.
3090  *        endif.
3091  *      elseif ld_rsdsexpr-fieldname = 'MONAT'.
3092  *        if ld_rsdsexpr-option = 'BT'.
3093  *          ld_rsdsexpr-high = so_yrper-high+5.
3094  *          modify ld_rsds_expr-expr_tab from ld_rsdsexpr.
3095  *        endif.
3096  *      endif.
3097  *    endloop.
3098  *    modify it_dyn_texpr from ld_rsds_expr.
3099  *  endloop.
3100  *
3101  *endform.                    "sdf_dynamics
3102
3103  *&---------------------------------------------------------------------*
3104  *&      Form  fill_sdf_callback
3105  *&---------------------------------------------------------------------*
3106  *       text
3107  *----------------------------------------------------------------------*
3108  *form fill_sdf_callback.
3109  *  refresh: it_callback.
3110  *  it_callback-ldbnode = 'BSIS'.
3111  *  it_callback-get     = 'X'.
3112  *  it_callback-cb_prog = c_repid_fagl_gl.
3113  *  it_callback-cb_form = 'CB_SDF_GET_BSIS'.
3114  *  append it_callback.
3115  *endform.                    "fill_sdf_callback
3116
3117  *&---------------------------------------------------------------------*
3118  *&      Form  cb_sdf_get_bsis
3119  *&---------------------------------------------------------------------*
3120  *       text
3121  *----------------------------------------------------------------------*
3122  *      -->NAME       text
3123  *      -->LS_BSIS    text
3124  *      -->MODE       text
3125  *      -->SELECTED   text
3126  *----------------------------------------------------------------------*
3127  *form cb_sdf_get_bsis  using  name     like ldbn-ldbnode
3128  *                             ls_bsis  like bsis
3129  *                             mode     type c
3130  *                             selected type c.
```

```
3131  *  bsis = ls_bsis.
3132  *  perform pos_table_fill  changing  x_stop.
3133  *endform.                    "cb_sdf_get_bsis
3134
3135
3136  *&---------------------------------------------------------------------*
3137  *&     Include RFITEM_INC
3138  *&---------------------------------------------------------------------*
3139  INCLUDE fagl_account_items_inc.
3140
3141  *---------------------------------------------------------------------*
3142  *      New Forms
3143  *---------------------------------------------------------------------*
3144
3145  *&---------------------------------------------------------------------*
3146  *&      Form  FS_SET_SSCRTEXTS_DYNSEL
3147  *&---------------------------------------------------------------------*
3148  *       Text fÃ¼r Drucktaste
3149  *---------------------------------------------------------------------*
3150  *  ---->  p1        text
3151  *  <----  p2        text
3152  *---------------------------------------------------------------------*
3153  FORM fs_set_sscrtexts_dynsel CHANGING p_text " LIKE SSCRTEXTS-DYNSEL.
3154                                            LIKE smp_dyntxt.
3155    DATA l_text LIKE smp_dyntxt.
3156
3157    MOVE: icon_fencing TO l_text-icon_id,
3158          'Free selections'(270) TO l_text-text.              "#EC *
3159
3160    IF fs_num > 0.
3161      WRITE fs_num TO l_text-icon_text(2).
3162      MOVE 'active'(271) TO l_text-icon_text+3.               "#EC *
3163    ENDIF.
3164    p_text = l_text.
3165
3166  ENDFORM.                               " SET_SSCRTEXTS_DYNSEL
3167  *&---------------------------------------------------------------------*
3168  *&      Form  FS_SET_SSCRTEXTS_DYNSEL3
3169  *&---------------------------------------------------------------------*
3170  *       Text fÃ¼r Drucktaste
3171  *---------------------------------------------------------------------*
3172  *  ---->  p1        text
3173  *  <----  p2        text
3174  *---------------------------------------------------------------------*
3175  FORM fs_set_sscrtexts_dynsel3 CHANGING p_text " LIKE SSCRTEXTS-DYNSEL.
3176                                            LIKE smp_dyntxt.
3177    DATA l_text LIKE smp_dyntxt.
3178
3179    MOVE: icon_other_object TO l_text-icon_id,
3180          TEXT-272 TO l_text-text.                            "#EC *
3181    MOVE TEXT-272 TO l_text-icon_text.
3182
3183    p_text = l_text.
3184
3185  ENDFORM.                               " SET_SSCRTEXTS_DYNSEL
3186  *&---------------------------------------------------------------------*
3187  *&      Form  FS_SET_SSCRTEXTS_DYNSEL4
3188  *&---------------------------------------------------------------------*
3189  *       Text fÃ¼r Drucktaste
3190  *---------------------------------------------------------------------*
3191  *  ---->  p1        text
```

```
3192  *  <——  p2        text
3193  *———————————————————————————————————————————————————————————————————*
3194  FORM fs_set_sscrtexts_dynsel4 CHANGING p_text " LIKE SSCRTEXTS-DYNSEL.
3195                                              LIKE smp_dyntxt.
3196    DATA l_text LIKE smp_dyntxt.
3197    DATA ld_text LIKE l_text-text.
3198
3199    IF gv_xerfs IS INITIAL.
3200      ld_text = TEXT-278.
3201    ELSE.
3202      ld_text = TEXT-277.
3203    ENDIF.
3204
3205    MOVE: icon_summarize_undo TO l_text-icon_id,
3206          ld_text TO l_text-text.                              "#EC *
3207    MOVE  ld_text TO l_text-icon_text.
3208
3209    p_text = l_text.
3210
3211  ENDFORM.                              " SET_SSCRTEXTS_DYNSEL
3212
3213  *&———————————————————————————————————————————————————————————————————*
3214  *&      Form  choose_ledger
3215  *&———————————————————————————————————————————————————————————————————*
3216  FORM choose_ledger
3217              TABLES it_chosen_values TYPE ty_sval
3218              USING uc_relevant_ledger TYPE rldnr_flex
3219              CHANGING cc_new_ledger TYPE rldnr_flex.
3220
3221    INCLUDE <icon>.
3222
3223    DATA ic_icon_okay LIKE icon-name.
3224    DATA ic_icon_save LIKE icon-name.
3225
3226  * set exporting parameters for function modul
3227  * POPUP_GET_VALUES_USER_BUTTONS
3228  *  it_chosen_values-tabname = 'T881'.
3229  *  it_chosen_values-fieldname = 'RLDNR'.
3230    LOG-POINT ID fins_udoc.
3231    IF cl_fins_acdoc_util=>get_acdoc_activation_status( ) = if_fins_acdoc_
      constants=>gc_acdoc_activation_status-inactive.
3232      it_chosen_values-tabname = 'BKPF'.
3233    ELSE.
3234      it_chosen_values-tabname = if_fins_acdoc_constants=>gc_tabname_acdoc
      a.
3235    ENDIF.
3236    it_chosen_values-fieldname = 'RLDNR'.
3237    it_chosen_values-field_obl = 'X'.
3238    it_chosen_values-value     = uc_relevant_ledger.
3239    APPEND it_chosen_values.
3240
3241    ic_icon_okay = icon_okay.
3242    ic_icon_save = icon_system_save.
3243
3244  * call function modul
3245
3246    CALL FUNCTION 'POPUP_GET_VALUES_USER_BUTTONS'
3247      EXPORTING
3248  *     F1_FORMNAME        = ' '
3249  *     F1_PROGRAMNAME     = ' '
3250        f4_formname        = 'F4_LEDGER'
```

```
3251         f4_programname     = 'FAGL_ACCOUNT_BALANCE'
3252         formname           = 'CHECK_OK_CODES'
3253         programname        = 'FAGL_ACCOUNT_BALANCE'
3254         popup_title        = 'Ledger setzen'(275)
3255         ok_pushbuttontext  = ''
3256         icon_ok_push       = ic_icon_okay
3257         quickinfo_ok_push  = TEXT-273
3258         first_pushbutton   = ' '
3259         icon_button_1      = ic_icon_save
3260         quickinfo_button_1 = TEXT-274
3261   *     SECOND_PUSHBUTTON  = ' '
3262   *     ICON_BUTTON_2      =
3263   *     QUICKINFO_BUTTON_2 = ' '
3264         start_column       = '25'
3265         start_row          = '7'
3266   *     NO_CHECK_FOR_FIXED_VALUES       = ' '
3267   * IMPORTING
3268   *     RETURNCODE         =
3269       TABLES
3270         fields             = it_chosen_values
3271       EXCEPTIONS
3272         error_in_fields    = 1
3273         OTHERS             = 2.
3274
3275     IF sy-subrc <> 0.
3276       deco_message_sy.
3277     ENDIF.
3278
3279     cc_new_ledger = it_chosen_values-value.
3280
3281   ENDFORM.                    " choose_ledger
3282
3283   *---------------------------------------------------------------------*
3284   *   form worklist_on_off_new
3285   *   set worklist flag on <-> off, set button text accordingly
3286   *---------------------------------------------------------------------*
3287   *form worklist_on_off_new.
3288   *
3289   *  data: ld_value   type c.
3290   *
3291   *  if gd_wl_on = c_x.
3292   *    perform wl_memory_set_get using c_x
3293   *                                    ld_value.
3294   *  else.
3295   *    ld_value = c_x.
3296   *    perform wl_memory_set_get using c_x
3297   *                                    ld_value.
3298   *  endif.
3299   *  perform wl_flag_and_button.
3300   *
3301   *endform.                    "worklist_on_off
3302
3303   *&---------------------------------------------------------------------*
3304   *&      Form  find_relevant_ledger
3305   *&---------------------------------------------------------------------*
3306   FORM find_relevant_ledger CHANGING cc_relevant_ledger TYPE fagl_rldnr.
3307
3308   *  DATA lv_return_code TYPE syst-subrc.
3309   *  DATA lv_ledger_user_format TYPE usr05-parva.
3310
3311   * Access to user master data for the ledger.
```

```
3312  * open question: Is the parameter ID correct?
3313
3314  *  CALL FUNCTION 'G_GET_USER_PARAMETER'
3315  *    EXPORTING
3316  *      parameter_id    = 'GLN_FLEX'
3317  *    IMPORTING
3318  *      parameter_value = lv_ledger_user_format
3319  *      rc              = lv_return_code.
3320
3321    CALL FUNCTION 'GL_USER_DATA_GET'
3322      EXPORTING
3323        owner          = gc_owner
3324        name           = gc_parameter_id
3325      IMPORTING
3326        value          = cc_relevant_ledger
3327      EXCEPTIONS
3328        invalid_owner  = 1
3329        name_not_found = 2.
3330
3331  *  cc_relevant_ledger = lv_ledger_user_format.
3332
3333  * If program runs in decouple mode use ledger from API
3334    IF cl_fin_ui_deco=>mv_mode = abap_true.
3335      IF NOT gv_deco_ledger IS INITIAL.
3336        cc_relevant_ledger = gv_deco_ledger.
3337        sy-subrc = 0.
3338      ENDIF.
3339    ENDIF.
3340
3341    IF sy-subrc NE 0.
3342  * default ledger.
3343      CALL FUNCTION 'FAGL_GET_LEADING_LEDGER'
3344        IMPORTING
3345          e_rldnr   = cc_relevant_ledger
3346        EXCEPTIONS
3347          not_found = 1.
3348
3349      IF sy-subrc <> 0.
3350  * MESSAGE ID SY-MSGID TYPE SY-MSGTY NUMBER SY-MSGNO
3351  *         WITH SY-MSGV1 SY-MSGV2 SY-MSGV3 SY-MSGV4.
3352      ENDIF.
3353    ENDIF.
3354
3355  ENDFORM.                    " find_relevant_ledger
3356
3357  *&---------------------------------------------------------------------*
3358  *&      Form  FREE_SELECTIONS
3359  *&---------------------------------------------------------------------*
3360  *       free selections
3361  *----------------------------------------------------------------------*
3362  *  -->  p1        text
3363  *  <--  p2        text
3364  *----------------------------------------------------------------------*
3365  FORM free_selections.
3366  *? DATA BEGIN OF P_TABLES OCCURS 10.
3367  *?          INCLUDE STRUCTURE RSDSTABS.
3368  *?  DATA END   OF P_TABLES.
3369    DATA BEGIN OF tables_tab OCCURS 10.
3370    INCLUDE STRUCTURE rsdstabs.
3371    DATA END   OF tables_tab.
3372  *Felder obiger Tabellen
```

```
3373    DATA: BEGIN OF fields OCCURS 10.
3374            INCLUDE STRUCTURE rsdsfields.
3375    DATA: END   OF fields.
3376    DATA: BEGIN OF fields_not OCCURS 10.
3377            INCLUDE STRUCTURE rsdsfields.
3378    DATA: END   OF fields_not.
3379
3380    DATA:ds_expr_tab TYPE rsds_expr_tab.
3381    DATA:ds_expr_tab_line LIKE rsdsexpr.
3382
3383    DATA: ds_texpr  TYPE rsds_expr.
3384    DATA: ld_rep_ldgr TYPE rldnr.
3385    DATA: ls_tab TYPE  fagl_tabnames.
3386    DATA: selection_id LIKE  rsdynsel-selid.
3387    DATA: ls_dfies TYPE dfies.
3388    DATA: lt_dfies TYPE TABLE OF dfies.
3389    DATA: lt_incl TYPE TABLE OF dfies.
3390    DATA: ld_lines LIKE sy-tabix.
3391
3392  ** new declaration
3393  *  DATA: ls_tablenames  TYPE fagl_tabnames.
3394  **  DATA: ds_trange  TYPE RSDS_TRANGE.
3395  **  data: ds_trange_tab type RSDS_TRANGE.
3396  *  data: ds_trange_tab_line type RSDS_RANGE.
3397  **  Data: ds_Frange_tab type RSDS_FRANGE_T.
3398  *  data: ds_frange_tab_line type RSDS_FRANGE.
3399    DATA: ld_tablename TYPE rsds_expr-tablename.
3400  *  data: ds_Texpr_tab_line type RSDS_EXPR.
3401  *  data: ds_expr_tab_tab_line like RSDSEXPR.
3402  *  data: ld_fieldname type RSDSEXPR-fieldname.
3403  *  data: lt_excl_fields like fields occurs 0 with header line.
3404  *  data: ls_texpr_tab_line type RSDS_EXPR.
3405  *  data: ls_expr_tab_line LIKE RSDSEXPR.
3406  *  data: ls_num like sy-tfill.
3407  *  data: ls_field like rsdsfields.
3408  *  data: begin of lt_tab_fld_active occurs 0,
3409  *          tablename LIKE RSDSTABS-PRIM_TAB,
3410  *          fieldname like RSDSEXPR-fieldname,
3411  *        end of lt_tab_fld_active.
3412  *  data: ld_xdel type c.
3413
3414
3415  * new free selection
3416    tables_tab-prim_tab = 'SKA1_FS'. APPEND tables_tab.
3417    tables_tab-prim_tab = 'SKB1_FS'. APPEND tables_tab.
3418  *tables_tab-prim_tab = 'BKPF_FS'. APPEND tables_tab.
3419  *tables_tab-prim_tab = 'BSEG_FS'. APPEND tables_tab.
3420    tables_tab-prim_tab = 'BSIS_FS'. APPEND tables_tab.
3421    IF  gv_xerfs IS INITIAL.
3422      CONCATENATE gv_si_table '_FS' INTO ld_tablename.
3423      tables_tab-prim_tab = ld_tablename. APPEND tables_tab.
3424    ENDIF.
3425
3426    fields[] = fs_field[].
3427
3428    CALL FUNCTION 'FREE_SELECTIONS_INIT'
3429      EXPORTING
3430  *     kind                       = 'G'
3431        kind                       = 'T'
3432  *     kind                       = 'F'
3433  *         Alte Abgrenzungen erscheinen wieder
```

```
3434         expressions              = fs_dyns-texpr
3435   *     FIELD_GROUPS_KEY         = ds_groups_key
3436   *     RESTRICTION              =
3437   *     ALV                      =
3438   *     CURR_QUAN_PROG           = SY-CPROG
3439   *     CURR_QUAN_RELATION       =
3440       IMPORTING
3441         selection_id             = selection_id
3442         where_clauses            = fs_dyns-clauses
3443         expressions              = fs_dyns-texpr
3444         field_ranges             = fs_dyns-trange
3445         number_of_active_fields  = fs_num
3446       TABLES
3447         tables_tab               = tables_tab
3448         fields_tab               = fields
3449   *     FIELD_DESC               =
3450   *     FIELD_TEXTS              =
3451   *     EVENTS                   =
3452   *     EVENT_FIELDS             =
3453         fields_not_selected      = fields_not
3454       EXCEPTIONS
3455         fields_incomplete        = 1
3456         fields_no_join           = 2
3457         field_not_found          = 3
3458         no_tables                = 4
3459         table_not_found          = 5
3460         expression_not_supported = 6
3461         incorrect_expression     = 7
3462         illegal_kind             = 8
3463         area_not_found           = 9
3464         inconsistent_area        = 10
3465         kind_f_no_fields_left    = 11
3466         kind_f_no_fields         = 12
3467         too_many_fields          = 13
3468         dup_field                = 14
3469         field_no_type            = 15
3470         field_ill_type           = 16
3471         dup_event_field          = 17
3472         node_not_in_ldb          = 18
3473         area_no_field            = 19
3474         OTHERS                   = 20.
3475     IF sy-subrc <> 0.
3476   *    MESSAGE ID sy-msgid TYPE sy-msgty NUMBER sy-msgno
3477   *            WITH sy-msgv1 sy-msgv2 sy-msgv3 sy-msgv4.
3478       deco_message_0 e104.
3479     ENDIF.
3480   * endif.
3481
3482     CHECK sy-subrc = 0.
3483     CALL FUNCTION 'FREE_SELECTIONS_DIALOG'
3484       EXPORTING
3485         selection_id           = selection_id
3486         title                  = 'Freie Selektion '(276)
3487   *     FRAME_TEXT             = ' '
3488   *     STATUS                 =
3489   *     AS_WINDOW              = ' '
3490   *     START_ROW              = 2
3491   *     START_COL              = 2
3492   *     NO_INTERVALS           = ' '
3493   *     JUST_DISPLAY           = ' '
3494   *     PFKEY                  =
```

```
3495  *      ALV                   = ' '
3496         tree_visible          = ' '
3497  *      DIAG_TEXT_1           =
3498  *      DIAG_TEXT_2           =
3499  *      WARNING_TITLE         =
3500       IMPORTING
3501         where_clauses         = fs_dyns-clauses
3502         expressions           = fs_dyns-texpr
3503         field_ranges          = fs_dyns-trange
3504         number_of_active_fields = fs_num
3505       TABLES
3506         fields_tab            = fields
3507  *      FCODE_TAB             =
3508         fields_not_selected   = fields_not
3509       EXCEPTIONS
3510         internal_error        = 1
3511  *      no_action             =
3512         selid_not_found       = 2
3513         illegal_status        = 3
3514         OTHERS                = 0.
3515
3516    IF sy-subrc <> 0.
3517  *    MESSAGE ID sy-msgid TYPE sy-msgty NUMBER sy-msgno
3518  *            WITH sy-msgv1 sy-msgv2 sy-msgv3 sy-msgv4.
3519       deco_message_0 e104.
3520    ENDIF.
3521
3522    PERFORM check_reporting_fields.
3523
3524    fs_field[] = fields[].
3525
3526  ENDFORM.                    "free_selections
3527  *&---------------------------------------------------------------------*
3528  *&      Form  set_dyn_sel_ldb
3529  *&---------------------------------------------------------------------*
3530  *       Modify dynamic Selections of Log. DB
3531  *----------------------------------------------------------------------*
3532  *  --->  p1        text
3533  *  <---  p2        text
3534  *----------------------------------------------------------------------*
3535  FORM set_dyn_sel_ldb .
3536    DATA: ls_dyns TYPE rsds_type.
3537    DATA: ls_clauses_tab_line TYPE rsds_where.
3538    DATA: ls_texpr_tab_line TYPE rsds_expr.
3539    DATA: ls_trange_tab_line TYPE rsds_range.
3540
3541    ls_dyns = fs_dyns.
3542    LOOP AT ls_dyns-clauses INTO ls_clauses_tab_line.
3543      IF ls_clauses_tab_line-tablename = 'SKA1_FS'
3544      OR ls_clauses_tab_line-tablename = 'SKB1_FS'.
3545        IF ls_clauses_tab_line-tablename = 'SKA1_FS'.
3546          ls_clauses_tab_line-tablename = 'SKA1'.
3547          MODIFY ls_dyns-clauses FROM ls_clauses_tab_line.
3548        ELSE.
3549          ls_clauses_tab_line-tablename = 'SKB1'.
3550          MODIFY ls_dyns-clauses FROM ls_clauses_tab_line.
3551        ENDIF.
3552      ELSE.
3553        DELETE ls_dyns-clauses.
3554      ENDIF.
3555    ENDLOOP.
```

```
3556
3557    LOOP AT ls_dyns-texpr INTO ls_texpr_tab_line.
3558      IF ls_texpr_tab_line-tablename = 'SKA1_FS'
3559      OR ls_texpr_tab_line-tablename = 'SKB1_FS'.
3560        IF ls_texpr_tab_line-tablename = 'SKA1_FS'.
3561          ls_texpr_tab_line-tablename = 'SKA1'.
3562          MODIFY ls_dyns-texpr FROM ls_texpr_tab_line.
3563        ELSE.
3564          ls_texpr_tab_line-tablename = 'SKB1'.
3565          MODIFY ls_dyns-texpr FROM ls_texpr_tab_line.
3566        ENDIF.
3567      ELSE.
3568        DELETE ls_dyns-texpr.
3569      ENDIF.
3570    ENDLOOP.
3571
3572    LOOP AT ls_dyns-trange INTO ls_trange_tab_line.
3573      IF ls_trange_tab_line-tablename = 'SKA1_FS'
3574      OR ls_trange_tab_line-tablename = 'SKB1_FS'.
3575        IF ls_trange_tab_line-tablename = 'SKA1_FS'.
3576          ls_trange_tab_line-tablename = 'SKA1'.
3577          MODIFY ls_dyns-trange FROM ls_trange_tab_line.
3578        ELSE.
3579          ls_trange_tab_line-tablename = 'SKB1'.
3580          MODIFY ls_dyns-trange FROM ls_trange_tab_line.
3581        ENDIF.
3582      ELSE.
3583        DELETE ls_dyns-trange.
3584      ENDIF.
3585    ENDLOOP.
3586
3587
3588    PERFORM modif_dyn_sel(sapdbsdf) USING ls_dyns.
3589    gs_dyns = ls_dyns.
3590  ENDFORM.                    " set_dyn_sel_ldb
3591  *&---------------------------------------------------------------------*
3592  *&      Form  find_si_table
3593  *&---------------------------------------------------------------------*
3594  *       text
3595  *----------------------------------------------------------------------*
3596  *      -->P_GV_RELEVANT_LEDGER  text
3597  *      <--P_GV_SI_TABLE  text
3598  *----------------------------------------------------------------------*
3599  FORM find_si_table  USING    p_rldnr TYPE rldnr_flex
3600                      CHANGING p_si_table LIKE fagl_tabnames-si_table.
3601
3602    DATA: ls_tablenames  TYPE fagl_tabnames.
3603
3604    CALL FUNCTION 'FAGL_GET_TABLENAMES'
3605      EXPORTING
3606        i_ledger                    = p_rldnr
3607        ib_acdoc_compatibility_mode = abap_false
3608      IMPORTING
3609        es_tabnames                 = ls_tablenames.
3610
3611    p_si_table = ls_tablenames-si_table.
3612
3613  ENDFORM.                    " find_si_table
3614  *&---------------------------------------------------------------------*
3615  *&      Form  del_free_selections_si_tab
3616  *&---------------------------------------------------------------------*
```

```
3617  *       text
3618  *----------------------------------------------------------------------*
3619  *  ---->  p1        text
3620  *  <----  p2        text
3621  *----------------------------------------------------------------------*
3622  FORM del_free_selections_si_tab
3623                           CHANGING p_gd_xdels TYPE xfeld.
3624
3625    DATA: ls_clauses_tab_line TYPE rsds_where.
3626    DATA: ls_texpr_tab_line TYPE rsds_expr.
3627    DATA: ls_expr_tab_line LIKE rsdsexpr.
3628    DATA: ls_trange_tab_line TYPE rsds_range.
3629    DATA: ls_num LIKE sy-tfill.
3630    DATA: ls_field LIKE rsdsfields.
3631    DATA: BEGIN OF lt_tab_fld_active OCCURS 0,
3632            tablename LIKE rsdstabs-prim_tab,
3633            fieldname LIKE rsdsexpr-fieldname,
3634          END OF lt_tab_fld_active.
3635    DATA: ls_si_table_fs TYPE tabname.
3636    DATA: ld_xdels TYPE xfeld.
3637
3638    CONCATENATE gv_save_si_table '_FS' INTO ls_si_table_fs.
3639
3640    LOOP AT fs_dyns-clauses INTO ls_clauses_tab_line.
3641      IF ls_clauses_tab_line-tablename = gv_save_si_table.
3642        DELETE fs_dyns-clauses.
3643        ld_xdels = 'X'.
3644      ENDIF.
3645    ENDLOOP.
3646
3647    LOOP AT fs_dyns-texpr INTO ls_texpr_tab_line.
3648      IF ls_texpr_tab_line-tablename = ls_si_table_fs.
3649        DELETE fs_dyns-texpr.
3650        ld_xdels = 'X'.
3651      ENDIF.
3652    ENDLOOP.
3653
3654    LOOP AT fs_dyns-trange INTO ls_trange_tab_line.
3655      IF ls_trange_tab_line-tablename = ls_si_table_fs.
3656        DELETE fs_dyns-trange.
3657        ld_xdels = 'X'.
3658      ENDIF.
3659    ENDLOOP.
3660
3661    LOOP AT fs_field INTO ls_field.
3662      IF ls_field-tablename = ls_si_table_fs.
3663        DELETE fs_field.
3664        ld_xdels = 'X'.
3665      ENDIF.
3666    ENDLOOP.
3667
3668    LOOP AT fs_dyns-texpr INTO ls_texpr_tab_line.
3669      lt_tab_fld_active-tablename = ls_texpr_tab_line-tablename.
3670      LOOP AT ls_texpr_tab_line-expr_tab INTO ls_expr_tab_line.
3671        IF NOT ls_expr_tab_line-fieldname IS INITIAL.
3672          lt_tab_fld_active-fieldname = ls_expr_tab_line-fieldname.
3673          COLLECT lt_tab_fld_active.
3674        ENDIF.
3675      ENDLOOP.
3676    ENDLOOP.
3677    DESCRIBE TABLE lt_tab_fld_active LINES ls_num.
```

```
3678    IF ls_num NE fs_num.
3679      fs_num = ls_num.
3680    ENDIF.
3681
3682    p_gd_xdels = ld_xdels.
3683
3684  ENDFORM.                    " change_free_selections
3685
3686  *&---------------------------------------------------------------------*
3687  *&      Form  GET_GENERAL_PARAM_NEW
3688  *&---------------------------------------------------------------------*
3689  *form get_general_param_new.
3690  *  data: charh(16)   type c,
3691  *        ld_num(16)  type n.
3692  *  data: ls_rfopt  type rfopt.
3693  *  data: ld_parid type usr05-parid.
3694  *
3695  **... get checkbox stuff depending on acc.type:
3696  *  case sy-repid.
3697  *    when c_repid_ar.
3698  *      get parameter id 'FIT_X_AR' field charh.
3699  *      get parameter id 'FIT_ALV_AR' field pa_vari.
3700  *      ld_parid = 'FIT_ALV_AR'.
3701  *    when c_repid_ap.
3702  *      get parameter id 'FIT_X_AP' field charh.
3703  *      get parameter id 'FIT_ALV_AP' field pa_vari.
3704  *      ld_parid = 'FIT_ALV_AP'.
3705  *    when c_repid_gl.
3706  *      get parameter id 'FIT_X_GL' field charh.
3707  *      get parameter id 'FIT_ALV_GL' field pa_vari.
3708  *      ld_parid = 'FIT_ALV_GL'.
3709  *    when c_repid_FAGl_GL.                        "Param IDs in Arbeit
3710  *      get parameter id 'FIT_X_GL' field charh.
3711  *      get parameter id 'FIT_ALV_GL' field pa_vari.
3712  *      ld_parid = 'FIT_ALV_GL'.
3713  *    when others.
3714  **      horror case!
3715  *      exit.
3716  *  endcase.
3717  *  if not charh is initial.
3718  *    x_norm = charh(1).
3719  *    x_shbv = charh+1(1).
3720  *    x_merk = charh+2(1).
3721  *    x_park = charh+3(1).
3722  *    x_apar = charh+4(1).
3723  *  endif.
3724  *
3725  **...changing of pa_vari (default value FIT_ALV_**)
3726  *  get parameter id 'FOP' field ls_rfopt.
3727  *  if not ls_rfopt-xarbv is initial.
3728  *    select single parva from usr05 into (pa_vari)
3729  *                        where bname = sy-uname
3730  *                        and parid = ld_parid.
3731  *    if sy-subrc <> 0.
3732  *      clear pa_vari.
3733  *    endif.
3734  *  endif.
3735  *
3736  **... get item number limit:
3737  *  get parameter id 'FIT_X_NMAX' field ld_num.
3738  *  if ld_num co ' 0123456789'.
```

```
3739  *    pa_nmax = ld_num.
3740  *  endif.
3741  *
3742  *endform.                                " GET_GENERAL_PARAM
3743  *
3744  **&---------------------------------------------------------------------*
3745  **
3746  **&      Form  SET_GENERAL_PARAM_NEW
3747  **&---------------------------------------------------------------------*
3748  **
3749  *form set_general_param_new.
3750  *
3751  *  data: ls_rfopt  type rfopt.
3752  *  data: charh(16) type c,
3753  *        ld_num(16)  type n,
3754  *        ls_usr05  like usr05.
3755  *  data: ld_refresh_count   type i,
3756  *        ld_memory_id(40)   type c.
3757  *
3758  *  check sy-calld is initial.
3759  *  check sy-tcode cp 'FBL+N' or
3760  *      ( sy-tcode cp 'S+38' and sy-repid cp 'RFITEM++' ) or
3761  *      ( sy-tcode eq 'SE80' and sy-repid cp 'RFITEM++' ).
3762  *
3763  *  charh(1)   = x_norm.
3764  *  charh+1(1) = x_shbv.
3765  *  charh+2(1) = x_merk.
3766  *  charh+3(1) = x_park.
3767  *  charh+4(1) = x_apar.
3768  ** if in refresh call, then do not update layout parameter:
3769  *  ld_memory_id = sy-repid.
3770  *  write '/REFR' to ld_memory_id+35(5).
3771  *  condense ld_memory_id no-gaps.
3772  *  import counter to ld_refresh_count
3773  *         from memory id ld_memory_id.
3774  *  case sy-repid.
3775  *    when c_repid_ar.
3776  *      set parameter id 'FIT_X_AR'   field charh.
3777  *      if ld_refresh_count = 0.
3778  *        get parameter id 'FOP' field ls_rfopt.
3779  *        if ls_rfopt-xarbv is initial.
3780  *          set parameter id 'FIT_ALV_AR' field pa_vari.
3781  *          write 'FIT_ALV_AR' to ls_usr05-parid.
3782  *        else.
3783  *          set parameter id 'FIT_ALV_AR' field space.
3784  *        endif.
3785  *      endif.
3786  *    when c_repid_ap.
3787  *      set parameter id 'FIT_X_AP'   field charh.
3788  *      if ld_refresh_count = 0.
3789  *        get parameter id 'FOP' field ls_rfopt.
3790  *        if ls_rfopt-xarbv is initial.
3791  *          set parameter id 'FIT_ALV_AP' field pa_vari.
3792  *          write 'FIT_ALV_AP' to ls_usr05-parid.
3793  *        else.
3794  *          set parameter id 'FIT_ALV_AP' field space.
3795  *        endif.
3796  *      endif.
3797  *    when c_repid_gl.
3798  *      set parameter id 'FIT_X_GL'   field charh.
3799  *      if ld_refresh_count = 0.
```

```
3800  *        get parameter id 'FOP' field ls_rfopt.
3801  *        if ls_rfopt-xarbv is initial.
3802  *          set parameter id 'FIT_ALV_GL' field pa_vari.
3803  *          write 'FIT_ALV_GL' to ls_usr05-parid.
3804  *        else.
3805  *          set parameter id 'FIT_ALV_GL' field space.
3806  *        endif.
3807  *      endif.
3808  *    when c_repid_FAGl_GL.
3809  *      set parameter id 'FIT_X_GL'   field charh.
3810  *      if ld_refresh_count = 0.
3811  *        get parameter id 'FOP' field ls_rfopt.
3812  *        if ls_rfopt-xarbv is initial.
3813  *          set parameter id 'FIT_ALV_GL' field pa_vari.
3814  *          write 'FIT_ALV_GL' to ls_usr05-parid.
3815  *        else.
3816  *          set parameter id 'FIT_ALV_GL' field space.
3817  *        endif.
3818  *      endif.
3819  *    when others.
3820  **     horror case!
3821  *      exit.
3822  *  endcase.
3823  *  if ld_refresh_count = 0.
3824  *    get parameter id 'FOP' field ls_rfopt.
3825  *    if ls_rfopt-xarbv is initial.
3826  *      ls_usr05-bname = sy-uname.
3827  *      write pa_vari to ls_usr05-parva.
3828  *      if ls_usr05-parva is initial.
3829  *        delete from usr05 where bname = ls_usr05-bname
3830  *                          and   parid = ls_usr05-parid.
3831  *      else.
3832  *        modify usr05 from ls_usr05.
3833  *      endif.
3834  *      commit work.
3835  *    endif.
3836  *  endif.
3837  *  ld_num = pa_nmax.
3838  *  set parameter id 'FIT_X_NMAX' field ld_num.
3839  *
3840  *endform.                                " SET_GENERAL_PARAM
3841
3842  *&---------------------------------------------------------------------*
3843  *&      Form  MAKE_FIELDCATALOG_NEW
3844  *&---------------------------------------------------------------------*
3845  *form make_fieldcatalog_new.
3846  *  data: lt_dfies_rfposx   like dfies occurs  50 with header line,
3847  *        lt_dfies_bsix     like dfies occurs 100 with header line,
3848  *        lt_duplicates     type tpit_t_fname with header line,
3849  *        ld_fname          like dfies-fieldname,
3850  *        ld_name_bsix      like dcobjdef-name,
3851  *        lt_fmrfc          like fmrfc occurs 1.
3852  *
3853  **...define icon fields:
3854  *
3855  *  clear gt_fieldcat.
3856  *  gt_fieldcat-fieldname = 'SYM_AUGP'.
3857  *  gt_fieldcat-symbol    = 'X'.
3858  *  append gt_fieldcat.
3859  *  clear gt_fieldcat.
3860  *  gt_fieldcat-fieldname = 'ICO_AUGP'.
```

```
3861  *  gt_fieldcat-icon      = 'X'.
3862  *  append gt_fieldcat.
3863  *  clear gt_fieldcat.
3864  *  gt_fieldcat-fieldname = 'ICO_DUE'.
3865  *  gt_fieldcat-icon      = 'X'.
3866  *  append gt_fieldcat.
3867  *  clear gt_fieldcat.
3868  *  gt_fieldcat-fieldname = 'ICO_DUE1'.
3869  *  gt_fieldcat-icon      = 'X'.
3870  *  append gt_fieldcat.
3871  *
3872  **...define technical and obsolete fields (static):
3873  *
3874  *  clear gt_fieldcat.
3875  *  gt_fieldcat-fieldname = 'ACTIV'.
3876  *  gt_fieldcat-tech      = 'X'.
3877  *  append gt_fieldcat.
3878  *  clear gt_fieldcat.
3879  *  gt_fieldcat-fieldname = 'STAKZ'.
3880  *  gt_fieldcat-tech      = 'X'.
3881  *  append gt_fieldcat.
3882  *  clear gt_fieldcat.
3883  *  gt_fieldcat-fieldname = 'XAUGP'.
3884  *  gt_fieldcat-tech      = 'X'.
3885  *  append gt_fieldcat.
3886  *  clear gt_fieldcat.
3887  *  gt_fieldcat-fieldname = 'COLOR'.
3888  *  gt_fieldcat-tech      = 'X'.
3889  *  append gt_fieldcat.
3890  *  clear gt_fieldcat.
3891  *  gt_fieldcat-fieldname = 'XZAHL'.
3892  *  gt_fieldcat-tech      = 'X'.
3893  *  append gt_fieldcat.
3894  ** hide GKONT and GKART, if OPEN FI exit 1650 not used:
3895  *  call function 'BF_FUNCTIONS_FIND'
3896  *    EXPORTING
3897  *      i_event       = '00001650'
3898  *    TABLES
3899  *      t_fmrfc       = lt_fmrfc
3900  *    EXCEPTIONS
3901  *      nothing_found = 1
3902  *      others        = 2.
3903  *  if sy-subrc ne 0.
3904  *    clear gt_fieldcat.
3905  *    gt_fieldcat-fieldname = 'GKONT'.
3906  *    gt_fieldcat-tech      = 'X'.
3907  *    append gt_fieldcat.
3908  *    clear gt_fieldcat.
3909  *    gt_fieldcat-fieldname = 'GKART'.
3910  *    gt_fieldcat-tech      = 'X'.
3911  *    append gt_fieldcat.
3912  *  endif.
3913  *
3914  **...define obsolete fields (dynamic):
3915  *
3916  ** check for existence of special fields:
3917  *  read table it_t021s index 1.
3918  *  check sy-subrc = 0.
3919  *
3920  ** get RFPOSX field list:
3921  *  call function 'DDIF_NAMETAB_GET'
```

```
3922  *       exporting
3923  *            tabname     = 'RFPOSX'
3924  *       tables
3925  **           X031L_TAB   =
3926  *            dfies_tab   = lt_dfies_rfposx
3927  *       exceptions
3928  *            not_found   = 1
3929  *            others      = 2
3930  *            .
3931  *  check sy-subrc = 0.
3932  *
3933  ** collect duplicates = special fields which are contained in RFPOSX:
3934  *  loop at it_t021s.
3935  *    check not it_t021s-tname is initial.
3936  *    read table lt_dfies_rfposx with key
3937  *                               fieldname = it_t021s-fname.
3938  *    check sy-subrc = 0.
3939  *    lt_duplicates-fname = it_t021s-fname.
3940  *    append lt_duplicates.
3941  *  endloop.
3942  *
3943  ** found duplicates?
3944  *  read table lt_duplicates index 1.
3945  *  check sy-subrc = 0.
3946  *
3947  ** get BSIx field list:
3948  *  case g_repid.
3949  *    when c_repid_ar.
3950  *      ld_name_bsix = 'BSID'.
3951  *    when c_repid_ap.
3952  *      ld_name_bsix = 'BSIK'.
3953  *    when c_repid_gl.
3954  *      ld_name_bsix = 'BSIS'.
3955  *  endcase.
3956  *  call function 'DDIF_NAMETAB_GET'
3957  *       exporting
3958  *            tabname     = ld_name_bsix
3959  *       tables
3960  **           X031L_TAB   =
3961  *            dfies_tab   = lt_dfies_bsix
3962  *       exceptions
3963  *            not_found   = 1
3964  *            others      = 2
3965  *            .
3966  *  check sy-subrc = 0.
3967  *
3968  ** check whether duplicates are contained in BSIx:
3969  *  loop at lt_duplicates.
3970  *    clear ld_fname.
3971  *    read table lt_dfies_bsix with key
3972  *                               fieldname = lt_duplicates-fname.
3973  *    if sy-subrc = 0.
3974  **     standard field ok. set special field to 'tech':
3975  *      ld_fname = 'U_'.
3976  *      write lt_duplicates-fname to ld_fname+2.
3977  *    else.
3978  **     special field ok. set standard field to 'tech':
3979  *      ld_fname = lt_duplicates-fname.
3980  *    endif.
3981  *    clear gt_fieldcat.
3982  *    gt_fieldcat-fieldname = ld_fname.
```

```
*    gt_fieldcat-tech      = 'X'.
*    append gt_fieldcat.
*  endloop.
*
*endform.                                " MAKE_FIELDCATALOG

**&---------------------------------------------------------------------*
**
**&      Form  SPECIAL_FIELDS_new
**&---------------------------------------------------------------------*
**
*form special_fields_init_new using   rs_variant like disvariant.
*  data: it_var_fieldcat  type slis_t_fieldcat_alv with header line,
*        it_max_fieldcat  type slis_t_fieldcat_alv,
*        dummy_layout     type slis_layout_alv.
*
** ... get existing variant:
*  perform get_existing_variant using rs_variant.
**... get fieldcat of initial variant:
*  perform get_initial_fieldcat_new   tables it_var_fieldcat
*                                   using  rs_variant.
**... analyze initial fieldcat:
*  perform analyze_act_fieldcat  tables it_var_fieldcat.
**... read required special tables and fill item fields:
*  perform read_special_tables_new.
*
*endform.                                " SPECIAL_FIELDS_INIT

*&---------------------------------------------------------------------*
*&      Form GET_INITIAL_FIELDCAT
*&---------------------------------------------------------------------*
*form get_initial_fieldcat_new  tables rt_var_fieldcat
*                                      type slis_t_fieldcat_alv
*                            using  rs_variant like disvariant.
*  data: it_max_fieldcat  type slis_t_fieldcat_alv,
*        dummy_layout     type slis_layout_alv,
*        ls_variant       type disvariant,
*        ld_exit          type c.
*
*  ls_variant = rs_variant.
*
**... build maximal fieldcat:
*  call function 'REUSE_ALV_FIELDCATALOG_MERGE'
*    EXPORTING
*      i_buffer_active        = 'X'
*      i_program_name         = g_repid
*      i_structure_name       = 'FAGLPOSX'
*    CHANGING
*      ct_fieldcat            = it_max_fieldcat
*    EXCEPTIONS
*      inconsistent_interface = 1
*      program_error          = 2
*      others                 = 3.
*  if sy-subrc <> 0.
*    message a011.
*  endif.
**... get variant fieldcat:
*  call function 'REUSE_ALV_VARIANT_SELECT'
*    EXPORTING
*      i_dialog            = ' '
*      i_user_specific     = 'X'
```

```
4044  *      it_default_fieldcat = it_max_fieldcat
4045  *      i_layout            = dummy_layout
4046  *      i_buffer_active     = 'X'
4047  *    IMPORTING
4048  *      e_exit              = ld_exit
4049  *      et_fieldcat         = rt_var_fieldcat[]
4050  *    CHANGING
4051  *      cs_variant          = ls_variant
4052  *    EXCEPTIONS
4053  *      wrong_input         = 1
4054  *      fc_not_complete     = 2
4055  *      not_found           = 3
4056  *      program_error       = 4
4057  *      others              = 5.
4058  *  if sy-subrc <> 0.
4059  **... take maximal fieldcat for special fields:
4060  *    rt_var_fieldcat[] = it_max_fieldcat[].
4061  *  endif.
4062  *
4063  *endform.                                " GET_INITIAL_FIELDCAT
4064
4065  *&---------------------------------------------------------------------*
4066  *&      Form  READ_SPECIAL_TABLES
4067  *&---------------------------------------------------------------------*
4068  *form read_special_tables_new.
4069  *  data: posindex   like sy-tabix.
4070  *
4071  **... check item table filled?
4072  *  read table it_pos index 1.
4073  *  check sy-subrc = 0.
4074  **... loop over all pending tables:
4075  *  loop at it_spectab where tstat = 'P'.
4076  ** building additional help tables for BKPF and BSEG for optimized
4077  ** database selections.
4078  *    perform create_performance_tables_new using it_spectab-tname.
4079  *    case it_spectab-tname.
4080  *      when 'BKPF'.
4081  *        PERFORM SPECIAL_FIELDS_USED USING it_spectab-tname.
4082  *        if sy-subrc = 0.
4083  *          perform complete_bkpf.
4084  *        endif.
4085  *      when 'BSEG'.
4086  *        perform complete_bseg.
4087  *      when 'BSEGC'.
4088  *        PERFORM SPECIAL_FIELDS_USED USING it_spectab-tname.
4089  *        if sy-subrc = 0.
4090  *          perform complete_bsegc.
4091  *        endif.
4092  *      when 'BSEC'.
4093  *        perform complete_bsec.
4094  *      when 'BSED'.
4095  *        perform complete_bsed.
4096  *      when 'BSBV'.
4097  *        perform complete_bsbv.
4098  *      when 'PAYR'.
4099  *        perform complete_payr.
4100  *      when '_TEXT'.
4101  *        perform complete_document_text.
4102  *    endcase.
4103  *  endloop.
4104  **... final bookkeeping: status of special tables & fields
```

```
4105  *  perform upd_special_status.
4106  *
4107  *endform.                                " READ_SPECIAL_TABLES
4108
4109  *---------------------------------------------------------------------*
4110  *       FORM create_performance_tables                                *
4111  *---------------------------------------------------------------------*
4112  *       ........                                                      *
4113  *---------------------------------------------------------------------*
4114  *  -->  id_tablename                                                  *
4115  *---------------------------------------------------------------------*
4116  *form create_performance_tables
4117  *                            using id_tablename type dd03p-tabname .
4118  *
4119  *  field-symbols : <lfs_pos>              type FAGLPOSX.
4120  *  statics :       lst_performance_tables type c.
4121  *
4122  *  if lst_performance_tables is initial.
4123  *    if id_tablename eq 'BKPF' or id_tablename eq 'BSEG' .
4124  *
4125  ** Creating additional tables for speeding up the database selects
4126  *      refresh: gt_pos_parked, gt_pos_posted, gt_pos_archived.
4127  *
4128  *      loop at it_pos assigning <lfs_pos>.
4129  *        gd_pos_reduced-bukrs = <lfs_pos>-bukrs.
4130  *        gd_pos_reduced-belnr = <lfs_pos>-belnr.
4131  *        gd_pos_reduced-gjahr = <lfs_pos>-gjahr.
4132  *        gd_pos_reduced-buzei = <lfs_pos>-buzei.
4133  *        gd_pos_reduced-budat = <lfs_pos>-budat.
4134  *
4135  *        if <lfs_pos>-bstat ca 'VWZ'.
4136  *          insert gd_pos_reduced into table gt_pos_parked.
4137  *        elseif <lfs_pos>-xarch eq 'X' and gd_usear = 'X'.
4138  *          insert gd_pos_reduced into table gt_pos_archived.
4139  *        else.
4140  *          insert gd_pos_reduced into table gt_pos_posted.
4141  *        endif.
4142  *      endloop.
4143  *
4144  *      lst_performance_tables = 'X'.
4145  *    endif.
4146  *  endif.
4147  *
4148  *endform.
4149
4150  *&---------------------------------------------------------------------*
4151  *&      Form  CHECK_ITEM_OK
4152  *&---------------------------------------------------------------------*
4153  *form check_item_ok_new using    p_norm
4154  *                             p_shbv
4155  *                             p_merk
4156  *                             p_park
4157  *                             p_item    like faglposx
4158  *                    changing p_okay.
4159  *
4160  *  clear p_okay.
4161  ** normal document:
4162  *  if p_norm = 'X'.
4163  *    if ( p_item-bstat = space ) and ( p_item-umskz = space ).
4164  *      p_okay = 'X'.
4165  *    endif.
```

```
4166  *  endif.
4167  ** SHB item:
4168  *  if p_shbv = 'X'.
4169  *    if ( p_item-umskz ne space ) and ( p_item-bstat ne 'S' ) and
4170  *       ( p_item-bstat ne 'V' )   and ( p_item-bstat ne 'W' ) and
4171  *       ( p_item-bstat ne 'Z' ).
4172  *      p_okay = 'X'.
4173  *    endif.
4174  *  endif.
4175  ** Merkposten:
4176  *  if p_merk = 'X'.
4177  *    if p_item-bstat = 'S'.
4178  *      p_okay = 'X'.
4179  *    endif.
4180  *  endif.
4181  ** parked document:
4182  *  if p_park = 'X'.
4183  *    if ( p_item-bstat = 'V' ) or ( p_item-bstat = 'W' ).
4184  *      p_okay = 'X'.
4185  *    endif.
4186  *  endif.
4187  *
4188  *endform.                               " CHECK_ITEM_OK
4189
4190  *&---------------------------------------------------------------------*
4191  *&      Form  CB_ITEM_ALV_USERCOMM
4192  *&---------------------------------------------------------------------*
4193  *form cb_item_alv_usercomm_new  using  r_ucomm     like sy-ucomm
4194  *                                  rs_selfield type slis_selfield.
4195  *  data: rt_fieldcat type slis_t_fieldcat_alv.
4196  *  data: ld_refresh_count    type i,
4197  *        ld_memory_id(40)    type c.
4198  *  data: ls_variant          type disvariant,
4199  *        lt_selscreen        like rsparams occurs 5 with header line,
4200  *        ld_memory_id2(40)   type c.
4201  *
4202  *  case r_ucomm.
4203  *    when '&OL0' or '&OAD' or '&OLX'.
4204  **.... ALV change display variant. get new field catalog:
4205  *      if x_grid is initial.
4206  *        call function 'REUSE_ALV_LIST_LAYOUT_INFO_GET'
4207  *          IMPORTING
4208  *            et_fieldcat   = rt_fieldcat
4209  *          EXCEPTIONS
4210  *            no_infos      = 1
4211  *            program_error = 2
4212  *            others        = 3.
4213  *      else.
4214  *        call function 'REUSE_ALV_GRID_LAYOUT_INFO_GET'
4215  *          IMPORTING
4216  *            et_fieldcat   = rt_fieldcat
4217  *          EXCEPTIONS
4218  *            no_infos      = 1
4219  *            program_error = 2
4220  *            others        = 3.
4221  *      endif.
4222  *      if sy-subrc <> 0.
4223  *        exit.
4224  *      endif.
4225  **     analyze new fieldcat:
4226  *      perform analyze_act_fieldcat  tables rt_fieldcat.
```

```
4227  **     treat new special fields:
4228  *      perform read_special_tables.
4229  *    when 'REFR' or 'LTOG'.
4230  **.... list refresh:
4231  *      if x_grid is initial.
4232  *        call function 'REUSE_ALV_LIST_LAYOUT_INFO_GET'
4233  *          IMPORTING
4234  *            es_variant    = ls_variant
4235  *          EXCEPTIONS
4236  *            no_infos      = 1
4237  *            program_error = 2
4238  *            others        = 3.
4239  *      else.
4240  *        call function 'REUSE_ALV_GRID_LAYOUT_INFO_GET'
4241  *          IMPORTING
4242  *            es_variant    = ls_variant
4243  *          EXCEPTIONS
4244  *            no_infos      = 1
4245  *            program_error = 2
4246  *            others        = 3.
4247  *      endif.
4248  *      if sy-subrc = 0 and not ls_variant-variant is initial.
4249  *        ld_memory_id2 = g_repid.
4250  *        write '/LAYO' to ld_memory_id2+35(5).
4251  *        condense ld_memory_id2 no-gaps.
4252  *        export layout from ls_variant-variant
4253  *               to memory id ld_memory_id2.
4254  *      endif.
4255  *      ld_memory_id = g_repid.
4256  *      write '/REFR' to ld_memory_id+35(5).
4257  *      condense ld_memory_id no-gaps.
4258  *      import counter to ld_refresh_count
4259  *             from memory id ld_memory_id.
4260  *      add 1 to ld_refresh_count.
4261  *      export counter from ld_refresh_count
4262  *             to memory id ld_memory_id.
4263  *      if ld_refresh_count > 1.
4264  *        leave program.
4265  *      endif.
4266  *      while ld_refresh_count > 0.
4267  *        clear ls_variant.
4268  *        lt_selscreen[] = gt_selscreen[].
4269  *        import layout to ls_variant-variant
4270  *               from memory id ld_memory_id2.
4271  *        if sy-subrc = 0 and not ls_variant-variant is initial.
4272  *          read table lt_selscreen with key selname = 'PA_VARI'.
4273  *          if sy-subrc = 0.
4274  *            lt_selscreen-low = ls_variant-variant.
4275  *            modify lt_selscreen index sy-tabix.
4276  *          endif.
4277  *        endif.
4278  *        submit (g_repid) with selection-table lt_selscreen
4279  *                         with free selections gt_dyn_texpr
4280  *                         with kd_index eq gt_searchpattern
4281  *                         with dd_index eq gt_searchpattern
4282  *                         with sd_index eq gt_searchpattern
4283  *                         and return.
4284  *        clear ld_refresh_count.
4285  *        import counter to ld_refresh_count
4286  *               from memory id ld_memory_id.
4287  *        if ld_refresh_count > 0.
```

```
4288  *          subtract 1 from ld_refresh_count.
4289  *        endif.
4290  *        export counter from ld_refresh_count
4291  *               to memory id ld_memory_id.
4292  *      endwhile.
4293  *      if gd_dynp_val = 0.
4294  *        submit (g_repid) with selection-table gt_selscreen
4295  *                         with free selections gt_dyn_texpr
4296  *                         with kd_index eq gt_searchpattern
4297  *                         with dd_index eq gt_searchpattern
4298  *                         with sd_index eq gt_searchpattern
4299  *                         via selection-screen.
4300  *      else.
4301  *        leave program.
4302  *      endif.
4303  *
4304  *  endcase.
4305  *
4306  *endform.                                " CB_ITEM_ALV_USERCOMM
4307
4308  *&---------------------------------------------------------------------*
4309  *&      Form  CHANGE_STATUS_NEW
4310  *&---------------------------------------------------------------------*
4311  *form change_status_NEW.
4312  *
4313  *  data: begin of lt_exclude occurs 5,
4314  *          fcode like sy-ucomm,
4315  *        end of lt_exclude.
4316  *  data: ld_flag(1) type c.
4317  *  data: ls_rssubinfo like rssubinfo.
4318  *
4319  *
4320  *  call function 'RS_SUBMIT_INFO'
4321  *       importing
4322  *            p_submit_info = ls_rssubinfo
4323  *       exceptions
4324  *            others        = 1.
4325  *
4326  *  if not ls_rssubinfo-mode_norml is initial.
4327  ** check: worklist functionality active?
4328  **  get parameter id 'FI_WORKLISTS_FLAG' field ld_flag.
4329  **  if ld_flag is initial.
4330  **    lt_exclude-fcode = 'FC01'.
4331  **    append lt_exclude.
4332  ***   clear user preference parameter:
4333  **    set parameter id 'FIT_WL' field space.
4334  **  endif.
4335  **  lt_exclude-fcode = 'FC01'.
4336  **  append lt_exclude.
4337  *  get parameter id 'FI_WORKLISTS_FLAG' field ld_flag.
4338  *  if ld_flag is initial.
4339  *    lt_exclude-fcode = 'FC02'.
4340  *    append lt_exclude.
4341  **   clear user preference parameter:
4342  *    set parameter id 'FIT_WL' field space.
4343  *  endif.
4344  **  lt_exclude-fcode = 'FC03'.
4345  **  append lt_exclude.
4346  *   lt_exclude-fcode = 'FC04'.
4347  *   append lt_exclude.
4348  **  lt_exclude-fcode = 'FC05'.
```

```
4349  **  append lt_exclude.
4350  *
4351  ** deactivate the dyn. Selections of Log. DB
4352  *  lt_exclude-fcode = 'DYNS'.
4353  *  append lt_exclude.
4354  *
4355  *  call function 'RS_SET_SELSCREEN_STATUS'
4356  *    EXPORTING
4357  *      p_status  = sy-pfkey
4358  *    TABLES
4359  *      p_exclude = lt_exclude.
4360  *
4361  *  endif.
4362  *
4363  *endform.                                " CHANGE_STATUS_NEW
4364
4365  *---------------------------------------------------------------------*
4366  *   form wl_flag_and_button_new
4367  *   set global flag and button text according to static memory
4368  *---------------------------------------------------------------------*
4369  *form wl_flag_and_button_new.
4370  *
4371  ** icon freie selection
4372  *  perform fs_set_sscrtexts_dynsel changing fs_f1_icon.
4373  *  sscrfields-functxt_01 = fs_f1_icon.
4374  *
4375  ** text worklist button
4376  *  perform wl_memory_set_get using space
4377  *                                  gd_wl_on.
4378  *  if gd_wl_on is initial.
4379  *    sscrfields-functxt_02 = text-w01.
4380  *  else.
4381  *    sscrfields-functxt_02 = text-w04.
4382  *  endif.
4383  *
4384  *  sscrfields-functxt_03 = text-272.
4385  *
4386  *endform.                    "wl_flag_and_button_new
4387  *&---------------------------------------------------------------------*
4388  *&      Form  setlmnt_accounts
4389  *&---------------------------------------------------------------------*
4390  *       text
4391  *---------------------------------------------------------------------*
4392  *  -->  p1        text
4393  *  <--  p2        text
4394  *---------------------------------------------------------------------*
4395  FORM setlmnt_accounts .
4396
4397    DATA: ls_account_slt TYPE gty_acccoco.
4398    DATA: ls_acc_debit   TYPE acct_det_c_c_bf.
4399    DATA: ls_acc_credit  TYPE acct_det_c_c_bf.
4400  *  data: ld_bukrs       type bukrs.
4401
4402    REFRESH gt_account_slt.
4403
4404    DATA: lt_bukrs TYPE bukrs OCCURS 0 WITH HEADER LINE.
4405
4406    SELECT bukrs FROM t001 INTO TABLE lt_bukrs
4407      WHERE bukrs IN sd_bukrs.
4408    LOOP AT lt_bukrs.
4409      CALL FUNCTION 'FI_ACCT_DET_CLEARING'
```

```
4410        EXPORTING
4411          id_company_code = lt_bukrs
4412        IMPORTING
4413          es_acc_debit    = ls_acc_debit
4414          es_acc_credit   = ls_acc_credit.
4415  *   TABLES
4416  *     RETURN                =
4417
4418      IF NOT ls_acc_debit-hkont IS INITIAL.
4419        IF ls_acc_debit-hkont IN sd_saknr.
4420          ls_account_slt-bukrs = lt_bukrs.
4421          ls_account_slt-saknr = ls_acc_debit-hkont.
4422          APPEND ls_account_slt TO gt_account_slt.
4423        ENDIF.
4424      ENDIF.
4425      IF NOT ls_acc_credit-hkont IS INITIAL.
4426        IF ls_acc_credit-hkont IN sd_saknr.
4427          ls_account_slt-bukrs = lt_bukrs.
4428          ls_account_slt-saknr = ls_acc_credit-hkont.
4429          APPEND ls_account_slt TO gt_account_slt.
4430        ENDIF.
4431      ENDIF.
4432    ENDLOOP.
4433  *  endselect.
4434    SORT gt_account_slt.
4435
4436  ENDFORM.                    " setlmnt_accounts
4437  *&---------------------------------------------------------------------*
4438  *&      Form  get_leading_ledger
4439  *&---------------------------------------------------------------------*
4440  *       text
4441  *----------------------------------------------------------------------*
4442  *      <--P_GV_LEADING_LEDGER  text
4443  *----------------------------------------------------------------------*
4444  FORM get_leading_ledger  CHANGING cc_leading_ledger TYPE fagl_rldnr.
4445
4446    CALL FUNCTION 'FAGL_GET_LEADING_LEDGER'
4447      IMPORTING
4448        e_rldnr   = cc_leading_ledger
4449      EXCEPTIONS
4450        not_found = 1.
4451
4452    IF sy-subrc <> 0.
4453  * MESSAGE ID SY-MSGID TYPE SY-MSGTY NUMBER SY-MSGNO
4454  *         WITH SY-MSGV1 SY-MSGV2 SY-MSGV3 SY-MSGV4.
4455    ENDIF.
4456
4457  ENDFORM.                    " get_leading_ledger
4458  *&---------------------------------------------------------------------*
4459  *&      Form  check_reporting_fields
4460  *&---------------------------------------------------------------------*
4461  *       text
4462  *----------------------------------------------------------------------*
4463  *  -->  p1        text
4464  *  <--  p2        text
4465  *----------------------------------------------------------------------*
4466  FORM check_reporting_fields .
4467
4468  *Felder obiger Tabellen
4469    DATA: BEGIN OF fields OCCURS 10.
4470            INCLUDE STRUCTURE rsdsfields.
```

```
4471    DATA: END   OF fields.
4472  *  DATA: BEGIN OF fields_not OCCURS 10.
4473  *          INCLUDE STRUCTURE rsdsfields.
4474  *  DATA: END   OF fields_not.
4475
4476  * new declaration
4477    DATA: ls_tablenames  TYPE fagl_tabnames.
4478  *  DATA: ds_trange  TYPE RSDS_TRANGE.
4479  *  data: ds_trange_tab type RSDS_TRANGE.
4480    DATA: ds_trange_tab_line TYPE rsds_range.
4481  *  Data: ds_Frange_tab type RSDS_FRANGE_T.
4482    DATA: ds_frange_tab_line TYPE rsds_frange.
4483    DATA: ld_tablename TYPE rsds_expr-tablename.
4484    DATA: ds_texpr_tab_line TYPE rsds_expr.
4485    DATA: ds_expr_tab_tab_line LIKE rsdsexpr.
4486    DATA: ld_fieldname TYPE rsdsexpr-fieldname.
4487    DATA: lt_excl_fields LIKE fields OCCURS 0 WITH HEADER LINE.
4488    DATA: ls_texpr_tab_line TYPE rsds_expr.
4489    DATA: ls_expr_tab_line LIKE rsdsexpr.
4490    DATA: ls_num LIKE sy-tfill.
4491    DATA: ls_field LIKE rsdsfields.
4492    DATA: BEGIN OF lt_tab_fld_active OCCURS 0,
4493            tablename LIKE rsdstabs-prim_tab,
4494            fieldname LIKE rsdsexpr-fieldname,
4495          END OF lt_tab_fld_active.
4496    DATA: ld_xdel TYPE c.
4497
4498  * check selected fields against reporting fields of ledger
4499    CONCATENATE gv_si_table '_FS' INTO ld_tablename.
4500
4501    CALL FUNCTION 'FAGL_GET_REPORTING_FIELDS'
4502      EXPORTING
4503        i_rldnr   = gv_relevant_ledger
4504      IMPORTING
4505        et_fields = gt_reporting_fields
4506        et_dfies  = gt_dfies_reporting_fields.
4507
4508    gs_reporting_fields = 'BELNR'.
4509    APPEND gs_reporting_fields TO gt_reporting_fields.
4510    gs_reporting_fields = 'DOCLN'.
4511    APPEND gs_reporting_fields TO gt_reporting_fields.
4512    gs_reporting_fields = 'POPER'.
4513    APPEND gs_reporting_fields TO gt_reporting_fields.
4514    gs_reporting_fields = 'KOKRS'.
4515    APPEND gs_reporting_fields TO gt_reporting_fields.
4516    gs_reporting_fields = 'RMVCT'.
4517    APPEND gs_reporting_fields TO gt_reporting_fields.
4518
4519    LOOP AT fs_dyns-texpr INTO ds_texpr_tab_line
4520      WHERE tablename = ld_tablename.
4521      LOOP AT ds_texpr_tab_line-expr_tab INTO ds_expr_tab_tab_line.
4522        ld_fieldname = ds_expr_tab_tab_line-fieldname.
4523        IF NOT ld_fieldname IS INITIAL.
4524          READ TABLE gt_reporting_fields INTO gs_reporting_fields
4525          WITH KEY ld_fieldname.
4526          IF sy-subrc NE 0.
4527            lt_excl_fields-tablename = ld_tablename.
4528            lt_excl_fields-fieldname = ld_fieldname.
4529            COLLECT lt_excl_fields.
4530          ENDIF.
4531        ENDIF.
```

```
4532      ENDLOOP.
4533    ENDLOOP.
4534    LOOP AT fs_dyns-trange INTO ds_trange_tab_line
4535      WHERE tablename = ld_tablename.
4536      LOOP AT ds_trange_tab_line-frange_t INTO ds_frange_tab_line.
4537        ld_fieldname = ds_frange_tab_line-fieldname.
4538        LOOP AT lt_excl_fields
4539          WHERE fieldname = ld_fieldname.
4540          EXIT.
4541        ENDLOOP.
4542        IF sy-subrc = 0.
4543          DELETE ds_trange_tab_line-frange_t.
4544          ld_xdel = 'X'.
4545        ENDIF.
4546      ENDLOOP.
4547      IF NOT ld_xdel IS INITIAL.
4548        MODIFY fs_dyns-trange FROM ds_trange_tab_line.
4549        deco_message_0 i103.
4550      ENDIF.
4551    ENDLOOP.
4552
4553    IF NOT ld_xdel IS INITIAL.
4554      CALL FUNCTION 'FREE_SELECTIONS_RANGE_2_EX'
4555        EXPORTING
4556          field_ranges = fs_dyns-trange
4557        IMPORTING
4558          expressions  = fs_dyns-texpr.
4559
4560      CALL FUNCTION 'FREE_SELECTIONS_RANGE_2_WHERE'
4561        EXPORTING
4562          field_ranges  = fs_dyns-trange
4563        IMPORTING
4564          where_clauses = fs_dyns-clauses.
4565
4566      LOOP AT fs_dyns-texpr INTO ls_texpr_tab_line.
4567        lt_tab_fld_active-tablename = ls_texpr_tab_line-tablename.
4568        LOOP AT ls_texpr_tab_line-expr_tab INTO ls_expr_tab_line.
4569          IF NOT ls_expr_tab_line-fieldname IS INITIAL.
4570            lt_tab_fld_active-fieldname = ls_expr_tab_line-fieldname.
4571            COLLECT lt_tab_fld_active.
4572          ENDIF.
4573        ENDLOOP.
4574      ENDLOOP.
4575      DESCRIBE TABLE lt_tab_fld_active LINES ls_num.
4576      IF ls_num NE fs_num.
4577        fs_num = ls_num.
4578      ENDIF.
4579    ENDIF.
4580
4581  *           fs_field[] = fields[].
4582
4583  ENDFORM.                    " check_reporting_fields
4584  *&---------------------------------------------------------------------*
4585  *&      Form  map_sel_to_items_mast_dat
4586  *&---------------------------------------------------------------------*
4587  *       text
4588  *----------------------------------------------------------------------*
4589  *  -->  p1        text
4590  *  <--  p2        text
4591  *----------------------------------------------------------------------*
4592  FORM map_sel_to_items_mast_dat USING
```

```
4593                                p_gb_xplacc         TYPE  boolean
4594                                p_gt_selection_part TYPE  gusl_t_selection
4595                                  CHANGING
4596                                p_gt_selection_full TYPE  gusl_t_selection.
4597
4598    DATA: lt_selection   TYPE  gusl_t_selection.
4599    DATA: ls_selection   TYPE  gusl_s_selection.
4600    FIELD-SYMBOLS: <wa1> TYPE any.
4601    DATA: ls_selopt      TYPE rsdsselopt.
4602    DATA: ls_selopt_t    TYPE rsds_selopt_t.
4603    DATA: ls_si_table_fs TYPE tabname.
4604    DATA: ls_bsis_fs     TYPE tabname VALUE 'BSIS_FS'.
4605    DATA: ls_trange_tab_line TYPE rsds_range.
4606    DATA: ls_frange_tab_line TYPE rsds_frange.
4607    DATA: ld_stida       TYPE budat.
4608    DATA: ld_stida_2     TYPE budat.
4609    DATA: lt_augdt   TYPE range_date OCCURS 0 WITH HEADER LINE.
4610    DATA: lt_budat   TYPE range_date OCCURS 0 WITH HEADER LINE.
4611    RANGES: lt_saknr     FOR skb1-saknr.
4612    RANGES: lt_bukrs     FOR skb1-bukrs.
4613    RANGES: lt_rldnr     FOR faglflexa-rldnr.
4614
4615    REFRESH p_gt_selection_full.
4616
4617  * masterdata
4618    lt_saknr-sign = 'I'.
4619    lt_saknr-option = 'EQ'.
4620    lt_saknr-low    = skb1-saknr.
4621    APPEND lt_saknr.
4622    lt_bukrs-sign = 'I'.
4623    lt_bukrs-option = 'EQ'.
4624    lt_bukrs-low    = skb1-bukrs.
4625    APPEND lt_bukrs.
4626
4627    REFRESH ls_selopt_t.
4628    LOOP AT lt_bukrs ASSIGNING <wa1>.
4629      MOVE-CORRESPONDING <wa1> TO ls_selopt.
4630      APPEND ls_selopt TO ls_selopt_t.
4631    ENDLOOP.
4632    ls_selection-fieldname = 'BUKRS'.
4633    ls_selection-t_range  = ls_selopt_t.
4634    APPEND ls_selection TO lt_selection.
4635    REFRESH ls_selopt_t.
4636    LOOP AT lt_bukrs ASSIGNING <wa1>.
4637      MOVE-CORRESPONDING <wa1> TO ls_selopt.
4638      APPEND ls_selopt TO ls_selopt_t.
4639    ENDLOOP.
4640    ls_selection-fieldname = 'RBUKRS'.
4641    ls_selection-t_range  = ls_selopt_t.
4642    APPEND ls_selection TO lt_selection.
4643
4644    REFRESH ls_selopt_t.
4645    LOOP AT lt_saknr ASSIGNING <wa1>.
4646      MOVE-CORRESPONDING <wa1> TO ls_selopt.
4647      APPEND ls_selopt TO ls_selopt_t.
4648    ENDLOOP.
4649    ls_selection-fieldname = 'HKONT'.
4650    ls_selection-t_range  = ls_selopt_t.
4651    APPEND ls_selection TO lt_selection.
4652    REFRESH ls_selopt_t.
4653    LOOP AT lt_saknr ASSIGNING <wa1>.
```

```
4654      MOVE-CORRESPONDING <wa1> TO ls_selopt.
4655      APPEND ls_selopt TO ls_selopt_t.
4656    ENDLOOP.
4657    ls_selection-fieldname = 'RACCT'.
4658    ls_selection-t_range  = ls_selopt_t.
4659    APPEND ls_selection TO lt_selection.
4660
4661    LOOP AT p_gt_selection_part INTO ls_selection.
4662      APPEND ls_selection TO lt_selection.
4663    ENDLOOP.
4664
4665  * Begin of note 2697319:
4666  * Select only ACDOCA line items with MIG_SOURCE SPACE, G and F
4667  * and 'E' (Balance Enrichment) "n3394830
4668  * so that only 'normal' line items (not e.g. MIG_SOURCE A and R)
4669  * are displayed
4670  * If also MIG_SOURCE A and R records are displayed, there is the
4671  * danger that e.g. the drilldown from FAGLB03 from 'Debit'
4672  * or 'Credit' doesn't display the correct line items
4673    DATA: ld_param_name  LIKE fagl_settings-param_name,
4674          ld_param_value LIKE fagl_settings-param_value.
4675    ld_param_name = 'FAGLL03_RESTRICT_MIG'.
4676    CALL FUNCTION 'FAGL_GET_FAGL_SETTINGS'
4677      EXPORTING
4678        id_param_name = ld_param_name
4679      IMPORTING
4680        ed_value      = ld_param_value.
4681    IF NOT ld_param_value IS INITIAL.
4682      CLEAR ls_selopt.
4683      REFRESH ls_selopt_t.
4684      ls_selopt-sign = 'I'.
4685      ls_selopt-option = 'EQ'.
4686      ls_selopt-low = ' '.
4687      APPEND ls_selopt TO ls_selopt_t.
4688      ls_selopt-low = 'F'.
4689      APPEND ls_selopt TO ls_selopt_t.
4690      ls_selopt-low = 'G'.
4691      APPEND ls_selopt TO ls_selopt_t.
4692      ls_selopt-low = 'E'.                "n3394830
4693      APPEND ls_selopt TO ls_selopt_t.    "n3394830
4694      ls_selection-fieldname = 'MIG_SOURCE'.
4695      ls_selection-t_range  = ls_selopt_t.
4696      APPEND ls_selection TO lt_selection.
4697    ENDIF.
4698  * End of note 2697319
4699
4700    p_gt_selection_full[] = lt_selection[].
4701
4702  ENDFORM.                    " map_sel_to_items_mast_dat
4703  *&---------------------------------------------------------------------*
4704  *&      Form  select_items
4705  *&---------------------------------------------------------------------*
4706  *       text
4707  *----------------------------------------------------------------------*
4708  *  -->  p1        text
4709  *  <--  p2        text
4710  *----------------------------------------------------------------------*
4711  FORM select_items .
4712    CHECK x_stop IS INITIAL.
4713
4714    EXPORT SKB1-XOPVW TO MEMORY ID 'REAL_OI_FAGLL03'.            "3571534
```

```
4715    " External Auditor: Time Restrictions for FAGLB03 and FAGLL03 - SAP No
      te 2950454
4716    cl_fagl_time_restriction_mgr=>restrict_reporting_time_frame( "
         2950454
4717      CHANGING ct_selection = gt_selection_full ).            "
         2950454
4718
4719    DATA(lo_settings) = cl_fagl_item_select_settings=>get_instance( ).
4720
4721    lo_settings->set_view_kind( x_erfs ).
4722    lo_settings->set_select_options( gt_selection_full ).
4723    lo_settings->set_key_date( p_keydate ).
4724    lo_settings->set_data_temperature( lv_data_temperature ).
4725    lo_settings->set_fiscal_year_period_range( so_yrper[] ).
4726    lo_settings->set_maximum_number_of_items( COND #( WHEN gd_max_lines =
      0 THEN 999999999 ELSE gd_max_lines ) ).
4727    lo_settings->set_what_should_be_selected( VALUE #( all_items     = x_a
      isel
4728                                                       open_items    = x_o
      psel
4729                                                       cleared_items = x_c
      lsel
4730                                                       normal_items  = x_n
      orm
4731                                                       noted_items   = x_m
      erk
4732                                                       parked_items  = x_p
      ark
4733                                                       closing_items = x_g
      lyec
4734                                                       from_database = gd_
      usedb
4735                                                       from_archive  = gd_
      usear ) ).
4736    IF gd_usear = abap_true.
4737      data(lt_archive_files) = GT_FILES[].
4738      lo_settings->set_archive_files( lt_archive_files ).
4739    ENDIF.
4740
4741    PERFORM set_dark_select_options USING lo_settings.
4742
4743    DATA(lo_item_selector) = cl_fagl_item_selector_factory=>get_instance(
      )->get_item_selector( lo_settings ).
4744    DATA(lo_converter) = cl_fagl_item_result_converter=>get_instance( ).
4745    DATA(lo_result) = lo_item_selector->get_items( ).
4746
4747
4748    ybkpf = lo_result->get_header_bkpf( ).
4749    ybseg = lo_result->get_items_bseg( ).
4750
4751    PERFORM import_arch_from_itab.
4752
4753    lo_converter->convert( EXPORTING it_line_items = lo_result->get_items_
      faglpose( ) io_settings = lo_settings
4754                             CHANGING it_pos = it_pos[] ).
4755
4756    gd_count_noauth = lo_converter->get_num_of_unauthorized_items( ).
4757    gd_count_arch   = lo_converter->get_number_of_archived_items( ).
4758    gd_max_lines    = lo_settings->get_maximum_number_of_items( ).
4759
4760    cl_fagl_item_select_settings=>clear_instance( ).
```

```
4761
4762    IF gd_max_lines = 0.
4763      x_stop = 'X'.
4764    ENDIF.
4765
4766
4767    ##TODO
4768    "Will be deleted after refactoring is complete
4769    GET PARAMETER ID 'FAGLL03_TEST' FIELD DATA(lv_test_mode).
4770    CHECK lv_test_mode = abap_true.
4771
4772    "Remember what has been selected by the refactored code
4773    DATA(lt_pos) = it_pos[].
4774    DATA(lt_ybkpf) = ybkpf.
4775    DATA(lt_ybseg) = ybseg.
4776
4777    CLEAR: it_pos[], ybkpf, ybseg.
4778
4779  *  This is the old code. It will set it_pos, ybkpf and ybseg
4780    PERFORM select_items_old.
4781
4782    SET PARAMETER ID 'FAGLL03_TEST' FIELD abap_false.
4783
4784    SORT: it_pos[] BY belnr buzei, lt_pos BY belnr buzei.
4785    SORT: lt_ybkpf, lt_ybseg, ybkpf, ybseg.
4786
4787  ENDFORM.                    " select_items
4788
4789  FORM set_dark_select_options USING io_settings TYPE REF TO if_fagl_item_
      select_settings.
4790    io_settings->add_dark_select_option( iv_name = 'so_bwwrt'  it_select_o
      ption = so_bwwrt[]
4791              )->add_dark_select_option( iv_name = 'so_bwwr2'  it_select_o
      ption = so_bwwr2[]
4792              )->add_dark_select_option( iv_name = 'so_bwwr3'  it_select_o
      ption = so_bwwr3[]
4793              )->add_dark_select_option( iv_name = 'so_dmshb'  it_select_o
      ption = so_dmshb[]
4794              )->add_dark_select_option( iv_name = 'so_hwaer'  it_select_o
      ption = so_hwaer[]
4795              )->add_dark_select_option( iv_name = 'so_dmbe2'  it_select_o
      ption = so_dmbe2[]
4796              )->add_dark_select_option( iv_name = 'so_hwae2'  it_select_o
      ption = so_hwae2[]
4797              )->add_dark_select_option( iv_name = 'so_dmbe3'  it_select_o
      ption = so_dmbe3[]
4798              )->add_dark_select_option( iv_name = 'so_hwae3'  it_select_o
      ption = so_hwae3[]
4799              )->add_dark_select_option( iv_name = 'so_faedt'  it_select_o
      ption = so_faedt[]
4800              )->add_dark_select_option( iv_name = 'so_jamon'  it_select_o
      ption = so_jamon[]
4801              )->add_dark_select_option( iv_name = 'so_koart'  it_select_o
      ption = so_koart[]
4802              )->add_dark_select_option( iv_name = 'so_qbshb'  it_select_o
      ption = so_qbshb[]
4803              )->add_dark_select_option( iv_name = 'so_qsfbt'  it_select_o
      ption = so_qsfbt[]
4804              )->add_dark_select_option( iv_name = 'so_qsshb'  it_select_o
      ption = so_qsshb[]
4805              )->add_dark_select_option( iv_name = 'so_skfbt'  it_select_o
```

```
      ption = so_skfbt[]
4806              )->add_dark_select_option( iv_name = 'so_wskto'  it_select_o
      ption = so_wskto[]
4807              )->add_dark_select_option( iv_name = 'so_sknto'  it_select_o
      ption = so_sknto[]
4808              )->add_dark_select_option( iv_name = 'so_verz1'  it_select_o
      ption = so_verz1[]
4809              )->add_dark_select_option( iv_name = 'so_verzn'  it_select_o
      ption = so_verzn[]
4810              )->add_dark_select_option( iv_name = 'so_waers'  it_select_o
      ption = so_waers[]
4811              )->add_dark_select_option( iv_name = 'so_zaldt'  it_select_o
      ption = so_zaldt[]
4812              )->add_dark_select_option( iv_name = 'so_zinsz'  it_select_o
      ption = so_zinsz[]
4813              )->add_dark_select_option( iv_name = 'so_askto'  it_select_o
      ption = so_askto[]
4814              )->add_dark_select_option( iv_name = 'so_kurse'  it_select_o
      ption = so_kurse[]
4815              )->add_dark_select_option( iv_name = 'so_kkbwr'  it_select_o
      ption = so_kkbwr[]
4816              )->add_dark_select_option( iv_name = 'so_gsber'  it_select_o
      ption = so_gsber[]
4817              )->add_dark_select_option( iv_name = 'so_shkzg'  it_select_o
      ption = so_shkzg[] ).
4818  ENDFORM.
4819
4820  FORM select_items_old.
4821    TYPES: BEGIN OF ts_what_has_been_called,
4822             get_items_bsis     TYPE abap_bool,
4823             get_items_vbsegs   TYPE abap_bool,
4824             get_items_bsas     TYPE abap_bool,
4825             get_items_bseg     TYPE abap_bool,
4826             get_arch_item_bsis TYPE abap_bool,
4827             get_arch_item_bseg TYPE abap_bool,
4828             get_items_faglbsis TYPE abap_bool,
4829             get_items_faglbsas TYPE abap_bool,
4830             get_arch_faglbsas  TYPE abap_bool,
4831           END OF ts_what_has_been_called.
4832    DATA: ld_read_bseg.
4833    DATA: ld_lines LIKE sy-tabix.
4834    DATA: lt_files TYPE  as_t_rng_archiv.
4835
4836    DATA:
4837      ls_function_called TYPE ts_what_has_been_called.
4838
4839    STATICS lv_reset_init TYPE gusl_bool. "correction archive and non arch
      ive
4840
4841    GET PARAMETER ID 'FAGLL03_TEST' FIELD DATA(lv_test_mode).
4842    IF lv_test_mode = abap_true.
4843      lv_reset_init = abap_true.
4844    ENDIF.
4845
4846    CHECK x_stop IS INITIAL.
4847
4848    IF NOT x_opsel IS INITIAL
4849    OR NOT x_aisel IS INITIAL.
4850      IF ( NOT skb1-xopvw IS INITIAL
4851      OR ( NOT skb1-xkres IS INITIAL
4852           AND NOT x_erfs IS INITIAL )
```

```
4853      OR ( NOT skb1-xkres IS INITIAL
4854           AND x_erfs IS INITIAL
4855           AND ( skb1-mitkz EQ 'D' OR skb1-mitkz EQ 'K' ) ) ).
4856        IF gv_special_account IS INITIAL.                    "1004374
4857          IF NOT x_merk IS INITIAL
4858          OR NOT x_norm IS INITIAL.
4859            IF NOT gd_usedb IS INITIAL.
4860  * recon. account + line item display -> read additional data
4861              IF ( skb1-mitkz EQ 'D' OR skb1-mitkz EQ 'K' )
4862              AND skb1-xkres NE space AND x_erfs IS INITIAL.  "1699013
4863                ld_read_bseg = 'X'.
4864              ENDIF.
4865
4866              gd_cursor_open = 'X'.                          "987950
4867              WHILE gd_cursor_open = 'X' AND x_stop IS INITIAL. "987950
4868  *             check x_stop is initial.                      "987950
4869              REFRESH it_sel.
4870              IF gd_cursor_open = 'X'.
4871                ls_function_called-get_items_bsis =  abap_true.
4872                CALL FUNCTION 'FAGL_GET_ITEMS_BSIS'
4873                  EXPORTING
4874                    iv_acdoc_compatibility_mode = abap_false
4875                    iv_data_temperature         = lv_data_temperature
      "FAGLL03 4./5. HW
4876                    i_selection                 = gt_selection_full
4877  *                 I_FIELDLIST                 =
4878                    i_blocksize                 = gd_max_pack_lines
      "987950
4879                    i_block_data                = gusl_c_true
      "987950
4880  *                 I_AGGREGATION               = GUSL_C_FALSE
4881                    i_read_bseg                 = ld_read_bseg
4882                    i_max_lines                 = gd_max_lines
4883                    i_reset_init                = lv_reset_init  "correc
    tion archive and non archive
4884                  IMPORTING                                    "987950
4885                    flag_cursor_open            = gd_cursor_open
      "987950
4886                  CHANGING
4887                    c_t_faglpose                = it_sel.
4888
4889                CLEAR lv_reset_init. "correction archive and non archive
4890              ENDIF.
4891
4892              CLEAR gd_xarch.
4893              LOOP AT it_sel INTO wa_sel.
4894  * relevant basically if report is called from balance display.
4895                IF NOT x_merk IS INITIAL
4896                OR NOT x_erfs IS INITIAL.
4897                  gp_yrper(4) = wa_sel-gjahr.
4898                  gp_yrper+5  = wa_sel-monat.
4899                ELSE.
4900                  gp_yrper(4) = wa_sel-ryear.
4901                  gp_yrper+4 = wa_sel-poper.
4902                ENDIF.
4903                CHECK gp_yrper IN so_yrper.
4904                IF  NOT gd_usear IS INITIAL
4905                AND NOT x_erfs IS INITIAL
4906                AND NOT wa_sel-xarch IS INITIAL.
4907                ELSE.
4908  * writing items into it_pos.
```

```
4909                   PERFORM pos_table_fill  CHANGING  x_stop.
4910                 ENDIF.
4911                 IF NOT x_stop IS INITIAL.
4912                   CLEAR gd_cursor_open.                    "987950
4913                   EXIT.
4914                 ENDIF.
4915               ENDLOOP.
4916               IF NOT gd_max_lines IS INITIAL.
4917                 DESCRIBE TABLE it_sel LINES ld_lines.
4918                 gd_max_lines = gd_max_lines - ld_lines.
4919               ENDIF.
4920             ENDWHILE.                                      "987950
4921             FREE MEMORY ID 'FAGLL03_REMAIN_SEL'.           "1105201
4922             FREE MEMORY ID 'FAGLL03_REMAIN_SEL_BSEG'.
4923
4924  * tmp skip select FAGL_GET_ITEMS_BSAS for option all items (x_aisel) as
      all items are selected with FAGL_GET_ITEMS_BSIS
4925             IF NOT ( x_aisel = abap_true
4926             AND ( cl_fins_acdoc_util=>is_simulation_ledger( iv_rldnr = g
      v_relevant_ledger ) = abap_true
4927             OR cl_fins_acdoc_util=>is_prediction_ledger( iv_rldnr = gv_r
      elevant_ledger ) = abap_true ) ).
4928
4929               gd_cursor_open = 'X'.                        "987950
4930               WHILE gd_cursor_open = 'X' AND x_stop IS INITIAL. "987950
4931  *            check x_stop is initial.                      "987950
4932                 REFRESH it_sel.
4933                 ls_function_called-get_items_bsas = abap_true.
4934                 CALL FUNCTION 'FAGL_GET_ITEMS_BSAS'
4935                   EXPORTING
4936                     iv_acdoc_compatibility_mode = abap_false
4937                     iv_data_temperature         = lv_data_temperature
       "FAGLL03 4./5. HW
4938                     i_selection                 = gt_selection_full
4939                     i_blocksize                 = gd_max_pack_lines
       "987950
4940                     i_block_data                = gusl_c_true
       "987950
4941                     i_read_bseg                 = ld_read_bseg
4942                     i_max_lines                 = gd_max_lines
4943                     i_reset_init                = lv_reset_init  "correc
      tion archive and non archive
4944                   IMPORTING                                  "987950
4945                     flag_cursor_open            = gd_cursor_open
       "987950
4946                   CHANGING
4947                     c_t_faglpose                = it_sel.
4948
4949                 CLEAR lv_reset_init.  "correction archive and non archiv
      e
4950
4951                 CLEAR gd_xarch.
4952                 LOOP AT it_sel INTO wa_sel.
4953  * relevant basically if report is called from balance display.
4954                   IF NOT x_merk IS INITIAL
4955                   OR NOT x_erfs IS INITIAL.
4956                     gp_yrper(4) = wa_sel-gjahr.
4957                     gp_yrper+5  = wa_sel-monat.
4958                   ELSE.
4959                     gp_yrper(4) = wa_sel-ryear.
4960                     gp_yrper+4 = wa_sel-poper.
```

```
4961                    ENDIF.
4962                    CHECK gp_yrper IN so_yrper.
4963    * writing items into it_pos.
4964                    IF  NOT gd_usear IS INITIAL
4965                    AND NOT x_erfs IS INITIAL
4966                    AND NOT wa_sel-xarch IS INITIAL.
4967                    ELSE.
4968                      PERFORM pos_table_fill  CHANGING  x_stop.
4969                    ENDIF.
4970                    IF NOT x_stop IS INITIAL.
4971                      CLEAR gd_cursor_open.                    "987950
4972                      EXIT.
4973                    ENDIF.
4974                  ENDLOOP.
4975                  IF NOT gd_max_lines IS INITIAL.
4976                    DESCRIBE TABLE it_sel LINES ld_lines.
4977                    gd_max_lines = gd_max_lines - ld_lines.
4978                  ENDIF.
4979                ENDWHILE.                                      "987950
4980                FREE MEMORY ID 'FAGLL03_REMAIN_SEL'.           "1105201
4981              ENDIF.
4982            ENDIF.
4983            EXPORT what_has_been_called = ls_function_called TO MEMORY ID
      'faglut' COMPRESSION OFF.
4984            IF NOT gd_usear IS INITIAL.
4985              CHECK x_stop IS INITIAL.
4986              REFRESH it_sel.
4987              lt_files[] = gt_files[].
4988              ls_function_called-get_arch_item_bsis = abap_true.
4989              CALL FUNCTION 'FAGL_GET_ARCH_ITEMS_BSIS'
4990                EXPORTING
4991                  i_selection  = gt_selection_full
4992                  i_read_bseg  = ld_read_bseg
4993                  i_arch_sel   = lt_files
4994                  i_max_lines  = gd_max_lines
4995                CHANGING
4996                  c_t_faglpose = it_sel
4997                  c_t_bkpf     = ybkpf
4998                  c_t_bseg     = ybseg.
4999
5000              lv_reset_init = abap_true. "correction archive and non archi
      ve
5001              gd_xarch = 'X'.
5002              LOOP AT it_sel INTO wa_sel.
5003                wa_sel-xarch = 'X'.
5004    * relevant basically if report is called from balance display.
5005                IF NOT x_merk IS INITIAL
5006                OR NOT x_erfs IS INITIAL.
5007                  gp_yrper(4) = wa_sel-gjahr.
5008                  gp_yrper+5  = wa_sel-monat.
5009                ELSE.
5010                  gp_yrper(4) = wa_sel-ryear.
5011                  gp_yrper+4 = wa_sel-poper.
5012                ENDIF.
5013                CHECK gp_yrper IN so_yrper.
5014    * writing items into it_pos.
5015                PERFORM pos_table_fill  CHANGING  x_stop.
5016                IF NOT x_stop IS INITIAL.
5017                  EXIT.
5018                ENDIF.
5019              ENDLOOP.
```

```
5020              PERFORM import_arch_from_itab.
5021              IF NOT gd_max_lines IS INITIAL.
5022                DESCRIBE TABLE it_sel LINES ld_lines.
5023                gd_max_lines = gd_max_lines - ld_lines.
5024              ENDIF.
5025              FREE MEMORY ID 'FAGLL03_REMAIN_SEL'.            "1105201
5026            ENDIF.
5027          ENDIF.
5028        ENDIF.                                                "1004374
5029      ENDIF.
5030
5031      IF ( skb1-xopvw IS INITIAL
5032      AND ( ( NOT skb1-xkres IS INITIAL
5033            AND skb1-mitkz NE 'D'
5034            AND skb1-mitkz NE 'K')
5035            OR skb1-xkres IS INITIAL )
5036      AND skb1-xlgclr IS INITIAL
5037      AND NOT x_norm IS INITIAL
5038      AND x_merk IS INITIAL
5039      AND x_park IS INITIAL
5040      AND x_erfs IS INITIAL )
5041      OR NOT gv_special_account IS INITIAL.                   "1004374
5042        IF NOT gd_usedb IS INITIAL.
5043          gd_cursor_open = 'X'.                               "987950
5044          WHILE gd_cursor_open = 'X' AND x_stop IS INITIAL.   "987950
5045  *         check x_stop is initial.                            "987950
5046            REFRESH it_sel.
5047            ls_function_called-get_items_bseg = abap_true.
5048            CALL FUNCTION 'FAGL_GET_ITEMS_BSEG'
5049              EXPORTING
5050                iv_acdoc_compatibility_mode = abap_false
5051                iv_data_temperature         = lv_data_temperature
5052                i_selection                 = gt_selection_full
5053  *             I_FIELDLIST                 =
5054                i_blocksize                 = gd_max_pack_lines         "
      987950
5055                i_block_data                = gusl_c_true               "
      987950
5056  *             I_AGGREGATION               = GUSL_C_FALSE
5057                i_max_lines                 = gd_max_lines
5058                i_reset_init                = lv_reset_init  "correction a
      rchive and non archive
5059              IMPORTING                                         "987950
5060                flag_cursor_open            = gd_cursor_open            "
      987950
5061              CHANGING
5062                c_t_faglpose                = it_sel.
5063
5064            CLEAR lv_reset_init.  "correction archive and non archive
5065
5066            CLEAR gd_xarch.
5067            LOOP AT it_sel INTO wa_sel.
5068  * relevant basically if report is called from balance display.
5069              IF NOT x_merk IS INITIAL
5070              OR NOT x_erfs IS INITIAL.
5071                gp_yrper(4) = wa_sel-gjahr.
5072                gp_yrper+5  = wa_sel-monat.
5073              ELSE.
5074                gp_yrper(4) = wa_sel-ryear.
5075                gp_yrper+4 = wa_sel-poper.
5076              ENDIF.
```

```
5077              CHECK gp_yrper IN so_yrper.
5078  * writing items into it_pos.
5079              PERFORM pos_table_fill  CHANGING  x_stop.
5080              IF NOT x_stop IS INITIAL.
5081                CLEAR gd_cursor_open.                          "987950
5082                EXIT.
5083              ENDIF.
5084            ENDLOOP.
5085            IF NOT gd_max_lines IS INITIAL.
5086              DESCRIBE TABLE it_sel LINES ld_lines.
5087              gd_max_lines = gd_max_lines - ld_lines.
5088            ENDIF.
5089          ENDWHILE.                                            "987950
5090          FREE MEMORY ID 'FAGLL03_REMAIN_SEL_BSEG'.
5091        ENDIF.
5092        EXPORT what_has_been_called = ls_function_called TO MEMORY ID 'fag
      lut' COMPRESSION OFF.
5093        IF NOT gd_usear IS INITIAL.
5094          CHECK x_stop IS INITIAL.
5095          REFRESH it_sel.
5096          lt_files[] = gt_files[].
5097          ls_function_called-get_arch_item_bseg = abap_true.
5098          CALL FUNCTION 'FAGL_GET_ARCH_ITEMS_BSEG'
5099            EXPORTING
5100              i_selection  = gt_selection_full
5101              i_arch_sel   = lt_files
5102              i_max_lines  = gd_max_lines
5103            CHANGING
5104              c_t_faglpose = it_sel
5105              c_t_bkpf     = ybkpf
5106              c_t_bseg     = ybseg.
5107
5108          lv_reset_init = abap_true. "correction archive and non archive
5109          gd_xarch = 'X'.
5110          LOOP AT it_sel INTO wa_sel.
5111            wa_sel-xarch = 'X'.
5112  * relevant basically if report is called from balance display.
5113            IF NOT x_merk IS INITIAL
5114            OR NOT x_erfs IS INITIAL.
5115              gp_yrper(4) = wa_sel-gjahr.
5116              gp_yrper+5  = wa_sel-monat.
5117            ELSE.
5118              gp_yrper(4) = wa_sel-ryear.
5119              gp_yrper+4 = wa_sel-poper.
5120            ENDIF.
5121            CHECK gp_yrper IN so_yrper.
5122  * writing items into it_pos.
5123            PERFORM pos_table_fill  CHANGING  x_stop.
5124            IF NOT x_stop IS INITIAL.
5125              EXIT.
5126            ENDIF.
5127          ENDLOOP.
5128          PERFORM import_arch_from_itab.
5129          IF NOT gd_max_lines IS INITIAL.
5130            DESCRIBE TABLE it_sel LINES ld_lines.
5131            gd_max_lines = gd_max_lines - ld_lines.
5132          ENDIF.
5133          FREE MEMORY ID 'FAGLL03_REMAIN_SEL'.                 "1105201
5134        ENDIF.
5135      ENDIF.
5136
```

```
5137  * read items from faglbsis if ledger group specific clearing
5138      IF skb1-xlgclr = 'X'.
5139        IF gv_special_account IS INITIAL.                    "1004374
5140  *        IF NOT x_merk IS INITIAL
5141  *        OR NOT x_norm IS INITIAL.
5142          IF NOT x_norm IS INITIAL.
5143            IF NOT gd_usedb IS INITIAL.
5144              gd_cursor_open = 'X'.                          "987950
5145              WHILE gd_cursor_open = 'X' AND x_stop IS INITIAL. "987950
5146  *             check x_stop is initial.                      "987950
5147                REFRESH it_sel.
5148                ls_function_called-get_items_faglbsis = abap_true.
5149                CALL FUNCTION 'FAGL_GET_ITEMS_FAGLBSIS'
5150                  EXPORTING
5151                    iv_acdoc_compatibility_mode = abap_false
5152                    iv_data_temperature         = lv_data_temperature    "
      FAGLL03 4./5. HW
5153                    i_selection                 = gt_selection_full
5154                    i_blocksize                 = gd_max_pack_lines      "
      987950
5155                    i_block_data                = gusl_c_true            "
      987950
5156                    i_read_bseg                 = ld_read_bseg
5157                    i_max_lines                 = gd_max_lines
5158                    i_reset_init                = lv_reset_init  "correcti
      on archive and non archive
5159                  IMPORTING                                  "987950
5160                    flag_cursor_open            = gd_cursor_open         "
      987950
5161                  CHANGING
5162                    c_t_faglpose                = it_sel.
5163
5164                CLEAR lv_reset_init.  "correction archive and non archive
5165
5166                CLEAR gd_xarch.
5167                LOOP AT it_sel INTO wa_sel.
5168  * relevant basically if report is called from balance display.
5169                  IF NOT x_merk IS INITIAL
5170                  OR NOT x_erfs IS INITIAL.
5171                    gp_yrper(4) = wa_sel-gjahr.
5172                    gp_yrper+5  = wa_sel-monat.
5173                  ELSE.
5174                    gp_yrper(4) = wa_sel-ryear.
5175                    gp_yrper+4 = wa_sel-poper.
5176                  ENDIF.
5177                  CHECK gp_yrper IN so_yrper.
5178                  IF  NOT gd_usear IS INITIAL
5179                  AND NOT x_erfs IS INITIAL
5180                  AND NOT wa_sel-xarch IS INITIAL.
5181                  ELSE.
5182  * writing items into it_pos.
5183                    PERFORM pos_table_fill  CHANGING  x_stop.
5184                  ENDIF.
5185                  IF NOT x_stop IS INITIAL.
5186                    CLEAR gd_cursor_open.                    "987950
5187                    EXIT.
5188                  ENDIF.
5189                ENDLOOP.
5190                IF NOT gd_max_lines IS INITIAL.
5191                  DESCRIBE TABLE it_sel LINES ld_lines.
5192                  gd_max_lines = gd_max_lines - ld_lines.
```

```
5193                ENDIF.
5194              ENDWHILE.                                          "987950
5195              FREE MEMORY ID 'FAGLL03_REMAIN_SEL'.               "1105201
5196
5197  * Note 2914454
5198  * tmp skip select FAGL_GET_ITEMS_BSAS for option all items (x_aisel) as
      all items are selected with FAGL_GET_ITEMS_BSIS
5199              IF NOT ( x_aisel = abap_true
5200              AND ( cl_fins_acdoc_util=>is_simulation_ledger( iv_rldnr = g
      v_relevant_ledger ) = abap_true
5201              OR cl_fins_acdoc_util=>is_prediction_ledger( iv_rldnr = gv_r
      elevant_ledger ) = abap_true ) ).
5202
5203                gd_cursor_open = 'X'.                            "987950
5204                WHILE gd_cursor_open = 'X' AND x_stop IS INITIAL. "987950
5205  *             check x_stop is initial.                          "987950
5206                  REFRESH it_sel.
5207                  ls_function_called-get_items_faglbsas = abap_true.
5208                  CALL FUNCTION 'FAGL_GET_ITEMS_FAGLBSAS'
5209                    EXPORTING
5210                      iv_acdoc_compatibility_mode = abap_false
       "FAGLL03 4./5. HW
5211                      iv_data_temperature         = lv_data_temperature
       "FAGLL03 4./5. HW
5212                      i_selection                 = gt_selection_full
5213  *                   I_FIELDLIST                 =
5214                      i_blocksize                 = gd_max_pack_lines
       "987950
5215                      i_block_data                = gusl_c_true
       "987950
5216  *                   I_AGGREGATION               = GUSL_C_FALSE
5217                      i_read_bseg                 = ld_read_bseg
5218                      i_max_lines                 = gd_max_lines
5219                      i_reset_init                = lv_reset_init  "correc
      tion archive and non archive
5220                    IMPORTING                                      "987950
5221                      flag_cursor_open            = gd_cursor_open
       "987950
5222                    CHANGING
5223                      c_t_faglpose                = it_sel.
5224
5225                  CLEAR lv_reset_init.  "correction archive and non archiv
      e
5226
5227                  CLEAR gd_xarch.
5228                  LOOP AT it_sel INTO wa_sel.
5229  * relevant basically if report is called from balance display.
5230                    IF NOT x_merk IS INITIAL
5231                    OR NOT x_erfs IS INITIAL.
5232                      gp_yrper(4) = wa_sel-gjahr.
5233                      gp_yrper+5  = wa_sel-monat.
5234                    ELSE.
5235                      gp_yrper(4) = wa_sel-ryear.
5236                      gp_yrper+4 = wa_sel-poper.
5237                    ENDIF.
5238                    CHECK gp_yrper IN so_yrper.
5239  * writing items into it_pos.
5240                    IF  NOT gd_usear IS INITIAL
5241                    AND NOT x_erfs IS INITIAL
5242                    AND NOT wa_sel-xarch IS INITIAL.
5243                    ELSE.
```

```
5244                    PERFORM pos_table_fill  CHANGING  x_stop.
5245                  ENDIF.
5246                  IF NOT x_stop IS INITIAL.
5247                    CLEAR gd_cursor_open.                   "987950
5248                    EXIT.
5249                  ENDIF.
5250                ENDLOOP.
5251                IF NOT gd_max_lines IS INITIAL.
5252                  DESCRIBE TABLE it_sel LINES ld_lines.
5253                  gd_max_lines = gd_max_lines - ld_lines.
5254                ENDIF.
5255              ENDWHILE.                                     "987950
5256              FREE MEMORY ID 'FAGLL03_REMAIN_SEL'.          "1105201
5257            ENDIF.
5258          ENDIF.
5259          EXPORT what_has_been_called = ls_function_called TO MEMORY ID
      'faglut' COMPRESSION OFF.
5260    ** If items are archieved
5261          IF NOT gd_usear IS INITIAL.
5262            CHECK x_stop IS INITIAL.
5263            REFRESH it_sel.
5264            lt_files[] = gt_files[].
5265            ls_function_called-get_arch_faglbsas = abap_true.
5266            CALL FUNCTION 'FAGL_GET_ARCH_ITEMS_FAGLBSAS'
5267              EXPORTING
5268                i_selection  = gt_selection_full
5269    *           I_FIELDLIST  =
5270    *           I_BLOCKSIZE  =
5271    *           I_BLOCK_DATA = GUSL_C_FALSE
5272    *           I_AGGREGATION          = GUSL_C_FALSE
5273                i_read_bseg  = ld_read_bseg
5274                i_arch_sel   = lt_files
5275                i_max_lines  = gd_max_lines
5276    *         IMPORTING
5277    *           FLAG_CURSOR_OPEN       =
5278              CHANGING
5279                c_t_faglpose = it_sel
5280                c_t_bkpf     = ybkpf
5281                c_t_bseg     = ybseg.
5282
5283            lv_reset_init = abap_true. "correction archive and non archi
      ve
5284            gd_xarch = 'X'.
5285            LOOP AT it_sel INTO wa_sel.
5286              wa_sel-xarch = 'X'.
5287    * relevant basically if report is called from balance display.
5288              IF NOT x_merk IS INITIAL
5289              OR NOT x_erfs IS INITIAL.
5290                gp_yrper(4) = wa_sel-gjahr.
5291                gp_yrper+5  = wa_sel-monat.
5292              ELSE.
5293                gp_yrper(4) = wa_sel-ryear.
5294                gp_yrper+4 = wa_sel-poper.
5295              ENDIF.
5296              CHECK gp_yrper IN so_yrper.
5297    * writing items into it_pos.
5298              PERFORM pos_table_fill  CHANGING  x_stop.
5299              IF NOT x_stop IS INITIAL.
5300                EXIT.
5301              ENDIF.
5302            ENDLOOP.
```

```
5303              PERFORM import_arch_from_itab.
5304              IF NOT gd_max_lines IS INITIAL.
5305                DESCRIBE TABLE it_sel LINES ld_lines.
5306                gd_max_lines = gd_max_lines - ld_lines.
5307              ENDIF.
5308              FREE MEMORY ID 'FAGLL03_REMAIN_SEL'.           "1105201
5309            ENDIF.
5310          ENDIF.
5311        ENDIF.
5312      ENDIF.
5313      EXPORT what_has_been_called = ls_function_called TO MEMORY ID 'faglu
      t' COMPRESSION OFF.
5314      IF NOT x_park IS INITIAL.
5315        IF NOT gd_usedb IS INITIAL.
5316          CHECK x_stop IS INITIAL.
5317          REFRESH it_sel.
5318          ls_function_called-get_items_vbsegs = abap_true.
5319          CALL FUNCTION 'FAGL_GET_ITEMS_VBSEGS'
5320            EXPORTING
5321              i_selection  = gt_selection_full
5322  *           I_FIELDLIST  =
5323  *           I_BLOCKSIZE  =
5324  *           I_BLOCK_DATA = GUSL_C_FALSE
5325  *           I_AGGREGATION          = GUSL_C_FALSE
5326              i_max_lines  = gd_max_lines
5327              i_reset_init = lv_reset_init  "correction archive and non ar
      chive
5328  *         IMPORTING
5329  *           FLAG_CURSOR_OPEN       =
5330            CHANGING
5331              c_t_faglpose = it_sel.
5332
5333
5334          CLEAR lv_reset_init. "correction archive and non archive
5335
5336          CLEAR gd_xarch.
5337          LOOP AT it_sel INTO wa_sel.
5338  * relevant basically if report is called from balance display.
5339            gp_yrper(4) = wa_sel-gjahr.
5340            gp_yrper+5  = wa_sel-monat.
5341            CHECK gp_yrper IN so_yrper.
5342  * writing items into it_pos.
5343            PERFORM pos_table_fill  CHANGING  x_stop.
5344            IF NOT x_stop IS INITIAL.
5345              EXIT.
5346            ENDIF.
5347          ENDLOOP.
5348          IF NOT gd_max_lines IS INITIAL.
5349            DESCRIBE TABLE it_sel LINES ld_lines.
5350            gd_max_lines = gd_max_lines - ld_lines.
5351          ENDIF.
5352        ENDIF.
5353      ENDIF.
5354    ELSE.
5355      EXPORT what_has_been_called = ls_function_called TO MEMORY ID 'faglu
      t' COMPRESSION OFF.
5356  *cleared items
5357      IF ( NOT skb1-xopvw IS INITIAL
5358      OR ( NOT skb1-xkres IS INITIAL
5359           AND NOT x_erfs IS INITIAL )
5360      OR ( NOT skb1-xkres IS INITIAL
```

```
5361           AND x_erfs IS INITIAL
5362           AND ( gv_relevant_ledger = gv_leading_ledger
5363           OR skb1-mitkz EQ 'D' OR skb1-mitkz EQ 'K' ) ) ).
5364         IF NOT x_norm IS INITIAL.
5365           IF NOT gd_usedb IS INITIAL.
5366             IF ( skb1-mitkz EQ 'D' OR skb1-mitkz EQ 'K' )     "1107639
5367             AND skb1-xkres NE space AND x_erfs IS INITIAL.    "1699013
5368               ld_read_bseg = 'X'.                             "1107639
5369             ENDIF.                                            "1107639
5370             gd_cursor_open = 'X'.                             "987950
5371             WHILE gd_cursor_open = 'X' AND x_stop IS INITIAL. "987950
5372 *             check x_stop is initial.                          "987950
5373               REFRESH it_sel.
5374               ls_function_called-get_items_bsas = abap_true.
5375               CALL FUNCTION 'FAGL_GET_ITEMS_BSAS'
5376                 EXPORTING
5377                   iv_acdoc_compatibility_mode = abap_false
5378                   iv_data_temperature         = lv_data_temperature    "FA
      GLL03 4./5. HW
5379                   i_selection                 = gt_selection_full
5380 *                 I_FIELDLIST                 =
5381                   i_blocksize                 = gd_max_pack_lines       "
      987950
5382                   i_block_data                = gusl_c_true             "
      987950
5383 *                 I_AGGREGATION               = GUSL_C_FALSE
5384                   i_read_bseg                 = ld_read_bseg            "
      1107639
5385                   i_max_lines                 = gd_max_lines
5386                   i_reset_init                = lv_reset_init  "correction
       archive and non archive
5387                 IMPORTING                                           "987950
5388                   flag_cursor_open            = gd_cursor_open          "
      987950
5389                 CHANGING
5390                   c_t_faglpose                = it_sel.
5391
5392               CLEAR lv_reset_init. "correction archive and non archive
5393
5394               CLEAR gd_xarch.
5395               LOOP AT it_sel INTO wa_sel.
5396 * relevant basically if report is called from balance display.
5397                 IF NOT x_merk IS INITIAL
5398                 OR NOT x_erfs IS INITIAL.
5399                   gp_yrper(4) = wa_sel-gjahr.
5400                   gp_yrper+5  = wa_sel-monat.
5401                 ELSE.
5402                   gp_yrper(4) = wa_sel-ryear.
5403                   gp_yrper+4 = wa_sel-poper.
5404                 ENDIF.
5405                 CHECK gp_yrper IN so_yrper.
5406 * writing items into it_pos.
5407                 IF  NOT gd_usear IS INITIAL
5408                 AND NOT x_erfs IS INITIAL
5409                 AND NOT wa_sel-xarch IS INITIAL.
5410                 ELSE.
5411                   PERFORM pos_table_fill  CHANGING  x_stop.
5412                 ENDIF.
5413                 IF NOT x_stop IS INITIAL.
5414                   CLEAR gd_cursor_open.                           "987950
5415                   EXIT.
```

```
5416              ENDIF.
5417            ENDLOOP.
5418            IF NOT gd_max_lines IS INITIAL.
5419              DESCRIBE TABLE it_sel LINES ld_lines.
5420              gd_max_lines = gd_max_lines - ld_lines.
5421            ENDIF.
5422          ENDWHILE.                                          "987950
5423          FREE MEMORY ID 'FAGLL03_REMAIN_SEL'.               "1105201
5424        ENDIF.
5425        EXPORT what_has_been_called = ls_function_called TO MEMORY ID 'f
      aglut' COMPRESSION OFF.
5426        IF NOT gd_usear IS INITIAL.
5427          CHECK x_stop IS INITIAL.
5428          REFRESH it_sel.
5429          lt_files[] = gt_files[].
5430          ls_function_called-get_arch_item_bsis = abap_true.
5431          CALL FUNCTION 'FAGL_GET_ARCH_ITEMS_BSIS'
5432            EXPORTING
5433              i_selection  = gt_selection_full
5434  *           I_FIELDLIST  =
5435  *           I_BLOCKSIZE  =
5436  *           I_BLOCK_DATA = GUSL_C_FALSE
5437  *           I_AGGREGATION          = GUSL_C_FALSE
5438              i_read_bseg  = ld_read_bseg         "1107639
5439              i_arch_sel   = lt_files
5440              i_max_lines  = gd_max_lines
5441  *         IMPORTING
5442  *           FLAG_CURSOR_OPEN       =
5443            CHANGING
5444              c_t_faglpose = it_sel
5445              c_t_bkpf     = ybkpf
5446              c_t_bseg     = ybseg.
5447
5448
5449          lv_reset_init = abap_true. "correction archive and non archive
5450          gd_xarch = 'X'.
5451          LOOP AT it_sel INTO wa_sel.
5452            wa_sel-xarch = 'X'.
5453  * relevant basically if report is called from balance display.
5454            IF NOT x_merk IS INITIAL
5455            OR NOT x_erfs IS INITIAL.
5456              gp_yrper(4) = wa_sel-gjahr.
5457              gp_yrper+5  = wa_sel-monat.
5458            ELSE.
5459              gp_yrper(4) = wa_sel-ryear.
5460              gp_yrper+4 = wa_sel-poper.
5461            ENDIF.
5462            CHECK gp_yrper IN so_yrper.
5463  * writing items into it_pos.
5464            PERFORM pos_table_fill  CHANGING  x_stop.
5465            IF NOT x_stop IS INITIAL.
5466              EXIT.
5467            ENDIF.
5468          ENDLOOP.
5469          PERFORM import_arch_from_itab.
5470          IF NOT gd_max_lines IS INITIAL.
5471            DESCRIBE TABLE it_sel LINES ld_lines.
5472            gd_max_lines = gd_max_lines - ld_lines.
5473          ENDIF.
5474        ENDIF.
5475      ENDIF.
```

```
5476      ELSEIF skb1-xlgclr = 'X'.
5477        IF NOT x_norm IS INITIAL.
5478          IF NOT gd_usedb IS INITIAL.
5479            gd_cursor_open = 'X'.                                  "987950
5480            WHILE gd_cursor_open = 'X' AND x_stop IS INITIAL. "987950
5481  *           check x_stop is initial.                               "987950
5482              REFRESH it_sel.
5483              ls_function_called-get_items_faglbsas = abap_true.
5484              CALL FUNCTION 'FAGL_GET_ITEMS_FAGLBSAS'
5485                EXPORTING
5486                  iv_acdoc_compatibility_mode = abap_false     "FAGLL03 4.
      /5. HW
5487                  iv_data_temperature         = lv_data_temperature    "FA
      GLL03 4./5. HW
5488                  i_selection                 = gt_selection_full
5489                  i_blocksize                 = gd_max_pack_lines        "
      987950
5490                  i_block_data                = gusl_c_true              "
      987950
5491                  i_max_lines                 = gd_max_lines
5492                  i_reset_init                = lv_reset_init  "correction
       archive and non archive
5493                IMPORTING                                          "987950
5494                  flag_cursor_open            = gd_cursor_open           "
      987950
5495                CHANGING
5496                  c_t_faglpose                = it_sel.
5497
5498              CLEAR lv_reset_init. "correction archive and non archive
5499
5500              CLEAR gd_xarch.
5501              LOOP AT it_sel INTO wa_sel.
5502  * relevant basically if report is called from balance display.
5503                IF NOT x_merk IS INITIAL
5504                OR NOT x_erfs IS INITIAL.
5505                  gp_yrper(4) = wa_sel-gjahr.
5506                  gp_yrper+5  = wa_sel-monat.
5507                ELSE.
5508                  gp_yrper(4) = wa_sel-ryear.
5509                  gp_yrper+4 = wa_sel-poper.
5510                ENDIF.
5511                CHECK gp_yrper IN so_yrper.
5512  * writing items into it_pos.
5513                IF  NOT gd_usear IS INITIAL
5514                AND NOT x_erfs IS INITIAL
5515                AND NOT wa_sel-xarch IS INITIAL.
5516                ELSE.
5517                  PERFORM pos_table_fill  CHANGING  x_stop.
5518                ENDIF.
5519                IF NOT x_stop IS INITIAL.
5520                  CLEAR gd_cursor_open.                            "987950
5521                  EXIT.
5522                ENDIF.
5523              ENDLOOP.
5524              IF NOT gd_max_lines IS INITIAL.
5525                DESCRIBE TABLE it_sel LINES ld_lines.
5526                gd_max_lines = gd_max_lines - ld_lines.
5527              ENDIF.
5528            ENDWHILE.                                              "987950
5529            FREE MEMORY ID 'FAGLL03_REMAIN_SEL'.                   "1105201
5530          ENDIF.
```

```
5531          EXPORT what_has_been_called = ls_function_called TO MEMORY ID 'f
      aglut' COMPRESSION OFF.
5532          IF NOT gd_usear IS INITIAL.
5533            CHECK x_stop IS INITIAL.
5534            REFRESH it_sel.
5535            lt_files[] = gt_files[].
5536            ls_function_called-get_arch_faglbsas = abap_true.
5537            CALL FUNCTION 'FAGL_GET_ARCH_ITEMS_FAGLBSAS'
5538              EXPORTING
5539                i_selection  = gt_selection_full
5540                i_arch_sel   = lt_files
5541                i_max_lines  = gd_max_lines
5542              CHANGING
5543                c_t_faglpose = it_sel
5544                c_t_bkpf     = ybkpf
5545                c_t_bseg     = ybseg.
5546
5547            lv_reset_init = abap_true. "correction archive and non archive
5548            gd_xarch = 'X'.
5549            LOOP AT it_sel INTO wa_sel.
5550              wa_sel-xarch = 'X'.
5551  * relevant basically if report is called from balance display.
5552              IF NOT x_merk IS INITIAL
5553              OR NOT x_erfs IS INITIAL.
5554                gp_yrper(4) = wa_sel-gjahr.
5555                gp_yrper+5  = wa_sel-monat.
5556              ELSE.
5557                gp_yrper(4) = wa_sel-ryear.
5558                gp_yrper+4 = wa_sel-poper.
5559              ENDIF.
5560              CHECK gp_yrper IN so_yrper.
5561  * writing items into it_pos.
5562              PERFORM pos_table_fill  CHANGING  x_stop.
5563              IF NOT x_stop IS INITIAL.
5564                EXIT.
5565              ENDIF.
5566            ENDLOOP.
5567            PERFORM import_arch_from_itab.
5568            IF NOT gd_max_lines IS INITIAL.
5569              DESCRIBE TABLE it_sel LINES ld_lines.
5570              gd_max_lines = gd_max_lines - ld_lines.
5571            ENDIF.
5572            FREE MEMORY ID 'FAGLL03_REMAIN_SEL'.              "1105201
5573          ENDIF.
5574        ENDIF.
5575      ENDIF.
5576    ENDIF.
5577
5578    EXPORT what_has_been_called = ls_function_called TO MEMORY ID 'faglut'
       COMPRESSION OFF.
5579  ENDFORM.
5580
5581
5582  *&---------------------------------------------------------------------*
5583  *&      Form  check_auth_ledger
5584  *&---------------------------------------------------------------------*
5585  *       text
5586  *----------------------------------------------------------------------*
5587  *  -->  p1        text
5588  *  <--  p2        text
5589  *----------------------------------------------------------------------*
```

```
5590  FORM check_auth_ledger .
5591    DATA: lt_bukrs TYPE bukrs OCCURS 0 WITH HEADER LINE.
5592
5593    IF NOT sd_bukrs[] IS INITIAL.
5594      SELECT bukrs FROM t001 INTO TABLE lt_bukrs
5595        WHERE bukrs IN sd_bukrs.
5596    ELSE.
5597  *  Possibly the worklist function is active
5598      SELECT bukrs FROM t001 INTO TABLE lt_bukrs
5599        WHERE bukrs IN so_wlbuk.
5600    ENDIF.
5601    LOOP AT lt_bukrs.
5602      CALL FUNCTION 'FAGL_AUTHORITY_LEDGER'
5603        EXPORTING
5604          i_bukrs = lt_bukrs
5605          i_rldnr = gv_relevant_ledger
5606          i_actvt = '03'.
5607    ENDLOOP.
5608  ENDFORM.                    " check_auth_ledger
5609  *&---------------------------------------------------------------------*
5610  *&      Form  init_rri_data
5611  *&---------------------------------------------------------------------*
5612  *       text
5613  *----------------------------------------------------------------------*
5614  *  -->  p1        text
5615  *  <--  p2        text
5616  *----------------------------------------------------------------------*
5617  FORM init_rri_data .
5618    DATA:   ls_fieldr TYPE rstifields
5619            , lt_fieldr       TYPE TABLE OF rstifields.
5620    DATA: ls_sel          TYPE rstisel
5621        , ls_sel_2        TYPE rstisel
5622        , lt_sel          TYPE TABLE OF rstisel
5623        , ls_fields       TYPE rstifields
5624        , lt_fields       TYPE TABLE OF rstifields.
5625    DATA: ld_rldnr        TYPE rldnr.
5626    DATA: ld_si_table     TYPE tabname.
5627    DATA: ld_tabname      TYPE tabname.
5628    DATA: lt_x031l_1  LIKE x031l OCCURS 0 WITH HEADER LINE.
5629    DATA: lt_x031l_2  LIKE x031l OCCURS 0 WITH HEADER LINE.
5630    FIELD-SYMBOLS: <wa1> TYPE any.
5631    FIELD-SYMBOLS: <wa2> TYPE table.
5632    DATA: ld_t_range_name TYPE string.
5633    DATA: BEGIN OF lt_selection OCCURS 0,
5634            tablename TYPE tabname,
5635            fieldname TYPE fieldname.
5636            INCLUDE TYPE gusl_s_range.
5637    DATA: END   OF lt_selection.
5638    DATA: ls_tablenames  TYPE fagl_tabnames.
5639    DATA: ds_trange_tab_line TYPE rsds_range.
5640    DATA: ds_frange_tab_line TYPE rsds_frange.
5641    DATA: ds_texpr_tab_line TYPE rsds_expr.
5642    DATA: ds_expr_tab_tab_line LIKE rsdsexpr.
5643    DATA: ld_fieldname TYPE rsdsexpr-fieldname.
5644    DATA: ls_texpr_tab_line TYPE rsds_expr.
5645    DATA: ls_expr_tab_line LIKE rsdsexpr.
5646    DATA: ls_num LIKE sy-tfill.
5647    DATA: ls_field LIKE rsdsfields.
5648    DATA: ls_selopt      TYPE rsdsselopt.
5649    DATA: ls_selopt_t    TYPE rsds_selopt_t.
5650    DATA: a_rec          LIKE rstirec.
```

```
5651    DATA: rri_subrc      LIKE sy-subrc.
5652    DATA: ld_versn       LIKE t011-versn.
5653    DATA: ld_lines       TYPE i.
5654    DATA: ld_ergsl       LIKE rf011p-ergsl.
5655    DATA: ld_poper       TYPE poper.
5656    DATA: ld_ryear       TYPE gjahr.
5657    DATA: ld_yrper       LIKE gp_yrper.
5658    DATA: ld_curtp       TYPE curtp.
5659    DATA: ld_variant     TYPE slis_vari.
5660    CONSTANTS:
5661      lc_variant_s_fc TYPE slis_vari VALUE '1SAP-FC',
5662      lc_variant_s_lc TYPE slis_vari VALUE '1SAP-LC'.
5663    DATA: ld_kennzahl_set   TYPE boolean.                    "1093945
5664    DATA: ld_xbilk_acc      TYPE boolean.                    "1692932
5665    DATA: ld_xbilk_cnt      TYPE i.                          "1692932
5666    DATA: ld_ktopl          TYPE ska1-ktopl.                 "1692932
5667    DATA: ld_account_found  TYPE boolean.                    "1093945
5668    DATA: ld_cum_display    TYPE boolean.                    "1093945
5669    DATA: ld_saknr          LIKE ska1-saknr.                 "2034540
5670    DATA: lt_set_sel        TYPE TABLE OF rgsb4.             "1417516
5671    DATA: ls_set_sel        TYPE rgsb4.                      "1417516
5672    DATA: setnr(50)         TYPE c.                          "1417516
5673    DATA  ls_sender         TYPE rstirec.                    "n2075846
5674    DATA ld_item_status_set.                                 "n2954418
5675
5676
5677    CALL FUNCTION 'RSTI_APPL_STACK_POP'
5678      IMPORTING
5679        i_rec                     = a_rec
5680        i_sen                     = ls_sender                "n2075846
5681      EXCEPTIONS
5682        appl_stack_not_initialized = 1.
5683    rri_subrc = sy-subrc.
5684    IF rri_subrc = 0.
5685  *    check Receiver is this report
5686      IF NOT ( ( a_rec-rtool = 'RT'
5687                 AND a_rec-ronam = 'FAGL_ACCOUNT_ITEMS_GL' )
5688             OR ( a_rec-rtool = 'TR' AND a_rec-ronam = 'FAGLL03' ) ) .
5689        rri_subrc = 2.
5690      ENDIF.
5691    ENDIF.
5692    IF rri_subrc = 0.
5693      gd_rri_sender = ls_sender-ronam.                       "n2075846
5694      REFRESH sd_bukrs.
5695      REFRESH sd_saknr.
5696      CLEAR   gd_wl_on.
5697      CLEAR x_opsel.                                         "n2075846
5698      CLEAR pa_stida.                                        "n2075846
5699      CLEAR sd_stida.                                        "n2075846
5700
5701      CLEAR x_clsel.                                         "n2075846
5702      REFRESH so_augdt.                                      "n2075846
5703      REFRESH sd_augdt[].                                    "n2075846
5704      CLEAR pa_stid2.                                        "n2075846
5705
5706      CLEAR x_aisel.                                         "n2075846
5707      REFRESH so_budat[].                                    "n2075846
5708      gd_rri_sender = ls_sender-ronam.                       "n2075846
5709
5710      gv_xrric = 'X'.
5711
```

```
5712      CALL FUNCTION 'RSTI_SELECTION_IMPORT'
5713        TABLES
5714          it_sel            = lt_sel
5715          it_fields         = lt_fields
5716        EXCEPTIONS
5717          no_selection_data = 1
5718          OTHERS            = 2.
5719      IF sy-subrc <> 0.
5720        CALL FUNCTION 'RSTI_APPL_STACK_INITIALIZE'
5721          EXPORTING
5722            e_tool = a_rec-rtool
5723            e_onam = a_rec-ronam.
5724        EXIT.
5725      ENDIF.
5726
5727  * delete selection which are not given correctly            "984308
5728      LOOP AT lt_sel INTO ls_sel                             "984308
5729        WHERE sign   IS INITIAL                              "984308
5730          AND option IS INITIAL                              "984308
5731          AND low    IS INITIAL.                             "984308
5732        DELETE lt_sel.                                       "984308
5733      ENDLOOP.                                               "984308
5734
5735  * deleted restrictions to USEDB, USEAR and USEAS              "1599984
5736      LOOP AT lt_sel INTO ls_sel
5737        WHERE field = 'USEDB'
5738           OR field = 'USEAR'
5739           OR field = 'USEAS'.
5740        DELETE lt_sel.
5741      ENDLOOP.
5742
5743  * customer could have used the special selection field 'PERIO'   "170318
     1
5744      LOOP AT lt_sel INTO ls_sel
5745        WHERE field = 'RYEAR'.
5746      ENDLOOP.
5747      IF sy-subrc NE 0.
5748        LOOP AT lt_sel INTO ls_sel
5749          WHERE field = 'POPER'.
5750        ENDLOOP.
5751        IF sy-subrc NE 0.
5752          LOOP AT lt_sel INTO ls_sel
5753            WHERE field = 'PERIO'.
5754          ENDLOOP.
5755          IF sy-subrc = 0.
5756            ls_sel_2-sign   = 'I'.
5757            ls_sel_2-option = 'EQ'.
5758            ls_sel_2-field  = 'RYEAR'.
5759            ls_sel_2-low    = ls_sel-low(4).
5760            APPEND ls_sel_2 TO lt_sel.
5761            ls_sel_2-field  = 'POPER'.
5762            ls_sel_2-low    = ls_sel-low+4(3).
5763            APPEND ls_sel_2 TO lt_sel.
5764          ENDIF.
5765        ENDIF.
5766      ENDIF.
5767  * customer could have used the special selection field 'PERIO'   "170318
     1
5768
5769  * If program is called from RFBILA00/N the posting date range should be
     used.
```

```
5770  * Additional restriction of the year is not necessary.
5771      IF ls_sender-ronam CS 'RFBILA00'.                          "n2075846
5772        DELETE lt_sel WHERE field = 'RYEAR'.                     "n2075846
5773        DELETE lt_sel WHERE field = 'POPER'.                     "n2075846
5774      ENDIF.                                                     "n2075846
5775
5776  * determine versn for recherche
5777      LOOP AT lt_sel INTO ls_sel
5778        WHERE field = 'VERSN'.
5779        ld_versn = ls_sel-low.
5780        DELETE lt_sel.
5781      ENDLOOP.
5782      IF ld_versn IS INITIAL.
5783        LOOP AT lt_sel INTO ls_sel
5784          WHERE field = 'SAK_PLUMI'.
5785          ld_versn = ls_sel-hienm.
5786        ENDLOOP.
5787      ENDIF.
5788      LOOP AT lt_sel INTO ls_sel
5789        WHERE field = 'RACCT'.
5790        EXIT.
5791      ENDLOOP.
5792      IF sy-subrc = 0.
5793        LOOP AT lt_sel INTO ls_sel
5794          WHERE field = 'SAK_PLUMI'
5795          OR field = 'ERGSL'
5796          OR field = 'SAKFB_LONG'.
5797          DELETE lt_sel.
5798        ENDLOOP.
5799      ENDIF.
5800
5801      CLEAR ld_ktopl.                                            "1764246
5802      LOOP AT lt_sel INTO ls_sel WHERE field = 'KTOPL'.           "1692932
5803        ld_ktopl = ls_sel-low.                                   "1692932
5804      ENDLOOP.                                                   "1692932
5805      IF ld_ktopl IS INITIAL.                                    "1764246
5806        LOOP AT lt_sel INTO ls_sel WHERE field = 'RKTPL'.        "1764246
5807          ld_ktopl = ls_sel-low.                                 "1764246
5808        ENDLOOP.                                                 "1764246
5809      ENDIF.                                                     "1764246
5810      IF ld_ktopl IS INITIAL.                                    "1764246
5811        ld_ktopl = ld_versn.                                     "1692932
5812      ENDIF.                                                     "1692932
5813      LOOP AT lt_sel INTO ls_sel WHERE field = '&KKENNZAHL'.     "1692932
5814        IF ls_sel-low = 'BILWERT'.                               "1692932
5815          ld_kennzahl_set = 'X'.                                 "1692932
5816        ENDIF.                                                   "1692932
5817      ENDLOOP.                                                   "1692932
5818      CLEAR ld_xbilk_acc.                                        "1692932
5819
5820
5821  * Begin of Note "1093945
5822      LOOP AT lt_sel INTO ls_sel.
5823  * Determine Comp Code
5824        CASE ls_sel-field.                              "begin "n2075846
5825          WHEN 'BUKRS'
5826            OR 'RBUKRS'.
5827            MOVE-CORRESPONDING ls_sel TO sd_bukrs.
5828            APPEND sd_bukrs.
5829  * Determine Posting Date
5830          WHEN 'BUDAT'.
```

```
5831            MOVE-CORRESPONDING ls_sel TO so_budat.
5832            IF so_budat-option = 'EQ'.                          "2600639
5833              CLEAR so_budat-high.                              "2600639
5834            ENDIF.                                              "2600639
5835            APPEND so_budat.
5836    * Determine Business Areas
5837          WHEN 'GSBER'
5838            OR 'BUSA'
5839            OR 'RBUSA'.
5840            MOVE-CORRESPONDING ls_sel TO so_gsber.
5841            APPEND so_gsber.
5842          WHEN OTHERS.                                  "end "n2075846
5843    * determine GL Accounts
5844            IF ls_sel-field = 'RACCT'
5845            OR ls_sel-field = 'SAKNR'
5846            OR ls_sel-field = 'HKONT'
5847            OR ( ls_sel-field = 'COST_ELEM'                     "1847834
5848                         AND NOT ls_sel-type = 'SET' ).         "1847834
5849              IF ls_sel-field = 'COST_ELEM'.                    "2034540
5850                IF NOT ld_ktopl IS INITIAL.                     "2501428
5851              IF ls_sel-option = 'BT'.                                "3
     479498
5852                SELECT SINGLE SAKNR FROM SKA1 INTO ld_saknr           "3
     479498
5853                  WHERE ktopl = ld_ktopl                              "3
     479498
5854                  AND saknr BETWEEN ls_sel-low(10) AND ls_sel-high(10).  "3
     479498
5855              ELSE.                                                   "3
     479498
5856                SELECT SINGLE SAKNR FROM SKA1 INTO ld_saknr       "2501428
5857                  WHERE ktopl = ld_ktopl                          "2501428
5858                    AND saknr = ls_sel-low(10).                   "2501428
5859              ENDIF.                                                  "3
     479498
5860                ELSE.                                           "2501428
5861              IF ls_sel-option = 'BT'.                                "3
     479498
5862                SELECT SINGLE SAKNR FROM SKA1 INTO ld_saknr           "3
     479498
5863                  WHERE saknr BETWEEN ls_sel-low(10) AND ls_sel-high(10)."3
     479498
5864              ELSE.                                                   "3
     479498
5865                SELECT SINGLE SAKNR FROM SKA1 INTO ld_saknr       "2501428
5866                  WHERE saknr = ls_sel-low(10).                   "2501428
5867              ENDIF.                                                  "3
     479498
5868                ENDIF.                                          "2501428
5869                IF sy-subrc NE 0.                               "2034540
5870                  DELETE lt_sel.                                        "3
     479498
5871                  CONTINUE.                                     "2034540
5872                ENDIF.                                          "2034540
5873              ENDIF.                                            "2034540
5874              IF NOT ls_sel-type = 'SET'.                       "2170719
5875                CLEAR lt_selection.
5876                ls_sel-field = 'SD_SAKNR'.
5877                MOVE-CORRESPONDING ls_sel TO lt_selection.
5878                lt_selection-fieldname = ls_sel-field.
5879                APPEND lt_selection.
```

```
5880                IF ld_kennzahl_set = 'X'.                          "1692932
5881                  SELECT SINGLE saknr FROM ska1 INTO ld_sknr_dummy "169293
     2
5882                                       WHERE ktopl = ld_ktopl  "1692932
5883                                         AND saknr = ls_sel-low(10) "169293
     2
5884                                         AND xbilk = 'X'.      "1692932
5885                  IF sy-subrc = 0.                               "1692932
5886                    ld_xbilk_acc = 'X'.                          "1692932
5887                  ENDIF.                                         "1692932
5888                ENDIF.                                           "1692932
5889                DELETE lt_sel.
5890                ld_account_found = 'X'.                          "1093945
5891
5892              ELSE.                                              "2170719
5893                REFRESH lt_set_sel.                              "2170719
5894                CLEAR ls_set_sel.                                "2170719
5895                CLEAR setnr.                                     "2170719
5896                setnr = ls_sel-low.                              "2170719
5897                CALL FUNCTION 'G_SET_GET_ALL_VALUES'                   "2170
     719
5898                  EXPORTING                                            "2170
     719
5899                    setnr      = setnr                                 "2170
     719
5900                  TABLES                                               "2170
     719
5901                    set_values = lt_set_sel.                           "2170
     719
5902                IF sy-subrc = 0.                                 "2170719
5903                  LOOP AT lt_set_sel INTO ls_set_sel.            "2170719
5904                    CLEAR lt_selection.                          "2170719
5905                    ls_sel-sign = 'I'.                           "2170719
5906                    ls_sel-option = 'BT'.                        "2170719
5907                    ls_sel-low    = ls_set_sel-from.             "2170719
5908                    ls_sel-high   = ls_set_sel-to.               "2170719
5909                    ls_sel-hienm  = ls_set_sel-setnr.            "2170719
5910                    ls_sel-hclass = ls_set_sel-setclass.         "2170719
5911                    ls_sel-type   = ls_set_sel-setclass.         "2170719
5912                    IF ls_sel-low = ls_sel-high.                 "2170719
5913                      CLEAR ls_sel-high.                         "2170719
5914                      ls_sel-option = 'EQ'.                      "2170719
5915                    ENDIF.                                       "2170719
5916                    MOVE-CORRESPONDING ls_sel TO lt_selection. "2170719
5917                    ls_sel-field = 'SD_SAKNR'.                   "2170719
5918                    lt_selection-fieldname = ls_sel-field.       "2170719
5919                    APPEND lt_selection.                         "2170719
5920                  ENDLOOP.                                       "2170719
5921                ENDIF.                                           "2170719
5922              ENDIF.                                             "2170719
5923            ENDIF.
5924        ENDCASE.                                                 "n2075846
5925      ENDLOOP.
5926
5927      IF ld_account_found IS INITIAL.
5928        LOOP AT lt_sel INTO ls_sel.
5929  * determine Accounts from FI recherche
5930          IF ( ls_sel-field = 'SAK_PLUMI'
5931          OR   ls_sel-field = 'ERGSL'
5932          OR   ls_sel-field = 'SAKFB_LONG' ).                    "1107224
5933            IF ld_versn IS INITIAL.                              "1107224
```

```
5934              ld_versn = ls_sel-hienm.                              "1107224
5935            ENDIF.                                                  "1107224
5936            IF NOT ld_versn IS INITIAL.                             "1107224
5937              DATA: lt_accounts TYPE rfaccinf OCCURS 0 WITH HEADER LINE.
5938              ld_ergsl = ls_sel-low.
5939              CALL FUNCTION 'RGRE_ERGSL_TO_RACCT_CONVERT'
5940                EXPORTING
5941                  i_bs_version         = ld_versn
5942                  i_bs_item            = ld_ergsl
5943                TABLES
5944                  t_accounts           = lt_accounts
5945                EXCEPTIONS
5946                  bs_version_not_exist = 1
5947                  OTHERS               = 2.
5948              IF sy-subrc <> 0.
5949  *             MESSAGE ID SY-MSGID TYPE SY-MSGTY NUMBER SY-MSGNO
5950  *             WITH SY-MSGV1 SY-MSGV2 SY-MSGV3 SY-MSGV4.
5951              ENDIF.
5952
5953              DESCRIBE TABLE lt_accounts LINES ld_lines.
5954              CLEAR lt_selection.                                   "1093945
5955              IF ld_lines GT 0.
5956                IF ld_kennzahl_set = 'X'.                           "1692932
5957                  READ TABLE lt_accounts INDEX 1.                   "1692932
5958                  SELECT SINGLE saknr FROM ska1                     "1692932
5959                           INTO ld_sknr_dummy                       "1692932
5960                           WHERE ktopl = lt_accounts-ktopl          "2253641
5961                             AND saknr = lt_accounts-acct_from      "1692932
5962                             AND xbilk = 'X'.                       "1692932
5963                  IF sy-subrc = 0.                                  "1692932
5964                    ld_xbilk_acc = 'X'.                             "1692932
5965                  ENDIF.                                            "1692932
5966                ENDIF.                                              "1692932
5967                LOOP AT lt_accounts.
5968                  ls_sel-field = 'SD_SAKNR'.
5969                  lt_selection-sign = 'I'.
5970                  lt_selection-option = 'EQ'.
5971                  lt_selection-low = lt_accounts-acct_from.
5972                  lt_selection-fieldname = ls_sel-field.
5973                  APPEND lt_selection.
5974                ENDLOOP.
5975                DELETE lt_sel.
5976              ELSE.
5977                CLEAR lt_selection.
5978                ls_sel-field = 'SD_SAKNR'.
5979                MOVE-CORRESPONDING ls_sel TO lt_selection.
5980                lt_selection-fieldname = ls_sel-field.
5981                APPEND lt_selection.
5982                IF ld_kennzahl_set = 'X'.                           "1692932
5983                  SELECT SINGLE saknr FROM ska1                     "1692932
5984                           INTO ld_sknr_dummy                       "1692932
5985                           WHERE ktopl = ld_ktopl                   "1692932
5986                             AND saknr = ls_sel-low(10)             "1692932
5987                             AND xbilk = 'X'.                       "1692932
5988                  IF sy-subrc = 0.                                  "1692932
5989                    ld_xbilk_acc = 'X'.                             "1692932
5990                  ENDIF.                                            "1692932
5991                ENDIF.                                              "1692932
5992                DELETE lt_sel.
5993              ENDIF.
5994            ENDIF.                                                  "1107224
```

```
5995          ENDIF.
5996        ENDLOOP.
5997      ENDIF.
5998
5999    * Rewritten: Determine if a cumulated display is needed        "1363278
6000    * determine kumsl for recherche
6001        CLEAR ld_kennzahl_set.                                    "1692932
6002        LOOP AT lt_sel INTO ls_sel
6003          WHERE field = 'KUMSL'.
6004          DELETE lt_sel.
6005        ENDLOOP.
6006        IF sy-subrc NE 0.
6007          LOOP AT lt_sel INTO ls_sel
6008            WHERE field = '&KKENNZAHL'.
6009            IF   ls_sel-low = 'KUMSL'
6010              OR ( ls_sel-low = 'BILWERT'                         "1692932
6011                        AND ld_xbilk_acc = 'X' )                  "1692932
6012              OR ls_sel-low = 'KUMSOLL'
6013              OR ls_sel-low = 'KUMHABEN'.
6014              ld_kennzahl_set = 'X'.
6015            ENDIF.
6016          ENDLOOP.
6017        ELSE.
6018          ld_kennzahl_set = 'X'.
6019        ENDIF.
6020        IF ld_kennzahl_set = 'X'.
6021          LOOP AT lt_sel INTO ls_sel
6022            WHERE field = 'POPER'.
6023            IF NOT ls_sel-high IS INITIAL.
6024              ld_poper = ls_sel-high.
6025            ELSE.
6026              ld_poper = ls_sel-low.
6027            ENDIF.
6028            DELETE lt_sel.
6029          ENDLOOP.
6030          LOOP AT lt_sel INTO ls_sel
6031            WHERE field = 'RYEAR'.
6032            ld_ryear = ls_sel-low.
6033            DELETE lt_sel.
6034          ENDLOOP.
6035          ls_sel-field = 'SO_YRPER'.
6036          lt_selection-sign = 'I'.
6037          lt_selection-option = 'BT'.
6038          lt_selection-low = '1950001'.
6039          ld_yrper(4) = ld_ryear.
6040          ld_yrper+4(3) = ld_poper.
6041          lt_selection-high = ld_yrper.
6042          lt_selection-fieldname = ls_sel-field.
6043          APPEND lt_selection.
6044          gd_rri_so_set = 'X'.
6045          ls_sel-field = 'POPER'.
6046          lt_selection-sign = 'I'.
6047          lt_selection-option = 'BT'.
6048          lt_selection-low = '001'.
6049          lt_selection-high = ld_poper.
6050          lt_selection-fieldname = ls_sel-field.
6051          APPEND lt_selection.
6052          ls_sel-field = 'RYEAR'.
6053          lt_selection-sign = 'I'.
6054          lt_selection-option = 'BT'.
6055          lt_selection-low = '1950'.
```

```
6056        lt_selection-high = ld_ryear.
6057        lt_selection-fieldname = ls_sel-field.
6058        APPEND lt_selection.
6059      ENDIF.
6060      CLEAR ld_kennzahl_set.
6061
6062  * determine umsav recherche
6063      LOOP AT lt_sel INTO ls_sel
6064          WHERE field = 'UMSAV'.
6065        DELETE lt_sel.
6066      ENDLOOP.
6067      IF sy-subrc NE 0.
6068        LOOP AT lt_sel INTO ls_sel
6069          WHERE field = '&KKENNZAHL'.
6070          IF   ls_sel-low = 'UMSAV'.
6071            ld_kennzahl_set = 'X'.
6072          ENDIF.
6073        ENDLOOP.
6074      ELSE.
6075        ld_kennzahl_set = 'X'.
6076      ENDIF.
6077      IF ld_kennzahl_set = 'X'.
6078        LOOP AT lt_sel INTO ls_sel
6079          WHERE field = 'POPER'.
6080          DELETE lt_sel.
6081        ENDLOOP.
6082        LOOP AT lt_sel INTO ls_sel
6083          WHERE field = 'RYEAR'.
6084          ld_ryear = ls_sel-low.
6085          DELETE lt_sel.
6086        ENDLOOP.
6087        gv_xumsav = 'X'.
6088        ls_sel-field = 'SO_YRPER'.
6089        lt_selection-sign = 'I'.
6090        lt_selection-option = 'BT'.
6091        lt_selection-low = '1950001'.
6092        ld_yrper(4) = ld_ryear - 1.
6093        ld_yrper+4(3) = '016'.
6094        lt_selection-high = ld_yrper.
6095        lt_selection-fieldname = ls_sel-field.
6096        APPEND lt_selection.
6097        gd_rri_so_set = 'X'.
6098        ls_sel-field = 'RYEAR'.
6099        lt_selection-sign = 'I'.
6100        lt_selection-option = 'BT'.
6101        lt_selection-low = '1950'.
6102        lt_selection-high = ld_ryear - 1.
6103        lt_selection-fieldname = ls_sel-field.
6104        APPEND lt_selection.
6105      ENDIF.
6106      CLEAR ld_kennzahl_set.
6107  * Rewritten: Determine if a cumulated display is needed        "1363278
6108
6109  * Begin of Note 1093945
6110      LOOP AT lt_sel INTO ls_sel
6111        WHERE field = '&PARAMETER\MT-DATE'.
6112        EXIT.
6113      ENDLOOP.
6114      IF sy-subrc = 0.
6115        LOOP AT lt_sel INTO ls_sel
6116          WHERE field = 'RPMAX'
```

```
6117             OR field = 'POPER'.
6118           IF ls_sel-low = '000'.
6119             ld_cum_display = 'X'.
6120           ENDIF.
6121         ENDLOOP.
6122         IF ld_cum_display = 'X'.
6123           LOOP AT lt_sel INTO ls_sel
6124             WHERE field = 'RPMAX'
6125                OR field = 'POPER'.
6126             IF NOT ls_sel-high IS INITIAL.
6127               IF ld_poper LE ls_sel-high.
6128                 ld_poper = ls_sel-high.
6129               ENDIF.
6130             ELSE.
6131               IF ld_poper LE ls_sel-low.
6132                 ld_poper = ls_sel-low.
6133               ENDIF.
6134             ENDIF.
6135             DELETE lt_sel.
6136           ENDLOOP.
6137           LOOP AT lt_sel INTO ls_sel
6138             WHERE field = 'RYEAR'.
6139             IF NOT ls_sel-high IS INITIAL.
6140               ld_ryear = ls_sel-high.
6141             ELSE.
6142               ld_ryear = ls_sel-low.
6143             ENDIF.
6144             DELETE lt_sel.
6145           ENDLOOP.
6146           ls_sel-field = 'SO_YRPER'.
6147           lt_selection-sign = 'I'.
6148           lt_selection-option = 'BT'.
6149           lt_selection-low = '1950001'.
6150           ld_yrper(4) = ld_ryear.
6151           ld_yrper+4(3) = ld_poper.
6152           lt_selection-high = ld_yrper.
6153           lt_selection-fieldname = ls_sel-field.
6154           APPEND lt_selection.
6155           gd_rri_so_set = 'X'.                                    "1093945
6156           ls_sel-field = 'POPER'.
6157           lt_selection-sign = 'I'.
6158           lt_selection-option = 'BT'.
6159           lt_selection-low = '001'.
6160           lt_selection-high = ld_poper.
6161           lt_selection-fieldname = ls_sel-field.
6162           APPEND lt_selection.
6163           ls_sel-field = 'RYEAR'.
6164           lt_selection-sign = 'I'.
6165           lt_selection-option = 'BT'.
6166           lt_selection-low = '1950'.
6167           lt_selection-high = ld_ryear.
6168           lt_selection-fieldname = ls_sel-field.
6169           APPEND lt_selection.
6170         ENDIF.
6171       ENDIF.
6172   * End of Note 1093945
6173
6174   * determine curtp recherche
6175       LOOP AT lt_sel INTO ls_sel
6176         WHERE field = 'CURTP'.
6177         ld_curtp = ls_sel-low.
```

```
6178        DELETE lt_sel.
6179      ENDLOOP.
6180      IF sy-subrc = 0.
6181        IF ld_curtp NE '00'.
6182          LOOP AT lt_sel INTO ls_sel
6183            WHERE field = 'WAERS'.
6184            DELETE lt_sel.
6185          ENDLOOP.
6186          ld_variant = lc_variant_s_lc.
6187        ELSE.
6188          ld_variant = lc_variant_s_fc.
6189        ENDIF.
6190        CLEAR ls_sel.
6191        ls_sel-field = 'PA_VARI'.
6192        ls_sel-sign    = 'I'.
6193        ls_sel-option  = 'EQ'.
6194        ls_sel-low     = ld_variant.
6195        APPEND ls_sel TO lt_sel.
6196      ENDIF.
6197
6198      LOOP AT lt_sel INTO ls_sel.
6199  * determine relevant ledger
6200        IF ls_sel-field = 'RLDNR'.
6201          CLEAR lt_selection.
6202          ld_rldnr = ls_sel-low.
6203          MOVE-CORRESPONDING ls_sel TO lt_selection.
6204          lt_selection-fieldname = ls_sel-field.
6205          APPEND lt_selection.
6206          DELETE lt_sel.
6207        ENDIF.
6208  * determine D/C Indicator
6209        IF ls_sel-field = 'SHKZG'
6210        OR ls_sel-field = 'DRCRK'
6211        OR ls_sel-field = 'SO_SHKZG'.
6212          CLEAR lt_selection.
6213          ls_sel-field = 'SO_SHKZG'.
6214          MOVE-CORRESPONDING ls_sel TO lt_selection.
6215          lt_selection-fieldname = ls_sel-field.
6216          APPEND lt_selection.
6217          DELETE lt_sel.
6218        ENDIF.
6219        IF ls_sel-field = '&KKENNZAHL'.
6220          IF ls_sel-low = 'UMSOL'.
6221            CLEAR lt_selection.
6222            ls_sel-field = 'SO_SHKZG'.
6223            lt_selection-sign = 'I'.
6224            lt_selection-option = 'EQ'.
6225            lt_selection-low = 'S'.
6226            lt_selection-fieldname = ls_sel-field.
6227            APPEND lt_selection.
6228            DELETE lt_sel.
6229          ENDIF.
6230          IF ls_sel-low = 'UMHAB'.
6231            CLEAR lt_selection.
6232            ls_sel-field = 'SO_SHKZG'.
6233            lt_selection-sign = 'I'.
6234            lt_selection-option = 'EQ'.
6235            lt_selection-low = 'H'.
6236            lt_selection-fieldname = ls_sel-field.
6237            APPEND lt_selection.
6238            DELETE lt_sel.
```

```
6239          ENDIF.
6240        ENDIF.
6241      ENDLOOP.
6242
6243    *** Process SETS for Profit Center or Cost Element if given    "1417516
6244      LOOP AT lt_sel INTO ls_sel
6245        WHERE ( field = 'PRCTR' OR field = 'COST_ELEM' )
6246          AND type  = 'SET'.
6247        REFRESH lt_set_sel.
6248        CLEAR ls_set_sel.
6249        CLEAR setnr.
6250        setnr = ls_sel-low.
6251        CALL FUNCTION 'G_SET_GET_ALL_VALUES'
6252          EXPORTING
6253            setnr      = setnr
6254          TABLES
6255            set_values = lt_set_sel.
6256        IF sy-subrc = 0.
6257          DELETE lt_sel.
6258          LOOP AT lt_set_sel INTO ls_set_sel.
6259            ls_sel-sign = 'I'.
6260            ls_sel-option = 'BT'.
6261            ls_sel-low    = ls_set_sel-from.
6262            ls_sel-high   = ls_set_sel-to.
6263            ls_sel-hienm  = ls_set_sel-setnr.
6264            ls_sel-hclass = ls_set_sel-setclass.
6265            ls_sel-type   = ls_set_sel-setclass.
6266            IF ls_sel-low = ls_sel-high.
6267              CLEAR ls_sel-high.
6268              ls_sel-option = 'EQ'.
6269            ENDIF.
6270            APPEND ls_sel TO lt_sel.
6271          ENDLOOP.
6272        ENDIF.
6273      ENDLOOP.
6274    *** Process SETS for Profit Center or Cost Element if given    "1417516
6275      IF ld_rldnr IS INITIAL.                          "begin "n1723146
6276        CLEAR ls_fields.
6277        READ TABLE lt_fields
6278          INTO ls_fields
6279          WITH KEY rfield = 'RLDNR'.
6280        IF sy-subrc <> 0.
6281          READ TABLE lt_fields
6282            INTO ls_fields
6283            WITH KEY rollname = 'RLDNR'.
6284          IF sy-subrc <> 0.
6285            READ TABLE lt_fields
6286              INTO ls_fields
6287              WITH KEY rollname = 'FAGL_RLDNR'.
6288          ENDIF.
6289        ENDIF.
6290        IF ls_fields IS NOT INITIAL.
6291          LOOP AT lt_sel
6292            INTO  ls_sel
6293            WHERE field = ls_fields-field.
6294            CLEAR lt_selection.
6295            ld_rldnr = ls_sel-low.
6296            MOVE-CORRESPONDING ls_sel TO lt_selection.
6297            lt_selection-fieldname = ls_sel-field.
6298            APPEND lt_selection.
6299            DELETE lt_sel.
```

```
6300          ENDLOOP.
6301        ENDIF.
6302      ENDIF.                                          "end "n1723146
6303
6304    * add fields to free selection
6305      IF NOT ld_rldnr IS INITIAL.
6306        PERFORM find_si_table
6307             USING    ld_rldnr
6308             CHANGING ld_si_table.
6309        gv_relevant_ledger = ld_rldnr.
6310        gv_si_table = ld_si_table.
6311
6312    * fields FlexGl
6313        CONCATENATE ld_si_table '_FS' INTO ld_tabname.
6314
6315        CALL FUNCTION 'DDIF_NAMETAB_GET'
6316          EXPORTING
6317            tabname   = ld_tabname
6318          TABLES
6319            x031l_tab = lt_x031l_1
6320    *       DFIES_TAB =
6321          EXCEPTIONS
6322            not_found = 1
6323            OTHERS    = 2.
6324
6325        LOOP AT lt_x031l_1.
6326          LOOP AT lt_sel INTO ls_sel
6327            WHERE field = lt_x031l_1-fieldname.
6328            CLEAR lt_selection.
6329            MOVE-CORRESPONDING ls_sel TO lt_selection.
6330            lt_selection-fieldname = ls_sel-field.
6331            lt_selection-tablename = lt_x031l_1-tabname.
6332            APPEND lt_selection.
6333            DELETE lt_sel.
6334          ENDLOOP.
6335          IF sy-subrc <> 0.                           "begin "n1723146
6336            LOOP AT lt_fields
6337              INTO  ls_fields
6338              WHERE rollname = lt_x031l_1-rollname.
6339              LOOP AT lt_sel
6340                INTO  ls_sel
6341                WHERE field = ls_fields-field.
6342                CLEAR lt_selection.
6343                MOVE-CORRESPONDING ls_sel TO lt_selection.
6344                lt_selection-fieldname = lt_x031l_1-fieldname.   "3496575
6345                lt_selection-tablename = lt_x031l_1-tabname.
6346                APPEND lt_selection.
6347                DELETE lt_sel.
6348              ENDLOOP.
6349            ENDLOOP.
6350          ENDIF.                                      "end "n1723146
6351        ENDLOOP.
6352      ELSE.
6353    *     clear gv_relevant_ledger.
6354        x_erfs = 'X'.
6355      ENDIF.
6356
6357    * fields BSIS
6358      CALL FUNCTION 'DDIF_NAMETAB_GET'
6359        EXPORTING
6360          tabname   = 'BSIS_FS'
```

```
6361        TABLES
6362          x031l_tab = lt_x031l_2
6363  *       DFIES_TAB =
6364        EXCEPTIONS
6365          not_found = 1
6366          OTHERS    = 2.
6367
6368      CLEAR lt_selection.
6369      LOOP AT lt_x031l_2.
6370        LOOP AT lt_sel INTO ls_sel
6371          WHERE field = lt_x031l_2-fieldname.
6372          CLEAR lt_selection.
6373          MOVE-CORRESPONDING ls_sel TO lt_selection.
6374          lt_selection-fieldname = ls_sel-field.
6375          lt_selection-tablename = lt_x031l_2-tabname.
6376          APPEND lt_selection.
6377          DELETE lt_sel.
6378        ENDLOOP.
6379      ENDLOOP.
6380
6381      CALL FUNCTION 'RSTI_REPORT_FIELDS_FIND'
6382        EXPORTING
6383          e_repid   = 'FAGL_ACCOUNT_ITEMS_GL'
6384          e_type    = 'R'
6385        TABLES
6386          it_fields = lt_fieldr.
6387
6388  * add rest of the fields
6389      CLEAR lt_selection.
6390      LOOP AT lt_sel INTO ls_sel.
6391        CLEAR lt_selection.
6392        MOVE-CORRESPONDING ls_sel TO lt_selection.
6393        lt_selection-fieldname = ls_sel-field.
6394        APPEND lt_selection.
6395      ENDLOOP.
6396
6397      SORT lt_selection BY tablename fieldname.
6398
6399  * map rri selection to Report selections
6400      CLEAR lt_selection.
6401      LOOP AT lt_selection.
6402        AT NEW tablename.
6403          IF NOT lt_selection-tablename IS INITIAL.
6404            CLEAR ls_selopt.
6405            CLEAR ls_selopt_t.
6406            CLEAR ds_frange_tab_line.
6407            CLEAR ds_trange_tab_line.
6408          ENDIF.
6409        ENDAT.
6410
6411        AT NEW fieldname.
6412          IF lt_selection-tablename IS INITIAL.
6413            LOOP AT lt_fieldr INTO ls_fieldr
6414              WHERE rfield = lt_selection-fieldname.
6415              EXIT.
6416            ENDLOOP.
6417            IF sy-subrc <> 0.                          "begin "n1723146
6418              READ TABLE lt_fields
6419                INTO ls_fields
6420                WITH KEY
6421                  field = lt_selection-fieldname.
```

```
6422              IF  sy-subrc = 0
6423              AND ls_fields-field <> ls_fieldr-rfield.        "2150276
6424                LOOP AT lt_fieldr INTO ls_fieldr
6425                  WHERE rfield = ls_fields-rfield.
6426                  EXIT.
6427                ENDLOOP.
6428              ENDIF.
6429            ENDIF.                                      "end "n1723146
6430            IF sy-subrc = 0.
6431              IF ls_fieldr-kind = 'S'.
6432                CONCATENATE ls_fieldr-rfield '[]'             "n1723146
6433                  INTO ld_t_range_name.
6434                ASSIGN (ld_t_range_name) TO <wa2>.
6435              ENDIF.
6436            ELSE.
6437              CLEAR ls_fieldr.
6438              CLEAR ls_selopt.
6439              CLEAR ls_selopt_t.
6440              CLEAR ds_frange_tab_line.
6441            ENDIF.   "end xrp
6442          ELSE.
6443            CLEAR ls_fieldr.
6444            CLEAR ls_selopt.
6445            CLEAR ls_selopt_t.
6446            CLEAR ds_frange_tab_line.
6447          ENDIF.
6448        ENDAT.
6449
6450        IF lt_selection-tablename IS INITIAL.
6451          IF ls_fieldr-kind = 'P'.
6452            ASSIGN (ls_fieldr-rfield) TO <wa1>.              "n1723146
6453            IF <wa1> IS ASSIGNED.                           "n1723146
6454              MOVE lt_selection-low TO <wa1>.
6455            ENDIF.                                          "n1723146
6456          ENDIF.
6457          IF ls_fieldr-kind = 'S'.
6458            ASSIGN (ls_fieldr-rfield) TO <wa1>.              "n1723146
6459            IF  <wa1> IS ASSIGNED                           "n1723146
6460            AND <wa2> IS ASSIGNED.                          "n1723146
6461              MOVE-CORRESPONDING lt_selection TO <wa1>.
6462              APPEND <wa1> TO <wa2>.
6463            ENDIF.                                          "n1723146
6464          ENDIF.
6465        ENDIF.
6466        IF lt_selection-tablename = 'BSIS_FS'
6467        OR ( lt_selection-tablename = ld_tabname
6468        AND NOT ld_tabname IS INITIAL ).
6469          MOVE-CORRESPONDING lt_selection TO ls_selopt.
6470          APPEND ls_selopt TO ls_selopt_t.
6471        ENDIF.
6472
6473        AT END OF fieldname.
6474          IF NOT lt_selection-tablename IS INITIAL.
6475            ds_frange_tab_line-fieldname = lt_selection-fieldname.
6476            ds_frange_tab_line-selopt_t[] = ls_selopt_t[].
6477            APPEND ds_frange_tab_line TO ds_trange_tab_line-frange_t.
6478          ENDIF.
6479        ENDAT.
6480
6481        AT END OF tablename.
6482          IF NOT lt_selection-tablename IS INITIAL.
```

```
6483            ds_trange_tab_line-tablename = lt_selection-tablename.
6484            APPEND ds_trange_tab_line TO fs_dyns-trange.
6485          ENDIF.
6486        ENDAT.
6487      ENDLOOP.
6488
6489      CASE 'X'.                                          "begin "n2954418
6490        WHEN x_opsel.
6491          IF  x_clsel IS INITIAL
6492          AND x_aisel IS INITIAL.
6493            ld_item_status_set = 'X'.
6494          ENDIF.
6495        WHEN x_clsel.
6496          IF  x_opsel IS INITIAL
6497          AND x_aisel IS INITIAL.
6498            ld_item_status_set = 'X'.
6499          ENDIF.
6500        WHEN x_aisel.
6501          IF  x_clsel IS INITIAL
6502          AND x_opsel IS INITIAL.
6503            ld_item_status_set = 'X'.
6504          ENDIF.
6505      ENDCASE.
6506
6507      IF ld_item_status_set IS INITIAL.                  "end "n2954418
6508        IF sd_stida IS NOT INITIAL                         "begin "n2075846
6509      OR pa_stida IS NOT INITIAL.
6510          x_opsel = 'X'.
6511          x_clsel = space.
6512          x_aisel = space.
6513        ELSEIF so_augdt[] IS NOT INITIAL
6514        OR     sd_augdt[] IS NOT INITIAL
6515        OR     pa_stid2   IS NOT INITIAL.
6516          x_opsel = space.
6517          x_clsel = 'X'.
6518          x_aisel = space.
6519        ELSE.
6520          x_opsel = space.
6521          x_clsel = space.
6522          x_aisel = 'X'.
6523        ENDIF.                                             "end "n2075846
6524      ENDIF.                                                     "n2954418
6525
6526      gv_xerfs = x_erfs.                                         "n2437262
6527      IF x_erfs > space.                                         "n2437262
6528        CLEAR gv_relevant_ledger.                                "n2437262
6529      ENDIF.                                                     "n2437262
6530
6531      IF gd_rri_so_set = 'X'.                                    "1093945
6532        EXPORT so_yrper TO MEMORY ID 'FAGLL03_SO_YRPER'.         "1093945
6533      ENDIF.                                                     "1093945
6534
6535      CALL FUNCTION 'FREE_SELECTIONS_RANGE_2_EX'
6536        EXPORTING
6537          field_ranges = fs_dyns-trange
6538        IMPORTING
6539          expressions  = fs_dyns-texpr.
6540
6541      CALL FUNCTION 'FREE_SELECTIONS_RANGE_2_WHERE'
6542        EXPORTING
6543          field_ranges  = fs_dyns-trange
```

```
6544        IMPORTING
6545          where_clauses = fs_dyns-clauses.
6546
6547    ELSE.
6548      IF rri_subrc = 2.
6549        CALL FUNCTION 'RSTI_APPL_STACK_INITIALIZE'
6550          EXPORTING
6551            e_tool = a_rec-rtool
6552            e_onam = a_rec-ronam.
6553      ENDIF.
6554    ENDIF.
6555  ENDFORM.                    " init_rri_data
6556  *&---------------------------------------------------------------------*
6557  *&      Form  get_free_selections
6558  *&---------------------------------------------------------------------*
6559  *       Transfer free selections
6560  *       normally used for extrenal callers like item display module
6561  *---------------------------------------------------------------------*
6562  *  --->  p1        text
6563  *  <---  p2        text
6564  *---------------------------------------------------------------------*
6565  FORM get_free_selections CHANGING
6566                           p_fs_dyns TYPE rsds_type.
6567    p_fs_dyns = fs_dyns.
6568  ENDFORM.                      "get_free_selections
6569
6570  *&---------------------------------------------------------------------*
6571  *&      Form  check_acc_for_trm                                "1004374
6572  *&---------------------------------------------------------------------*
6573  *       Special check for the items only in non-leading Leader
6574  *       with an 'open item' or 'line item' managed account     "1052688
6575  *---------------------------------------------------------------------*
6576  FORM check_trtm_account.
6577    DATA: ls_tablenames TYPE fagl_tabnames,
6578          ld_counter    TYPE i.
6579
6580    IF NOT gv_relevant_ledger IS INITIAL.
6581      IF  gv_relevant_ledger NE gv_leading_ledger
6582      AND ( skb1-xopvw = 'X' OR skb1-xkres = 'X' ).          "1052688
6583
6584        LOG-POINT ID fins_udoc  SUBKEY 'FGL_FAGLFLEXT'.
6585
6586
6587        fins_set_stack_temperaturecold.                      "190814
6588
6589  *  FAGLFLEXT begin ------------------------------------------
6590
6591  *      CALL FUNCTION 'FAGL_GET_TABLENAMES'
6592  *        EXPORTING
6593  *          i_ledger    = gv_relevant_ledger
6594  *        IMPORTING
6595  *          es_tabnames = ls_tablenames.
6596  *
6597  *      SELECT COUNT( * ) FROM (ls_tablenames-tot_table) INTO ld_counter
6598  *                     UP TO 1 ROWS
6599  *                     WHERE racct  = skb1-saknr
6600  *                       AND rbukrs = skb1-bukrs
6601  *                       AND rldnr  = gv_relevant_ledger
6602  *                       AND awtyp  = 'TR-TM'.
6603  *      IF ld_counter = 1.
6604  *        gv_special_account = 'X'.
```

```
6605  *      ENDIF.
6606
6607
6608        DATA: lo_select     TYPE REF TO if_fins_acdoc_select,
6609              lo_cx         TYPE REF TO cx_fins_acdoc_select,
6610              lv_msg        TYPE string,
6611              lt_projection TYPE gusl_t_fields,
6612              ls_projection TYPE dfies-fieldname,
6613              lt_filter     TYPE rsds_trange,
6614              ls_filter     TYPE rsds_range,
6615              l_tablename   TYPE tabname,
6616              lt_frange     TYPE rsds_frange_t,
6617              ls_frange     TYPE rsds_frange,
6618              l_fieldname   TYPE fieldname,
6619              lt_selopt     TYPE rsds_selopt_t,
6620              ls_selopt     TYPE rsdsselopt,
6621              ld_subrc      TYPE sy-subrc,
6622              ld_count      TYPE decfloat34.
6623
6624
6625        CLEAR ls_filter. REFRESH lt_filter.
6626        ls_filter-tablename = space.
6627
6628
6629        CLEAR ls_frange. REFRESH lt_frange.
6630        ls_frange-fieldname = 'RACCT'.
6631        CLEAR ls_selopt. REFRESH lt_selopt.
6632        ls_selopt-sign   = 'I'.
6633        ls_selopt-option = 'EQ'.
6634        ls_selopt-low   =  skb1-saknr.
6635        ls_selopt-high   = space.
6636        APPEND ls_selopt TO lt_selopt.
6637        ls_frange-selopt_t = lt_selopt.
6638        APPEND ls_frange TO lt_frange.
6639
6640        CLEAR ls_frange.
6641        ls_frange-fieldname = 'RBUKRS'.
6642        CLEAR ls_selopt. REFRESH lt_selopt.
6643        ls_selopt-sign   = 'I'.
6644        ls_selopt-option = 'EQ'.
6645        ls_selopt-low   =  skb1-bukrs.
6646        ls_selopt-high   = space.
6647        APPEND ls_selopt TO lt_selopt.
6648        ls_frange-selopt_t = lt_selopt.
6649        APPEND ls_frange TO lt_frange.
6650
6651        CLEAR ls_frange.
6652        ls_frange-fieldname = 'RLDNR'.
6653        CLEAR ls_selopt. REFRESH lt_selopt.
6654        ls_selopt-sign   = 'I'.
6655        ls_selopt-option = 'EQ'.
6656        ls_selopt-low    = gv_relevant_ledger.
6657        ls_selopt-high   = space.
6658        APPEND ls_selopt TO lt_selopt.
6659        ls_frange-selopt_t = lt_selopt.
6660        APPEND ls_frange TO lt_frange.
6661
6662        CLEAR ls_frange.
6663        ls_frange-fieldname = 'AWTYP'.
6664        CLEAR ls_selopt. REFRESH lt_selopt.
6665        ls_selopt-sign   = 'I'.
```

```
6666        ls_selopt-option = 'EQ'.
6667        ls_selopt-low    = 'TR-TM'.
6668        ls_selopt-high   = space.
6669        APPEND ls_selopt TO lt_selopt.
6670        ls_selopt-low    = 'LOANS'.                              "2191278
6671        ls_selopt-high   = space.                                "2191278
6672        APPEND ls_selopt TO lt_selopt.                           "2191278
6673        ls_selopt-low    = 'COPCA'.                              "3314307
6674        ls_selopt-high   = space.                                "3314307
6675        APPEND ls_selopt TO lt_selopt.                           "3314307
6676        ls_frange-selopt_t = lt_selopt.
6677        APPEND ls_frange TO lt_frange.
6678
6679        ls_filter-frange_t = lt_frange.
6680        APPEND ls_filter TO lt_filter.
6681
6682        lo_select ?= cl_fins_acdoc_factory=>create_for_select( ).
6683
6684        TRY.
6685            ld_counter = lo_select->count( it_filter = lt_filter ).
6686
6687          CATCH cx_fins_acdoc_select  INTO lo_cx.
6688
6689
6690        ENDTRY.
6691
6692        IF ld_counter > 0.
6693          gv_special_account = 'X'.
6694        ENDIF.
6695
6696  *  FAGLFLEXT end ------------------------------------------------------
6697
6698      ENDIF.
6699    ENDIF.
6700    cl_fagl_item_select_settings=>get_instance( )->set_account_has_treasur
      y_items( gv_special_account ).
6701  ENDFORM.                    " check_trtm_account                 "1004374
6702
6703  *&---------------------------------------------------------------------*
6704  *&      Form  check_glyec_account
6705  *&---------------------------------------------------------------------*
6706  *       Special check for whether the selection should be switched
6707  *       so the GLYEC postings are also selected
6708  *---------------------------------------------------------------------*
6709  FORM check_glyec_account.
6710    DATA: ls_tablenames TYPE fagl_tabnames,
6711          ld_counter    TYPE i.
6712
6713    IF NOT gv_relevant_ledger IS INITIAL.
6714  *   Check the selection screen field, which is only visible with the
6715  *   related switch active (FAGL_FIN_GL_CP_RS)
6716      IF NOT x_glyec IS INITIAL OR
6717        ( cl_fagl_switch_check=>fagl_fin_gl_cp_rs( ) IS INITIAL
6718          AND gv_relevant_ledger NE gv_leading_ledger ).
6719
6720        LOG-POINT ID fins_udoc  SUBKEY 'FGL_FAGLFLEXT'.
6721
6722        fins_set_stack_temperaturecold.                          "190814
6723
6724  *  FAGLFLEXT begin ----------------------------------------
6725  *
```

```
6726  *      CALL FUNCTION 'FAGL_GET_TABLENAMES'
6727  *        EXPORTING
6728  *          i_ledger    = gv_relevant_ledger
6729  *        IMPORTING
6730  *          es_tabnames = ls_tablenames.
6731  *
6732  *      SELECT COUNT( * ) FROM (ls_tablenames-tot_table) INTO ld_counter
6733  *                     UP TO 1 ROWS
6734  *                     WHERE racct  = skb1-saknr
6735  *                       AND rbukrs = skb1-bukrs
6736  *                       AND rldnr  = gv_relevant_ledger
6737  *                       AND awtyp = 'GLYEC'.
6738  *      IF ld_counter = 1.
6739  *        gv_special_account = 'X'.
6740  *      ENDIF.
6741  *
6742
6743        DATA: lo_select     TYPE REF TO if_fins_acdoc_select,
6744              lo_cx         TYPE REF TO cx_fins_acdoc_select,
6745              lv_msg        TYPE string,
6746              lt_projection TYPE gusl_t_fields,
6747              ls_projection TYPE dfies-fieldname,
6748              lt_filter     TYPE rsds_trange,
6749              ls_filter     TYPE rsds_range,
6750              l_tablename   TYPE tabname,
6751              lt_frange     TYPE rsds_frange_t,
6752              ls_frange     TYPE rsds_frange,
6753              l_fieldname   TYPE fieldname,
6754              lt_selopt     TYPE rsds_selopt_t,
6755              ls_selopt     TYPE rsdsselopt,
6756              ld_subrc      TYPE sy-subrc,
6757              ld_count      TYPE decfloat34.
6758
6759
6760        CLEAR ls_filter. REFRESH lt_filter.
6761        ls_filter-tablename = space.
6762
6763
6764        CLEAR ls_frange. REFRESH lt_frange.
6765        ls_frange-fieldname = 'RACCT'.
6766        CLEAR ls_selopt. REFRESH lt_selopt.
6767        ls_selopt-sign   = 'I'.
6768        ls_selopt-option = 'EQ'.
6769        ls_selopt-low   =  skb1-saknr.
6770        ls_selopt-high   = space.
6771        APPEND ls_selopt TO lt_selopt.
6772        ls_frange-selopt_t = lt_selopt.
6773        APPEND ls_frange TO lt_frange.
6774
6775        CLEAR ls_frange.
6776        ls_frange-fieldname = 'RBUKRS'.
6777        CLEAR ls_selopt. REFRESH lt_selopt.
6778        ls_selopt-sign   = 'I'.
6779        ls_selopt-option = 'EQ'.
6780        ls_selopt-low   =  skb1-bukrs.
6781        ls_selopt-high   = space.
6782        APPEND ls_selopt TO lt_selopt.
6783        ls_frange-selopt_t = lt_selopt.
6784        APPEND ls_frange TO lt_frange.
6785
6786        CLEAR ls_frange.
```

```
6787        ls_frange-fieldname = 'RLDNR'.
6788        CLEAR ls_selopt. REFRESH lt_selopt.
6789        ls_selopt-sign   = 'I'.
6790        ls_selopt-option = 'EQ'.
6791        ls_selopt-low    = gv_relevant_ledger.
6792        ls_selopt-high   = space.
6793        APPEND ls_selopt TO lt_selopt.
6794        ls_frange-selopt_t = lt_selopt.
6795        APPEND ls_frange TO lt_frange.
6796
6797        CLEAR ls_frange.
6798        ls_frange-fieldname = 'AWTYP'.
6799        CLEAR ls_selopt. REFRESH lt_selopt.
6800        ls_selopt-sign   = 'I'.
6801        ls_selopt-option = 'EQ'.
6802        ls_selopt-low    = 'GLYEC'.
6803        ls_selopt-high   = space.
6804        APPEND ls_selopt TO lt_selopt.
6805        ls_frange-selopt_t = lt_selopt.
6806        APPEND ls_frange TO lt_frange.
6807
6808        ls_filter-frange_t = lt_frange.
6809        APPEND ls_filter TO lt_filter.
6810
6811        lo_select ?= cl_fins_acdoc_factory=>create_for_select( ).
6812
6813        TRY.
6814            ld_counter = lo_select->count( it_filter = lt_filter ).
6815
6816          CATCH cx_fins_acdoc_select  INTO lo_cx.
6817
6818        ENDTRY.
6819
6820        IF ld_counter > 0.
6821          gv_special_account = 'X'.
6822        ENDIF.
6823
6824  *  FAGLFLEXT end ------------------------------------------------------------
6825
6826
6827      ENDIF.
6828    ENDIF.
6829    GET PARAMETER ID 'FAGLL03_TEST' FIELD DATA(lv_test_mode).
6830    IF lv_test_mode = abap_true.
6831      GET PARAMETER ID 'FAGLL03_TEST_SPECIAL' FIELD gv_special_account.
6832    ENDIF.
6833    cl_fagl_item_select_settings=>get_instance( )->set_account_has_glyec_i
     tems( gv_special_account ).
6834  ENDFORM.                    " check_glyec_account
6835
6836  INCLUDE fagl_account_items_deco.
6837  *&---------------------------------------------------------------------*
6838  *&      Form  CHECK_SELECTIONS
6839  *&---------------------------------------------------------------------*
6840  *       Check if the number of selections is not large. (note 1578577)
6841  *----------------------------------------------------------------------*
6842  *      -->P_GT_SELECTION_PART  text
6843  *----------------------------------------------------------------------*
6844  FORM check_selections  USING    p_gt_selection_part.
6845
6846  * data declaration
```

```
6847    DATA: lt_selection_check TYPE  gusl_t_selection,
6848          ls_selection_check TYPE  gusl_s_selection,
6849          ld_num             TYPE i.
6850    CONSTANTS:  ld_nummax TYPE i VALUE 500.
6851
6852    GET PARAMETER ID 'FAGLL03_DEL_MAX_SEL' FIELD ld_del_nummax. "1893789
6853
6854    lt_selection_check[] = p_gt_selection_part.
6855
6856    LOOP AT lt_selection_check INTO ls_selection_check.
6857      DESCRIBE TABLE ls_selection_check-t_range LINES ld_num.
6858      IF ( ld_num > ld_nummax ) AND ld_del_nummax IS INITIAL. "1893789
6859        MESSAGE e123.
6860      ENDIF.
6861    ENDLOOP.
6862
6863  ENDFORM.                    " CHECK_SELECTIONS
6864
6865  FORM enforce_outer_join.                                    " 2857175
6866                                                              " 2857175
6867    DATA: lv_enforce_outer_join TYPE abap_bool.               " 2857175
6868                                                              " 2857175
6869    lv_enforce_outer_join = x_enfoj.                          " 2857175
6870  * lv_enforce_outer_join = abap_true.                        " 2857175
6871  * lv_enforce_outer_join = abap_false.                       " 2857175
6872                                                              " 2857175
6873    " This call sequence is nececessary since the             " 2857175
6874    " setting may be superseded by user parameter             " 2857175
6875                                                              " 2857175
6876    CALL FUNCTION 'FAGL_GET_ITEMS_GLOBALS_SET'                " 2857175
6877      EXPORTING                                               " 2857175
6878        iv_enforce_outer_join = lv_enforce_outer_join.        " 2857175
6879                                                              " 2857175
6880    CALL FUNCTION 'FAGL_GET_ITEMS_GLOBALS_GET'                " 2857175
6881      IMPORTING                                               " 2857175
6882        ev_enforce_outer_join = gd_enforce_outer_join.        " 2857175
6883                                                              " 2857175
6884  ENDFORM.                                                    " 2857175
6885
6886  INCLUDE fagl_account_items_forms.
```

```
*&---------------------------------------------------------------------*
*&  Include           FAGL_ACCOUNT_ITEMS_DECO
*&---------------------------------------------------------------------*

*&---------------------------------------------------------------------*
*&      Form  deco_init
*&---------------------------------------------------------------------*
*       text
*----------------------------------------------------------------------*
*      -->I_LEDGER   text
*----------------------------------------------------------------------*
FORM deco_init USING i_ledger TYPE fagl_rldnr.

*  sy-repid = 'FAGL_ACCOUNT_ITEMS_GL'.
  sy-dbnam = 'SDF'.
  sy-calld = 'X'.
  sy-tcode = 'FAGLL03'.
  cl_fin_ui_deco=>set_deco_mode( ).
  gv_deco_ledger = i_ledger.

  CALL FUNCTION 'FIN_UI_DECO_MESSAGE_SET_ID'
    EXPORTING
      iv_msgid = 'MSITEM'.

  PERFORM deco_at_initialization.
  PERFORM deco_at_sel_screen_output.

ENDFORM.                    "init_deco

*&---------------------------------------------------------------------*
*&      Form  supply_selection_deco
*&---------------------------------------------------------------------*
FORM deco_supply_selection USING ir_saknr TYPE rsdsselopt_t
                                 ir_bukrs TYPE rsdsselopt_t
                                 ir_augdt TYPE rsdsselopt_t
                                 ir_budat TYPE rsdsselopt_t
                                 iv_xopsel TYPE itemset-xopsel
                                 iv_stida  TYPE rfpdo-allgstid
                                 iv_xclsel TYPE  itemset-xclsel
                                 iv_stid2  TYPE  rfpdo-allgstid
                                 iv_xaisel TYPE  itemset-xaisel
                                 iv_xnorm TYPE  itemset-xnorm
                                 iv_rldnr TYPE  faglflext-rldnr
                                 iv_erfs  TYPE  boole_d
                                 it_freesel TYPE rsdsfrange_t_ssel.

  DATA:   sel_opt  TYPE rsdsselopt,
          free_sel TYPE rsdsfrange_s_ssel,
          t_range  TYPE rsds_range.

  REFRESH sd_saknr.
  LOOP AT ir_saknr INTO sel_opt.
    MOVE-CORRESPONDING sel_opt TO sd_saknr.
    APPEND sd_saknr.
  ENDLOOP.

  REFRESH sd_bukrs.
  LOOP AT ir_bukrs INTO sel_opt.
    MOVE-CORRESPONDING sel_opt TO sd_bukrs.
    APPEND sd_bukrs.
  ENDLOOP.
```

```
  62
  63    REFRESH so_budat.
  64    LOOP AT ir_budat INTO sel_opt.
  65      MOVE-CORRESPONDING sel_opt TO so_budat.
  66      APPEND so_budat.
  67    ENDLOOP.
  68
  69    REFRESH so_augdt.
  70    LOOP AT ir_augdt INTO sel_opt.
  71      MOVE-CORRESPONDING sel_opt TO so_augdt.
  72      APPEND so_augdt.
  73    ENDLOOP.
  74
  75    x_opsel  = iv_xopsel.
  76    pa_stida = iv_stida.
  77    x_clsel = iv_xclsel.
  78    pa_stid2 = iv_stid2.
  79    x_aisel = iv_xaisel.
  80    x_erfs   = iv_erfs.                                          "2692942
  81
  82  *  sd_budat[] = ir_budat.
  83
  84    x_norm = iv_xnorm.
  85    IF iv_rldnr IS NOT INITIAL.
  86      rldnr = iv_rldnr.
  87    ENDIF.
  88
  89  *---------------------------------------------------------------------*
  90  * Build free selections
  91  *---------------------------------------------------------------------*
  92    IF NOT it_freesel IS INITIAL.
  93      t_range-tablename = gv_si_table && '_FS'.
  94      t_range-frange_t  = it_freesel.
  95      INSERT t_range INTO TABLE fs_dyns-trange.
  96
  97      CALL FUNCTION 'FREE_SELECTIONS_RANGE_2_EX'
  98        EXPORTING
  99          field_ranges = fs_dyns-trange
 100        IMPORTING
 101          expressions  = fs_dyns-texpr.
 102
 103      CALL FUNCTION 'FREE_SELECTIONS_RANGE_2_WHERE'
 104        EXPORTING
 105          field_ranges  = fs_dyns-trange
 106        IMPORTING
 107          where_clauses = fs_dyns-clauses.
 108    ENDIF.
 109
 110  *-------------------------------------
 111    sd_usedb = 'X'.
 112
 113  ENDFORM.                    "supply_selection_deco
 114
 115  *&---------------------------------------------------------------------*
 116  *&      Form  selection_deco
 117  *&---------------------------------------------------------------------*
 118  *       text
 119  *---------------------------------------------------------------------*
 120  FORM deco_selection.
 121
 122    DATA: lt_ldb_dynsel TYPE rsds_texpr,
```

```
123          lt_callback   TYPE TABLE OF ldbcb,
124          ls_callback   LIKE LINE OF lt_callback,
125          lt_selections TYPE TABLE OF rsparams,  "TYPE TABLE OF rsparams.
126          ls_selection  LIKE LINE OF lt_selections.
127
128    PERFORM deco_at_sel_screen_glaccount.
129    PERFORM deco_at_sel_screen_company.
130    PERFORM deco_at_sel_screen.
131    PERFORM deco_at_sel_screen_type.
132
133
134    PERFORM deco_at_selection_start.
135
136    ls_callback-ldbnode  = 'SKB1'.
137    ls_callback-get      = 'X'.
138    ls_callback-cb_prog  = 'FAGL_ACCOUNT_ITEMS_GL'.
139    ls_callback-cb_form  = 'DECO_LDB_CALLBACK'.
140    INSERT ls_callback INTO TABLE lt_callback.
141
142    LOOP AT sd_saknr.
143      ls_selection-selname = 'SD_SAKNR'.
144      ls_selection-kind    = 'S'.
145      MOVE-CORRESPONDING sd_saknr TO ls_selection.
146      INSERT ls_selection INTO TABLE lt_selections.
147    ENDLOOP.
148
149    LOOP AT sd_bukrs.
150      ls_selection-selname = 'SD_BUKRS'.
151      ls_selection-kind    = 'S'.
152      MOVE-CORRESPONDING sd_bukrs TO ls_selection.
153      INSERT ls_selection INTO TABLE lt_selections.
154    ENDLOOP.
155
156    CALL FUNCTION 'LDB_PROCESS'
157      EXPORTING
158        ldbname                     = 'SDF'
159        expressions                 = gs_dyns-texpr
160      TABLES
161        callback                    = lt_callback
162        selections                  = lt_selections
163      EXCEPTIONS
164        ldb_not_reentrant           = 1
165        ldb_incorrect               = 2
166        ldb_already_running         = 3
167        ldb_error                   = 4
168        ldb_selections_error        = 5
169        ldb_selections_not_accepted = 6
170        variant_not_existent        = 7
171        variant_obsolete            = 8
172        variant_error               = 9
173        free_selections_error       = 10
174        callback_no_event           = 11
175        callback_node_duplicate     = 12
176        callback_no_program         = 13
177        callback_no_cbform          = 14
178        dyn_node_no_type            = 15
179        dyn_node_invalid_type       = 16
180        OTHERS                      = 17.
181    IF sy-subrc <> 0.
182      deco_message_sy.
183    ENDIF.
```

```
184
185   PERFORM deco_at_end_of_selection.
186
187 ENDFORM.                    "selection_deco
188
189 *&---------------------------------------------------------------------*
190 *&      Form  get_result_deco
191 *&---------------------------------------------------------------------*
192 *       text
193 *----------------------------------------------------------------------*
194 *      -->ET_FAGLPOSX  text
195 *----------------------------------------------------------------------*
196 FORM deco_get_result TABLES et_faglposx STRUCTURE faglposx.
197
198   et_faglposx[] = it_pos[].
199
200 ENDFORM.                    "get_result_deco
201
202
203 *&---------------------------------------------------------------------*
204 *&      Form  DECO_LDB_CALLBACK
205 *&---------------------------------------------------------------------*
206 FORM deco_ldb_callback USING cb_node LIKE ldbn-ldbnode
207                              cb_skb1 LIKE skb1
208                              cb_mode
209                              cb_selected.
210
211   skb1 = cb_skb1.
212   PERFORM deco_at_selection_get.
213
214 ENDFORM.                    " DECO_LDB_CALLBACK
215
216
217 *&---------------------------------------------------------------------*
218 *&      Form  DECO_CLEAR_GLOBAL_VARIABLES
219 *&---------------------------------------------------------------------*
220 FORM deco_clear_global_variables.
221
222   CLEAR:
223     b0sg,
224     rfxpo,
225     itemset,
226     sscrfields,
227     g_repid,
228     gs_variant,
229     gs_layout,
230     s_epositem,
231     gd_title,
232     gd_save_title,
233     gc_title,
234     gd_expcur_flag,
235     gd_expcur_proc,
236     gd_expcur_bseg,
237     x_stop,
238     x_gl_opvw,
239     gd_saknr_msg,
240     gd_bukrs_msg,
241     n_lines,
242     p_keydate,
243     gd_alvsave,
244     gd_wl_on,
```

```
245       gd_ddsel_on,
246       gt_cosel[], gt_cosel,
247       x_grid,
248       gd_dynp_fun,
249       gd_dynp_val,
250       subrc,
251       x_change,
252       it_t021s[], it_t021s,
253       it_spectab[], it_spectab,
254       it_specfld[], it_specfld,
255       it_movefld[], it_movefld,
256       it_moveref[], it_moveref,
257       it_kkcurr[], it_kkcurr,
258       it_selscreen[], it_selscreen,
259       it_dyn_trange[], it_dyn_trange,
260       it_dyn_texpr[], it_dyn_texpr,
261       it_callback[], it_callback,
262       gt_selscreen[], gt_selscreen,
263       gt_dyn_trange,
264       gt_searchpattern,
265       gt_dyn_texpr,
266       it_central,
267       gt_fieldcat,
268       it_accts[], it_accts,
269       it_comps[], it_comps,
270       wa_pos,
271       it_pos[], it_pos,
272       it_sel,
273       wa_sel,
274       it_tp_auth[], it_tp_auth,
275       it_tp_fields[], it_tp_fields,
276       it_tp_field2[], it_tp_field2,
277       gt_tp_dmbex,
278       gt_bukrs_tp_curtp,
279       gd_tpauth,
280       wa_bkpf,
281       wa_vbkpf,
282       wa_bseg,
283       wa_bseg_add,
284       wa_vbsegd,
285       wa_vbsegk,
286       wa_vbsegs,
287       wa_bsegc,
288       wa_bsec,
289       wa_vbsec,
290       wa_bsed,
291       wa_bsbv,
292       wa_payr,
293       wa_bsegp,
294       wa_x001,
295       gt_spec_fields,
296       pa_stida_default,
297       ybseg[], ybseg,
298       xbseg,
299       ybkpf[], ybkpf,
300       xbkpf,
301       gt_pos_posted,
302       gt_pos_parked,
303       gt_pos_archived,
304       gd_pos_reduced.
305     IF <gfs_pos_reduced> IS ASSIGNED.
```

```
306       CLEAR <gfs_pos_reduced>.
307     ENDIF.
308     CLEAR:
309       gd_read_memo_done,
310       gd_usear,
311       gd_usedb,
312       gd_memor,
313       gd_objec,
314       gd_useas,
315       gt_files[], gt_files,
316       gd_iarch,
317       gd_rri_so_set,
318       gd_ref_to_auth_s,
319       gd_ref_to_auth_c,
320       gd_act_imp_auth_s,
321       gd_act_imp_auth_c,
322       gd_origin,
323       gv_deco_ledger,
324       gs_dyns,
325       gt_add_curr_column_text.
326     CLEAR:
327       skat,
328       skb1,
329       bsis,
330       admi_files,
331       t001,
332       it_h_t001[], it_h_t001,
333       it_h_skat[], it_h_skat,
334       it_h_skb1[], it_h_skb1,
335       gt_saknr[], gt_saknr,
336       gd_saknr,
337       gt_bukrs[], gt_bukrs,
338       gt_gjahr[], gt_gjahr,
339       gd_gjahr,
340       gt_monat[], gt_monat,
341       gd_monat,
342       gt_yrper[], gt_yrper,
343       gd_yrper,
344       gt_yrper_old[], gt_yrper_old,
345       gt_accoco,
346       gt_account_slt,
347       gt_accoco_slt,
348       gd_accoco,
349       fs_f1_icon,
350       fs_f3_icon,
351       fs_f4_icon,
352       gv_relevant_ledger,
353       gv_leading_ledger,
354       gv_si_table,
355       gv_save_si_table,
356       gv_xumsav,
357       gv_xerfs,
358       gv_xrric,
359       gv_ucomm,
360       gv_special_account,
361       gb_flex,
362       gb_xplacc,
363       gd_max_lines.
364     gd_max_pack_lines = '50000'.
365     CLEAR:
366       gd_cursor_open,
```

```
367       gd_xdels,
368       gt_reporting_fields,
369       gs_reporting_fields,
370       gt_dfies_reporting_fields,
371       gt_selection_part,
372       gt_selection_full,
373       gb_xmig,
374       gb_xmig_tpara,
375       gb_xerfs,
376       gb_xuncpl,
377       gt_fagl_mig_001,
378       gs_fagl_mig_001,
379       gs_mig_date_2,
380       gd_alw_waers,
381       gd_alw_date,
382       gt_alw_fieldlist[], gt_alw_fieldlist,
383       gt_log_handle,
384       gs_log_handle,
385       gs_log,
386       gd_xmsg,
387       gd_dummy,
388       gd_xarch,
389       gd_count_arch,
390       gd_count_noauth,
391       l_auth,
392       ld_del_nummax,
393       gd_rri_sender,
394       lv_data_temperature,
395       ld_sknr_dummy,
396       pa_wlsak,
397       so_wlsak[], so_wlsak,
398       pa_wlbuk,
399       so_wlbuk[], so_wlbuk,
400       x_opsel,
401       pa_stida,
402       x_clsel,
403       so_augdt[], so_augdt,
404       pa_stid2,
405       x_aisel,
406       so_budat[], so_budat,
407       x_norm,
408       x_erfs,
409       x_shbv,
410       x_merk,
411       x_park,
412       x_apar,
413       rldnr,
414       x_glyec,
415       pa_bwber,
416       pa_vari,
417       pa_nmax,
418       pa_cent,
419       so_faedt[], so_faedt,
420       gp_yrper,
421       so_yrper[], so_yrper,
422       pa_inet,
423       pa_grid.
424     CLEAR:
425       so_bwwrt[], so_bwwrt,
426       so_bwwr2[], so_bwwr2,
427       so_bwwr3[], so_bwwr3,
```

```
428       so_dmshb[], so_dmshb,
429       so_hwaer[], so_hwaer,
430       so_dmbe2[], so_dmbe2,
431       so_hwae2[], so_hwae2,
432       so_dmbe3[], so_dmbe3,
433       so_hwae3[], so_hwae3,
434       so_jamon[], so_jamon,
435       so_koart[], so_koart,
436       so_qbshb[], so_qbshb,
437       so_qsfbt[], so_qsfbt,
438       so_qsshb[], so_qsshb,
439       so_skfbt[], so_skfbt,
440       so_wskto[], so_wskto,
441       so_sknto[], so_sknto,
442       so_verz1[], so_verz1,
443       so_verzn[], so_verzn,
444       so_wrshb[], so_wrshb,
445       so_waers[], so_waers,
446       so_zaldt[], so_zaldt,
447       so_zinsz[], so_zinsz,
448       so_askto[], so_askto,
449       so_kurse[], so_kurse,
450       so_kkbwr[], so_kkbwr,
451       so_gsber[], so_gsber,
452       so_shkzg[], so_shkzg,
453       fs_dyns.
454
455  ENDFORM.
```

```
1  *---------------------------------------------------------------------*
2  *   INCLUDE FAGL_ACCOUNT_ITEMS_DEF                                    *
3  *---------------------------------------------------------------------*
4
5  TYPE-POOLS: slis, tpit, rsds, gusl.
6
7  TABLES: b0sg, rfxpo, itemset.
8  TABLES: sscrfields.
9
10 DATA: g_repid    LIKE sy-repid,
11       gs_variant LIKE disvariant,
12       gs_layout  TYPE slis_layout_alv,
13       s_epositem TYPE tpit_epositem.
14 DATA: gd_title      LIKE sy-title,
15       gd_save_title LIKE sy-title,
16       gc_title      TYPE gui_title VALUE '%_T'.
17
18 * expiring currencies:
19 DATA: gd_expcur_flag TYPE c,
20       gd_expcur_proc TYPE tprcproc,
21       gd_expcur_bseg TYPE c.
22
23 DATA: x_stop       TYPE c,
24       x_gl_opvw    TYPE c,
25       gd_saknr_msg TYPE saknr,
26       gd_bukrs_msg TYPE bukrs,
27       n_lines      LIKE sy-tabix,
28       p_keydate    LIKE sy-datum,
29       gd_alvsave   TYPE c VALUE 'A',
30       gd_wl_on     TYPE c,
31       gd_ddsel_on  TYPE c,
32       gt_cosel     TYPE tcosel WITH HEADER LINE,
33       x_grid       TYPE c,
34       gd_dynp_fun  TYPE i,
35       gd_dynp_val  TYPE i,
36       subrc        LIKE sy-subrc,
37       x_change     TYPE c.
38
39 DATA: it_t021s         LIKE t021s OCCURS 1     WITH HEADER LINE,
40       it_spectab       TYPE tpit_t_tabstat     WITH HEADER LINE,
41       it_specfld       TYPE tpit_t_fldstat     WITH HEADER LINE,
42       it_movefld       TYPE tpit_t_fldmove     WITH HEADER LINE,
43       it_moveref       TYPE tpit_t_refmove     WITH HEADER LINE,
44       it_kkcurr        TYPE tpit_t_kkcurr      WITH HEADER LINE,
45       it_selscreen     LIKE rsparams OCCURS 10 WITH HEADER LINE,
46       it_dyn_trange    TYPE rsds_trange        WITH HEADER LINE,
47       it_dyn_texpr     TYPE rsds_texpr         WITH HEADER LINE,
48       it_callback      LIKE ldbcb OCCURS 10    WITH HEADER LINE,
49       gt_selscreen     LIKE rsparams OCCURS 10,
50       gt_dyn_trange    TYPE rsds_trange,
51       gt_searchpattern TYPE syldb_sp,
52       gt_dyn_texpr     TYPE rsds_texpr,
53       it_central       TYPE tpit_t_central     WITH HEADER LINE,
54       gt_fieldcat      TYPE slis_t_fieldcat_alv WITH HEADER LINE.
55
56 *... accounts and companies tables, needed for open FI:
57 DATA: it_accts TYPE tpit_t_rfepk   WITH HEADER LINE,
58       it_comps TYPE tpit_t_rfepb   WITH HEADER LINE.
59
60 DATA: wa_pos LIKE faglposx,
61       it_pos LIKE faglposx OCCURS 1 WITH HEADER LINE.
```

```
  62  DATA: it_sel         TYPE faglpose_t.
  63  DATA: wa_sel         TYPE faglpose.
  64
  65  *... tables needed for transfer prices:
  66  DATA: it_tp_auth   TYPE tpit_t_tp_auth_t001a WITH HEADER LINE,
  67        it_tp_fields TYPE tpit_t_tpfields WITH HEADER LINE,
  68        it_tp_field2 TYPE tpit_t_tpfields WITH HEADER LINE.
  69  DATA  gt_tp_dmbex  TYPE tpit_t_tpfields. "Transfer Pricing authority che
      ck
  70  DATA  gt_bukrs_tp_curtp TYPE cl_fins_acdoc_util=>tt_bukrs_tp_curtp. "Tra
      nsfer Pricing authority check
  71
  72  DATA: gd_tpauth      TYPE c.
  73
  74  DATA: wa_acdoca   like acdoca,
  75        wa_bkpf     LIKE bkpf,
  76        wa_vbkpf    LIKE vbkpf,
  77        wa_bseg     LIKE bseg,
  78        wa_bseg_add LIKE bseg_add,                                  "1117587
  79        wa_vbsegd   LIKE vbsegd,
  80        wa_vbsegk   LIKE vbsegk,
  81        wa_vbsegs   LIKE vbsegs,
  82        wa_bsegc    LIKE bsegc,
  83        wa_bsec     LIKE bsec,
  84        wa_vbsec    LIKE vbsec,
  85        wa_bsed     LIKE bsed,
  86        wa_bsbv     LIKE bsbv,
  87        wa_payr     LIKE payr,
  88        wa_bsegp    LIKE bsegp,
  89        wa_x001     LIKE x001.
  90
  91  * Table for optimized reading of special fields.
  92  * Same structure as analogous table (TRATAB) in the old line item
  93  * display (RFEPOS00)
  94  TYPES: BEGIN OF type_spec_fields,
  95           fname TYPE dfies-lfieldname,
  96         END OF type_spec_fields.
  97  DATA: gt_spec_fields TYPE STANDARD TABLE OF type_spec_fields
  98                       WITH KEY fname.
  99
 100  * Copy of pa_stida for fixing keydates.
 101  DATA: pa_stida_default LIKE sy-datum.
 102
 103  CONSTANTS:
 104        c_guicnt       like sy-tabix    value 10000,              "2715970
 105    c_repid_ar LIKE sy-repid
 106                         VALUE 'FAGL_ACCOUNT_ITEMS_AR',
 107    c_repid_ap LIKE sy-repid
 108                         VALUE 'FAGL_ACCOUNT_ITEMS_AP',
 109    c_repid_gl LIKE sy-repid
 110                         VALUE 'FAGL_ACCOUNT_ITEMS_GL',
 111    c_koart_ar LIKE bseg-koart  VALUE 'D',
 112    c_koart_ap LIKE bseg-koart  VALUE 'K',
 113    c_koart_gl LIKE bseg-koart  VALUE 'S',
 114    c_true     TYPE c           VALUE 'X',
 115    c_false    TYPE c           VALUE space,
 116    c_x        TYPE c           VALUE 'X'.
 117
 118
 119  * Tables for reading from archive:
 120  DATA: ybseg TYPE bseg OCCURS 10,
```

```
121       xbseg TYPE HASHED TABLE OF bseg
122             WITH UNIQUE KEY bukrs belnr gjahr buzei,
123       ybkpf TYPE bkpf OCCURS 10,
124       xbkpf TYPE HASHED TABLE OF bkpf
125             WITH UNIQUE KEY bukrs belnr gjahr.
126
127 TYPES : BEGIN OF gty_pos_reduced,
128           bukrs TYPE skb1-bukrs,
129           belnr TYPE bseg-belnr,
130           gjahr TYPE bseg-gjahr,
131           buzei TYPE bseg-buzei,
132           budat TYPE bkpf-budat,
133           bstat TYPE bkpf-bstat,                              "1117587
134           docln TYPE faglpose-docln,                          "1237389
135           rldnr type acdoca-rldnr,                            "2521327
136         END OF gty_pos_reduced.
137 DATA: gt_pos_posted   TYPE SORTED TABLE OF gty_pos_reduced
138                        WITH UNIQUE KEY                      "1237389
139                           bukrs belnr gjahr buzei budat docln, "1237389
140       gt_pos_parked   TYPE SORTED TABLE OF gty_pos_reduced
141                        WITH UNIQUE KEY bukrs belnr gjahr buzei budat,
142       gt_pos_archived TYPE SORTED TABLE OF gty_pos_reduced
143                        with unique key
144                           bukrs belnr gjahr buzei budat docln, "2622861
145       gd_pos_reduced  TYPE gty_pos_reduced.
146
147 FIELD-SYMBOLS : <gfs_pos_reduced> TYPE gty_pos_reduced.
148
149 DATA: gd_read_memo_done  TYPE c.
150
151 DATA: gd_usear TYPE boole_d,
152       gd_usedb TYPE boole_d,
153       gd_memor TYPE c.
154
155 DATA: gd_objec TYPE arch_obj-object,
156 *       gd_usedb  TYPE boole_d,
157 *       gd_usear  TYPE boole_d
158       gd_useas TYPE boole_d.
159 *       gd_memor  TYPE C
160 RANGES: gt_files FOR admi_files-archiv_key.
161 DATA:   gd_iarch  TYPE c.
162 DATA:   gd_rri_so_set TYPE boolean.                           "1093945
163
164 *---------------------------------------------------------------------*
165 *    Class for Business Add Ins (BADI)                                *
166 *---------------------------------------------------------------------*
167 CLASS cl_exithandler DEFINITION LOAD.
168
169 * "Interfacereferenz
170 DATA: gd_ref_to_auth_s TYPE REF TO if_ex_fagl_auth_add_data_s.
171 DATA: gd_ref_to_auth_c TYPE REF TO if_ex_fagl_auth_add_data_c.
172 DATA: gd_act_imp_auth_s TYPE c.
173 DATA: gd_act_imp_auth_c TYPE c.
174
175 DATA: gd_origin TYPE rsdswhere-line.
176
177 *---------------------------------------------------------------------*
178 *    Decoupling: Additional Global Variables                          *
179 *---------------------------------------------------------------------*
180 DATA: gv_deco_ledger TYPE fagl_rldnr,
181       gs_dyns        TYPE rsds_type.
```

```
182
183 *_____________________________________________________________________*
184 *    Freely defined currencies: global table for column texts          *
185 *_____________________________________________________________________*
186 TYPES: BEGIN OF ty_add_curr_column_text,
187          fieldname TYPE fieldname,
188          seltext_l LIKE dd03p-scrtext_l,
189          seltext_m LIKE dd03p-scrtext_m,
190          seltext_s LIKE dd03p-scrtext_s,
191        END OF ty_add_curr_column_text.
192
193 DATA gt_add_curr_column_text TYPE TABLE OF ty_add_curr_column_text.
```

```
*---------------------------------------------------------------------*
***INCLUDE FAGL_ACCOUNT_ITEMS_FORMS.
*---------------------------------------------------------------------*
*&---------------------------------------------------------------------*
*& Form FILL_TP_DMBEX
*&---------------------------------------------------------------------*
*& text
*&---------------------------------------------------------------------*
*& --->  p1        text
*& <---  p2        text
*&---------------------------------------------------------------------*
FORM fill_tp_dmbex .



ENDFORM.

*&---------------------------------------------------------------------*
*& Class ltc_select_jtems
*&---------------------------------------------------------------------*
*&
*&---------------------------------------------------------------------*
CLASS ltc_select_jtems DEFINITION for  testing FINAL.

ENDCLASS.
```

```
*&---------------------------------------------------------------------*
*&      Form  ALV_VARIANT_F4
*&---------------------------------------------------------------------*
FORM alv_variant_f4 CHANGING pa_vari.
  DATA: rs_variant LIKE disvariant.
  DATA nof4 TYPE c.

  CLEAR nof4.
  LOOP AT SCREEN.
    IF screen-name = 'PA_VARI'.
      IF screen-input = 0.
        nof4 = 'X'.
      ENDIF.
    ENDIF.
  ENDLOOP.

  rs_variant-report   = g_repid.
  rs_variant-username = sy-uname.
  CALL FUNCTION 'REUSE_ALV_VARIANT_F4'
    EXPORTING
      is_variant = rs_variant
      i_save     = 'A'
    IMPORTING
      es_variant = rs_variant
    EXCEPTIONS
      OTHERS     = 1.
  IF sy-subrc = 0 AND nof4 EQ space.
    pa_vari = rs_variant-variant.
  ENDIF.
ENDFORM.                                    " ALV_VARIANT_F4

*&---------------------------------------------------------------------*
*&      Form  CB_ITEM_ALV_USERCOMM
*&---------------------------------------------------------------------*
FORM cb_item_alv_usercomm  USING  r_ucomm     LIKE sy-ucomm
                                  rs_selfield TYPE slis_selfield.
  DATA: rt_fieldcat TYPE slis_t_fieldcat_alv.
*  data: Lt_fieldcat type slis_t_fieldcat_alv.
  DATA: ld_refresh_count TYPE i,
        ld_memory_id(40) TYPE c.
  DATA: ls_variant        TYPE disvariant,
        lt_selscreen      LIKE rsparams OCCURS 5 WITH HEADER LINE,
        ld_memory_id2(40) TYPE c.

  CASE r_ucomm.
    WHEN '&OL0' OR '&OAD' OR '&OLX'.
*.... ALV change display variant. get new field catalog:
      IF x_grid IS INITIAL.
        CALL FUNCTION 'REUSE_ALV_LIST_LAYOUT_INFO_GET'
          IMPORTING
            et_fieldcat   = rt_fieldcat
          EXCEPTIONS
            no_infos      = 1
            program_error = 2
            OTHERS        = 3.
      ELSE.
        CALL FUNCTION 'REUSE_ALV_GRID_LAYOUT_INFO_GET'
          IMPORTING
            et_fieldcat   = rt_fieldcat
          EXCEPTIONS
            no_infos      = 1
```

```
 62              program_error = 2
 63              OTHERS        = 3.
 64        ENDIF.
 65        IF sy-subrc <> 0.
 66          EXIT.
 67        ENDIF.
 68        PERFORM analyze_act_fieldcat  TABLES rt_fieldcat.
 69  *     treat new special fields:
 70        PERFORM read_special_tables.
 71      WHEN 'REFR' OR 'LTOG'.
 72  *.... list refresh:
 73        IF x_grid IS INITIAL.
 74          CALL FUNCTION 'REUSE_ALV_LIST_LAYOUT_INFO_GET'
 75            IMPORTING
 76              es_variant    = ls_variant
 77            EXCEPTIONS
 78              no_infos      = 1
 79              program_error = 2
 80              OTHERS        = 3.
 81        ELSE.
 82          CALL FUNCTION 'REUSE_ALV_GRID_LAYOUT_INFO_GET'
 83            IMPORTING
 84              es_variant    = ls_variant
 85            EXCEPTIONS
 86              no_infos      = 1
 87              program_error = 2
 88              OTHERS        = 3.
 89        ENDIF.
 90        IF sy-subrc = 0 AND NOT ls_variant-variant IS INITIAL.
 91          ld_memory_id2 = g_repid.
 92          WRITE '/LAYO' TO ld_memory_id2+35(5).
 93          CONDENSE ld_memory_id2 NO-GAPS.
 94          EXPORT layout FROM ls_variant-variant
 95                  TO MEMORY ID ld_memory_id2.
 96        ENDIF.
 97        ld_memory_id = g_repid.
 98        WRITE '/REFR' TO ld_memory_id+35(5).
 99        CONDENSE ld_memory_id NO-GAPS.
100        IMPORT counter TO ld_refresh_count
101               FROM MEMORY ID ld_memory_id.
102        ADD 1 TO ld_refresh_count.
103        EXPORT counter FROM ld_refresh_count
104               TO MEMORY ID ld_memory_id.
105        IF ld_refresh_count > 1.
106          PERFORM deco_leave_program.
107        ENDIF.
108        WHILE ld_refresh_count > 0.
109          CLEAR ls_variant.
110          lt_selscreen[] = gt_selscreen[].
111          IMPORT layout TO ls_variant-variant
112                 FROM MEMORY ID ld_memory_id2.
113          IF sy-subrc = 0 AND NOT ls_variant-variant IS INITIAL.
114            READ TABLE lt_selscreen WITH KEY selname = 'PA_VARI'.
115            IF sy-subrc = 0.
116              lt_selscreen-low = ls_variant-variant.
117              MODIFY lt_selscreen INDEX sy-tabix.
118            ENDIF.
119          ENDIF.
120          FREE it_h_t001.                                    "2756111
121          FREE it_h_skat.                                    "2756111
122          FREE it_h_skb1.                                    "2756111
```

```
123           FREE it_pos.                                          "2756111
124           SUBMIT (g_repid) WITH SELECTION-TABLE lt_selscreen
125   *                        with free selections gt_dyn_texpr
126                            WITH fs_dyns  EQ fs_dyns
127                            WITH kd_index EQ gt_searchpattern
128                            WITH dd_index EQ gt_searchpattern
129                            WITH sd_index EQ gt_searchpattern
130                            AND RETURN.
131           CLEAR ld_refresh_count.
132           IMPORT counter TO ld_refresh_count
133                  FROM MEMORY ID ld_memory_id.
134           IF ld_refresh_count > 0.
135             SUBTRACT 1 FROM ld_refresh_count.
136           ENDIF.
137           EXPORT counter FROM ld_refresh_count
138                  TO MEMORY ID ld_memory_id.
139         ENDWHILE.
140         IF gd_dynp_val = 0.
141           SUBMIT (g_repid) WITH SELECTION-TABLE gt_selscreen
142                            WITH FREE SELECTIONS fs_dyns-texpr "2085656
143                            WITH fs_dyns  EQ fs_dyns           "2085656
144                            WITH fs_field EQ fs_field          "2085656
145                            WITH kd_index EQ gt_searchpattern
146                            WITH dd_index EQ gt_searchpattern
147                            WITH sd_index EQ gt_searchpattern
148                            VIA SELECTION-SCREEN.
149         ELSE.
150           PERFORM deco_leave_program.
151         ENDIF.
152
153     ENDCASE.
154
155   ENDFORM.                                 " CB_ITEM_ALV_USERCOMM
156
157   *&---------------------------------------------------------------------*
158   *&      Form  SPECIAL_FIELDS
159   *&---------------------------------------------------------------------*
160   FORM special_fields_init USING   rs_variant LIKE disvariant.
161     DATA: it_var_fieldcat TYPE slis_t_fieldcat_alv WITH HEADER LINE,
162           it_max_fieldcat TYPE slis_t_fieldcat_alv,
163           dummy_layout    TYPE slis_layout_alv.
164
165   * ... get existing variant:
166     PERFORM get_existing_variant USING rs_variant.
167   *... get fieldcat of initial variant:
168     PERFORM get_initial_fieldcat   TABLES it_var_fieldcat
169                                    USING  rs_variant.
170   *... analyze initial fieldcat:
171     PERFORM analyze_act_fieldcat  TABLES it_var_fieldcat.
172   *... read required special tables and fill item fields:
173     PERFORM read_special_tables.
174
175   ENDFORM.                                 " SPECIAL_FIELDS_INIT
176
177   *&---------------------------------------------------------------------*
178   *&      Form GET_INITIAL_FIELDCAT
179   *&---------------------------------------------------------------------*
180   FORM get_initial_fieldcat  TABLES rt_var_fieldcat
181                                     TYPE slis_t_fieldcat_alv
182                              USING  rs_variant LIKE disvariant.
183     DATA: it_max_fieldcat TYPE slis_t_fieldcat_alv,
```

```
184          dummy_layout    TYPE slis_layout_alv,
185          ls_variant      TYPE disvariant,
186          ld_exit         TYPE c.
187
188    ls_variant = rs_variant.
189
190  *... build maximal fieldcat:
191    CALL FUNCTION 'REUSE_ALV_FIELDCATALOG_MERGE'
192      EXPORTING
193        i_buffer_active        = 'X'
194        i_program_name         = g_repid
195        i_structure_name       = 'FAGLPOSX'
196      CHANGING
197        ct_fieldcat            = it_max_fieldcat
198      EXCEPTIONS
199        inconsistent_interface = 1
200        program_error          = 2
201        OTHERS                 = 3.
202    IF sy-subrc <> 0.
203      deco_message_0 a011.
204    ENDIF.
205  *... get variant fieldcat:
206    CALL FUNCTION 'REUSE_ALV_VARIANT_SELECT'
207      EXPORTING
208        i_dialog            = ' '
209        i_user_specific     = 'X'
210        it_default_fieldcat = it_max_fieldcat
211        i_layout            = dummy_layout
212        i_buffer_active     = 'X'
213      IMPORTING
214        e_exit              = ld_exit
215        et_fieldcat         = rt_var_fieldcat[]
216      CHANGING
217        cs_variant          = ls_variant
218      EXCEPTIONS
219        wrong_input         = 1
220        fc_not_complete     = 2
221        not_found           = 3
222        program_error       = 4
223        OTHERS              = 5.
224    IF sy-subrc <> 0.
225  *... take maximal fieldcat for special fields:
226      rt_var_fieldcat[] = it_max_fieldcat[].
227    ENDIF.
228
229  ENDFORM.                                " GET_INITIAL_FIELDCAT
230
231  *&---------------------------------------------------------------------*
232  *&      Form  INIT_ADMIN_TABLES
233  *&---------------------------------------------------------------------*
234  FORM init_admin_tables.
235    DATA: field_n   LIKE dd03p-fieldname,
236          field_n_2 LIKE dd03p-fieldname,
237          table_n   LIKE dd03p-tabname.
238    FIELD-SYMBOLS: <fsym>.
239    DATA: ld_object       TYPE ddobjname,
240          ld_field        LIKE dfies-lfieldname,
241          ls_dfies        TYPE dfies,
242          ld_reffield(10) TYPE c.
243
244
```

```
245
246  *... first get T021S:
247    SELECT * FROM t021s INTO TABLE it_t021s WHERE lstcl = 'D'.
248
249
250    DATA: lr_struct        TYPE REF TO cl_abap_structdescr.   "n2570981
251    TRY.                                                     "n2570981
252        lr_struct ?= cl_abap_typedescr=>describe_by_name( 'ACDOCA' ). "n25
     70981
253        DATA(lt_fields) = lr_struct->get_ddic_field_list( ).  "n2570981
254      CATCH cx_root.                                         "n2570981
255  *       Should not occur                                          "n2
     570981
256    ENDTRY.                                                  "n2570981
257    SORT lt_fields BY fieldname.                             "n2570981
258
259
260  * Add ACDOCA for G/L documents only
261    LOOP AT it_t021s INTO DATA(ls_t021s)
262                         WHERE tname = 'BSEG'.
263
264      READ TABLE lt_fields TRANSPORTING NO FIELDS            "n2570981
265                           WITH KEY fieldname = ls_t021s-fname "n2570981
266                           BINARY SEARCH.                    "n2570981
267      CHECK sy-subrc EQ 0.                                   "n2570981
268
269      ls_t021s-tname = 'ACDOCA'.
270      APPEND ls_t021s TO it_t021s.
271    ENDLOOP.
272
273  * Add field for bill of exchange problems.
274    PERFORM init_admin_tables_boe.
275    SORT it_t021s BY tname fname.
276    DELETE ADJACENT DUPLICATES FROM it_t021s COMPARING tname fname.
277  *... now make other administration tables:
278    CLEAR it_spectab.
279    CLEAR it_specfld.
280    CLEAR it_movefld.
281    CLEAR it_moveref.
282    LOOP AT it_t021s.
283      CHECK NOT it_t021s-tname IS INITIAL.
284  *... status table of special tables:
285      WRITE it_t021s-tname TO table_n.
286      it_spectab-tname = table_n.
287      APPEND it_spectab.
288      IF it_t021s-tname = 'BSEG'.                            "1117587
289        it_spectab-tname = 'BSEG_ADD'.                       "1117587
290      ENDIF.                                                 "1117587
291      APPEND it_spectab.                                     "1117587
292  *   expiring currencies:
293      IF it_t021s-tname = 'BSEG'.
294        ld_object = it_t021s-tname.
295        ld_field  = it_t021s-fname.
296        CALL FUNCTION 'DDIF_NAMETAB_GET'
297          EXPORTING
298            tabname    = ld_object
299            lfieldname = ld_field
300          IMPORTING
301            dfies_wa   = ls_dfies
302          EXCEPTIONS
303            not_found  = 1
```

```
304           OTHERS     = 2.
305       IF sy-subrc = 0.
306         CLEAR ld_reffield.
307         WRITE ls_dfies-reftable TO ld_reffield.
308         WRITE '-'               TO ld_reffield+4.
309         WRITE ls_dfies-reffield TO ld_reffield+5.
310         IF  ls_dfies-datatype = 'CURR'
311         AND ( ld_reffield = 'BKPF-WAERS' OR
312               ld_reffield = 'BSEG-PYCUR' ).
313           gd_expcur_bseg = 'X'.
314         ENDIF.
315       ENDIF.
316     ENDIF.
317 *... status table of field names:
318     it_specfld-tname = table_n.
319     WRITE it_t021s-fname TO field_n.
320     it_specfld-fname = field_n.
321     PERFORM extnd_fieldname USING    field_n
322                             CHANGING it_specfld-fname_2.
323     APPEND it_specfld.
324 *... move map table:
325     it_movefld-tname = table_n.
326     WRITE it_t021s-fname TO field_n.
327     PERFORM extnd_fieldname USING    field_n
328                             CHANGING field_n_2.
329 *   original table work area field:
330     WRITE 'WA_'   TO it_movefld-tabfld.
331     WRITE table_n TO it_movefld-tabfld+4.
332     WRITE '-'     TO it_movefld-tabfld+14.                  "2614094
333     WRITE field_n TO it_movefld-tabfld+15.                  "2614094
334     CONDENSE it_movefld-tabfld NO-GAPS.
335 *   item table work area field:
336     WRITE 'IT_POS'   TO it_movefld-tabfld_2.
337     WRITE '-'        TO it_movefld-tabfld_2+14.             "2614094
338     WRITE field_n_2  TO it_movefld-tabfld_2+15.             "2614094
339     CONDENSE it_movefld-tabfld_2 NO-GAPS.
340     APPEND it_movefld.
341 *... move reference table (parallel to move map table):
342     it_moveref-tname = it_movefld-tname.
343     ASSIGN (it_movefld-tabfld) TO <fsym>.
344     GET REFERENCE OF <fsym> INTO it_moveref-fldref.
345     ASSIGN (it_movefld-tabfld_2) TO <fsym>.
346     GET REFERENCE OF <fsym> INTO it_moveref-fldref_2.
347     APPEND it_moveref.
348 *... that's it.
349 *...Special case for the non leading ledger special fields     "1117587
350     IF table_n = 'BSEG'.
351 *...  move map table:
352       it_movefld-tname = 'BSEG_ADD'.
353       table_n          = 'BSEG_ADD'.
354       WRITE it_t021s-fname TO field_n.
355       PERFORM extnd_fieldname USING    field_n
356                               CHANGING field_n_2.
357 *     Special case for FKBER_LONG, as in BSEG_ADD, this is     "1684946
358 *     defined as FKBER                                         "1684946
359       IF field_n = 'FKBER_LONG'.                              "1684946
360         CLEAR field_n.                                        "1684946
361         field_n = 'FKBER'.                                    "1684946
362       ENDIF.                                                  "1684946
363 *...  original table work area field:
364       WRITE 'WA_'   TO it_movefld-tabfld.
```

```
365         WRITE table_n TO it_movefld-tabfld+4.
366         WRITE '-'     TO it_movefld-tabfld+19.
367         WRITE field_n TO it_movefld-tabfld+20.
368         CONDENSE it_movefld-tabfld NO-GAPS.
369   *...  item table work area field:
370         WRITE 'IT_POS'   TO it_movefld-tabfld_2.
371         WRITE '-'        TO it_movefld-tabfld_2+19.
372         WRITE field_n_2  TO it_movefld-tabfld_2+20.
373         CONDENSE it_movefld-tabfld_2 NO-GAPS.
374         APPEND it_movefld.
375   *...  move reference table (parallel to move map table):
376         it_moveref-tname = it_movefld-tname.
377         ASSIGN (it_movefld-tabfld) TO <fsym>.
378         GET REFERENCE OF <fsym> INTO it_moveref-fldref.
379         ASSIGN (it_movefld-tabfld_2) TO <fsym>.
380         GET REFERENCE OF <fsym> INTO it_moveref-fldref_2.
381         APPEND it_moveref.
382   *...  that's it.
383       ENDIF.
384   *...Special case for the non leading ledger special fields      "1117587
385     ENDLOOP.
386   *... document text fields:
387     it_spectab-tname = '_TEXT'.
388     APPEND it_spectab.
389
390     SORT it_spectab.                                            "2574355
391     DELETE ADJACENT DUPLICATES FROM it_spectab.
392
393   ENDFORM.                                " INIT_ADMIN_TABLES
394
395   *&---------------------------------------------------------------------*
396   *&      Form  EXTND_FIELDNAME
397   *&---------------------------------------------------------------------*
398   FORM extnd_fieldname USING    p_fieldn
399                        CHANGING p_fieldn_ext.
400     DATA: text(40)  TYPE c.
401
402     WRITE 'U_' TO text.
403     WRITE p_fieldn TO text+2.
404     WRITE text TO p_fieldn_ext.
405
406   ENDFORM.                                " EXTND_FIELDNAME
407
408   *&---------------------------------------------------------------------*
409   *&      Form  ANALYZE_ACT_FIELDCAT
410   *&      Updating status tables it_specfld and it_spectab.
411   *&---------------------------------------------------------------------*
412   FORM analyze_act_fieldcat TABLES   rt_var_fieldcat
413                                      TYPE slis_t_fieldcat_alv.
414     DATA: prefix(2)    TYPE c,
415           ld_lines     TYPE i,
416           ld_count     TYPE i,
417           ld_bukrs(25) TYPE c.
418
419   *  statics ld_supress_msg(1) type c.
420     DATA    ld_noauth(1)      TYPE c.
421     DATA lv_no_transfer_price_auth TYPE abap_bool. "Transfer Pricing autho
      rity check
422
423     DATA: lt_specfld   TYPE tpit_t_fldstat.
424
```

```
425  * company code with tranfer prices
426    DESCRIBE TABLE it_tp_auth LINES ld_lines.
427
428    LOOP AT rt_var_fieldcat.
429  *   field displayed in variant?
430      IF rt_var_fieldcat-no_out = 'X'.
431  *   or rt_var_fieldcat-tech   = 'X'.
432        CONTINUE.
433      ENDIF.
434  *   company code with tp
435  *    if ld_lines gt 0 and ld_supress_msg = space.
436      IF ld_lines GT 0.
437  *     tp fields 2. HW
438        READ TABLE it_tp_fields WITH KEY
439                                fname = rt_var_fieldcat-fieldname.
440        IF sy-subrc EQ 0.
441  *       check authority
442  *       ld_count = 0. ld_bukrs = space.
443  *       loop at it_tp_auth where auth = space.
444  *          concatenate ld_bukrs it_tp_auth-bukrs into ld_bukrs
445  *                                                 separated by space.
446  *          ld_count = ld_count + 1.
447  *          if ld_count > 3. exit. endif.
448  *        endloop.
449  *        if ld_count > 0.
450  *          ld_supress_msg = 'X'.
451  *          message i036 with ld_bukrs.
452  *        endif.
453  *       endloop.
454          CASE gd_tpauth.
455            WHEN space.
456              IF rt_var_fieldcat-tech IS INITIAL.
457                rt_var_fieldcat-tech = 'X'.
458                MODIFY rt_var_fieldcat.
459              ENDIF.
460              ld_noauth = 'X'.
461            WHEN '2'.
462              IF rt_var_fieldcat-tech IS INITIAL.
463                rt_var_fieldcat-tech = 'X'.
464                MODIFY rt_var_fieldcat.
465              ENDIF.
466              IF x_erfs IS INITIAL.                              "3251156
467                ld_noauth = '1'.
468              ENDIF.                                             "3251156
469            ENDCASE.
470        ENDIF.
471  * tp fields 3. HW
472        READ TABLE it_tp_field2 WITH KEY
473                                  fname = rt_var_fieldcat-fieldname.
474        IF sy-subrc EQ 0.
475          CASE gd_tpauth.
476            WHEN space.
477              IF rt_var_fieldcat-tech IS INITIAL.
478                rt_var_fieldcat-tech = 'X'.
479                MODIFY rt_var_fieldcat.
480              ENDIF.
481              ld_noauth = 'X'.
482            WHEN '1'.
483              IF rt_var_fieldcat-tech IS INITIAL.
484                rt_var_fieldcat-tech = 'X'.
485                MODIFY rt_var_fieldcat.
```

```
486                IF x_erfs IS INITIAL.                                "3251156
487                  ld_noauth = '2'.
488                ENDIF.                                               "3251156
489              ENDIF.
490          ENDCASE.
491        ENDIF.
492      ENDIF.
493
494  *   document text field?
495      IF rt_var_fieldcat-fieldname = 'TDID' OR
496         rt_var_fieldcat-fieldname = 'XTEXT'.
497        READ TABLE it_spectab WITH KEY tname = '_TEXT'.
498        IF sy-subrc = 0 AND it_spectab-tstat = space.
499          it_spectab-tstat = 'P'.
500          MODIFY it_spectab INDEX sy-tabix.
501        ENDIF.
502        CONTINUE.
503      ENDIF.
504  *   special field?
505      WRITE rt_var_fieldcat-fieldname TO prefix(2).
506      IF prefix NE 'U_'.
507        CONTINUE.
508      ENDIF.
509  *   check special field status table:
510      READ TABLE it_specfld WITH KEY fname_2 = rt_var_fieldcat-fieldname.
511      IF sy-subrc NE 0.
512  *     horror case:
513        deco_message_0 w012.
514        EXIT.
515      ENDIF.
516  *   special field status = initial?
517      IF it_specfld-fstat = space.
518  *     set status 'pending':
519        it_specfld-fstat = 'P'.
520        LOOP AT it_specfld WHERE fname_2 = rt_var_fieldcat-fieldname.
521          MODIFY it_specfld TRANSPORTING fstat.
522          APPEND it_specfld TO lt_specfld.
523        ENDLOOP.
524  *     check corresponding table status:
525        READ TABLE it_spectab WITH KEY tname = it_specfld-tname.
526        IF sy-subrc NE 0.
527  *       horror case:
528          deco_message_0 w012.
529          EXIT.
530        ENDIF.
531        IF it_spectab-tstat = space.
532          it_spectab-tstat = 'P'.
533          LOOP AT lt_specfld INTO DATA(ls_specfld).
534            MODIFY it_spectab TRANSPORTING tstat WHERE tname = ls_specfld-
    tname.
535            IF ls_specfld-tname = 'BSEG'.
536              MODIFY it_spectab TRANSPORTING tstat WHERE tname = 'ACDOCA'.
537            ENDIF.
538          ENDLOOP.
539        ENDIF.
540      ENDIF.
541
542  * begin of Transfer Pricing authority check
543      CLEAR lv_no_transfer_price_auth.
544      LOOP AT gt_bukrs_tp_curtp TRANSPORTING NO FIELDS
545                                WHERE ( field_dmbe = rt_var_fieldcat-field
```

```
     name OR field_hwae = rt_var_fieldcat-fieldname )
546                                    AND auth = space.
547         lv_no_transfer_price_auth = abap_true.
548         EXIT.
549       ENDLOOP.
550
551       IF lv_no_transfer_price_auth = abap_true.
552         IF rt_var_fieldcat-tech IS INITIAL.
553           rt_var_fieldcat-tech = 'X'.
554           MODIFY rt_var_fieldcat.
555         ENDIF.
556         ld_noauth = 'X'.
557       ENDIF.
558   * end of Transfer Pricing authority check
559
560   *   Handle additional special fields for bill of exchange.
561       PERFORM analyze_act_fieldcat_boe.
562     ENDLOOP.
563
564   * analyze whether LDGRP needs to be read
565     PERFORM analyze_act_fieldcat_ldgrp  TABLES rt_var_fieldcat.
566
567     CASE ld_noauth.
568       WHEN 'X'.
569         deco_message_0 i114.
570       WHEN '1'.
571         deco_message_0 i115.
572       WHEN '2'.
573         deco_message_0 i116.
574     ENDCASE.
575   ENDFORM.                                  " ANALYZE_ACT_FIELDCAT
576
577   *&---------------------------------------------------------------------*
578   *&      Form  READ_SPECIAL_TABLES
579   *&---------------------------------------------------------------------*
580   FORM read_special_tables.
581     DATA: posindex   LIKE sy-tabix.
582
583   *... check item table filled?
584     READ TABLE it_pos INDEX 1.
585     CHECK sy-subrc = 0.
586   *... loop over all pending tables:
587     LOOP AT it_spectab WHERE tstat = 'P'.
588   * building additional help tables for BKPF and BSEG for optimized
589   * database selections.
590       PERFORM create_performance_tables USING it_spectab-tname.
591       CASE it_spectab-tname.
592         WHEN 'BKPF'.
593           PERFORM special_fields_used USING it_spectab-tname.
594           IF sy-subrc = 0.
595             PERFORM complete_bkpf.
596           ENDIF.
597         WHEN 'BSEG' OR 'ACDOCA'.
598           PERFORM special_fields_used USING it_spectab-tname.    "3138815
599           IF sy-subrc = 0.                                       "3138815
600             PERFORM complete_bseg.
601           ENDIF.                                                 "3138815
602         WHEN 'BSEGC'.
603           PERFORM special_fields_used USING it_spectab-tname.
604           IF sy-subrc = 0.
605             PERFORM complete_bsegc.
```

```
606         ENDIF.
607       WHEN 'BSEC'.
608         PERFORM complete_bsec.
609       WHEN 'BSED'.
610         PERFORM complete_bsed.
611       WHEN 'BSBV'.
612         PERFORM complete_bsbv.
613       WHEN 'PAYR'.
614         PERFORM complete_payr.
615       WHEN '_TEXT'.
616         PERFORM complete_document_text.
617     ENDCASE.
618   ENDLOOP.
619 *... final bookkeeping: status of special tables & fields
620   PERFORM upd_special_status.
621
622 ENDFORM.                                    " READ_SPECIAL_TABLES
623
624 *&---------------------------------------------------------------------*
625 *&      Form  COMPLETE_BKPF
626 *&---------------------------------------------------------------------*
627 FORM complete_bkpf.
628
629   DATA: ld_index  LIKE sy-tabix,
630         ld_indexh LIKE sy-tabix.
631   STATICS: sd_logsys TYPE logsystem,
632            sd_lsflag TYPE c.
633   DATA:    ld_tablename LIKE dd03p-tabname.
634
635
636   CLEAR: wa_bkpf, wa_vbkpf.
637
638   SORT it_pos STABLE BY bukrs belnr gjahr.
639
640 * get logical system id once:
641   IF sd_lsflag IS INITIAL.
642     CALL FUNCTION 'OWN_LOGICAL_SYSTEM_GET'
643       IMPORTING
644         own_logical_system = sd_logsys
645       EXCEPTIONS
646         OTHERS             = 1.
647     sd_lsflag = 'X'.
648   ENDIF.
649
650 * reading special fields from archiv infos.
651   IF gd_usear = 'X'.
652     LOOP AT gt_pos_archived ASSIGNING <gfs_pos_reduced>.
653       READ TABLE xbkpf INTO wa_bkpf
654                        WITH TABLE KEY bukrs = <gfs_pos_reduced>-bukrs
655                                       belnr = <gfs_pos_reduced>-belnr
656                                       gjahr = <gfs_pos_reduced>-gjahr.
657 *   fill logical system id:
658       IF wa_bkpf-awsys IS INITIAL.
659         wa_bkpf-awsys = sd_logsys.
660       ENDIF.
661 *   expiring currencies:
662       IF NOT gd_expcur_flag IS INITIAL.
663         PERFORM item_curr_convert_bkpf USING    gd_expcur_proc
664                                                 gd_alw_date
665                                        CHANGING wa_bkpf.
666       ENDIF.
```

```
667  *   fill special fields for all lines of document:
668        READ TABLE it_pos WITH KEY bukrs = wa_bkpf-bukrs
669                                   belnr = wa_bkpf-belnr
670                                   gjahr = wa_bkpf-gjahr
671                                   BINARY SEARCH.
672        IF sy-subrc EQ 0.
673          PERFORM move_special_fields USING it_spectab-tname.
674          MODIFY it_pos INDEX sy-tabix.
675          ld_index = sy-tabix + 1.
676          LOOP AT it_pos FROM ld_index.
677            IF it_pos-bukrs = wa_bkpf-bukrs AND
678               it_pos-belnr = wa_bkpf-belnr AND
679               it_pos-gjahr = wa_bkpf-gjahr.
680              PERFORM move_special_fields USING it_spectab-tname.
681              MODIFY it_pos.
682            ELSE.
683              EXIT.
684            ENDIF.
685          ENDLOOP.
686        ENDIF.
687      ENDLOOP.
688    ENDIF.
689  * reading special fields from database
690    IF gd_usedb = 'X'.
691  *... first posted items table:
692      if ( b0sg-xstan = 'X' or b0sg-xstas = 'X' )                "2846477
693                            and not gt_pos_posted[] is initial.
694        SELECT       (gt_spec_fields)
695                FROM bkpf
696                INTO CORRESPONDING FIELDS OF wa_bkpf
697                 FOR ALL ENTRIES IN gt_pos_posted
698                      WHERE bukrs = gt_pos_posted-bukrs
699                      AND   belnr = gt_pos_posted-belnr
700                      AND   gjahr = gt_pos_posted-gjahr.
701  *   fill logical system id:
702          IF wa_bkpf-awsys IS INITIAL.
703            wa_bkpf-awsys = sd_logsys.
704          ENDIF.
705  *   expiring currencies:
706          IF NOT gd_expcur_flag IS INITIAL.
707            PERFORM item_curr_convert_bkpf USING    gd_expcur_proc
708                                                    gd_alw_date
709                                            CHANGING wa_bkpf.
710          ENDIF.
711  *   fill special fields for all lines of document:
712          READ TABLE it_pos WITH KEY bukrs = wa_bkpf-bukrs
713                                     belnr = wa_bkpf-belnr
714                                     gjahr = wa_bkpf-gjahr
715                                     BINARY SEARCH.
716          IF sy-subrc EQ 0.
717            PERFORM move_special_fields USING it_spectab-tname.
718            MODIFY it_pos INDEX sy-tabix.
719            ld_index = sy-tabix + 1.
720            LOOP AT it_pos FROM ld_index.
721              IF it_pos-bukrs = wa_bkpf-bukrs AND
722                 it_pos-belnr = wa_bkpf-belnr AND
723                 it_pos-gjahr = wa_bkpf-gjahr.
724                PERFORM move_special_fields USING it_spectab-tname.
725                MODIFY it_pos.
726              ELSE.
727                EXIT.
```

```
728             ENDIF.
729           ENDLOOP.
730         ENDIF.
731
732       ENDSELECT.
733     ENDIF.
734
735 *... now parked items table:
736     IF b0sg-xstav = 'X' AND NOT gt_pos_parked[] IS INITIAL.
737       CLEAR wa_bkpf.
738       ld_tablename = 'VBKPF'.
739       PERFORM special_fields_modify USING ld_tablename.
740       SELECT       (gt_spec_fields)
741            FROM vbkpf
742            INTO CORRESPONDING FIELDS OF wa_bkpf
743                FOR ALL ENTRIES IN gt_pos_parked
744              WHERE bukrs = gt_pos_parked-bukrs
745                AND belnr = gt_pos_parked-belnr
746                AND gjahr = gt_pos_parked-gjahr
747                AND budat = gt_pos_parked-budat.
748 *   fill logical system id:
749         IF wa_bkpf-awsys IS INITIAL.
750           wa_bkpf-awsys = sd_logsys.
751         ENDIF.
752 *   expiring currencies:
753         IF NOT gd_expcur_flag IS INITIAL.
754           PERFORM item_curr_convert_bkpf USING    gd_expcur_proc
755                                                   gd_alw_date
756                                           CHANGING wa_bkpf.
757         ENDIF.
758 *   fill special fields for all lines of document:
759         READ TABLE it_pos WITH KEY bukrs = wa_bkpf-bukrs
760                                    belnr = wa_bkpf-belnr
761                                    gjahr = wa_bkpf-gjahr
762                                    budat = wa_bkpf-budat
763                                    BINARY SEARCH.
764         IF sy-subrc EQ 0.
765           PERFORM move_special_fields USING it_spectab-tname.
766           MODIFY it_pos INDEX sy-tabix.
767           ld_index = sy-tabix + 1 .
768           LOOP AT it_pos FROM ld_index.
769             IF it_pos-bukrs = wa_bkpf-bukrs AND
770                it_pos-belnr = wa_bkpf-belnr AND
771                it_pos-gjahr = wa_bkpf-gjahr AND
772                it_pos-budat = wa_bkpf-budat.
773               PERFORM move_special_fields USING it_spectab-tname.
774               MODIFY it_pos.
775             ELSE.
776               EXIT.
777             ENDIF.
778           ENDLOOP.
779         ENDIF.
780       ENDSELECT.
781     ENDIF.
782   ENDIF.
783
784   PERFORM it_pos_sort_back TABLES it_pos.
785
786 ENDFORM.                                " COMPLETE_BKPF
787
788
```

```
789  *&---------------------------------------------------------------------*
790  *&      Form  COMPLETE_BSEG
791  *&---------------------------------------------------------------------*
792  FORM complete_bseg.
793    DATA: posindex   LIKE sy-tabix.
794    DATA: ld_waers   TYPE waers.
795    DATA: ld_tablename LIKE dd03p-tabname.                          "3138815
796    DATA: ld_fname TYPE type_spec_fields.                           "3138815
797
798  * needed for join with VBKPF and VBSEGS
799    DATA: BEGIN OF wa_vbsegs_ext.
800            INCLUDE STRUCTURE wa_vbsegs.
801    DATA:   budat LIKE vbkpf-budat,
802            waers LIKE vbkpf-waers,
803          END OF wa_vbsegs_ext.
804  *...Special case for the non leading ledger special fields      "1117587
805    TYPES: BEGIN OF ty_pos_posted_nl,
806             bukrs TYPE skb1-bukrs,
807             belnr TYPE bseg-belnr,
808             gjahr TYPE bseg-gjahr,
809             buzei TYPE bseg_add-buzei,
810             budat TYPE bkpf-budat,
811             bstat TYPE bkpf-bstat,
812             docln TYPE faglpose-docln,                             "1237389
813           END OF ty_pos_posted_nl.
814
815  *...Special case for G/l postings only                           "2521327
816    TYPES: BEGIN OF ty_pos_posted_gl_only,                        "2521327
817             rldnr TYPE acdoca-rldnr,                             "2521327
818             bukrs TYPE bseg-bukrs,                               "2521327
819             belnr TYPE bseg-belnr,                               "2521327
820             gjahr TYPE bseg-gjahr,                               "2521327
821             buzei TYPE bseg-buzei,                               "2521327
822             budat TYPE bkpf-budat,                               "2521327
823             bstat TYPE bkpf-bstat,                               "2521327
824             docln TYPE acdoca-docln,                             "2521327
825           END OF ty_pos_posted_gl_only.                          "2521327
826
827    DATA: lt_acdoca   TYPE finst_acdoca.                          "2521327
828
829    DATA: lt_pos_posted    LIKE gt_pos_posted WITH HEADER LINE.
830    DATA: lt_pos_posted_nl TYPE SORTED TABLE OF ty_pos_posted_nl "2423559
831              WITH HEADER LINE                                    "2423559
832              WITH UNIQUE KEY                                     "2423559
833                    bukrs belnr gjahr buzei budat bstat docln. "2423559
834    DATA: lt_pos_posted_gl_only
835              TYPE SORTED TABLE OF ty_pos_posted_gl_only          "2521327
836              WITH HEADER LINE                                    "2521327
837              WITH UNIQUE KEY                                     "2521327
838                    bukrs belnr gjahr buzei budat bstat docln. "2521327
839    DATA: wa_pos_posted    LIKE LINE OF gt_pos_posted.
840    DATA: wa_pos_posted_nl TYPE TABLE OF ty_pos_posted_nl        "1355836
841                                          WITH HEADER LINE. "1355836
842    DATA: wa_pos_posted_gl_only
843                           TYPE TABLE OF ty_pos_posted_gl_only "2521327
844                                          WITH HEADER LINE. "2521327
845    DATA: ld_buzei_add     TYPE bseg_add-buzei.
846  *...Special case for the non leading ledger special fields      "1117587
847
848  * due to selection for parked documents
849    RANGES lt_seloption FOR vbkpf-ausbk.
```

```
850
851     CLEAR: wa_bseg, wa_vbsegd, wa_vbsegk, wa_vbsegs.
852
853   * sort due to binary search
854     SORT it_pos STABLE BY bukrs belnr gjahr buzei docln rldnr. "1237389
855
856     IF gd_usear = 'X'.
857       LOOP AT gt_pos_archived ASSIGNING <gfs_pos_reduced>.
858         READ TABLE xbseg INTO wa_bseg
859                          WITH TABLE KEY bukrs = <gfs_pos_reduced>-bukrs
860                                         belnr = <gfs_pos_reduced>-belnr
861                                         gjahr = <gfs_pos_reduced>-gjahr
862                                         buzei = <gfs_pos_reduced>-buzei.
863         IF wa_bseg-shkzg = 'H'.
864           wa_bseg-menge = wa_bseg-menge * ( -1 ).
865           wa_bseg-nebtr = wa_bseg-nebtr * ( -1 ).
866           wa_bseg-erfmg = wa_bseg-erfmg * ( -1 ).              "1582891
867           wa_bseg-klibt = wa_bseg-klibt * ( -1 ).              "1582891
868           wa_bseg-bdiff = wa_bseg-bdiff * ( -1 ).              "1117587
869           wa_bseg-bdif2 = wa_bseg-bdif2 * ( -1 ).              "1117587
870           wa_bseg-bdif3 = wa_bseg-bdif3 * ( -1 ).              "1117587
871           wa_bseg-rdiff = wa_bseg-rdiff * ( -1 ).              "1117587
872           wa_bseg-rdif2 = wa_bseg-rdif2 * ( -1 ).              "1117587
873           wa_bseg-rdif3 = wa_bseg-rdif3 * ( -1 ).              "1117587
874         ENDIF.
875
876         IF ( wa_bseg-squan = '-' AND wa_bseg-menge > 0 )          "3676362
877         OR ( wa_bseg-squan = '+' AND wa_bseg-menge < 0 ).         "3676362
878           wa_bseg-menge = wa_bseg-menge * ( -1 ).                 "3676362
879         ENDIF.                                                    "3676362
880         IF wa_bseg-squan = '0' AND wa_bseg-menge < 0.             "3676362
881           wa_bseg-menge = wa_bseg-menge * ( -1 ).                 "3676362
882         ENDIF.                                                    "3676362
883
884   *     invoice reference:
885         IF  wa_bseg-rebzg IS INITIAL
886         AND wa_bseg-rebzt IS INITIAL.
887           wa_bseg-rebzg = wa_bseg-belnr.
888         ENDIF.
889   *     fill special fields:
890         READ TABLE it_pos WITH KEY bukrs = wa_bseg-bukrs
891                                    belnr = wa_bseg-belnr
892                                    gjahr = wa_bseg-gjahr
893                                    buzei = wa_bseg-buzei
894                                    docln = <gfs_pos_reduced>-docln  "2622861
895                                    BINARY SEARCH.
896         IF sy-subrc = 0.
897           posindex = sy-tabix.
898   *       expiring currencies:
899           IF  NOT gd_expcur_flag IS INITIAL
900           AND NOT gd_expcur_bseg IS INITIAL.
901             READ TABLE xbkpf INTO wa_bkpf
902                              WITH TABLE KEY bukrs = wa_bseg-bukrs
903                                             belnr = wa_bseg-belnr
904                                             gjahr = wa_bseg-gjahr.
905             ld_waers = wa_bkpf-waers.
906             PERFORM item_curr_convert_bseg USING    gd_expcur_proc
907                                                     it_pos-hwaer
908                                                     ld_waers
909                                                     gd_alw_date
910                                            CHANGING wa_bseg.
```

```
911           ENDIF.
912           PERFORM move_special_fields USING it_spectab-tname.
913           MODIFY it_pos INDEX posindex.
914         ENDIF.
915       ENDLOOP.
916     ENDIF.
917     IF gd_usedb = 'X'.
918   *... first posted items table:
919   *...Special case for the non leading ledger special fields     "1117587
920       LOOP AT gt_pos_posted INTO wa_pos_posted.
921         IF wa_pos_posted-bstat = 'L'.
922           MOVE-CORRESPONDING wa_pos_posted TO lt_pos_posted_nl.
923           CONCATENATE '000' wa_pos_posted-buzei INTO ld_buzei_add.
924           lt_pos_posted_nl-buzei = ld_buzei_add.
925           APPEND lt_pos_posted_nl.
926
927   *... Special case for G/L postings only
928         ELSEIF wa_pos_posted-bstat = 'U'.                       "2521327
929           MOVE-CORRESPONDING wa_pos_posted TO lt_pos_posted_gl_only. "2521
    327
930           APPEND lt_pos_posted_gl_only.                         "2521327
931         ELSE.
932           MOVE-CORRESPONDING wa_pos_posted TO lt_pos_posted.
933           APPEND lt_pos_posted.
934         ENDIF.
935       ENDLOOP.
936   *...Special case for the non leading ledger special fields     "1117587
937       if ( b0sg-xstan = 'X' or b0sg-xstas = 'X' )               "2846477
938                               and not lt_pos_posted[] is initial.   "1117587
939         SELECT (gt_spec_fields) FROM bseg                       "3138815
940           INTO CORRESPONDING FIELDS OF wa_bseg                  "3138815
941           FOR ALL ENTRIES IN lt_pos_posted                    "2175765
942             WHERE bukrs = lt_pos_posted-bukrs                 "2175765
943             AND   belnr = lt_pos_posted-belnr                 "2175765
944             AND   gjahr = lt_pos_posted-gjahr                 "2175765
945             AND   buzei = lt_pos_posted-buzei.                "2175765
946           IF wa_bseg-shkzg = 'H'.
947             wa_bseg-menge = wa_bseg-menge * ( -1 ).
948             wa_bseg-nebtr = wa_bseg-nebtr * ( -1 ).
949             wa_bseg-erfmg = wa_bseg-erfmg * ( -1 ).           "1582891
950             wa_bseg-klibt = wa_bseg-klibt * ( -1 ).           "1582891
951             wa_bseg-bdiff = wa_bseg-bdiff * ( -1 ).           "1117587
952             wa_bseg-bdif2 = wa_bseg-bdif2 * ( -1 ).           "1117587
953             wa_bseg-bdif3 = wa_bseg-bdif3 * ( -1 ).           "1117587
954             wa_bseg-rdiff = wa_bseg-rdiff * ( -1 ).           "1117587
955             wa_bseg-rdif2 = wa_bseg-rdif2 * ( -1 ).           "1117587
956             wa_bseg-rdif3 = wa_bseg-rdif3 * ( -1 ).           "1117587
957           ENDIF.
958   *     invoice reference:
959           IF  wa_bseg-rebzg IS INITIAL
960           AND wa_bseg-rebzt IS INITIAL.
961             wa_bseg-rebzg = wa_bseg-belnr.
962           ENDIF.
963   *     fill special fields:
964           LOOP AT lt_pos_posted INTO wa_pos_posted            "2175765
965                                 WHERE bukrs = wa_bseg-bukrs   "2175765
966                                 AND   belnr = wa_bseg-belnr   "2175765
967                                 AND   gjahr = wa_bseg-gjahr   "2175765
968                                 AND   buzei = wa_bseg-buzei.  "2175765
969             READ TABLE it_pos WITH KEY bukrs = wa_bseg-bukrs
970                                 belnr = wa_bseg-belnr
```

```
 971                                gjahr = wa_bseg-gjahr
 972                                buzei = wa_bseg-buzei
 973                                docln = wa_pos_posted-docln    "1237389
 974                                BINARY SEARCH.
 975              IF sy-subrc = 0.
 976                posindex = sy-tabix.
 977  *           expiring currencies:
 978                IF  NOT gd_expcur_flag IS INITIAL
 979                AND NOT gd_expcur_bseg IS INITIAL.
 980                  SELECT SINGLE waers FROM bkpf INTO ld_waers
 981                                     WHERE bukrs = wa_bseg-bukrs
 982                                     AND   belnr = wa_bseg-belnr
 983                                     AND   gjahr = wa_bseg-gjahr.
 984                  PERFORM item_curr_convert_bseg USING    gd_expcur_proc
 985                                                          it_pos-hwaer
 986                                                          ld_waers
 987                                                          gd_alw_date
 988                                                 CHANGING wa_bseg.
 989                ENDIF.
 990                PERFORM move_special_fields USING it_spectab-tname.
 991                MODIFY it_pos INDEX posindex.
 992              ENDIF.
 993            ENDLOOP.                                              "2175765
 994          ENDSELECT.                                              "2175765
 995        ENDIF.
 996  *...Special case for the non leading ledger special fields      "1117587
 997        if ( b0sg-xstan = 'X' or b0sg-xstas = 'X' )               "2846477
 998                              and not lt_pos_posted_nl[] is initial.
 999          SELECT * FROM bseg_add INTO wa_bseg_add                 "2175765
1000            FOR ALL ENTRIES IN lt_pos_posted_nl                   "2175765
1001              WHERE bukrs = lt_pos_posted_nl-bukrs                "2175765
1002              AND   belnr = lt_pos_posted_nl-belnr                "2175765
1003              AND   gjahr = lt_pos_posted_nl-gjahr                "2175765
1004              AND   buzei = lt_pos_posted_nl-buzei.               "2175765
1005            IF wa_bseg-shkzg = 'H'.
1006              wa_bseg_add-menge = wa_bseg_add-menge * ( -1 ).
1007            ENDIF.
1008  *       fill special fields:
1009            LOOP AT lt_pos_posted_nl INTO wa_pos_posted_nl        "2175765
1010                                WHERE bukrs = wa_bseg_add-bukrs "2175765
1011                                AND   belnr = wa_bseg_add-belnr "2175765
1012                                AND   gjahr = wa_bseg_add-gjahr "2175765
1013                                AND   buzei = wa_bseg_add-buzei. "2175765
1014              READ TABLE it_pos WITH KEY bukrs = wa_bseg_add-bukrs
1015                                belnr = wa_bseg_add-belnr
1016                                gjahr = wa_bseg_add-gjahr
1017                                buzei = wa_bseg_add-buzei
1018                                docln = wa_pos_posted_nl-docln "1355836
1019                                BINARY SEARCH.
1020              IF sy-subrc = 0.
1021                posindex = sy-tabix.
1022                PERFORM move_special_fields USING 'BSEG_ADD'.
1023                MODIFY it_pos INDEX posindex.
1024              ENDIF.
1025            ENDLOOP.                                              "2175765
1026          ENDSELECT.                                              "2175765
1027        ENDIF.
1028  *...Special case for the non leading ledger special fields      "1117587
1029
1030  *... Special case for G/L postings only                         "2521327
1031        IF      b0sg-xstan = 'X'                                  "2521327
```

```
1032      AND NOT lt_pos_posted_gl_only[] IS INITIAL.                "2521327
1033        ld_fname-fname = 'RBUKRS'.                                  "3138815
1034        APPEND ld_fname TO gt_spec_fields.                          "3138815
1035        ld_fname-fname = 'DOCLN'.                                   "3138815
1036        APPEND ld_fname TO gt_spec_fields.                          "3138815
1037        ld_fname-fname = 'RLDNR'.                                   "3138815
1038        APPEND ld_fname TO gt_spec_fields.                          "3138815
1039        ld_tablename = 'ACDOCA'.                                    "3138815
1040        PERFORM SPECIAL_FIELDS_MODIFY USING ld_tablename.           "3138815
1041        SELECT (gt_spec_fields) FROM acdoca                         "3138815
1042          INTO CORRESPONDING FIELDS OF TABLE lt_acdoca              "3138815
1043                              ##DB_FEATURE_MODE[TABLE_LEN_MAX1] "2521327
1044          FOR ALL ENTRIES IN lt_pos_posted_gl_only                "2521327
1045            WHERE rldnr  = lt_pos_posted_gl_only-rldnr            "2521327
1046            AND   rbukrs = lt_pos_posted_gl_only-bukrs            "2521327
1047            AND   belnr  = lt_pos_posted_gl_only-belnr            "2521327
1048            AND   gjahr  = lt_pos_posted_gl_only-gjahr            "2521327
1049            AND   buzei  = lt_pos_posted_gl_only-buzei            "2521327
1050            AND   docln  = lt_pos_posted_gl_only-docln.           "2521327
1051  *    fill special fields:                                          "2521327
1052
1053        LOOP AT lt_acdoca INTO wa_acdoca.                           "2521327
1054          READ TABLE it_pos WITH KEY bukrs = wa_acdoca-rbukrs "2521327
1055                                     belnr = wa_acdoca-belnr  "2521327
1056                                     gjahr = wa_acdoca-gjahr  "2521327
1057                                     buzei = wa_acdoca-buzei  "2521327
1058                                     docln = wa_acdoca-docln  "2521327
1059                                     rldnr = wa_acdoca-rldnr  "2521327
1060                                     BINARY SEARCH.           "2521327
1061          IF sy-subrc = 0.
1062            posindex = sy-tabix.
1063            PERFORM move_special_fields USING 'ACDOCA'.           "2521327
1064            MODIFY it_pos  INDEX posindex.                        "2521327
1065          ENDIF.                                                  "2521327
1066        ENDLOOP.                                                  "2521327
1067        IF b0sg-xstav = 'X' AND NOT gt_pos_parked[] IS INITIAL.   "3138815
1068          "Parked items require field BUKRS, which is not available in ACD
      OCA
1069          ld_fname-fname = 'BUKRS'.                                 "3138815
1070          APPEND ld_fname TO gt_spec_fields.                        "3138815
1071        ENDIF.
1072      ENDIF.                                                      "2521327
1073  *... Special case for G/L postings only                           "2521327
1074
1075  *... now parked items tables:
1076      IF b0sg-xstav = 'X' AND NOT gt_pos_parked[] IS INITIAL.
1077        CLEAR wa_bseg.
1078        CASE it_pos-koart.
1079          WHEN c_koart_ar.
1080            ld_tablename = 'VBSEGD'.                                "3138815
1081            PERFORM SPECIAL_FIELDS_MODIFY USING ld_tablename.       "3138815
1082            SELECT (gt_spec_fields) FROM vbsegd                     "3138815
1083              INTO CORRESPONDING FIELDS OF wa_bseg                  "3138815
1084              FOR ALL ENTRIES IN gt_pos_parked                      "3138815
1085                  WHERE ausbk = gt_pos_parked-bukrs                 "3138815
1086                  AND   belnr = gt_pos_parked-belnr                 "3138815
1087                  AND   gjahr = gt_pos_parked-gjahr                 "3138815
1088                  AND   bukrs = gt_pos_parked-bukrs                 "3138815
1089                  AND   buzei = gt_pos_parked-buzei.                "3138815
1090              IF wa_bseg-shkzg = 'H'.
1091                wa_bseg-menge = wa_bseg-menge * ( -1 ).
```

```
1092                wa_bseg-erfmg = wa_bseg-erfmg * ( -1 ).       "1582891
1093                wa_bseg-klibt = wa_bseg-klibt * ( -1 ).       "1582891
1094                wa_bseg-nebtr = wa_bseg-nebtr * ( -1 ).       "1582891
1095                wa_bseg-bdiff = wa_bseg-bdiff * ( -1 ).       "1582891
1096                wa_bseg-bdif2 = wa_bseg-bdif2 * ( -1 ).       "1582891
1097                wa_bseg-bdif3 = wa_bseg-bdif3 * ( -1 ).       "1582891
1098                wa_bseg-rdiff = wa_bseg-rdiff * ( -1 ).       "1582891
1099                wa_bseg-rdif2 = wa_bseg-rdif2 * ( -1 ).       "1582891
1100                wa_bseg-rdif3 = wa_bseg-rdif3 * ( -1 ).       "1582891
1101              ENDIF.
1102              READ TABLE it_pos WITH KEY bukrs = wa_bseg-bukrs
1103                                         belnr = wa_bseg-belnr
1104                                         gjahr = wa_bseg-gjahr
1105                                         buzei = wa_bseg-buzei
1106                                         BINARY SEARCH.
1107              IF sy-subrc = 0.
1108                posindex = sy-tabix.
1109  *             expiring currencies:
1110                IF  NOT gd_expcur_flag IS INITIAL
1111                AND NOT gd_expcur_bseg IS INITIAL.
1112                  SELECT SINGLE waers FROM vbkpf INTO ld_waers
1113                                      WHERE ausbk = wa_bseg-bukrs
1114                                      AND   bukrs = wa_bseg-bukrs
1115                                      AND   belnr = wa_bseg-belnr
1116                                      AND   gjahr = wa_bseg-gjahr.
1117                  PERFORM item_curr_convert_bseg USING    gd_expcur_proc
1118                                                          it_pos-hwaer
1119                                                          ld_waers
1120                                                          gd_alw_date
1121                                                 CHANGING wa_bseg.
1122                ENDIF.
1123                PERFORM move_special_fields USING it_spectab-tname.
1124                MODIFY it_pos INDEX posindex.
1125              ENDIF.
1126            ENDSELECT.
1127          WHEN c_koart_ap.
1128            ld_tablename = 'VBSEGK'.                              "3138815
1129            PERFORM SPECIAL_FIELDS_MODIFY USING ld_tablename.     "3138815
1130            SELECT (gt_spec_fields) FROM vbsegk                   "3138815
1131              INTO CORRESPONDING FIELDS OF wa_bseg                "3138815
1132              FOR ALL ENTRIES IN gt_pos_parked                    "3138815
1133                  WHERE ausbk = gt_pos_parked-bukrs               "3138815
1134                  AND   belnr = gt_pos_parked-belnr               "3138815
1135                  AND   gjahr = gt_pos_parked-gjahr               "3138815
1136                  AND   bukrs = gt_pos_parked-bukrs               "3138815
1137                  AND   buzei = gt_pos_parked-buzei.              "3138815
1138              IF wa_bseg-shkzg = 'H'.
1139                wa_bseg-menge = wa_bseg-menge * ( -1 ).
1140                wa_bseg-erfmg = wa_bseg-erfmg * ( -1 ).       "1582891
1141                wa_bseg-klibt = wa_bseg-klibt * ( -1 ).       "1582891
1142                wa_bseg-nebtr = wa_bseg-nebtr * ( -1 ).       "1582891
1143                wa_bseg-bdiff = wa_bseg-bdiff * ( -1 ).       "1582891
1144                wa_bseg-bdif2 = wa_bseg-bdif2 * ( -1 ).       "1582891
1145                wa_bseg-bdif3 = wa_bseg-bdif3 * ( -1 ).       "1582891
1146                wa_bseg-rdiff = wa_bseg-rdiff * ( -1 ).       "1582891
1147                wa_bseg-rdif2 = wa_bseg-rdif2 * ( -1 ).       "1582891
1148                wa_bseg-rdif3 = wa_bseg-rdif3 * ( -1 ).       "1582891
1149              ENDIF.
1150              READ TABLE it_pos WITH KEY bukrs = wa_bseg-bukrs
1151                                         belnr = wa_bseg-belnr
1152                                         gjahr = wa_bseg-gjahr
```

```
1153                                  buzei = wa_bseg-buzei
1154                                  BINARY SEARCH.
1155            IF sy-subrc = 0.
1156              posindex = sy-tabix.
1157  *           expiring currencies:
1158              IF  NOT gd_expcur_flag IS INITIAL
1159              AND NOT gd_expcur_bseg IS INITIAL.
1160                SELECT SINGLE waers FROM vbkpf INTO ld_waers
1161                                    WHERE ausbk = wa_bseg-bukrs
1162                                    AND   bukrs = wa_bseg-bukrs
1163                                    AND   belnr = wa_bseg-belnr
1164                                    AND   gjahr = wa_bseg-gjahr.
1165                PERFORM item_curr_convert_bseg USING    gd_expcur_proc
1166                                                        it_pos-hwaer
1167                                                        ld_waers
1168                                                        gd_alw_date
1169                                               CHANGING wa_bseg.
1170              ENDIF.
1171              PERFORM move_special_fields USING it_spectab-tname.
1172              MODIFY it_pos INDEX posindex.
1173            ENDIF.
1174          ENDSELECT.
1175        WHEN c_koart_gl.
1176
1177          LOOP AT it_pos WHERE bstat = 'V'.
1178  *         ausbk and bukrs is equal if buzei = 001
1179            REFRESH lt_seloption.
1180            CLEAR lt_seloption.
1181            IF it_pos-buzei = '001'.
1182              lt_seloption-sign  = 'I'.
1183              lt_seloption-option = 'EQ'.
1184              lt_seloption-low = it_pos-bukrs.
1185              APPEND lt_seloption.
1186            ENDIF.
1187
1188            CLEAR: wa_vbsegs_ext, wa_bseg.
1189            SELECT SINGLE *
1190                    INTO CORRESPONDING FIELDS OF wa_vbsegs_ext
1191                    FROM vbsegs INNER JOIN vbkpf
1192                    ON   vbsegs~ausbk = vbkpf~ausbk
1193                    AND  vbsegs~bukrs = vbkpf~bukrs
1194                    AND  vbsegs~belnr = vbkpf~belnr
1195                    AND  vbsegs~gjahr = vbkpf~gjahr
1196                    WHERE vbsegs~ausbk IN lt_seloption
1197                    AND   vbsegs~belnr = it_pos-belnr
1198                    AND   vbsegs~gjahr = it_pos-gjahr
1199                    AND   vbsegs~bukrs = it_pos-bukrs
1200                    AND   vbsegs~buzei = it_pos-buzei
1201                    AND   vbkpf~budat  = it_pos-budat.
1202
1203            MOVE-CORRESPONDING wa_vbsegs_ext TO wa_bseg.
1204            wa_bseg-bewar = wa_vbsegs_ext-rmvct.
1205            IF wa_bseg-shkzg = 'H'.
1206              wa_bseg-menge = wa_bseg-menge * ( -1 ).
1207              wa_bseg-erfmg = wa_bseg-erfmg * ( -1 ).       "1582891
1208              wa_bseg-klibt = wa_bseg-klibt * ( -1 ).       "1582891
1209              wa_bseg-nebtr = wa_bseg-nebtr * ( -1 ).       "1582891
1210              wa_bseg-bdiff = wa_bseg-bdiff * ( -1 ).       "1582891
1211              wa_bseg-bdif2 = wa_bseg-bdif2 * ( -1 ).       "1582891
1212              wa_bseg-bdif3 = wa_bseg-bdif3 * ( -1 ).       "1582891
1213              wa_bseg-rdiff = wa_bseg-rdiff * ( -1 ).       "1582891
```

```
1214              wa_bseg-rdif2 = wa_bseg-rdif2 * ( -1 ).       "1582891
1215              wa_bseg-rdif3 = wa_bseg-rdif3 * ( -1 ).       "1582891
1216            ENDIF.
1217            READ TABLE it_pos WITH KEY bukrs = wa_bseg-bukrs
1218                                       belnr = wa_bseg-belnr
1219                                       gjahr = wa_bseg-gjahr
1220                                       buzei = wa_bseg-buzei
1221                                       budat = wa_vbsegs_ext-budat
1222                                       BINARY SEARCH.
1223            IF sy-subrc = 0.
1224              posindex = sy-tabix.
1225  *           expiring currencies:
1226              IF  NOT gd_expcur_flag IS INITIAL
1227              AND NOT gd_expcur_bseg IS INITIAL.
1228
1229                PERFORM item_curr_convert_bseg USING   gd_expcur_proc
1230                                                       it_pos-hwaer
1231                                                   wa_vbsegs_ext-waers
1232                                                     gd_alw_date
1233                                               CHANGING wa_bseg.
1234              ENDIF.
1235              PERFORM move_special_fields USING it_spectab-tname.
1236              MODIFY it_pos INDEX posindex.
1237            ENDIF.
1238          ENDLOOP.
1239
1240        ENDCASE.
1241      ENDIF.
1242    ENDIF.
1243
1244    PERFORM it_pos_sort_back TABLES it_pos.
1245
1246  ENDFORM.                                 " COMPLETE_BSEG
1247
1248
1249  *&---------------------------------------------------------------------*
1250  *&      Form  COMPLETE_BSEGC
1251  *&---------------------------------------------------------------------*
1252  FORM complete_bsegc.
1253
1254    CLEAR: wa_bsegc.
1255    SELECT       (gt_spec_fields)
1256          FROM bsegc
1257          INTO CORRESPONDING FIELDS OF wa_bsegc
1258           FOR ALL ENTRIES IN it_pos
1259         WHERE bukrs = it_pos-bukrs
1260           AND belnr = it_pos-belnr
1261           AND gjahr = it_pos-gjahr
1262           AND rfzei = it_pos-rfzei.
1263  *   fill special fields for all lines of document:
1264      LOOP AT it_pos WHERE bukrs = wa_bsegc-bukrs
1265                     AND   belnr = wa_bsegc-belnr
1266                     AND   gjahr = wa_bsegc-gjahr
1267                     AND   rfzei = wa_bsegc-rfzei.
1268        PERFORM move_special_fields USING it_spectab-tname.
1269        MODIFY it_pos.
1270      ENDLOOP.
1271    ENDSELECT.
1272
1273  ENDFORM.                                 " COMPLETE_BSEGC
1274
```

```
1275  *&---------------------------------------------------------------------*
1276  *&      Form  COMPLETE_BSEC
1277  *&---------------------------------------------------------------------*
1278  FORM complete_bsec.
1279    DATA: posindex   LIKE sy-tabix.
1280    DATA: f_bsec_sel(40) TYPE c,
1281          lt_bsec_sel    LIKE TABLE OF f_bsec_sel,
1282          lt_bsec        LIKE wa_bsec OCCURS 0 WITH HEADER LINE,
1283          lv_len         TYPE i.
1284
1285    CLEAR: wa_bsec, wa_vbsec.
1286
1287  *... posted items:
1288  *... NOTE 798255
1289    IF NOT it_pos[] IS INITIAL.
1290      FREE lt_bsec_sel.
1291      APPEND 'BUKRS' TO lt_bsec_sel.
1292      APPEND 'BELNR' TO lt_bsec_sel.
1293      APPEND 'GJAHR' TO lt_bsec_sel.
1294      APPEND 'BUZEI' TO lt_bsec_sel.
1295      SELECT fname FROM t021s APPENDING TABLE lt_bsec_sel
1296                                           WHERE lstcl = 2
1297                                             AND tname = 'BSEC'.
1298
1299      SELECT (lt_bsec_sel) FROM bsec
1300        INTO CORRESPONDING FIELDS OF TABLE lt_bsec
1301        FOR ALL ENTRIES IN it_pos
1302                  WHERE bukrs = it_pos-bukrs
1303                  AND   belnr = it_pos-belnr
1304                  AND   gjahr = it_pos-gjahr
1305                  AND   buzei = it_pos-buzei.
1306
1307      SORT lt_bsec BY bukrs belnr gjahr buzei.
1308
1309      LOOP AT it_pos.
1310        posindex = sy-tabix.
1311        READ TABLE lt_bsec INTO wa_bsec
1312          WITH KEY bukrs = it_pos-bukrs
1313                   belnr = it_pos-belnr
1314                   gjahr = it_pos-gjahr
1315                   buzei = it_pos-buzei
1316          BINARY SEARCH.
1317        IF sy-subrc = 0.
1318          PERFORM move_special_fields USING it_spectab-tname.
1319          MODIFY it_pos INDEX posindex.
1320        ENDIF.
1321      ENDLOOP.
1322    ENDIF.
1323  *... NOTE 798255
1324
1325  *... parked items:
1326    IF b0sg-xstav = 'X'.
1327      CLEAR wa_bsec.
1328      SELECT * FROM vbsec INTO wa_vbsec FOR ALL ENTRIES IN it_pos
1329                    WHERE ausbk = it_pos-bukrs
1330                    AND   belnr = it_pos-belnr
1331                    AND   gjahr = it_pos-gjahr
1332                    AND   buzei = it_pos-buzei.
1333        MOVE-CORRESPONDING wa_vbsec TO wa_bsec.
1334        wa_bsec-bukrs = wa_vbsec-ausbk.
1335        READ TABLE it_pos WITH KEY bukrs = wa_bsec-bukrs
```

```
1336                                belnr = wa_bsec-belnr
1337                                gjahr = wa_bsec-gjahr
1338                                buzei = wa_bsec-buzei.
1339        IF sy-subrc = 0.
1340          posindex = sy-tabix.
1341          PERFORM move_special_fields USING it_spectab-tname.
1342          MODIFY it_pos INDEX posindex.
1343        ENDIF.
1344      ENDSELECT.
1345    ENDIF.
1346
1347  ENDFORM.                                    " COMPLETE_BSEC
1348
1349  *&---------------------------------------------------------------------*
1350  *&      Form  COMPLETE_BSED
1351  *&---------------------------------------------------------------------*
1352  FORM complete_bsed.
1353    DATA: posindex   LIKE sy-tabix.
1354
1355    CLEAR: wa_bsed.
1356    SELECT * FROM bsed INTO wa_bsed FOR ALL ENTRIES IN it_pos
1357                  WHERE bukrs = it_pos-bukrs
1358                  AND   belnr = it_pos-belnr
1359                  AND   gjahr = it_pos-gjahr
1360                  AND   buzei = it_pos-buzei.
1361      READ TABLE it_pos WITH KEY bukrs = wa_bsed-bukrs
1362                                 belnr = wa_bsed-belnr
1363                                 gjahr = wa_bsed-gjahr
1364                                 buzei = wa_bsed-buzei.
1365      IF sy-subrc = 0.
1366        posindex = sy-tabix.
1367        PERFORM move_special_fields USING it_spectab-tname.
1368  *     Handle additional special fields for bill of exchange.
1369        PERFORM move_special_fields_boe.
1370        MODIFY it_pos INDEX posindex.
1371      ENDIF.
1372    ENDSELECT.
1373
1374  ENDFORM.                                    " COMPLETE_BSED
1375
1376  *&---------------------------------------------------------------------*
1377  *&      Form  COMPLETE_PAYR
1378  *&---------------------------------------------------------------------*
1379  FORM complete_payr.
1380    DATA: number LIKE bseg-augbl.
1381
1382    LOOP AT it_pos.
1383      CLEAR: wa_payr.
1384      SHIFT wa_payr-voidr BY 2 PLACES.
1385  *   prospects of finding something in PAYR:
1386      IF it_pos-koart CO 'DK'.
1387        IF it_pos-augbl IS INITIAL OR
1388           it_pos-shkzg EQ 'S' AND it_pos-xzahl EQ 'X'.
1389          number = it_pos-belnr.
1390        ELSE.
1391          number = it_pos-augbl.
1392        ENDIF.
1393      ELSE.
1394        number = it_pos-belnr.
1395      ENDIF.
1396      CALL FUNCTION 'GET_CHECK_INFORMATION'
```

```
1397        EXPORTING
1398          i_augbl = number
1399          i_augdt = it_pos-augdt
1400          i_belnr = it_pos-belnr
1401          i_bukrs = it_pos-bukrs
1402          i_gjahr = it_pos-gjahr
1403          i_shkzg = it_pos-shkzg
1404          i_xzahl = it_pos-xzahl
1405          i_call  = space
1406          i_koart = it_pos-koart
1407          i_umsks = it_pos-umsks                                  "N2308090
1408          i_umskz = it_pos-umskz                                  "N2308090
1409          i_auggj = it_pos-auggj                                  "N2308090
1410        IMPORTING
1411          e_payr  = wa_payr
1412        EXCEPTIONS
1413          OTHERS  = 4.
1414      IF wa_payr-voidr IS INITIAL.
1415        SHIFT wa_payr-voidr BY 2 PLACES.
1416      ENDIF.
1417      PERFORM move_special_fields USING it_spectab-tname.
1418      MODIFY it_pos.
1419    ENDLOOP.
1420
1421  ENDFORM.                                " COMPLETE_PAYR
1422
1423  *&---------------------------------------------------------------------*
1424  *&      Form  COMPLETE_BSBV
1425  *&---------------------------------------------------------------------*
1426  FORM complete_bsbv.
1427
1428    LOOP AT it_pos.
1429      CLEAR: wa_bsbv.
1430      CALL FUNCTION 'VALUATION_FETCH_BSBV'
1431        EXPORTING
1432          i_bukrs = it_pos-bukrs
1433          i_gjahr = it_pos-gjahr
1434          i_belnr = it_pos-belnr
1435          i_buzei = it_pos-buzei
1436          i_kunnr = it_pos-konto
1437        IMPORTING
1438          e_bsbv  = wa_bsbv.
1439      if it_pos-shkzg = 'H'.                                      "3022976
1440        wa_bsbv-bwshb1 = wa_bsbv-bwshb1 * ( -1 ).                 "3022976
1441        wa_bsbv-bwshb2 = wa_bsbv-bwshb2 * ( -1 ).                 "3022976
1442        wa_bsbv-bwshb3 = wa_bsbv-bwshb3 * ( -1 ).                 "3022976
1443        wa_bsbv-bwshb4 = wa_bsbv-bwshb4 * ( -1 ).                 "3022976
1444        wa_bsbv-bwshb5 = wa_bsbv-bwshb5 * ( -1 ).                 "3022976
1445        wa_bsbv-bwshb6 = wa_bsbv-bwshb6 * ( -1 ).                 "3022976
1446      endif.
1447      PERFORM move_special_fields USING it_spectab-tname.
1448      MODIFY it_pos.
1449    ENDLOOP.
1450
1451  ENDFORM.                                " COMPLETE_BSBV
1452
1453  *&---------------------------------------------------------------------*
1454  *&      Form  MOVE_SPECIAL_FIELDS
1455  *&---------------------------------------------------------------------*
1456  FORM move_special_fields  USING  p_tname LIKE dd03p-tabname.
1457    FIELD-SYMBOLS: <fs_sp_tab>, <fs_it_pos>.
```

```
1458
1459    LOOP AT it_moveref WHERE tname = p_tname.
1460      ASSIGN it_moveref-fldref->*   TO <fs_sp_tab>.
1461      ASSIGN it_moveref-fldref_2->* TO <fs_it_pos>.
1462      MOVE <fs_sp_tab> TO <fs_it_pos>.
1463    ENDLOOP.
1464
1465  ENDFORM.                                " MOVE_SPECIAL_FIELDS
1466
1467  *&---------------------------------------------------------------------*
1468  *&      Form  UPD_SPECIAL_STATUS
1469  *&---------------------------------------------------------------------*
1470  FORM upd_special_status.
1471
1472  *... special table status: pending -> processed
1473    LOOP AT it_spectab WHERE tstat = 'P'.
1474      it_spectab-tstat = 'X'.
1475      MODIFY it_spectab.
1476  *... corresponding special field status: ? -> processed
1477      LOOP AT it_specfld WHERE tname = it_spectab-tname.
1478        it_specfld-fstat = 'X'.
1479        MODIFY it_specfld.
1480      ENDLOOP.
1481    ENDLOOP.
1482
1483  ENDFORM.                                " UPD_SPECIAL_STATUS
1484
1485  *&---------------------------------------------------------------------*
1486  *&      Form  CHECK_ITEM_OK
1487  *&---------------------------------------------------------------------*
1488  FORM check_item_ok USING    p_norm
1489                              p_shbv
1490                              p_merk
1491                              p_park
1492                              p_item    LIKE faglposx
1493                     CHANGING p_okay.
1494
1495    CLEAR p_okay.
1496  * normal document:
1497    IF p_norm = 'X'.
1498      IF (   p_item-bstat = space OR p_item-bstat = 'L' OR    " 2857175
1499             p_item-bstat = if_fins_acdoc_constants=>gc_document_status-gl
      _only  OR " 2857175
1500             p_item-bstat = if_fins_acdoc_constants=>gc_document_status-gl
      _simulation OR " 2857175
1501             p_item-bstat = if_fins_acdoc_constants=>gc_document_status-pr
      edictive OR " 2857175
1502          ( p_item-bstat = if_fins_acdoc_constants=>gc_document_status-bc
      f_adjustment AND gd_enforce_outer_join = abap_true ) ) " 2857175
1503      AND ( p_item-umskz = space ).
1504        p_okay = 'X'.
1505      ENDIF.
1506      IF NOT x_glyec IS INITIAL AND p_item-bstat = 'J'.       "1779180
1507        p_okay = 'X'.                                         "1779180
1508      ENDIF.                                                  "1779180
1509    ENDIF.
1510  * SHB item:
1511    IF p_shbv = 'X'.
1512      IF ( p_item-umskz NE space ) AND ( p_item-bstat NE 'S' ) AND
1513         ( p_item-bstat NE 'V' )   AND ( p_item-bstat NE 'W' ) AND
1514         ( p_item-bstat NE 'Z' ).
```

```
1515        p_okay = 'X'.
1516      ENDIF.
1517    ENDIF.
1518  * Merkposten:
1519    IF p_merk = 'X'.
1520      IF p_item-bstat = 'S'.
1521        p_okay = 'X'.
1522      ENDIF.
1523    ENDIF.
1524  * parked document:
1525    IF p_park = 'X'.
1526      IF ( p_item-bstat = 'V' ) OR ( p_item-bstat = 'W' ).
1527        p_okay = 'X'.
1528      ENDIF.
1529    ENDIF.
1530
1531  ENDFORM.                                " CHECK_ITEM_OK
1532
1533  *&---------------------------------------------------------------------*
1534  *&      Form  INIT_KKCURR_TABLE
1535  *&---------------------------------------------------------------------*
1536  *form init_kkcurr_table.
1537  *  select kkber waers from t014 into it_kkcurr.
1538  *    insert table it_kkcurr.
1539  *  endselect.
1540  *endform.                                " INIT_KKCURR_TABLE
1541
1542  *&---------------------------------------------------------------------*
1543  *&      Form  ITEM_CURRENCY_FIELDS
1544  *&---------------------------------------------------------------------*
1545  *form item_currency_fields.
1546  * wa_pos-hwae2 = wa_x001-hwae2.
1547  * wa_pos-hwae3 = wa_x001-hwae3.
1548  *  read table it_kkcurr with key kkber = wa_pos-kkber
1549  *                        binary search.
1550  *  if sy-subrc = 0.
1551  *    wa_pos-kkbwr = it_kkcurr-curry.
1552  *  endif.
1553  *endform.                                " ITEM_CURRENCY_FIELDS
1554
1555  *&---------------------------------------------------------------------*
1556  *&      Form  ACCTAB_FILL
1557  *&---------------------------------------------------------------------*
1558  FORM fill_acct_table  USING  p_koart LIKE rfepk-koart
1559                               p_konto LIKE rfepk-konto
1560                               p_bukrs LIKE rfepk-bukrs
1561                               p_refkt LIKE rfepk-refkt
1562                               p_name1 LIKE rfepk-name1
1563                               p_sakan LIKE rfepk-sakan.
1564
1565    CLEAR: it_accts.
1566    it_accts-konto = p_konto.
1567    it_accts-bukrs = p_bukrs.
1568    it_accts-koart = p_koart.
1569    it_accts-refkt = p_refkt.
1570    it_accts-name1 = p_name1.
1571    it_accts-sakan = p_sakan.
1572    APPEND it_accts.
1573
1574  ENDFORM.                    "fill_acct_table
1575
```

```
1576  *&---------------------------------------------------------------------*
1577  *&      Form  COMTAB_FILL
1578  *&---------------------------------------------------------------------*
1579  FORM fill_comp_table  USING  p_bukrs LIKE rfepb-bukrs
1580                               p_waers LIKE rfepb-waers
1581                               p_kkber LIKE rfepb-kkber.
1582
1583    CLEAR: it_comps.
1584    CALL FUNCTION 'FI_PERIOD_DETERMINE'
1585      EXPORTING
1586        i_budat = sy-datlo
1587        i_bukrs = p_bukrs
1588  *     I_PERIV = ' '
1589  *     I_GJAHR = 0000
1590  *     I_MONAT = 00
1591  *     X_XMO16 = ' '
1592      IMPORTING
1593        e_gjahr = it_comps-gjahr.
1594    IF sy-subrc NE 0.
1595    ENDIF.
1596    it_comps-bukrs = p_bukrs.
1597    it_comps-waers = p_waers.
1598    it_comps-kkber = p_kkber.
1599    APPEND it_comps.
1600  ENDFORM.                    "fill_comp_table
1601
1602  *&---------------------------------------------------------------------*
1603  *&      Form  ITEM_CHECK_APPEND
1604  *&---------------------------------------------------------------------*
1605  FORM item_check_append.
1606    DATA: gtext(60) TYPE c,
1607          igui      TYPE i.
1608    DATA: ld_shkzg     LIKE bseg-shkzg.
1609    CONSTANTS:
1610      ldc_debit  LIKE bseg-shkzg VALUE 'S',
1611      ldc_credit LIKE bseg-shkzg VALUE 'H'.
1612
1613    CHECK: wa_pos-bwwrt IN so_bwwrt,
1614           wa_pos-bwwr2 IN so_bwwr2,
1615           wa_pos-bwwr3 IN so_bwwr3,
1616           wa_pos-dmshb IN so_dmshb,
1617           wa_pos-hwaer IN so_hwaer,
1618           wa_pos-dmbe2 IN so_dmbe2,
1619           wa_pos-hwae2 IN so_hwae2,
1620           wa_pos-dmbe3 IN so_dmbe3,
1621           wa_pos-hwae3 IN so_hwae3,
1622           wa_pos-faedt IN so_faedt,
1623           wa_pos-jamon IN so_jamon,
1624           wa_pos-koart IN so_koart.
1625
1626    CHECK: wa_pos-qbshb IN so_qbshb,
1627           wa_pos-qsfbt IN so_qsfbt,
1628           wa_pos-qsshb IN so_qsshb,
1629           wa_pos-skfbt IN so_skfbt,
1630           wa_pos-wskto IN so_wskto,
1631           wa_pos-sknto IN so_sknto,
1632           wa_pos-verz1 IN so_verz1,
1633           wa_pos-verzn IN so_verzn,
1634           wa_pos-wrshb IN so_wrshb,
1635           wa_pos-pswsl IN so_waers,
1636           wa_pos-zaldt IN so_zaldt,
```

```
1637           wa_pos-zinsz IN so_zinsz.
1638
1639     CHECK: wa_pos-askto IN so_askto,
1640            wa_pos-kurse IN so_kurse,
1641            wa_pos-kkbwr IN so_kkbwr.
1642
1643     CHECK: wa_pos-gsber IN so_gsber.
1644
1645
1646   * Removed with note 2230542 - Added to the selection directly
1647   *  IF gv_relevant_ledger NE gv_leading_ledger                  "1875681
1648   *                              AND x_glyec IS INITIAL.         "1875681
1649   *    IF wa_pos-awtyp = 'GLYEC'                                 "1875681
1650   *      AND NOT cl_fagl_switch_check=>fagl_fin_gl_cp_rs( )      "1875681
1651   *                                                 IS INITIAL."1875681
1652   *      EXIT.                                                   "1875681
1653   *    ENDIF.                                                    "1875681
1654   *  ENDIF.                                                      "1875681
1655   * Removed with note 2230542 - Added to the selection directly
1656
1657   * check debit/credit flag according to negative posting:
1658     IF wa_pos-xnegp IS INITIAL.
1659       ld_shkzg = wa_pos-shkzg.
1660     ELSE.
1661       IF wa_pos-shkzg = ldc_debit.
1662         ld_shkzg = ldc_credit.
1663       ELSE.
1664         ld_shkzg = ldc_debit.
1665       ENDIF.
1666     ENDIF.
1667
1668     CHECK: ld_shkzg IN so_shkzg.
1669
1670     IF gd_xarch IS INITIAL.
1671       IF NOT wa_pos-xarch IS INITIAL.
1672         gd_count_arch = gd_count_arch + 1.
1673       ENDIF.
1674     ELSE.
1675       gd_count_arch = gd_count_arch + 1.
1676     ENDIF.
1677
1678     APPEND wa_pos TO it_pos.
1679
1680   * number of selected items on gui:
1681     igui = sy-tabix MOD c_guicnt.
1682     IF igui = 0.
1683       WRITE sy-tabix TO gtext+0.
1684       CONDENSE gtext.
1685       WRITE TEXT-008 TO gtext+20.
1686       CONDENSE gtext.
1687       CALL FUNCTION 'SAPGUI_PROGRESS_INDICATOR'
1688         EXPORTING
1689           text = gtext.
1690     ENDIF.
1691
1692   ENDFORM.                                 " ITEM_CHECK_APPEND
1693
1694   *&---------------------------------------------------------------------*
1695   *&      Form  GET_GENERAL_PARAM
1696   *&---------------------------------------------------------------------*
1697   FORM get_general_param.
```

```
1698    DATA: charh(16)  TYPE c,
1699          ld_num(16) TYPE n.
1700    DATA: ls_rfopt  TYPE rfopt.
1701    DATA: ld_parid TYPE usr05-parid.
1702
1703  * in Arbeit
1704
1705  *... get checkbox stuff depending on acc.type:
1706    CASE sy-repid.
1707  *    when c_repid_ar.
1708  *      get parameter id 'FIT_X_AR' field charh.
1709  *      get parameter id 'FIT_ALV_AR' field pa_vari.
1710  *      ld_parid = 'FIT_ALV_AR'.
1711  *    when c_repid_ap.
1712  *      get parameter id 'FIT_X_AP' field charh.
1713  *      get parameter id 'FIT_ALV_AP' field pa_vari.
1714  *      ld_parid = 'FIT_ALV_AP'.
1715  *    when c_repid_gl.
1716  *      get parameter id 'FIT_X_GL' field charh.
1717  *      get parameter id 'FIT_ALV_GL' field pa_vari.
1718  *      ld_parid = 'FIT_ALV_GL'.
1719      WHEN c_repid_ar.
1720        GET PARAMETER ID 'FIT_X_FAGL_AR' FIELD charh.
1721        GET PARAMETER ID 'FIT_ALV_FAGL_AR' FIELD pa_vari.
1722        ld_parid = 'FIT_ALV_FAGL_AR'.
1723      WHEN c_repid_ap.
1724        GET PARAMETER ID 'FIT_X_FAGL_AP' FIELD charh.
1725        GET PARAMETER ID 'FIT_ALV_FAGL_AP' FIELD pa_vari.
1726        ld_parid = 'FIT_ALV_FAGL_AP'.
1727      WHEN c_repid_gl.
1728        GET PARAMETER ID 'FIT_X_FAGL_GL' FIELD charh.
1729        GET PARAMETER ID 'FIT_ALV_FAGL_GL' FIELD pa_vari.
1730        ld_parid = 'FIT_ALV_FAGL_GL'.
1731      WHEN OTHERS.
1732  *     horror case!
1733        EXIT.
1734    ENDCASE.
1735    TRANSLATE charh TO UPPER CASE.                                  "2889134
1736    IF NOT charh IS INITIAL.
1737      x_norm = charh(1).
1738      x_shbv = charh+1(1).
1739      x_merk = charh+2(1).
1740      x_park = charh+3(1).
1741      x_apar = charh+4(1).
1742      x_erfs = charh+5(1).
1743    ENDIF.
1744
1745  *...changing of pa_vari (default value FIT_ALV_**)
1746    GET PARAMETER ID 'FOP' FIELD ls_rfopt.
1747    IF NOT ls_rfopt-xarbv IS INITIAL.
1748      SELECT SINGLE parva FROM usr05 INTO (pa_vari)
1749                          WHERE bname = sy-uname
1750                          AND parid = ld_parid.
1751      IF sy-subrc <> 0.
1752        CLEAR pa_vari.
1753      ENDIF.
1754    ENDIF.
1755
1756  *... get item number limit:
1757    GET PARAMETER ID 'FIT_X_NMAX' FIELD ld_num.
1758    IF ld_num CO ' 0123456789'.
```

```
1759      pa_nmax = ld_num.
1760    ENDIF.
1761
1762  ENDFORM.                                " GET_GENERAL_PARAM
1763
1764  *&---------------------------------------------------------------------*
1765  *&      Form  SET_GENERAL_PARAM
1766  *&---------------------------------------------------------------------*
1767  FORM set_general_param.
1768
1769    DATA: ls_rfopt  TYPE rfopt.
1770    DATA: charh(16)  TYPE c,
1771          ld_num(16) TYPE n,
1772          ls_usr05   LIKE usr05.
1773    DATA: ld_refresh_count TYPE i,
1774          ld_memory_id(40) TYPE c.
1775
1776    CHECK sy-calld IS INITIAL.
1777  *  check sy-tcode cp 'FBL+N' or
1778  *       ( sy-tcode cp 'S+38' and sy-repid cp 'RFITEM++' ) or
1779  *       ( sy-tcode eq 'SE80' and sy-repid cp 'RFITEM++' ).
1780    CHECK sy-tcode CP 'FAGLL0+' OR
1781      ( sy-tcode CP 'S+38' AND sy-repid CP 'FAGL_ACCOUNT_ITEMS+++' ) OR
1782      ( sy-tcode EQ 'SE80' AND sy-repid CP 'FAGL_ACCOUNT_ITEMS+++' ).
1783
1784    charh(1)   = x_norm.
1785    charh+1(1) = x_shbv.
1786    charh+2(1) = x_merk.
1787    charh+3(1) = x_park.
1788    charh+4(1) = x_apar.
1789    charh+5(1) = x_erfs.
1790  * if in refresh call, then do not update layout parameter:
1791    ld_memory_id = sy-repid.
1792    WRITE '/REFR' TO ld_memory_id+35(5).
1793    CONDENSE ld_memory_id NO-GAPS.
1794    IMPORT counter TO ld_refresh_count
1795           FROM MEMORY ID ld_memory_id.
1796    CASE sy-repid.
1797  *    when c_repid_ar.
1798  *      set parameter id 'FIT_X_AR'   field charh.
1799  *      if ld_refresh_count = 0.
1800  *        get parameter id 'FOP' field ls_rfopt.
1801  *        if ls_rfopt-xarbv is initial.
1802  *          set parameter id 'FIT_ALV_AR' field pa_vari.
1803  *          write 'FIT_ALV_AR' to ls_usr05-parid.
1804  *        else.
1805  *          set parameter id 'FIT_ALV_AR' field space.
1806  *        endif.
1807  *      endif.
1808  *    when c_repid_ap.
1809  *      set parameter id 'FIT_X_AP'   field charh.
1810  *      if ld_refresh_count = 0.
1811  *        get parameter id 'FOP' field ls_rfopt.
1812  *        if ls_rfopt-xarbv is initial.
1813  *          set parameter id 'FIT_ALV_AP' field pa_vari.
1814  *          write 'FIT_ALV_AP' to ls_usr05-parid.
1815  *        else.
1816  *          set parameter id 'FIT_ALV_AP' field space.
1817  *        endif.
1818  *      endif.
1819  *    when c_repid_gl.
```

```
1820  *      set parameter id 'FIT_X_GL'   field charh.
1821  *      if ld_refresh_count = 0.
1822  *        get parameter id 'FOP' field ls_rfopt.
1823  *        if ls_rfopt-xarbv is initial.
1824  *          set parameter id 'FIT_ALV_GL' field pa_vari.
1825  *          write 'FIT_ALV_GL' to ls_usr05-parid.
1826  *        else.
1827  *          set parameter id 'FIT_ALV_GL' field space.
1828  *        endif.
1829  *      endif.
1830
1831      WHEN c_repid_ar.
1832        SET PARAMETER ID 'FIT_X_FAGL_AR'   FIELD charh.
1833        IF ld_refresh_count = 0.
1834          GET PARAMETER ID 'FOP' FIELD ls_rfopt.
1835          IF ls_rfopt-xarbv IS INITIAL.
1836            SET PARAMETER ID 'FIT_ALV_FAGL_AR' FIELD pa_vari.
1837            WRITE 'FIT_ALV_FAGL_AR' TO ls_usr05-parid.
1838          ELSE.
1839            SET PARAMETER ID 'FIT_ALV_FAGL_AR' FIELD space.
1840          ENDIF.
1841        ENDIF.
1842      WHEN c_repid_ap.
1843        SET PARAMETER ID 'FIT_X_FAGL_AP'   FIELD charh.
1844        IF ld_refresh_count = 0.
1845          GET PARAMETER ID 'FOP' FIELD ls_rfopt.
1846          IF ls_rfopt-xarbv IS INITIAL.
1847            SET PARAMETER ID 'FIT_ALV_FAGL_AP' FIELD pa_vari.
1848            WRITE 'FIT_ALV_AP' TO ls_usr05-parid.
1849          ELSE.
1850            SET PARAMETER ID 'FIT_ALV_FAGL_AP' FIELD space.
1851          ENDIF.
1852        ENDIF.
1853      WHEN c_repid_gl.
1854        SET PARAMETER ID 'FIT_X_FAGL_GL'   FIELD charh.
1855        IF ld_refresh_count = 0.
1856          GET PARAMETER ID 'FOP' FIELD ls_rfopt.
1857          IF ls_rfopt-xarbv IS INITIAL.
1858            SET PARAMETER ID 'FIT_ALV_FAGL_GL' FIELD pa_vari.
1859            WRITE 'FIT_ALV_FAGL_GL' TO ls_usr05-parid.
1860          ELSE.
1861            SET PARAMETER ID 'FIT_ALV_FAGL_GL' FIELD space.
1862          ENDIF.
1863        ENDIF.
1864  *    when c_repid_FAGl_GL.
1865  *      set parameter id 'FIT_X_FAGL_GL'   field charh.
1866  *      if ld_refresh_count = 0.
1867  *        get parameter id 'FOP' field ls_rfopt.
1868  *        if ls_rfopt-xarbv is initial.
1869  *          set parameter id 'FIT_ALV_FAGL_GL' field pa_vari.
1870  *          write 'FIT_ALV_GL' to ls_usr05-parid.
1871  *        else.
1872  *          set parameter id 'FIT_ALV_FAGL_GL' field space.
1873  *        endif.
1874  *      endif.
1875      WHEN OTHERS.
1876  *     horror case!
1877        EXIT.
1878    ENDCASE.
1879    IF ld_refresh_count = 0.
1880      GET PARAMETER ID 'FOP' FIELD ls_rfopt.
```

```
1881      IF ls_rfopt-xarbv IS INITIAL.
1882        ls_usr05-bname = sy-uname.
1883        WRITE pa_vari TO ls_usr05-parva.
1884        IF ls_usr05-parva IS INITIAL.
1885          DELETE FROM usr05 WHERE bname = ls_usr05-bname
1886                            AND   parid = ls_usr05-parid.
1887        ELSE.
1888          MODIFY usr05 FROM ls_usr05.
1889        ENDIF.
1890        COMMIT WORK.
1891      ENDIF.
1892    ENDIF.
1893    ld_num = pa_nmax.
1894    SET PARAMETER ID 'FIT_X_NMAX' FIELD ld_num.
1895
1896  ENDFORM.                                    " SET_GENERAL_PARAM
1897
1898  *&---------------------------------------------------------------------*
1899  *&      Form  SAVE_ALL_SELECTIONS
1900  *&---------------------------------------------------------------------*
1901  FORM save_all_selections.
1902
1903  *... set pa_cent finally:
1904    PERFORM set_pa_cent.
1905
1906  *... get report selection-screen selections:
1907    CALL FUNCTION 'RS_REFRESH_FROM_SELECTOPTIONS'
1908      EXPORTING
1909        curr_report     = g_repid
1910      IMPORTING
1911        sp              = gt_searchpattern
1912      TABLES
1913        selection_table = gt_selscreen
1914      EXCEPTIONS
1915        not_found       = 1
1916        no_report       = 2
1917        OTHERS          = 3.
1918    IF sy-subrc > 1.
1919      EXIT.
1920    ENDIF.
1921  *... get report dynamic selections:
1922    CALL FUNCTION 'RS_REFRESH_FROM_DYNAMICAL_SEL'
1923      EXPORTING
1924        curr_report        = g_repid
1925        mode_write_or_move = 'M'
1926      IMPORTING
1927        p_trange           = gt_dyn_trange
1928      EXCEPTIONS
1929        not_found          = 1
1930        wrong_type         = 2
1931        OTHERS             = 3.
1932    IF sy-subrc > 1.
1933      EXIT.
1934    ENDIF.
1935    CALL FUNCTION 'FREE_SELECTIONS_RANGE_2_EX'
1936      EXPORTING
1937        field_ranges = gt_dyn_trange
1938      IMPORTING
1939        expressions  = gt_dyn_texpr.
1940
1941  ENDFORM.                                    " SAVE_ALL_SELECTIONS
```

```
1942
1943  *&---------------------------------------------------------------------*
1944  *&      Form  SET_PA_CENT
1945  *&---------------------------------------------------------------------*
1946  FORM set_pa_cent.
1947    DATA: lb_cent_filled TYPE c,
1948          lb_cent_empty  TYPE c.
1949
1950  * already non initial => exit:
1951    CHECK pa_cent IS INITIAL.
1952  * no branch/central entries => exit:
1953    READ TABLE it_central INDEX 1.
1954    CHECK sy-subrc = 0.
1955  * check for either filled or empty central entries:
1956    LOOP AT it_central WHERE NOT central IS INITIAL.
1957      lb_cent_filled = c_true.
1958      EXIT.
1959    ENDLOOP.
1960    LOOP AT it_central WHERE central IS INITIAL.
1961      lb_cent_empty = c_true.
1962      EXIT.
1963    ENDLOOP.
1964    IF lb_cent_filled NE lb_cent_empty.
1965  *   pure case:
1966      IF lb_cent_filled = c_true.
1967  *     all filled, none empty
1968        pa_cent = 'Y'.
1969      ELSE.
1970  *     none filled, all empty
1971        pa_cent = 'N'.
1972      ENDIF.
1973    ELSE.
1974  *   mixed case: prompt user
1975      CLEAR pa_cent.
1976    ENDIF.
1977
1978  ENDFORM.                                    " SET_PA_CENT
1979
1980
1981  *&---------------------------------------------------------------------*
1982  *&      Form  display_grid_or_classic
1983  *&---------------------------------------------------------------------*
1984  FORM display_grid_or_classic.
1985    DATA: ld_param    TYPE c.
1986
1987  *... set list display type depending on parameters:
1988    CASE pa_grid.
1989      WHEN 'Y'.
1990        x_grid = c_x.
1991      WHEN 'N'.
1992        CLEAR x_grid.
1993      WHEN OTHERS.
1994        GET PARAMETER ID 'FIT_ALVC' FIELD ld_param.
1995        IF NOT ld_param IS INITIAL.
1996          x_grid = c_x.
1997        ELSE.
1998          CLEAR x_grid.
1999        ENDIF.
2000    ENDCASE.
2001
2002  *... if programm is running in batch, choose ALV list.
```

```
2003    IF sy-batch = 'X'.
2004      CLEAR x_grid.
2005    ENDIF.
2006
2007  ENDFORM.                                " display_grid_or_classic
2008
2009  *&---------------------------------------------------------------------*
2010  *&      Form  start_again
2011  *&---------------------------------------------------------------------*
2012  FORM start_again.
2013
2014    deco_message_0 s033.
2015    SUBMIT (g_repid) WITH SELECTION-TABLE gt_selscreen
2016                     WITH FREE SELECTIONS gt_dyn_texpr
2017                     WITH kd_index EQ gt_searchpattern
2018                     WITH dd_index EQ gt_searchpattern
2019                     WITH sd_index EQ gt_searchpattern
2020                     VIA SELECTION-SCREEN.
2021
2022  ENDFORM.                                " start_again
2023
2024  *&---------------------------------------------------------------------*
2025  *&      Form  MAKE_FIELDCATALOG
2026  *&---------------------------------------------------------------------*
2027  FORM make_fieldcatalog.
2028    DATA: lt_dfies_faglpose LIKE dfies OCCURS  50 WITH HEADER LINE,
2029          lt_dfies_bsix     LIKE dfies OCCURS 100 WITH HEADER LINE,
2030          lt_duplicates     TYPE tpit_t_fname WITH HEADER LINE,
2031          ld_fname          LIKE dfies-fieldname,
2032          ld_name_bsix      LIKE dcobjdef-name,
2033          lt_fmrfc          LIKE fmrfc OCCURS 1.
2034    DATA: ld_act_imp_ch_data TYPE c.
2035    DATA: ld_ref_to_ch_data TYPE REF TO if_ex_fagl_items_ch_data.
2036    DATA: lt_fields TYPE ttfieldname.
2037    DATA: ls_fieldname TYPE fieldname.
2038
2039  * in Arbeit
2040
2041  *...define icon fields:
2042
2043    CLEAR gt_fieldcat.
2044    gt_fieldcat-fieldname = 'SYM_AUGP'.
2045    gt_fieldcat-symbol    = 'X'.
2046    APPEND gt_fieldcat.
2047    CLEAR gt_fieldcat.
2048    gt_fieldcat-fieldname = 'ICO_AUGP'.
2049    gt_fieldcat-icon      = 'X'.
2050    APPEND gt_fieldcat.
2051    CLEAR gt_fieldcat.
2052    gt_fieldcat-fieldname = 'ICO_DUE'.
2053    gt_fieldcat-icon      = 'X'.
2054    APPEND gt_fieldcat.
2055    CLEAR gt_fieldcat.
2056    gt_fieldcat-fieldname = 'ICO_DUE1'.
2057    gt_fieldcat-icon      = 'X'.
2058    APPEND gt_fieldcat.
2059
2060  *...define technical and obsolete fields (static):
2061
2062    CLEAR gt_fieldcat.
2063    gt_fieldcat-fieldname = 'ACTIV'.
```

```
2064    gt_fieldcat-tech      = 'X'.
2065    APPEND gt_fieldcat.
2066    CLEAR gt_fieldcat.
2067    gt_fieldcat-fieldname = 'STAKZ'.
2068    gt_fieldcat-tech      = 'X'.
2069    APPEND gt_fieldcat.
2070    CLEAR gt_fieldcat.
2071    gt_fieldcat-fieldname = 'XAUGP'.
2072    gt_fieldcat-tech      = 'X'.
2073    APPEND gt_fieldcat.
2074    CLEAR gt_fieldcat.
2075    gt_fieldcat-fieldname = 'COLOR'.
2076    gt_fieldcat-tech      = 'X'.
2077    APPEND gt_fieldcat.
2078    CLEAR gt_fieldcat.
2079    gt_fieldcat-fieldname = 'XZAHL'.
2080    gt_fieldcat-tech      = 'X'.
2081    APPEND gt_fieldcat.
2082
2083  * DLA long procurement number longnum  ERP05
2084    DATA: cl_change TYPE REF TO /sappspro/cl_numbers,
2085          lf_active TYPE boolean.
2086    CREATE OBJECT cl_change.
2087    CALL METHOD cl_change->is_active
2088      RECEIVING
2089        rv_active = lf_active.
2090  * hide LONGNUM if DLA is not active
2091    IF lf_active = space.
2092      CLEAR gt_fieldcat.
2093      gt_fieldcat-fieldname = 'LONGNUM'.
2094      gt_fieldcat-tech      = 'X'.
2095      APPEND gt_fieldcat.
2096    ENDIF.
2097
2098  * hide GKONT and GKART, if BADI not used:
2099    CALL METHOD cl_exithandler=>get_instance
2100      EXPORTING
2101        exit_name             = 'FAGL_ITEMS_CH_DATA'
2102        null_instance_accepted = 'X'
2103      IMPORTING
2104        act_imp_existing      = ld_act_imp_ch_data
2105      CHANGING
2106        instance              = ld_ref_to_ch_data.
2107
2108    IF ld_act_imp_ch_data IS INITIAL.
2109  * Not needed anymore as the related GKONT is in ACDOCA already"2588267
2110  *    CLEAR gt_fieldcat.                                      "2588267
2111  *    gt_fieldcat-fieldname = 'GKONT'.                        "2588267
2112  *    gt_fieldcat-tech      = 'X'.                            "2588267
2113  *    APPEND gt_fieldcat.                                     "2588267
2114      CLEAR gt_fieldcat.
2115      gt_fieldcat-fieldname = 'GKART'.
2116      gt_fieldcat-tech      = 'X'.
2117      APPEND gt_fieldcat.
2118    ENDIF.
2119
2120  *...define obsolete fields (dynamic):
2121
2122  * check for existence of special fields:
2123    READ TABLE it_t021s INDEX 1.
2124    CHECK sy-subrc = 0.
```

```
2125
2126  * get RFPOSX field list (here RFPOSX was used):
2127    CALL FUNCTION 'DDIF_NAMETAB_GET'
2128      EXPORTING
2129        tabname   = 'FAGLPOSE'
2130      TABLES
2131  *     X031L_TAB =
2132        dfies_tab = lt_dfies_faglpose
2133      EXCEPTIONS
2134        not_found = 1
2135        OTHERS    = 2.
2136    CHECK sy-subrc = 0.
2137
2138  * eliminate T021s fields from faglposx
2139  *  loop at lt_dfies_faglposx.
2140  *    if lt_dfies_faglposx(2) = 'U_'.
2141  *      delete lt_dfies_faglposx.
2142  *    endif.
2143  *  endloop.
2144
2145  * collect duplicates = special fields which are contained in FAGLPOSX
2146  * (RFPOSX):
2147    LOOP AT it_t021s.
2148      CHECK NOT it_t021s-tname IS INITIAL.
2149      READ TABLE lt_dfies_faglpose WITH KEY
2150                                    fieldname = it_t021s-fname.
2151      CHECK sy-subrc = 0.
2152      lt_duplicates-fname = it_t021s-fname.
2153      APPEND lt_duplicates.
2154    ENDLOOP.
2155
2156  * found duplicates?
2157    READ TABLE lt_duplicates INDEX 1.
2158    CHECK sy-subrc = 0.
2159
2160  * get BSIx field list:
2161    CASE g_repid.
2162  *    when c_repid_ar.
2163  *      ld_name_bsix = 'BSID'.
2164  *    when c_repid_ap.
2165  *      ld_name_bsix = 'BSIK'.
2166  *    when c_repid_gl.
2167  *      ld_name_bsix = 'BSIS'.
2168      WHEN c_repid_ar.
2169        ld_name_bsix = 'BSID'.
2170      WHEN c_repid_ap.
2171        ld_name_bsix = 'BSIK'.
2172      WHEN c_repid_gl.
2173        ld_name_bsix = 'BSIS'.
2174  *    when c_repid_FAGl_GL.
2175  *      ld_name_bsix = 'BSIS'.
2176    ENDCASE.
2177    CALL FUNCTION 'DDIF_NAMETAB_GET'
2178      EXPORTING
2179        tabname   = ld_name_bsix
2180      TABLES
2181  *     X031L_TAB =
2182        dfies_tab = lt_dfies_bsix
2183      EXCEPTIONS
2184        not_found = 1
2185        OTHERS    = 2.
```

```
2186    CHECK sy-subrc = 0.
2187
2188    IF NOT gv_relevant_ledger IS INITIAL.
2189  * add converted FlexGl fields to fieldlist of BSIx
2190      PERFORM get_flexgl_fields USING gv_relevant_ledger
2191                                CHANGING lt_fields.
2192      LOOP AT lt_dfies_bsix.
2193        ls_fieldname = lt_dfies_bsix-fieldname.
2194        COLLECT ls_fieldname INTO lt_fields.
2195      ENDLOOP.
2196
2197  * check whether duplicates are contained in BSIx:
2198      LOOP AT lt_duplicates.
2199        CLEAR ld_fname.
2200        READ TABLE lt_fields INTO ls_fieldname
2201          WITH KEY lt_duplicates-fname.
2202        if sy-subrc = 0 and lt_duplicates-fname ne 'ZUONR'.
2203  *       standard field ok. set special field to 'tech':
2204          ld_fname = 'U_'.
2205          WRITE lt_duplicates-fname TO ld_fname+2.
2206        ELSE.
2207  *       special field ok. set standard field to 'tech':
2208          ld_fname = lt_duplicates-fname.
2209        ENDIF.
2210        CLEAR gt_fieldcat.
2211        gt_fieldcat-fieldname = ld_fname.
2212        gt_fieldcat-tech      = 'X'.
2213        APPEND gt_fieldcat.
2214      ENDLOOP.
2215    ELSE.
2216  * check whether duplicates are contained in BSIx:
2217      LOOP AT lt_duplicates.
2218        CLEAR ld_fname.
2219        READ TABLE lt_dfies_bsix WITH KEY
2220                                 fieldname = lt_duplicates-fname.
2221        if sy-subrc = 0 and lt_duplicates-fname ne 'ZUONR'.
2222  *       standard field ok. set special field to 'tech':
2223          ld_fname = 'U_'.
2224          WRITE lt_duplicates-fname TO ld_fname+2.
2225        ELSE.
2226  *       special field ok. set standard field to 'tech':
2227          ld_fname = lt_duplicates-fname.
2228        ENDIF.
2229        CLEAR gt_fieldcat.
2230        gt_fieldcat-fieldname = ld_fname.
2231        gt_fieldcat-tech      = 'X'.
2232        APPEND gt_fieldcat.
2233      ENDLOOP.
2234    ENDIF.
2235
2236  ENDFORM.                                " MAKE_FIELDCATALOG
2237
2238
2239  *&---------------------------------------------------------------------*
2240  *&      Form  make_fieldcatalog2
2241  *&      set TP-fields as TECH if TP-authority doesn't exists
2242  *&---------------------------------------------------------------------*
2243  FORM make_fieldcatalog2.
2244    DATA: ld_tpauth TYPE c.
2245
2246  * remove tp table entries
```

```
2247    LOOP AT it_tp_fields.
2248      DELETE gt_fieldcat WHERE fieldname = it_tp_fields-fname.
2249    ENDLOOP.
2250    LOOP AT it_tp_field2.
2251      DELETE gt_fieldcat WHERE fieldname = it_tp_field2-fname.
2252    ENDLOOP.
2253
2254  * check transfer price authority
2255    PERFORM check_tp_auth.
2256
2257  * if authority is missing: field = tech
2258    CLEAR gd_tpauth.
2259    ld_tpauth = 'X'.
2260    LOOP AT it_tp_auth.
2261      IF sy-tabix = 1.
2262        ld_tpauth = it_tp_auth-auth.
2263      ENDIF.
2264      IF it_tp_auth-auth IS INITIAL.
2265        CLEAR ld_tpauth.
2266      ENDIF.
2267      IF NOT ld_tpauth IS INITIAL.
2268        IF ld_tpauth = 'X'
2269        AND ( it_tp_auth-auth = '1'
2270        OR it_tp_auth-auth = '2' ).
2271          ld_tpauth = it_tp_auth-auth.
2272        ENDIF.
2273        IF ( ld_tpauth = '1'
2274        AND it_tp_auth-auth = '2' )
2275        OR ( ld_tpauth = '2'
2276        AND it_tp_auth-auth = '1' ).
2277          CLEAR ld_tpauth.
2278        ENDIF.
2279      ENDIF.
2280    ENDLOOP.
2281    gd_tpauth = ld_tpauth.
2282
2283    CASE gd_tpauth.
2284      WHEN space.
2285  * hide 2. and 3. HW
2286        LOOP AT it_tp_fields.
2287          CLEAR gt_fieldcat.
2288          gt_fieldcat-fieldname = it_tp_fields-fname.
2289          gt_fieldcat-tech = 'X'.
2290          APPEND gt_fieldcat.
2291        ENDLOOP.
2292        LOOP AT it_tp_field2.
2293          CLEAR gt_fieldcat.
2294          gt_fieldcat-fieldname = it_tp_field2-fname.
2295          gt_fieldcat-tech = 'X'.
2296          APPEND gt_fieldcat.
2297        ENDLOOP.
2298      WHEN '1'.
2299  * hide 3. HW
2300        LOOP AT it_tp_field2.
2301          CLEAR gt_fieldcat.
2302          gt_fieldcat-fieldname = it_tp_field2-fname.
2303          gt_fieldcat-tech = 'X'.
2304          APPEND gt_fieldcat.
2305        ENDLOOP.
2306      WHEN '2'.
2307  * hide 2. HW
```

```
2308        LOOP AT it_tp_fields.
2309          CLEAR gt_fieldcat.
2310          gt_fieldcat-fieldname = it_tp_fields-fname.
2311          gt_fieldcat-tech = 'X'.
2312          APPEND gt_fieldcat.
2313        ENDLOOP.
2314      ENDCASE.
2315    *  read table it_tp_auth with key auth = space.
2316    *  if sy-subrc eq 0.
2317    *    loop at it_tp_fields.
2318    *      clear gt_fieldcat.
2319    *      gt_fieldcat-fieldname = it_tp_fields-fname.
2320    *      gt_fieldcat-tech = 'X'.
2321    *      append gt_fieldcat.
2322    *    endloop.
2323    *  endif.
2324
2325    * begin of "note 2398274
2326      DATA  ls_bukrs_tp_curtp TYPE cl_fins_acdoc_util=>ts_bukrs_tp_curtp. "T
       ransfer Pricing authority check "note 2398274
2327
2328      LOOP AT gt_bukrs_tp_curtp INTO ls_bukrs_tp_curtp. "note 2398274
2329        IF ls_bukrs_tp_curtp-auth = abap_false.
2330          CLEAR gt_fieldcat.
2331          gt_fieldcat-fieldname = ls_bukrs_tp_curtp-field_dmbe.
2332          gt_fieldcat-tech = 'X'.
2333          APPEND gt_fieldcat.
2334
2335          CLEAR gt_fieldcat.
2336          gt_fieldcat-fieldname = ls_bukrs_tp_curtp-field_hwae.
2337          gt_fieldcat-tech = 'X'.
2338          APPEND gt_fieldcat.
2339
2340        ENDIF.
2341      ENDLOOP.
2342    *  end of "note 2398274.
2343
2344    ENDFORM.                    " make_fieldcatalog2
2345
2346
2347    *&---------------------------------------------------------------------*
2348    *&      Form  dynp_get_status
2349    *&---------------------------------------------------------------------*
2350    FORM dynp_get_status USING    id_fun
2351                         CHANGING cd_val.
2352      DATA: ld_val_save TYPE i.
2353
2354      ld_val_save = cd_val.
2355      CALL 'DYNP_GET_STATUS'
2356             ID 'FUNCTION' FIELD id_fun
2357             ID 'VALUE'    FIELD cd_val.
2358      IF sy-subrc NE 0.
2359        cd_val = ld_val_save.
2360      ENDIF.
2361
2362    ENDFORM.                                " dynp_get_status
2363
2364    *&---------------------------------------------------------------------*
2365    *&      Form  export_filitexts_data
2366    *&---------------------------------------------------------------------*
2367    FORM export_filitexts_data.
```

```
2368
2369    EXPORT stida FROM pa_stida_default
2370           TO MEMORY ID 'FILITXT01'.
2371
2372  ENDFORM.                                 " export_filitexts_data
2373  *---------------------------------------------------------------------*
2374  *   form wl_memory_set_get
2375  *   set/get parameter for worklist setting
2376  *---------------------------------------------------------------------*
2377  FORM wl_memory_set_get  USING    id_set    TYPE c
2378                          CHANGING cd_wl_on  TYPE c.
2379
2380    IF id_set IS INITIAL.
2381      GET PARAMETER ID 'FIT_WL' FIELD cd_wl_on.
2382    ELSE.
2383      SET PARAMETER ID 'FIT_WL' FIELD cd_wl_on.
2384    ENDIF.
2385
2386  ENDFORM.                    "wl_memory_set_get
2387
2388  *---------------------------------------------------------------------*
2389  *   form wl_flag_and_button
2390  *   set global flag and button text according to static memory
2391  *---------------------------------------------------------------------*
2392  FORM wl_flag_and_button.
2393
2394  * icon freie selection
2395    PERFORM fs_set_sscrtexts_dynsel CHANGING fs_f1_icon.
2396    sscrfields-functxt_01 = fs_f1_icon.
2397
2398  * text worklist button
2399    PERFORM wl_memory_set_get USING space
2400                                    gd_wl_on.
2401    IF gd_wl_on IS INITIAL.
2402      sscrfields-functxt_02 = TEXT-w01.
2403    ELSE.
2404      sscrfields-functxt_02 = TEXT-w04.
2405    ENDIF.
2406
2407  * icon change ledger
2408  *  sscrfields-functxt_03 = text-272.
2409    PERFORM fs_set_sscrtexts_dynsel3 CHANGING fs_f3_icon.
2410    sscrfields-functxt_03 = fs_f3_icon.
2411
2412  * icon change view
2413  *  if gv_xerfs is initial.
2414  *    sscrfields-functxt_04 = text-278.
2415  *  else.
2416  *    sscrfields-functxt_04 = text-277.
2417  *  endif.
2418    PERFORM fs_set_sscrtexts_dynsel4 CHANGING fs_f4_icon.
2419    sscrfields-functxt_04 = fs_f4_icon.
2420
2421  ENDFORM.                    "wl_flag_and_button
2422
2423  *---------------------------------------------------------------------*
2424  *   form worklist_on_off
2425  *   set worklist flag on <-> off, set button text accordingly
2426  *---------------------------------------------------------------------*
2427  FORM worklist_on_off.
2428
```

```
2429    DATA: ld_value   TYPE c.
2430
2431    IF gd_wl_on = c_x.
2432      PERFORM wl_memory_set_get USING c_x
2433                                      ld_value.
2434    ELSE.
2435      ld_value = c_x.
2436      PERFORM wl_memory_set_get USING c_x
2437                                      ld_value.
2438    ENDIF.
2439    PERFORM wl_flag_and_button.
2440
2441  ENDFORM.                    "worklist_on_off
2442
2443  *---------------------------------------------------------------------*
2444  *   form wl_modify_screen
2445  *   hide screen fields according to worklist flag setting
2446  *---------------------------------------------------------------------*
2447  FORM wl_modify_screen.
2448
2449    IF gd_wl_on IS INITIAL.
2450  *   worklist off -> hide worklist layout:
2451      IF screen-group1 = 'WKL'.
2452        screen-active    = 0.
2453        screen-invisible = 1.
2454      ENDIF.
2455    ELSE.
2456  *   worklist on -> hide LDB layout:
2457      IF screen-group2 = 'DBS'.
2458        screen-active    = 0.
2459        screen-invisible = 1.
2460      ENDIF.
2461    ENDIF.
2462
2463  ENDFORM.                    "wl_modify_screen
2464
2465  *---------------------------------------------------------------------*
2466  *   form resolve_worklist
2467  *   read worklist table entries for a given worklist identifier
2468  *---------------------------------------------------------------------*
2469  FORM resolve_worklist  TABLES et_cosel TYPE tcosel
2470                         USING  id_worklist LIKE tfav-ident
2471                                id_objct    LIKE tfav-objct.
2472
2473    DATA: lt_tfavw TYPE STANDARD TABLE OF tfavw WITH HEADER LINE.
2474
2475    REFRESH et_cosel.
2476    CALL FUNCTION 'RF_WORKLISTS_VALUES_GET'
2477      EXPORTING
2478        id_objct     = id_objct
2479        id_ident     = id_worklist
2480      TABLES
2481        et_tfavw     = lt_tfavw
2482      EXCEPTIONS
2483        not_existing = 1.
2484    IF sy-subrc NE 0.
2485      deco_message_1 e040(msitem) with id_worklist.
2486    ENDIF.
2487    et_cosel-field  = id_objct.
2488    et_cosel-sign   = 'I'.
2489    et_cosel-option = 'EQ'.
```

```
2490    CLEAR et_cosel-high.
2491    LOOP AT lt_tfavw.
2492      et_cosel-low = lt_tfavw-value.
2493      APPEND et_cosel.
2494    ENDLOOP.
2495
2496  ENDFORM.                    "resolve_worklist
2497
2498  *&---------------------------------------------------------------------*
2499  *&      Form  CHANGE_STATUS
2500  *&---------------------------------------------------------------------*
2501  FORM change_status.
2502
2503    DATA: BEGIN OF lt_exclude OCCURS 5,
2504            fcode LIKE sy-ucomm,
2505          END OF lt_exclude.
2506    DATA: ld_flag(1) TYPE c.
2507    DATA: ls_rssubinfo LIKE rssubinfo.
2508
2509
2510    CALL FUNCTION 'RS_SUBMIT_INFO'
2511      IMPORTING
2512        p_submit_info = ls_rssubinfo
2513      EXCEPTIONS
2514        OTHERS        = 1.
2515
2516    IF NOT ls_rssubinfo-mode_norml IS INITIAL.
2517  * check: worklist functionality active?
2518  *  get parameter id 'FI_WORKLISTS_FLAG' field ld_flag.
2519  *  if ld_flag is initial.
2520  *    lt_exclude-fcode = 'FC01'.
2521  *    append lt_exclude.
2522  **   clear user preference parameter:
2523  *    set parameter id 'FIT_WL' field space.
2524  *  endif.
2525  *  lt_exclude-fcode = 'FC01'.
2526  *  append lt_exclude.
2527      GET PARAMETER ID 'FI_WORKLISTS_FLAG' FIELD ld_flag.
2528      IF ld_flag IS INITIAL.
2529        lt_exclude-fcode = 'FC02'.
2530        APPEND lt_exclude.
2531  *   clear user preference parameter:
2532        SET PARAMETER ID 'FIT_WL' FIELD space.
2533      ENDIF.
2534      IF NOT gv_xerfs IS INITIAL.
2535        lt_exclude-fcode = 'FC03'.
2536        APPEND lt_exclude.
2537      ENDIF.
2538  *   lt_exclude-fcode = 'FC04'.
2539  *   append lt_exclude.
2540  *   lt_exclude-fcode = 'FC05'.
2541  *   append lt_exclude.
2542
2543  * deactivate the dyn. Selections of Log. DB
2544      lt_exclude-fcode = 'DYNS'.
2545      APPEND lt_exclude.
2546
2547      CALL FUNCTION 'RS_SET_SELSCREEN_STATUS'
2548        EXPORTING
2549          p_status  = sy-pfkey
2550        TABLES
```

```
2551          p_exclude = lt_exclude.
2552
2553    ENDIF.
2554
2555  ENDFORM.                                " CHANGE_STATUS
2556  *---------------------------------------------------------------------*
2557  *   form dd_get_flag
2558  *   get due date selection flag PID value
2559  *---------------------------------------------------------------------*
2560  FORM dd_get_flag.
2561
2562    GET PARAMETER ID 'FIT_DUE_DATE_SEL' FIELD gd_ddsel_on.
2563
2564  ENDFORM.                    "dd_get_flag
2565
2566  *---------------------------------------------------------------------*
2567  *   form dd_modify_screen
2568  *   hide screen fields according to due date selection PID setting
2569  *---------------------------------------------------------------------*
2570  FORM dd_modify_screen.
2571
2572    IF gd_ddsel_on IS INITIAL.
2573  *   parameter off -> hide due date selection field:
2574      IF screen-group1 = 'DUE'.
2575        screen-active    = 0.
2576        screen-invisible = 1.
2577      ENDIF.
2578    ENDIF.
2579
2580  ENDFORM.                    "dd_modify_screen
2581
2582  *&---------------------------------------------------------------------*
2583  *&      Form  COMPLETE_DOCUMENT_TEXT
2584  *&---------------------------------------------------------------------*
2585  FORM complete_document_text.
2586
2587    DATA: lt_ttxid      LIKE ttxid OCCURS 1 WITH HEADER LINE.
2588
2589    LOOP AT it_pos.
2590      CALL FUNCTION 'FI_TEXTS_DOCUMENT'
2591        EXPORTING
2592          i_aktyp         = 'A'
2593          i_belnr         = it_pos-belnr
2594          i_bukrs         = it_pos-bukrs
2595          i_check         = 'X'
2596          i_gjahr         = it_pos-gjahr
2597          i_object        = 'BELEG'
2598          i_give_back_ids = 'X'
2599        TABLES
2600          t_tdid          = lt_ttxid
2601        EXCEPTIONS
2602          no_texts_found  = 1
2603          OTHERS          = 2.
2604      IF sy-subrc = 0.
2605        it_pos-xtext = 'X'.
2606        READ TABLE lt_ttxid INDEX 1.
2607        IF sy-subrc = 0.
2608          it_pos-tdid = lt_ttxid-tdid.
2609        ENDIF.
2610      ENDIF.
2611      MODIFY it_pos.
```

```
2612    ENDLOOP.
2613
2614  ENDFORM.                    "complete_document_text
2615  *---------------------------------------------------------------------*
2616  *   INCLUDE RFITEM_INC                                                *
2617  *---------------------------------------------------------------------*
2618
2619  *&---------------------------------------------------------------------*
2620  *&      Expiring currencies: INCLUDE RFDBCONVCUR
2621  *&---------------------------------------------------------------------*
2622  INCLUDE rfdbexpc.
2623
2624  *&---------------------------------------------------------------------*
2625  *&      Form  init_expcur
2626  *&---------------------------------------------------------------------*
2627  FORM init_expcur.
2628
2629  * expiring currencies relevant for this process?
2630    gd_expcur_proc = 'SAPDB'.
2631    WRITE sy-dbnam TO gd_expcur_proc+5.
2632    CALL FUNCTION 'CURRENCY_CHECK_FOR_PROCESS'
2633      EXPORTING
2634        process                = gd_expcur_proc
2635      EXCEPTIONS
2636        process_not_maintained = 1
2637        OTHERS                 = 2.
2638    IF sy-subrc = 0.
2639      gd_expcur_flag = 'X'.
2640    ENDIF.
2641
2642  ENDFORM.                    " init_expcur
2643
2644  *&---------------------------------------------------------------------*
2645  *&      Form  insert_tp_fields
2646  *&      insert fields which are used for transfer prices
2647  *&---------------------------------------------------------------------*
2648  FORM insert_tp_fields.
2649
2650    CLEAR it_tp_fields. it_tp_fields-fname = 'DMBE2'. APPEND it_tp_fields.
2651    CLEAR it_tp_field2. it_tp_field2-fname = 'DMBE3'. APPEND it_tp_field2.
2652
2653    CLEAR it_tp_fields. it_tp_fields-fname = 'BWWR2'. APPEND it_tp_fields.
2654    CLEAR it_tp_field2. it_tp_field2-fname = 'BWWR3'. APPEND it_tp_field2.
2655
2656    CLEAR it_tp_fields. it_tp_fields-fname = 'HWAE2'. APPEND it_tp_fields.
2657    CLEAR it_tp_field2. it_tp_field2-fname = 'HWAE3'. APPEND it_tp_field2.
2658
2659    PERFORM check_tp_t021s USING 'DMBT2'.
2660    PERFORM check_tp_t021s_2 USING 'DMBT3'.
2661
2662    PERFORM check_tp_t021s USING 'DMB21'.
2663    PERFORM check_tp_t021s USING 'DMB22'.
2664    PERFORM check_tp_t021s USING 'DMB23'.
2665
2666    PERFORM check_tp_t021s_2 USING 'DMB31'.
2667    PERFORM check_tp_t021s_2 USING 'DMB32'.
2668    PERFORM check_tp_t021s_2 USING 'DMB33'.
2669
2670    PERFORM check_tp_t021s USING 'MWST2'.
2671    PERFORM check_tp_t021s_2 USING 'MWST3'.
2672
```

```
2673    PERFORM check_tp_t021s USING 'NAVH2'.
2674    PERFORM check_tp_t021s_2 USING 'NAVH3'.
2675
2676    PERFORM check_tp_t021s USING 'SKNT2'.
2677    PERFORM check_tp_t021s_2 USING 'SKNT3'.
2678
2679    PERFORM check_tp_t021s USING 'BDIF2'.
2680    PERFORM check_tp_t021s_2 USING 'BDIF3'.
2681
2682    PERFORM check_tp_t021s USING 'RDIF2'.
2683    PERFORM check_tp_t021s_2 USING 'RDIF3'.
2684
2685    PERFORM check_tp_t021s USING 'TXBH2'.
2686    PERFORM check_tp_t021s_2 USING 'TXBH3'.
2687
2688    PERFORM check_tp_t021s USING 'PPDIF2'.
2689    PERFORM check_tp_t021s_2 USING 'PPDIF3'.
2690
2691  ENDFORM.                    " insert_tp_fields
2692
2693
2694  *&---------------------------------------------------------------------*
2695  *&      Form  check_tp_t021s
2696  *&      insert tp fields if they exists in t021s
2697  *&---------------------------------------------------------------------*
2698  FORM check_tp_t021s USING tp_fname.
2699
2700    READ TABLE it_t021s WITH KEY fname = tp_fname.
2701    IF sy-subrc = 0.
2702      CLEAR it_tp_fields.
2703      CONCATENATE 'U_' tp_fname INTO it_tp_fields-fname.
2704      APPEND it_tp_fields.
2705    ENDIF.
2706
2707  ENDFORM.                    " check_tp_t021s
2708
2709  *&---------------------------------------------------------------------*
2710  *&      Form  check_tp_t021s_2
2711  *&      insert tp fields if they exists in t021s
2712  *&---------------------------------------------------------------------*
2713  FORM check_tp_t021s_2 USING tp_fname.
2714
2715    READ TABLE it_t021s WITH KEY fname = tp_fname.
2716    IF sy-subrc = 0.
2717      CLEAR it_tp_field2.
2718      CONCATENATE 'U_' tp_fname INTO it_tp_field2-fname.
2719      APPEND it_tp_field2.
2720    ENDIF.
2721
2722  ENDFORM.                    " check_tp_t021s_2
2723
2724  *&---------------------------------------------------------------------*
2725  *&      Form check_tp_auth.
2726  *&      determine auth and set auth-flag in it_tp_auth
2727  *&---------------------------------------------------------------------*
2728  FORM check_tp_auth.
2729
2730    DATA: ld_lines      TYPE i,
2731          ld_tpauth(1)  TYPE c,
2732          ld_tpauth1(1) TYPE c,
2733          ld_tpauth2(1) TYPE c.
```

```
2734
2735    DATA lv_tp_auth(1) TYPE c.
2736
2737  * company code with transfer prices
2738    DESCRIBE TABLE it_tp_auth LINES ld_lines.
2739
2740  * check transfer prices.
2741    IF ld_lines GT 0.
2742
2743  * check company codes.
2744      LOOP AT it_tp_auth.
2745        CLEAR ld_tpauth1.
2746        CLEAR ld_tpauth2.
2747        IF NOT it_tp_auth-curtp IS INITIAL.
2748          IF cl_fins_valutyp=>get_valutyp( it_tp_auth-curtp ) = '1'
2749            OR  cl_fins_valutyp=>get_valutyp( it_tp_auth-curtp ) = '2'.
2750            CALL FUNCTION 'TP_VALUATION_AUTHORITY'
2751              EXPORTING
2752                i_bukrs                          = it_tp_auth-bukrs
2753                i_cvtyp                          = it_tp_auth-curtp
2754                i_actvt                          = '03'
2755              IMPORTING
2756                e_xauth                          = ld_tpauth1
2757              EXCEPTIONS
2758                kokrs_finding_error              = 1
2759                valutyp_finding_error            = 2
2760                insufficient_input_for_kokrs     = 3
2761                insufficient_input_for_valutyp = 4
2762                OTHERS                           = 5.
2763            IF sy-subrc <> 0.
2764              deco_message_sy.
2765            ENDIF.
2766          ELSE.
2767            IF cl_fins_valutyp=>get_valutyp( it_tp_auth-curtp ) = '0'.
2768              ld_tpauth1 = 'X'.
2769            ENDIF.
2770          ENDIF.
2771        ELSE.                                                    "3251156
2772         IF X_ERFS IS INITIAL.                                   "3251156
2773           ld_tpauth1 = 'X'.                                     "3251156
2774         ENDIF.                                                  "3251156
2775        ENDIF.
2776        IF NOT it_tp_auth-curtp2 IS INITIAL.
2777          IF it_tp_auth-curtp2+1(1) = '1'
2778          OR it_tp_auth-curtp2+1(1) = '2'.
2779            CALL FUNCTION 'TP_VALUATION_AUTHORITY'
2780              EXPORTING
2781                i_bukrs                          = it_tp_auth-bukrs
2782                i_cvtyp                          = it_tp_auth-curtp2
2783                i_actvt                          = '03'
2784              IMPORTING
2785                e_xauth                          = ld_tpauth2
2786              EXCEPTIONS
2787                kokrs_finding_error              = 1
2788                valutyp_finding_error            = 2
2789                insufficient_input_for_kokrs     = 3
2790                insufficient_input_for_valutyp = 4
2791                OTHERS                           = 5.
2792            IF sy-subrc <> 0.
2793              deco_message_sy.
2794            ENDIF.
```

```
2795          ELSE.
2796            IF it_tp_auth-curtp2+1(1) = '0'.
2797              ld_tpauth2 = 'X'.
2798            ENDIF.
2799          ENDIF.
2800        ELSE.                                                    "3251156
2801         IF X_ERFS IS INITIAL.                                   "3251156
2802           ld_tpauth2 = 'X'.                                     "3251156
2803         ENDIF.                                                  "3251156
2804        ENDIF.
2805        IF  NOT ld_tpauth1 IS INITIAL
2806        AND NOT ld_tpauth2 IS INITIAL.
2807          ld_tpauth = 'X'.
2808        ELSE.
2809          IF NOT ld_tpauth1 IS INITIAL.
2810            ld_tpauth = '1'.
2811          ENDIF.
2812          IF NOT ld_tpauth2 IS INITIAL.
2813            ld_tpauth = '2'.
2814          ENDIF.
2815        ENDIF.
2816        it_tp_auth-auth = ld_tpauth.
2817        MODIFY it_tp_auth.
2818      ENDLOOP.
2819    ENDIF.
2820
2821  * begin of "Transfer Pricing authority check
2822    FIELD-SYMBOLS <bukrs_tp_curtp> TYPE cl_fins_acdoc_util=>ts_bukrs_tp_cu
     rtp.
2823
2824    LOOP AT gt_bukrs_tp_curtp ASSIGNING <bukrs_tp_curtp>.
2825
2826      CLEAR lv_tp_auth.
2827
2828      CALL FUNCTION 'TP_VALUATION_AUTHORITY'
2829        EXPORTING
2830          i_bukrs                          = <bukrs_tp_curtp>-bukrs
2831          i_cvtyp                          = <bukrs_tp_curtp>-tp_curtp
2832          i_actvt                          = '03'
2833        IMPORTING
2834          e_xauth                          = lv_tp_auth
2835        EXCEPTIONS
2836          kokrs_finding_error              = 1
2837          valutyp_finding_error            = 2
2838          insufficient_input_for_kokrs     = 3
2839          insufficient_input_for_valutyp = 4
2840          OTHERS                           = 5.
2841      IF sy-subrc <> 0.
2842        deco_message_sy.
2843      ENDIF.
2844      <bukrs_tp_curtp>-auth = lv_tp_auth.
2845
2846    ENDLOOP.
2847  * begin of "Transfer Pricing authority check
2848
2849  ENDFORM.                    " check_tp_auth
2850
2851
2852  *&---------------------------------------------------------------------*
2853  *&      Form  get_existing_variant
2854  *&
```

```
2855  *&---------------------------------------------------------------------*
2856  FORM get_existing_variant USING rs_variant LIKE disvariant.
2857
2858    DATA ld_use_default(1) TYPE c.
2859
2860    CLEAR ld_use_default.
2861
2862    IF rs_variant-variant = space.
2863  *   try ALV default variant
2864      CALL FUNCTION 'REUSE_ALV_VARIANT_DEFAULT_GET'
2865        EXPORTING
2866          i_save     = 'A'
2867        CHANGING
2868          cs_variant = rs_variant
2869        EXCEPTIONS
2870          OTHERS     = 4.
2871      IF sy-subrc NE 0.
2872        ld_use_default = 'X'.
2873      ENDIF.
2874    ENDIF.
2875
2876    IF ld_use_default = space.
2877      CALL FUNCTION 'REUSE_ALV_VARIANT_EXISTENCE'
2878        EXPORTING
2879          i_save        = 'A'
2880        CHANGING
2881          cs_variant    = rs_variant
2882        EXCEPTIONS
2883          wrong_input   = 1
2884          not_found     = 2
2885          program_error = 3
2886          OTHERS        = 4.
2887      IF sy-subrc <> 0.
2888        ld_use_default = 'X'.
2889      ENDIF.
2890    ENDIF.
2891
2892    IF ld_use_default = 'X'.
2893      rs_variant-report = g_repid.
2894      IF gv_xerfs = 'X'.                              "Transfer Pricing
2895        rs_variant-variant = '1SAP'.
2896      ELSE.                                           "Transfer Pricing
2897        rs_variant-variant = '1SAP-GLVIEW'.           "Transfer Pricing
2898      ENDIF.                                          "Transfer Pricing
2899      rs_variant-username = sy-uname.
2900      CALL FUNCTION 'REUSE_ALV_VARIANT_EXISTENCE'
2901        EXPORTING
2902          i_save        = 'A'
2903        CHANGING
2904          cs_variant    = rs_variant
2905        EXCEPTIONS
2906          wrong_input   = 1
2907          not_found     = 2
2908          program_error = 3
2909          OTHERS        = 4.
2910      IF sy-subrc <> 0.
2911        CLEAR rs_variant.
2912      ENDIF.
2913    ENDIF.
2914
2915  ENDFORM.                    " get_existing_variant
```

```
2916  *&---------------------------------------------------------------------*
2917  *&      Form  check_date
2918  *&---------------------------------------------------------------------*
2919  *       check
2920  *----------------------------------------------------------------------*
2921  *      -->P_KD_BUKRS  text
2922  *----------------------------------------------------------------------*
2923  FORM check_date TABLES   p_xd_bukrs
2924                  USING    p_xd_tpc.
2925
2926    DATA: ld_program LIKE sy-repid.
2927    STATICS: s_return TYPE char1.
2928    DATA: lt_bukrs TYPE bukrs OCCURS 0 WITH HEADER LINE.
2929
2930    p_xd_tpc = space.
2931
2932    IF  pa_stida_default IS INITIAL
2933    AND NOT x_opsel IS INITIAL.
2934      pa_stida_default = pa_stida.
2935    ENDIF.
2936
2937    CHECK s_return = 1 OR s_return IS INITIAL.
2938    CHECK x_opsel = 'X' AND ( NOT pa_stida_default IS INITIAL ) .
2939
2940    CASE g_repid.
2941  *    when 'RFITEMAP'.
2942  *      ld_program = 'SAPDBKDF'.
2943  *    when 'RFITEMAR'.
2944  *      ld_program = 'SAPDBDDF'.
2945  *    when 'RFITEMGL'.
2946  *      ld_program = 'SAPDBSDF'.
2947      WHEN 'FAGL_ACCOUNT_ITEMS_AP'.
2948        ld_program = 'SAPDBKDF'.
2949      WHEN 'FAGL_ACCOUNT_ITEMS_AR'.
2950        ld_program = 'SAPDBDDF'.
2951      WHEN 'FAGL_ACCOUNT_ITEMS_GL'.
2952        ld_program = 'SAPDBSDF'.
2953    ENDCASE.
2954
2955    SELECT bukrs FROM t001 INTO TABLE lt_bukrs
2956      WHERE bukrs IN p_xd_bukrs.
2957    LOOP AT lt_bukrs.
2958      p_xd_tpc = 'X'.
2959      CALL FUNCTION 'FI_CHECK_DATE'
2960        EXPORTING
2961          i_bukrs           = lt_bukrs
2962          i_user            = sy-uname
2963          i_program         = ld_program
2964          i_from_date       = pa_stida_default
2965        IMPORTING
2966          e_return          = s_return
2967        EXCEPTIONS
2968          no_authority_prog = 1
2969          no_authority_date = 2
2970          wrong_parameter   = 3
2971          OTHERS            = 4.
2972      IF sy-subrc <> 0.
2973        p_xd_tpc = space.
2974        deco_message_sy.
2975      ENDIF.
2976
```

```
2977    ENDLOOP.
2978  *  endselect.
2979
2980  ENDFORM.                    " check_date
2981
2982  *&---------------------------------------------------------------------*
2983  *&      Form  AUTHORITY_TCODE
2984  *&---------------------------------------------------------------------*
2985  *       Check authority to use transaction code
2986  *----------------------------------------------------------------------*
2987  *      -->P_Tcode
2988  *      -->P_SUBRC
2989  *----------------------------------------------------------------------*
2990  FORM authority_tcode USING    VALUE(p_tcode)
2991                                p_subrc.
2992
2993    STATICS: BEGIN OF s_tab OCCURS 0,
2994               tcode LIKE sy-tcode,
2995               subrc LIKE sy-subrc,
2996             END OF s_tab.
2997
2998    READ TABLE s_tab WITH KEY tcode = p_tcode.
2999    IF sy-subrc = 0.
3000      p_subrc = s_tab-subrc.
3001      EXIT.
3002    ENDIF.
3003    CLEAR s_tab.
3004    s_tab-tcode = p_tcode.
3005    CALL FUNCTION 'AUTHORITY_CHECK_TCODE'
3006      EXPORTING
3007        tcode  = s_tab-tcode
3008      EXCEPTIONS
3009        ok     = 0
3010        OTHERS = 4.
3011    MOVE sy-subrc TO: p_subrc, s_tab-subrc.
3012    APPEND s_tab.
3013
3014  ENDFORM.                    " AUTHORITY_TCODE
3015
3016
3017  *&---------------------------------------------------------------------*
3018  *&      Form  IT_POS_SORT_BACK
3019  *&---------------------------------------------------------------------*
3020  *       text
3021  *----------------------------------------------------------------------*
3022  *      -->RT_POS     text
3023  *----------------------------------------------------------------------*
3024  FORM it_pos_sort_back TABLES rt_pos.
3025  * basically the same form as used for sorting in the ALV.
3026  * (see FUNC K_KKB_OUTTAB_SORT, FORM FB_OUTTAB_SORT).
3027    DATA: lt_sort TYPE slis_t_sortinfo_alv,
3028          ld_sort TYPE slis_sortinfo_alv.
3029    DATA: a01(30) TYPE c,
3030          a02(30) TYPE c,
3031          a03(30) TYPE c,
3032          a04(30) TYPE c,
3033          a05(30) TYPE c,
3034          a06(30) TYPE c,
3035          a07(30) TYPE c,
3036          a08(30) TYPE c,
3037          a09(30) TYPE c,
```

```
3038          a10(30) TYPE c,
3039          a11(30) TYPE c,
3040          a12(30) TYPE c,
3041          a13(30) TYPE c,
3042          a14(30) TYPE c,
3043          a15(30) TYPE c.
3044    DATA: d01(30) TYPE c,
3045          d02(30) TYPE c,
3046          d03(30) TYPE c,
3047          d04(30) TYPE c,
3048          d05(30) TYPE c,
3049          d06(30) TYPE c,
3050          d07(30) TYPE c,
3051          d08(30) TYPE c,
3052          d09(30) TYPE c,
3053          d10(30) TYPE c,
3054          d11(30) TYPE c,
3055          d12(30) TYPE c,
3056          d13(30) TYPE c,
3057          d14(30) TYPE c,
3058          d15(30) TYPE c.
3059    DATA: l_pos(2) TYPE n.
3060    DATA: l_field(3) TYPE c.
3061    FIELD-SYMBOLS: <sf>.
3062
3063    IF x_grid IS INITIAL.
3064      CALL FUNCTION 'REUSE_ALV_LIST_LAYOUT_INFO_GET'
3065        IMPORTING
3066          et_sort       = lt_sort
3067        EXCEPTIONS
3068          no_infos      = 1
3069          program_error = 2
3070          OTHERS        = 3.
3071    ELSE.
3072      CALL FUNCTION 'REUSE_ALV_GRID_LAYOUT_INFO_GET'
3073        IMPORTING
3074          et_sort       = lt_sort
3075        EXCEPTIONS
3076          no_infos      = 1
3077          program_error = 2
3078          OTHERS        = 3.
3079    ENDIF.
3080
3081    SORT lt_sort BY spos.
3082    CLEAR l_pos.
3083    LOOP AT lt_sort INTO ld_sort.
3084      l_pos = l_pos + 1.
3085      IF ld_sort-up = 'X'.
3086        CONCATENATE 'A' l_pos
3087                        INTO l_field.
3088      ELSE.
3089        CONCATENATE 'D' l_pos
3090                        INTO l_field.
3091      ENDIF.
3092      ASSIGN (l_field) TO <sf>.
3093      <sf> = ld_sort-fieldname.
3094    ENDLOOP.
3095    IF sy-subrc = 0.
3096      SORT rt_pos STABLE
3097                   AS TEXT BY (a01) ASCENDING
3098                    (d01) DESCENDING
```

```
3099                    (a02) ASCENDING
3100                    (d02) DESCENDING
3101                    (a03) ASCENDING
3102                    (d03) DESCENDING
3103                    (a04) ASCENDING
3104                    (d04) DESCENDING
3105                    (a05) ASCENDING
3106                    (d05) DESCENDING
3107                    (a06) ASCENDING
3108                    (d06) DESCENDING
3109                    (a07) ASCENDING
3110                    (d07) DESCENDING
3111                    (a08) ASCENDING
3112                    (d08) DESCENDING
3113                    (a09) ASCENDING
3114                    (d09) DESCENDING
3115                    (a10) ASCENDING
3116                    (d10) DESCENDING
3117                    (a11) ASCENDING
3118                    (d11) DESCENDING
3119                    (a12) ASCENDING
3120                    (d12) DESCENDING
3121                    (a13) ASCENDING
3122                    (d13) DESCENDING
3123                    (a14) ASCENDING
3124                    (d14) DESCENDING
3125                    (a15) ASCENDING
3126                    (d15) DESCENDING .
3127    ENDIF.
3128
3129  ENDFORM.                    "IT_POS_SORT_BACK
3130
3131  *&---------------------------------------------------------------------*
3132  *&      Form  special_fields_used
3133  *&---------------------------------------------------------------------*
3134  *       text
3135  *----------------------------------------------------------------------*
3136  *      -->ID_TABLENAME
3137  *----------------------------------------------------------------------*
3138  FORM special_fields_used
3139       USING id_tablename LIKE dd03p-tabname.
3140
3141    DATA: ld_fname TYPE type_spec_fields.
3142
3143
3144    CLEAR gt_spec_fields.
3145    REFRESH gt_spec_fields.
3146
3147  * Always include the following infos ...
3148    ld_fname-fname = 'BUKRS'.
3149    APPEND ld_fname TO gt_spec_fields.
3150    ld_fname-fname = 'BELNR'.
3151    APPEND ld_fname TO gt_spec_fields.
3152    ld_fname-fname = 'GJAHR'.
3153    APPEND ld_fname TO gt_spec_fields.
3154  * ...plus some table specific additional infos...
3155    CASE id_tablename.
3156      WHEN 'BKPF'.
3157        ld_fname-fname = 'AWSYS'.
3158        APPEND ld_fname TO gt_spec_fields.
3159        ld_fname-fname = 'BUDAT'.
```

```
3160        APPEND ld_fname TO gt_spec_fields.
3161      WHEN 'BSEG' OR 'ACDOCA'.                                    "3138815
3162        ld_fname-fname = 'BUZEI'.                                 "3138815
3163        APPEND ld_fname TO gt_spec_fields.                        "3138815
3164        ld_fname-fname = 'SHKZG'.                                 "3138815
3165        APPEND ld_fname TO gt_spec_fields.                        "3138815
3166        ld_fname-fname = 'REBZT'.                                 "3138815
3167        APPEND ld_fname TO gt_spec_fields.                        "3138815
3168      WHEN 'BSEGC'.
3169        ld_fname-fname = 'RFZEI'.
3170        APPEND ld_fname TO gt_spec_fields.
3171  *     Needed for sign of MENGE related to note 3097838          "3676362
3172        ld_fname-fname = 'SQUAN'.                                 "3676362
3173        APPEND ld_fname TO gt_spec_fields.                        "3676362
3174    ENDCASE.
3175
3176  * ... plus special fields from internal table IT_T021S.
3177    LOOP AT it_t021s WHERE tname = id_tablename.
3178      APPEND it_t021s-fname TO gt_spec_fields.
3179    ENDLOOP.
3180
3181  ENDFORM.                    "special_fields_used
3182
3183  *---------------------------------------------------------------------*
3184  *       FORM special_fields_modify                                    *
3185  *---------------------------------------------------------------------*
3186  *       ........                                                      *
3187  *---------------------------------------------------------------------*
3188  *  ---->  ID_TABLENAME                                                *
3189  *---------------------------------------------------------------------*
3190  FORM special_fields_modify USING id_tablename LIKE dd03p-tabname.
3191
3192    DATA: ld_fname TYPE type_spec_fields.
3193
3194
3195  * Delete non-existent fields.
3196    LOOP AT gt_spec_fields INTO ld_fname.
3197
3198      CALL FUNCTION 'DDIF_NAMETAB_GET'
3199        EXPORTING
3200          tabname    = id_tablename
3201          lfieldname = ld_fname-fname
3202        EXCEPTIONS
3203          not_found  = 1.
3204
3205      IF sy-subrc = 1 .
3206        DELETE TABLE gt_spec_fields WITH TABLE KEY fname = ld_fname-fname
    .
3207      ENDIF.
3208
3209    ENDLOOP.
3210
3211  * Add new fields.
3212    CASE id_tablename.
3213      WHEN 'VBKPF'.
3214    ENDCASE.
3215
3216  ENDFORM.                    "special_fields_modify
3217
3218  *---------------------------------------------------------------------*
3219  *       FORM init_admin_tables_boe                                    *
```

```
3220  *---------------------------------------------------------------------*
3221  *       ........                                                      *
3222  *---------------------------------------------------------------------*
3223  FORM init_admin_tables_boe.
3224
3225    DATA: ls_t021s TYPE t021s.
3226
3227    LOOP AT it_t021s.
3228      IF it_t021s-tname = 'BSEC'.
3229        IF    it_t021s-fname = 'BANKS'
3230           OR it_t021s-fname = 'BANKL'
3231           OR it_t021s-fname = 'BANKN'
3232           OR it_t021s-fname = 'BKONT'.
3233          ls_t021s-lstcl = '2'.
3234          ls_t021s-tname = 'BSED'.
3235          ls_t021s-fname = 'WLZBP'.
3236          APPEND ls_t021s TO it_t021s.
3237          EXIT.
3238        ENDIF.
3239      ENDIF.
3240    ENDLOOP.
3241
3242  ENDFORM.                    "init_admin_tables_boe
3243
3244  *---------------------------------------------------------------------*
3245  *       FORM analyze_act_fieldcat_boe                                 *
3246  *---------------------------------------------------------------------*
3247  *       ........                                                      *
3248  *---------------------------------------------------------------------*
3249  FORM analyze_act_fieldcat_boe.
3250
3251    IF it_specfld-tname = 'BSEC' AND it_specfld-fstat = 'P'.
3252      IF    it_specfld-fname_2 = 'U_BANKS'
3253         OR it_specfld-fname_2 = 'U_BANKL'
3254         OR it_specfld-fname_2 = 'U_BANKN'
3255         OR it_specfld-fname_2 = 'U_BKONT' .
3256
3257        READ TABLE it_specfld WITH KEY fname_2 = 'U_WLZBP'.
3258        IF sy-subrc NE 0.
3259  *         horror case:
3260          deco_message_0 w012.
3261          EXIT.
3262        ENDIF.
3263  *       special field status = initial?
3264        IF it_specfld-fstat = space.
3265  *         set status 'pending':
3266          it_specfld-fstat = 'P'.
3267          MODIFY it_specfld INDEX sy-tabix.
3268  *         check corresponding table status:
3269          READ TABLE it_spectab WITH KEY tname = it_specfld-tname.
3270          IF sy-subrc NE 0.
3271  *         horror case:
3272            deco_message_0 w012.
3273            EXIT.
3274          ENDIF.
3275          IF it_spectab-tstat = space.
3276            it_spectab-tstat = 'P'.
3277            MODIFY it_spectab INDEX sy-tabix.
3278          ENDIF.
3279        ENDIF.
3280      ENDIF.
```

```
3281    ENDIF.
3282
3283  ENDFORM.                    "analyze_act_fieldcat_boe
3284
3285  *---------------------------------------------------------------------*
3286  *       FORM move_special_fields_boe                                  *
3287  *---------------------------------------------------------------------*
3288  *       ........                                                      *
3289  *---------------------------------------------------------------------*
3290  FORM move_special_fields_boe.
3291
3292    FIELD-SYMBOLS: <fs_sp_tab>, <fs_it_pos>.
3293
3294    IF NOT it_pos-umskz IS INITIAL AND NOT wa_bsed-wlzbp IS INITIAL.
3295      CLEAR wa_bsec.
3296      wa_bsec-banks = wa_bsed-wlzbp+1(3).
3297      wa_bsec-bankl = wa_bsed-wlzbp+5(15).
3298      wa_bsec-bankn = wa_bsed-wlzbp+21(18).
3299      wa_bsec-bkont = wa_bsed-wlzbp+40(2).
3300
3301      LOOP AT it_moveref WHERE tname = 'BSEC'.
3302        ASSIGN it_moveref-fldref->*   TO <fs_sp_tab>.
3303        ASSIGN it_moveref-fldref_2->* TO <fs_it_pos>.
3304        IF <fs_it_pos> IS INITIAL.
3305          MOVE <fs_sp_tab> TO <fs_it_pos>.
3306        ENDIF.
3307      ENDLOOP.
3308    ENDIF.
3309
3310  ENDFORM.                    "move_special_fields_boe
3311
3312
3313
3314
3315
3316  *---------------------------------------------------------------------*
3317  *       FORM create_performance_tables                                *
3318  *---------------------------------------------------------------------*
3319  *       ........                                                      *
3320  *---------------------------------------------------------------------*
3321  *  ---->  id_tablename                                                *
3322  *---------------------------------------------------------------------*
3323  FORM create_performance_tables USING id_tablename TYPE dd03p-tabname .
3324
3325    FIELD-SYMBOLS : <lfs_pos>              TYPE faglposx.
3326    STATICS :       lst_performance_tables TYPE c.
3327
3328    IF lst_performance_tables IS INITIAL.
3329      IF id_tablename EQ 'BKPF' OR id_tablename EQ 'BSEG'
3330                                OR id_tablename EQ 'ACDOCA' .
3331
3332  * Creating additional tables for speeding up the database selects
3333        REFRESH: gt_pos_parked, gt_pos_posted, gt_pos_archived.
3334
3335        LOOP AT it_pos ASSIGNING <lfs_pos>.
3336          gd_pos_reduced-bukrs = <lfs_pos>-bukrs.
3337          gd_pos_reduced-belnr = <lfs_pos>-belnr.
3338          gd_pos_reduced-gjahr = <lfs_pos>-gjahr.
3339          gd_pos_reduced-buzei = <lfs_pos>-buzei.
3340          gd_pos_reduced-budat = <lfs_pos>-budat.
3341          gd_pos_reduced-bstat = <lfs_pos>-bstat.             "1117587
```

```
3342          gd_pos_reduced-docln = <lfs_pos>-docln.              "1237389
3343          gd_pos_reduced-rldnr = <lfs_pos>-rldnr.              "2521327
3344
3345          IF <lfs_pos>-bstat CA 'VWZ'.
3346            INSERT gd_pos_reduced INTO TABLE gt_pos_parked.
3347          ELSEIF <lfs_pos>-xarch EQ 'X' AND gd_usear = 'X'.
3348            INSERT gd_pos_reduced INTO TABLE gt_pos_archived.
3349          ELSE.
3350            INSERT gd_pos_reduced INTO TABLE gt_pos_posted.
3351          ENDIF.
3352        ENDLOOP.
3353
3354        lst_performance_tables = 'X'.
3355      ENDIF.
3356    ENDIF.
3357
3358  ENDFORM.                    "create_performance_tables
3359
3360  *---------------------------------------------------------------------*
3361  *       FORM import_arch_from_memory                                  *
3362  *---------------------------------------------------------------------*
3363  *       ........                                                      *
3364  *---------------------------------------------------------------------*
3365  *FORM import_arch_from_memory.
3366  *
3367  *  FIELD-SYMBOLS: <ybkpf> TYPE bkpf,
3368  *                 <ybseg> TYPE bseg.
3369  *
3370  *  IF gd_usear = 'X' AND gd_memor = 'X'.
3371  *    import gd_read_memo_done from memory id 'GD_READ_MEMO_DONE'.
3372  *    if gd_read_memo_done eq 'N'.
3373  *      IMPORT ybkpf FROM MEMORY ID 'YBKPF'.
3374  *      IF xbkpf[] IS INITIAL.
3375  *        INSERT LINES OF ybkpf INTO TABLE xbkpf.
3376  *      ELSE.
3377  *        LOOP AT ybkpf ASSIGNING <ybkpf>.
3378  *          READ TABLE xbkpf WITH TABLE KEY bukrs = <ybkpf>-bukrs
3379  *                                          belnr = <ybkpf>-belnr
3380  *                                          gjahr = <ybkpf>-gjahr
3381  *                                   TRANSPORTING NO FIELDS.
3382  *          IF NOT sy-subrc IS INITIAL.
3383  *            INSERT <ybkpf> INTO TABLE xbkpf.
3384  *          ENDIF.
3385  *        ENDLOOP.
3386  *      ENDIF.
3387  *      IMPORT ybseg FROM MEMORY ID 'YBSEG'.
3388  *      IF xbseg[] IS INITIAL.
3389  *        INSERT LINES OF ybseg INTO TABLE xbseg.
3390  *      ELSE.
3391  *        LOOP AT ybseg ASSIGNING <ybseg>.
3392  *          READ TABLE xbseg WITH TABLE KEY bukrs = <ybseg>-bukrs
3393  *                                          belnr = <ybseg>-belnr
3394  *                                          gjahr = <ybseg>-gjahr
3395  *                                          buzei = <ybseg>-buzei
3396  *                                   TRANSPORTING NO FIELDS.
3397  *          IF NOT sy-subrc IS INITIAL.
3398  *            INSERT <ybseg> INTO TABLE xbseg.
3399  *          ENDIF.
3400  *        ENDLOOP.
3401  *      ENDIF.
3402  *
```

```
3403  **    INSERT LINES OF ybseg INTO TABLE xbseg.
3404  **    refresh ybseg.
3405  **    FREE MEMORY ID 'YBSEG'.
3406  *    endif.
3407  *  ENDIF.
3408  *
3409  *ENDFORM.                                " IMPORT_ARCH_FROM_MEMORY
3410  *&---------------------------------------------------------------------*
3411  *&      Form  get_flexgl_fields
3412  *&---------------------------------------------------------------------*
3413  *       text
3414  *----------------------------------------------------------------------*
3415  *      -->P_GV_RELEVANT_LEDGER  text
3416  *      <--P_LT_FIELDS  text
3417  *----------------------------------------------------------------------*
3418  FORM get_flexgl_fields  USING
3419                            p_gv_relevant_ledger TYPE rldnr
3420                          CHANGING
3421                            p_lt_fields TYPE ttfieldname.
3422
3423    DATA: lt_fields_bseg TYPE ttfieldname.
3424    DATA: lt_fields_bkpf TYPE ttfieldname.
3425    DATA: ls_fieldname   TYPE fieldname.
3426
3427  * fill buffer of function group
3428    CALL FUNCTION 'FAGL_GET_TABLE_FIELDS'
3429      EXPORTING
3430        i_rldnr = p_gv_relevant_ledger.
3431
3432  * convert fieldnames
3433    CALL FUNCTION 'FAGL_ITEMS_CONVERT_FIELDSNAMES'
3434      EXPORTING
3435        i_rldnr        = p_gv_relevant_ledger
3436  *     IT_FIELDS_FLEXGL =
3437      CHANGING
3438  *     CT_FIELDS_FLEXGL =
3439        ct_fields_bseg = lt_fields_bseg
3440        ct_fields_bkpf = lt_fields_bkpf.
3441
3442    LOOP AT lt_fields_bkpf INTO ls_fieldname.
3443      APPEND ls_fieldname TO lt_fields_bseg.
3444    ENDLOOP.
3445
3446    p_lt_fields[] = lt_fields_bseg.
3447
3448  ENDFORM.                    " get_flexgl_fields
3449  *&---------------------------------------------------------------------*
3450  *&      Form  item_check_auth
3451  *&---------------------------------------------------------------------*
3452  *       text
3453  *----------------------------------------------------------------------*
3454  *      <--P_X_NOAUT  text
3455  *----------------------------------------------------------------------*
3456  FORM item_check_auth  CHANGING
3457                          p_x_noauth TYPE c.
3458
3459    DATA: rtcode TYPE n.
3460
3461    IF NOT wa_pos-blart IS INITIAL
3462                AND NOT GD_AU_BLART IS INITIAL.                "2983368
3463      CALL FUNCTION 'FI_BLART_AUTH_CHECK'
```

```
3464        EXPORTING
3465  *       i_bukrs  =
3466          i_blart  = wa_pos-blart
3467          i_aktvt  = '03'
3468        IMPORTING
3469          e_rtcode = rtcode
3470        EXCEPTIONS
3471          OTHERS   = 0.
3472      IF rtcode = 4.
3473        p_x_noauth = 'X'.
3474      ENDIF.
3475    ENDIF.
3476
3477    IF NOT wa_pos-gsber IS INITIAL
3478                 AND NOT GD_AU_GSBER IS INITIAL.                "2983368
3479      CALL FUNCTION 'FI_GSBER_AUTH_CHECK'
3480        EXPORTING
3481  *       i_bukrs  =
3482          i_gsber  = wa_pos-gsber
3483          i_aktvt  = '03'
3484        IMPORTING
3485          e_rtcode = rtcode
3486        EXCEPTIONS
3487          OTHERS   = 0.
3488      IF rtcode = 4.
3489        p_x_noauth = 'X'.
3490      ENDIF.
3491    ENDIF.
3492
3493    " Initial value is also check and should be authorized if needs to be
     shown
3494    DATA(lv_func_area) = COND fkber( WHEN wa_pos-fkber_long IS INITIAL THE
     N wa_pos-fkber ELSE wa_pos-fkber_long ).
3495    DATA(lv_is_auth) = cl_fin_func_area_auth_check=>get_instance( )->is_va
     lue_allowed_by_sacf( iv_activity        = '03'
3496
                          iv_functional_area = lv_func_area ).
3497    IF lv_is_auth = abap_false.
3498        p_x_noauth = 'X'.
3499    ENDIF.
3500
3501    IF NOT wa_pos-segment IS INITIAL.
3502      CALL FUNCTION 'FAGL_AUTHORITY_SEGMENT'
3503        EXPORTING
3504          i_segment            = wa_pos-segment
3505          i_actvt              = '03'
3506        EXCEPTIONS
3507          no_authority_segment = 1
3508          OTHERS               = 3.
3509      IF sy-subrc <> 0.
3510        p_x_noauth = 'X'.
3511      ENDIF.
3512    ENDIF.
3513
3514    IF ( NOT wa_pos-prctr IS INITIAL
3515    OR NOT wa_pos-kostl IS INITIAL )
3516    AND wa_pos-kokrs IS INITIAL.
3517      CALL FUNCTION 'KOKRS_GET_FROM_BUKRS'
3518        EXPORTING
3519          i_bukrs        = wa_pos-bukrs
3520        IMPORTING
```

```
3521          e_kokrs        = wa_pos-kokrs
3522        EXCEPTIONS
3523          no_kokrs_found = 1
3524          OTHERS         = 2.
3525      IF sy-subrc NE 0.
3526        deco_message_1 e107 with wa_pos-bukrs.
3527      ENDIF.
3528    ENDIF.
3529
3530    IF NOT wa_pos-prctr IS INITIAL.
3531      l_auth-bukrs = wa_pos-bukrs.                          "1649564
3532      l_auth-rldnr = wa_pos-rldnr.                          "1649564
3533      CALL FUNCTION 'K_PCA_RESP_AUTHORITY_CHECK'
3534        EXPORTING
3535          i_kokrs             = wa_pos-kokrs
3536          i_prctr             = wa_pos-prctr
3537          i_kstar             = wa_pos-hkont
3538          i_vorgn             = 'REPP'
3539          i_actvt             = '28'
3540          i_no_dialog_message = 'X'
3541          i_auth              = l_auth                     "1649564
3542        EXCEPTIONS
3543          no_authority        = 1
3544          data_missing        = 2
3545          OTHERS              = 3.
3546      IF sy-subrc <> 0.
3547        p_x_noauth = 'X'.
3548      ENDIF.
3549    ENDIF.
3550
3551    IF NOT wa_pos-kostl IS INITIAL.
3552      CALL FUNCTION 'K_REPO_CCA_AUTHORITY_CHECK'
3553        EXPORTING
3554          actvt               = '28'
3555          kokrs               = wa_pos-kokrs
3556          kostl               = wa_pos-kostl
3557          kstar               = wa_pos-hkont
3558        EXCEPTIONS
3559          system_error        = 1
3560          user_not_authorized = 2
3561          OTHERS              = 3.
3562      IF sy-subrc <> 0.
3563        p_x_noauth = 'X'.
3564      ENDIF.
3565    ENDIF.
3566
3567  *... BADI: Authority SAP Modules
3568    IF  NOT gd_act_imp_auth_s IS INITIAL
3569    AND NOT gd_ref_to_auth_s  IS INITIAL.
3570      CALL METHOD gd_ref_to_auth_s->check_auth_item
3571        EXPORTING
3572          i_origin     = gd_origin
3573          is_item      = wa_pos
3574        EXCEPTIONS
3575          no_authority = 4.
3576      IF sy-subrc <> 0.
3577        p_x_noauth = 'X'.
3578      ENDIF.
3579    ENDIF.
3580  ENDFORM.                    " item_check_auth
3581  *&---------------------------------------------------------------------*
```

```
3582  *&      Form  curr_convert_item
3583  *&---------------------------------------------------------------------*
3584  *       convertion of ammounts for expiring currencies
3585  *---------------------------------------------------------------------*
3586  *  -->  p1        text
3587  *  <--  p2        text
3588  *---------------------------------------------------------------------*
3589  FORM curr_convert_item.
3590
3591    DATA: lt_dfies         LIKE dfies    OCCURS 0 WITH HEADER LINE.
3592    DATA: ld_fldlines      TYPE i.
3593  * data: ld_process LIKE  tprcd-process.
3594
3595    DESCRIBE TABLE gt_alw_fieldlist LINES ld_fldlines.
3596    IF ld_fldlines IS INITIAL.
3597      CALL FUNCTION 'DDIF_FIELDINFO_GET'
3598        EXPORTING
3599          tabname        = 'FAGLPOSX'
3600        TABLES
3601          dfies_tab      = lt_dfies
3602        EXCEPTIONS
3603          not_found      = 1
3604          internal_error = 2
3605          OTHERS         = 3.
3606      LOOP AT lt_dfies.
3607        IF  lt_dfies-datatype = 'CURR'
3608        AND lt_dfies-reftable = 'FAGLPOSX'
3609        AND lt_dfies-reffield = 'WAERS'.
3610          gt_alw_fieldlist-name = lt_dfies-fieldname.
3611          APPEND gt_alw_fieldlist.
3612        ENDIF.
3613      ENDLOOP.
3614    ENDIF.
3615
3616  * ld_process   = GD_ALW_report.
3617
3618    gd_alw_waers = wa_pos-waers.
3619    CALL FUNCTION 'CURRENCY_GET_SUBSEQUENT'
3620      EXPORTING
3621        currency     = gd_alw_waers
3622        process      = gd_expcur_proc
3623        date         = gd_alw_date
3624        bukrs        = wa_pos-bukrs
3625      IMPORTING
3626        currency_new = gd_alw_waers.
3627    IF gd_alw_waers NE wa_pos-waers.
3628      CALL FUNCTION 'CURRENCY_DOCUMENT_CONVERT'
3629        EXPORTING
3630  *       RATE_TYPE       = 'M'
3631          from_currency   = wa_pos-waers
3632          to_currency     = gd_alw_waers
3633          local_currency  = wa_pos-hwaer
3634          date            = gd_alw_date
3635  *       RATE            =
3636          conversion_mode = 'O'
3637        TABLES
3638          fieldlist       = gt_alw_fieldlist
3639  *       T_LINES         =
3640        CHANGING
3641          line            = wa_pos
3642        EXCEPTIONS
```

```
3643  *       FIELD_UNKNOWN   = 1
3644  *       FIELD_NOT_AMOUNT          = 2
3645  *       ERROR_IN_CONVERSION       = 3
3646  *       ILLEGAL_PARAMETERS        = 4
3647          OTHERS          = 5.
3648      IF sy-subrc <> 0.
3649  * MESSAGE ID SY-MSGID TYPE SY-MSGTY NUMBER SY-MSGNO
3650  *         WITH SY-MSGV1 SY-MSGV2 SY-MSGV3 SY-MSGV4.
3651      ENDIF.
3652
3653      wa_pos-waers = gd_alw_waers.
3654    ENDIF.
3655
3656    gd_alw_waers = wa_pos-pswsl.
3657    CALL FUNCTION 'CURRENCY_GET_SUBSEQUENT'
3658      EXPORTING
3659        currency     = gd_alw_waers
3660        process      = gd_expcur_proc
3661        date         = gd_alw_date
3662        bukrs        = wa_pos-bukrs
3663      IMPORTING
3664        currency_new = gd_alw_waers.
3665    IF gd_alw_waers NE wa_pos-pswsl.
3666      CALL FUNCTION 'CONVERT_FOREIGN_TO_FOREIGN_CUR'
3667        EXPORTING
3668  *       CLIENT          = SY-MANDT
3669          date            = gd_alw_date
3670  *       TYPE_OF_RATE    = 'M'
3671          from_amount     = wa_pos-pswbt
3672          from_currency   = wa_pos-pswsl
3673          to_currency     = gd_alw_waers
3674          local_currency  = wa_pos-hwaer
3675          conversion_mode = 'X'
3676        IMPORTING
3677          to_amount       = wa_pos-pswbt
3678  *     EXCEPTIONS
3679  *       NO_RATE_FOUND   = 1
3680  *       OVERFLOW        = 2
3681  *       NO_FACTORS_FOUND       = 3
3682  *       NO_SPREAD_FOUND = 4
3683  *       DERIVED_2_TIMES = 5
3684  *       OTHERS          = 6
3685        .
3686      IF sy-subrc <> 0.                                          "#EC *
3687        deco_message_sy.
3688      ENDIF.
3689
3690      wa_pos-pswsl = gd_alw_waers.
3691    ENDIF.
3692
3693    gd_alw_waers = wa_pos-pycur.
3694    CALL FUNCTION 'CURRENCY_GET_SUBSEQUENT'
3695      EXPORTING
3696        currency     = gd_alw_waers
3697        process      = gd_expcur_proc
3698        date         = gd_alw_date
3699        bukrs        = wa_pos-bukrs
3700      IMPORTING
3701        currency_new = gd_alw_waers.
3702    IF gd_alw_waers NE wa_pos-pycur.
3703      CALL FUNCTION 'CONVERT_FOREIGN_TO_FOREIGN_CUR'
```

```
3704        EXPORTING
3705  *       CLIENT           = SY-MANDT
3706          date             = gd_alw_date
3707  *       TYPE_OF_RATE     = 'M'
3708          from_amount      = wa_pos-pyamt
3709          from_currency    = wa_pos-pycur
3710          to_currency      = gd_alw_waers
3711          local_currency   = wa_pos-hwaer
3712          conversion_mode = 'X'
3713        IMPORTING
3714          to_amount        = wa_pos-pyamt
3715  *     EXCEPTIONS
3716  *       NO_RATE_FOUND    = 1
3717  *       OVERFLOW         = 2
3718  *       NO_FACTORS_FOUND       = 3
3719  *       NO_SPREAD_FOUND = 4
3720  *       DERIVED_2_TIMES = 5
3721  *       OTHERS           = 6
3722        .
3723      IF sy-subrc <> 0.                                          "#EC *
3724        deco_message_sy.
3725      ENDIF.
3726
3727      wa_pos-pycur = gd_alw_waers.
3728    ENDIF.
3729
3730  ENDFORM.                    " curr_convert_item
3731  *&---------------------------------------------------------------------*
3732  *&      Form  erfs_modify_screen
3733  *&---------------------------------------------------------------------*
3734  *       text
3735  *----------------------------------------------------------------------*
3736  *  -->  p1        text
3737  *  <--  p2        text
3738  *----------------------------------------------------------------------*
3739  FORM erfs_modify_screen .
3740    IF gv_xerfs IS INITIAL.
3741  *   worklist off -> hide worklist layout:
3742      IF screen-group1 = 'EFS'.
3743        screen-active    = 0.
3744        screen-invisible = 1.
3745      ENDIF.
3746      IF screen-group1 = 'GLS'.
3747        screen-input    = 0.
3748      ENDIF.
3749    ELSE.
3750      IF screen-group1 = 'GLS'.
3751        screen-active    = 0.
3752        screen-invisible = 1.
3753      ENDIF.
3754    ENDIF.
3755  ENDFORM.                    " erfs_modify_screen
3756  *&---------------------------------------------------------------------*
3757  *&      Form  glyec_modify_screen
3758  *&---------------------------------------------------------------------*
3759  *       balance carry forward posting selection
3760  *----------------------------------------------------------------------*
3761  FORM glyec_modify_screen.
3762    IF gv_xerfs IS INITIAL.
3763  *   Check the switch is active
3764      IF NOT cl_fagl_switch_check=>fagl_fin_gl_cp_rs( ) IS INITIAL.
```

```
3765        IF screen-group1 = 'GLY'.
3766          screen-active    = 1.
3767          screen-invisible = 0.
3768        ENDIF.
3769      ELSE.
3770        IF screen-group1 = 'GLY'.
3771          screen-active    = 0.
3772          screen-invisible = 1.
3773        ENDIF.
3774        CLEAR x_glyec.
3775      ENDIF.
3776    ELSE.
3777      IF screen-group1 = 'GLY'.
3778        screen-active    = 0.
3779        screen-invisible = 1.
3780      ENDIF.
3781      CLEAR x_glyec.
3782    ENDIF.
3783  ENDFORM.                    " glyec_modify_screen
3784  *&---------------------------------------------------------------------*
3785  *&      Form  erfs_on_off
3786  *&---------------------------------------------------------------------*
3787  *       text
3788  *----------------------------------------------------------------------*
3789  *  -->  p1        text
3790  *  <--  p2        text
3791  *----------------------------------------------------------------------*
3792  FORM erfs_on_off .
3793
3794    IF gv_xerfs IS INITIAL.
3795      gv_xerfs = 'X'.
3796      x_erfs   = 'X'.
3797      x_norm   = 'X'.
3798    ELSE.
3799      IF gb_xerfs IS INITIAL.
3800        CLEAR gv_xerfs.
3801        CLEAR x_erfs.
3802        CLEAR x_merk.
3803        CLEAR x_park.
3804        IF x_norm IS INITIAL.
3805          x_norm = 'X'.
3806        ENDIF.
3807      ENDIF.
3808    ENDIF.
3809
3810    PERFORM wl_flag_and_button.
3811
3812  ENDFORM.                    " erfs_on_off
3813  *&---------------------------------------------------------------------*
3814  *&      Form  create_log
3815  *&---------------------------------------------------------------------*
3816  *       text
3817  *----------------------------------------------------------------------*
3818  *  -->  p1        text
3819  *  <--  p2        text
3820  *----------------------------------------------------------------------*
3821  FORM create_log .
3822  * define header data of this log
3823    gs_log-extnumber = sy-repid.                          "#EC NOTEXT
3824    gs_log-aluser    = sy-uname.
3825    gs_log-alprog    = sy-repid.
```

```
3826  * ... see structure BAL_S_LOG for further data ...
3827  * ... which can be added to a log header       ...
3828
3829  * create a log
3830    CALL FUNCTION 'BAL_LOG_CREATE'
3831      EXPORTING
3832        i_s_log      = gs_log
3833      IMPORTING
3834        e_log_handle = gs_log_handle
3835      EXCEPTIONS
3836        OTHERS       = 1.
3837    IF sy-subrc <> 0.
3838      deco_message_sy.
3839    ENDIF.
3840  ENDFORM.                    " create_log
3841  *&---------------------------------------------------------------------*
3842  *&      Form  check_slt_ACCOUNT
3843  *&---------------------------------------------------------------------*
3844  *       text
3845  *----------------------------------------------------------------------*
3846  *      -->P_SKB1_SAKNR  text
3847  *      -->P_SKB1_BUKRS  text
3848  *      -->P_SKB1_XKRES  text
3849  *----------------------------------------------------------------------*
3850  FORM check_slt_account  USING
3851                            p_saknr TYPE saknr
3852                            p_bukrs TYPE bukrs
3853                            p_xkres TYPE xkres
3854                            p_xplacc TYPE boolean.
3855    DATA: ls_account_slt TYPE gty_acccoco.
3856    IF p_xplacc IS INITIAL.
3857      LOOP AT gt_account_slt INTO ls_account_slt
3858        WHERE saknr = p_saknr
3859        AND   bukrs = p_bukrs.
3860        EXIT.
3861      ENDLOOP.
3862    ELSE.
3863      LOOP AT gt_accoco_slt INTO ls_account_slt
3864        WHERE saknr = p_saknr
3865        AND   bukrs = p_bukrs.
3866        EXIT.
3867      ENDLOOP.
3868    ENDIF.
3869    IF  sy-subrc = 0
3870    AND NOT p_xkres IS INITIAL.
3871      MESSAGE e111 WITH p_saknr p_bukrs INTO gd_dummy.
3872      PERFORM msg_add_to_log.
3873    ENDIF.
3874  ENDFORM.                    " check_slt_ACCOUNT
3875  *&---------------------------------------------------------------------*
3876  *&      Form  msg_add_to_log
3877  *&---------------------------------------------------------------------*
3878  *       text
3879  *----------------------------------------------------------------------*
3880  *  -->  p1        text
3881  *  <--  p2        text
3882  *----------------------------------------------------------------------*
3883  FORM msg_add_to_log .
3884    DATA: ls_msg TYPE  bal_s_msg.
3885
3886    ls_msg-msgty = sy-msgty.
```

```
3887    ls_msg-msgid = sy-msgid.
3888    ls_msg-msgno = sy-msgno.
3889    ls_msg-msgv1 = sy-msgv1.
3890    ls_msg-msgv2 = sy-msgv2.
3891    ls_msg-msgv3 = sy-msgv3.
3892    ls_msg-msgv4 = sy-msgv4.
3893
3894    CALL FUNCTION 'BAL_LOG_MSG_ADD'
3895      EXPORTING
3896        i_log_handle     = gs_log_handle
3897        i_s_msg          = ls_msg
3898  *   IMPORTING
3899  *     E_S_MSG_HANDLE   =
3900  *     E_MSG_WAS_LOGGED =
3901  *     E_MSG_WAS_DISPLAYED       =
3902      EXCEPTIONS
3903        log_not_found    = 1
3904        msg_inconsistent = 2
3905        log_is_full      = 3
3906        OTHERS           = 4.
3907    IF sy-subrc <> 0.
3908      deco_message_sy.
3909    ENDIF.
3910  ENDFORM.                    " msg_add_to_log
3911  *&---------------------------------------------------------------------*
3912  *&      Form  check_msg_exist
3913  *&---------------------------------------------------------------------*
3914  *       text
3915  *----------------------------------------------------------------------*
3916  *      -->P_GS_LOG_HANDLE  text
3917  *      <--P_GD_XMSG  text
3918  *----------------------------------------------------------------------*
3919  FORM check_msg_exist  USING
3920                          p_log_handle TYPE balloghndl
3921                        CHANGING
3922                          p_xmsg TYPE xfeld.
3923    DATA: t_log_handle TYPE  bal_t_logh.
3924    APPEND p_log_handle TO t_log_handle.
3925
3926    CALL FUNCTION 'BAL_GLB_SEARCH_MSG'
3927      EXPORTING
3928  *     I_S_LOG_FILTER =
3929  *     I_T_LOG_CONTEXT_FILTER       =
3930        i_t_log_handle = t_log_handle
3931  *     I_S_MSG_FILTER =
3932  *     I_T_MSG_CONTEXT_FILTER       =
3933  *     I_T_MSG_HANDLE =
3934  *   IMPORTING
3935  *     E_T_LOG_HANDLE =
3936  *     E_T_MSG_HANDLE =
3937      EXCEPTIONS
3938        msg_not_found  = 1
3939        OTHERS         = 2.
3940    IF sy-subrc = 0.
3941      p_xmsg = 'X'.
3942    ENDIF.
3943  ENDFORM.                    " check_msg_exist
3944  *&---------------------------------------------------------------------*
3945  *&      Form  print_appl_log
3946  *&---------------------------------------------------------------------*
3947  *       text
```

```
3948  *---------------------------------------------------------------------*
3949  *  ---->  p1        text
3950  *  <----  p2        text
3951  *---------------------------------------------------------------------*
3952  FORM print_appl_log .
3953    DATA ls_profile TYPE bal_s_prof.
3954
3955  * To get standard display profile
3956    CALL FUNCTION 'BAL_DSP_PROFILE_NO_TREE_GET'
3957      IMPORTING
3958        e_s_display_profile = ls_profile
3959      EXCEPTIONS
3960        OTHERS              = 1.
3961    IF sy-subrc <> 0.
3962      deco_message_sy.
3963    ENDIF.
3964
3965  * modify the profile.
3966  * Application log should diplay a classic ALV list, because the protocol
3967  * should have non proportional letters.
3968  * 1. use the list display.
3969    CLEAR ls_profile-use_grid.
3970
3971  * 2. set the report to allow saving of variants.
3972    ls_profile-disvariant-report = sy-repid.
3973  * 3. Add title to application log
3974    ls_profile-title = sy-title.
3975
3976    IF sy-batch = 'X'
3977    OR gv_ucomm = 'PRIN'.
3978  *  or sy-ucomm = '&RNT'.
3979  ******** In background
3980      DATA: ls_print_options TYPE  slis_print_alv.
3981
3982      PERFORM set_print_list_options  CHANGING ls_print_options.
3983
3984      CALL FUNCTION 'BAL_DSP_LOG_PRINT'
3985        EXPORTING
3986          i_s_print_options   = ls_print_options
3987          i_s_list_append     = ' '
3988          i_s_display_profile = ls_profile
3989          i_t_log_handle      = gt_log_handle.
3990
3991    ELSE.
3992  ******** In foreground
3993      CALL FUNCTION 'BAL_DSP_LOG_DISPLAY'
3994        EXPORTING
3995          i_s_display_profile = ls_profile
3996          i_t_log_handle      = gt_log_handle.
3997    ENDIF.
3998  ENDFORM.                    " print_appl_log
3999  *&---------------------------------------------------------------------*
4000  *&      Form  set_print_list_options
4001  *&---------------------------------------------------------------------*
4002  *       text
4003  *---------------------------------------------------------------------*
4004  *      <----P_LS_PRINT_OPTIONS  text
4005  *---------------------------------------------------------------------*
4006  FORM set_print_list_options  CHANGING xs_print TYPE slis_print_alv.
4007
4008    xs_print-no_print_listinfos     = 'X'.
```

```
4009    xs_print-no_print_selinfos      = 'X'.
4010
4011  ENDFORM.                    " set_print_list_options
4012  *&---------------------------------------------------------------------*
4013  *&      Form  check_badi_active
4014  *&---------------------------------------------------------------------*
4015  *       text
4016  *----------------------------------------------------------------------*
4017  *  -->  p1        text
4018  *  <--  p2        text
4019  *----------------------------------------------------------------------*
4020  FORM check_badi_active .
4021
4022    gd_origin = sy-repid.
4023
4024  *...BADI FAGL_AUTH_ADD_DATA_S: Authority addi
4025    CALL METHOD cl_exithandler=>get_instance
4026      EXPORTING
4027        exit_name               = 'FAGL_AUTH_ADD_DATA_S'
4028        null_instance_accepted = 'X'
4029      IMPORTING
4030        act_imp_existing        = gd_act_imp_auth_s
4031      CHANGING
4032        instance                = gd_ref_to_auth_s.
4033  *...BADI FAGL_AUTH_ADD_DATA_C: menue extension
4034    CALL METHOD cl_exithandler=>get_instance
4035      EXPORTING
4036        exit_name               = 'FAGL_AUTH_ADD_DATA_C'
4037        null_instance_accepted = 'X'
4038      IMPORTING
4039        act_imp_existing        = gd_act_imp_auth_c
4040      CHANGING
4041        instance                = gd_ref_to_auth_c.
4042  ENDFORM.                    " check_badi_active
4043  *&---------------------------------------------------------------------*
4044  *&      Form  items_check_auth_c
4045  *&---------------------------------------------------------------------*
4046  *       text
4047  *----------------------------------------------------------------------*
4048  *  -->  p1        text
4049  *  <--  p2        text
4050  *----------------------------------------------------------------------*
4051  FORM items_check_auth_c .
4052    LOOP AT it_pos.
4053  *... BADI: Authority Customers
4054      CALL METHOD gd_ref_to_auth_c->check_auth_item
4055        EXPORTING
4056          i_origin     = gd_origin
4057          is_item      = it_pos
4058        EXCEPTIONS
4059          no_authority = 4.
4060      IF sy-subrc <> 0.
4061        DELETE it_pos.
4062      ENDIF.
4063    ENDLOOP.
4064  ENDFORM.                    " items_check_auth_c
4065  *&---------------------------------------------------------------------*
4066  *&      Form  import_arch_from_itab
4067  *&---------------------------------------------------------------------*
4068  *       text
4069  *----------------------------------------------------------------------*
```

```
4070  FORM import_arch_from_itab .
4071
4072    FIELD-SYMBOLS: <ybkpf> TYPE bkpf,
4073                   <ybseg> TYPE bseg.
4074
4075    IF gd_usear = 'X'.
4076      SORT ybkpf.
4077      IF xbkpf[] IS INITIAL.
4078        INSERT LINES OF ybkpf INTO TABLE xbkpf.
4079      ELSE.
4080        LOOP AT ybkpf ASSIGNING <ybkpf>.
4081          READ TABLE xbkpf WITH TABLE KEY bukrs = <ybkpf>-bukrs
4082                                          belnr = <ybkpf>-belnr
4083                                          gjahr = <ybkpf>-gjahr
4084                                   TRANSPORTING NO FIELDS.
4085          IF NOT sy-subrc IS INITIAL.
4086            INSERT <ybkpf> INTO TABLE xbkpf.
4087          ENDIF.
4088        ENDLOOP.
4089      ENDIF.
4090      SORT ybseg.
4091      IF xbseg[] IS INITIAL.
4092        INSERT LINES OF ybseg INTO TABLE xbseg.
4093      ELSE.
4094        LOOP AT ybseg ASSIGNING <ybseg>.
4095          READ TABLE xbseg WITH TABLE KEY bukrs = <ybseg>-bukrs
4096                                          belnr = <ybseg>-belnr
4097                                          gjahr = <ybseg>-gjahr
4098                                          buzei = <ybseg>-buzei
4099                                   TRANSPORTING NO FIELDS.
4100          IF NOT sy-subrc IS INITIAL.
4101            INSERT <ybseg> INTO TABLE xbseg.
4102          ENDIF.
4103        ENDLOOP.
4104      ENDIF.
4105    ENDIF.
4106
4107  ENDFORM.                    " import_arch_from_itab
4108  *&---------------------------------------------------------------------*
4109  *&      Form  ANALYZE_ACT_FIELDCAT_LDGRP
4110  *&---------------------------------------------------------------------*
4111  *       1) Do it only if leading ledger is required
4112  *       2) Do it only if LDGRP is shown in the ALV layout
4113  *          ( NO_OUT = ' ' )
4114  *       3) Then update status tables
4115  *                               it_t021s
4116  *                               it_spectab
4117  *                               it_specfld
4118  *                               it_movefld
4119  *                               it_moveref
4120  *
4121  * table it_t021s can already be filled if LDGRP is used as
4122  * special field (T021S).
4123  * table it_spectab can already be filled if LDGRP or another BKPF-field
4124  * is used as special field (T021S). This is done in form INIT_ADMIN_TABL
      ES.
4125  *
4126  * in this case either U_LDGRP or LDGRP is set to tech field in form
4127  * MAKE_FIELDCATALOG
4128  *
4129  * tables it_specfld,
```

```
4130  *        it_movefld
4131  *    and it_moveref need a separate entry for standard field LDGRP
4132  *
4133  *---------------------------------------------------------------------*
4134  FORM analyze_act_fieldcat_ldgrp TABLES rt_var_fieldcat
4135                                   TYPE slis_t_fieldcat_alv.
4136  * local data declarations
4137    FIELD-SYMBOLS <fsym>.
4138
4139  * 1) Do it only if leading ledger is required
4140    IF rldnr <> gv_leading_ledger.
4141      RETURN.  " >>> R E T U R N >>>>>>>>>
4142    ENDIF.
4143  * 2) Do it only if LDGRP is shown in the ALV layout ( NO_OUT = ' ' )
4144    LOOP AT rt_var_fieldcat TRANSPORTING NO FIELDS
4145                            WHERE fieldname = 'LDGRP'
4146                              AND no_out IS INITIAL
4147                              AND tech   IS INITIAL.
4148    ENDLOOP.
4149    IF sy-subrc <> 0.
4150      RETURN.  " >>> R E T U R N >>>>>>>>>
4151    ENDIF.
4152
4153  * clear header lines
4154    CLEAR it_t021s.
4155    CLEAR it_spectab.
4156    CLEAR it_specfld.
4157    CLEAR it_movefld.
4158    CLEAR it_moveref.
4159
4160  * check it_specfld
4161    READ TABLE it_specfld WITH KEY tname   = 'BKPF'
4162                                   fname   = 'LDGRP'
4163                                   fname_2 = 'LDGRP' TRANSPORTING NO FIELD
      S.
4164    IF sy-subrc <> 0.
4165  *   update it_t021s
4166      READ TABLE it_t021s WITH KEY lstcl = 'D'
4167                                   tname = 'BKPF'
4168                                   fname = 'LDGRP' TRANSPORTING NO FIELDS.
4169      IF sy-subrc <> 0.
4170        it_t021s-lstcl = 'D'.
4171        it_t021s-tname = 'BKPF'.
4172        it_t021s-fname = 'LDGRP'.
4173        APPEND it_t021s.
4174        SORT   it_t021s BY tname fname.
4175      ENDIF.
4176  *   update it_spectab
4177      READ TABLE it_spectab WITH KEY tname = 'BKPF'.
4178      IF sy-subrc = 0.
4179        it_spectab-tstat    = 'P'.
4180        MODIFY it_spectab INDEX sy-tabix.
4181      ELSE.
4182        it_spectab-tname    = 'BKPF'.
4183        it_spectab-tstat    = 'P'.
4184        APPEND it_spectab.
4185        SORT   it_spectab BY tname.
4186      ENDIF.
4187  *   update it_specfld
4188      it_specfld-tname    = 'BKPF'.
4189      it_specfld-fname    = 'LDGRP'.
```

```
4190      it_specfld-fname_2  = 'LDGRP'.
4191      it_specfld-fstat    = 'P'.
4192      APPEND it_specfld.
4193      SORT   it_specfld BY tname fname.
4194  *   update it_movefld
4195      it_movefld-tname    = 'BKPF'.
4196      it_movefld-tabfld   = 'WA_BKPF-LDGRP'.
4197      it_movefld-tabfld_2 = 'IT_POS-LDGRP'.
4198      APPEND it_movefld.
4199      SORT   it_movefld BY tname tabfld.
4200  *   update it_moveref
4201      it_moveref-tname    = 'BKPF'.
4202      ASSIGN (it_movefld-tabfld) TO <fsym>.
4203      GET REFERENCE OF <fsym> INTO it_moveref-fldref.
4204      ASSIGN (it_movefld-tabfld_2) TO <fsym>.
4205      GET REFERENCE OF <fsym> INTO it_moveref-fldref_2.
4206      APPEND it_moveref.
4207      SORT   it_moveref BY tname.
4208    ENDIF.
4209
4210  ENDFORM.                    " ANALYZE_ACT_FIELDCAT_LDGRP
4211  *&---------------------------------------------------------------------*
4212  *& Form ANALYZE_CURT
4213  *&---------------------------------------------------------------------*
4214  *& text
4215  *&---------------------------------------------------------------------*
4216  *& -->  p1        text
4217  *& <--  p2        text
4218  *&---------------------------------------------------------------------*
4219  FORM analyze_curt .
4220  * gv_relevant_ledger and sd_bukrs
4221  * -> define which columns can have text from currency type
4222  * provide global table
4223
4224  *   fieldname column_text
4225  *   DMBE3
4226  *   DMBE4
4227  *   ...
4228  *   DMBE10
4229
4230    DATA LT_BUKRS TYPE TABLE OF BUKRS.                          "3412695
4231    DATA LS_BUKRS TYPE BUKRS.                                   "3412695
4232    DATA ls_curtp TYPE cl_fins_acdoc_util=>currency_info_curtp.
4233    DATA lt_curtp LIKE TABLE OF ls_curtp.
4234
4235    DATA lv_curtp_k TYPE curtp. "3.HW or 1st free defined curr type
4236    DATA lv_curtp_o TYPE curtp. "3.HW or 1st free defined curr type
4237    DATA lv_curtp_v TYPE curtp. "4.HW or 2nd free defined curr type
4238    DATA lv_curtp_b TYPE curtp. "5.HW or 3rd free defined curr type
4239    DATA lv_curtp_c TYPE curtp. "6.HW or 4th free defined curr type
4240    DATA lv_curtp_d TYPE curtp. "7.HW or 5th free defined curr type
4241    DATA lv_curtp_e TYPE curtp. "8.HW or 6th free defined curr type
4242    DATA lv_curtp_f TYPE curtp. "9.HW or 7th free defined curr type
4243    DATA lv_curtp_g TYPE curtp. "10.HW or 8th free defined curr type
4244
4245    DATA lv_different_curtp_k TYPE abap_bool.
4246    DATA lv_different_curtp_o TYPE abap_bool.
4247    DATA lv_different_curtp_v TYPE abap_bool.
4248    DATA lv_different_curtp_b TYPE abap_bool.
4249    DATA lv_different_curtp_c TYPE abap_bool.
4250    DATA lv_different_curtp_d TYPE abap_bool.
```

```
4251    DATA lv_different_curtp_e TYPE abap_bool.
4252    DATA lv_different_curtp_f TYPE abap_bool.
4253    DATA lv_different_curtp_g TYPE abap_bool.
4254
4255    DATA ls_add_curr_column_text TYPE ty_add_curr_column_text.
4256
4257    IF sd_bukrs[] IS NOT INITIAL.
4258      SELECT t~BUKRS FROM t001 as t INNER JOIN FINSC_LD_CMP      "3412695
4259                AS l ON t~BUKRS = l~BUKRS  INTO TABLE lt_bukrs   "3412695
4260                  WHERE t~bukrs IN sd_bukrs                      "3412695
4261                    AND t~xtemplt NE 'X'                         "2697244
4262                    AND l~rldnr = gv_relevant_ledger             "3412695
4263                    ORDER BY t~BUKRS.                            "3412695
4264    ELSE.
4265      SELECT t~BUKRS FROM t001 as t INNER JOIN FINSC_LD_CMP      "3412695
4266                AS l ON t~BUKRS = l~BUKRS  INTO TABLE lt_bukrs   "3412695
4267                  WHERE t~bukrs IN gt_bukrs                      "3412695
4268                    AND t~xtemplt NE 'X'                         "2697244
4269                    AND l~rldnr = gv_relevant_ledger             "3412695
4270                    ORDER BY t~BUKRS.                            "3412695
4271    ENDIF.
4272
4273    LOOP AT lt_bukrs INTO ls_bukrs.                              "3412695
4274
4275      CLEAR ls_curtp.
4276
4277      CALL METHOD cl_fins_acdoc_util=>get_currency_info
4278        EXPORTING
4279          iv_bukrs  = ls_bukrs                                   "3412695
4280          iv_ledger = gv_relevant_ledger
4281        IMPORTING
4282          es_curtp  = ls_curtp
4283        EXCEPTIONS
4284          OTHERS    = 1.
4285      IF sy-subrc <> 0.
4286        MESSAGE ID sy-msgid
4287                TYPE sy-msgty
4288                NUMBER sy-msgno
4289                WITH sy-msgv1 sy-msgv2 sy-msgv3 sy-msgv4.
4290      ENDIF.
4291
4292      INSERT ls_curtp INTO TABLE lt_curtp.
4293
4294    ENDLOOP.
4295
4296    CLEAR lv_curtp_k.
4297    CLEAR lv_curtp_o.
4298    CLEAR lv_curtp_v.
4299    CLEAR lv_curtp_b.
4300    CLEAR lv_curtp_c.
4301    CLEAR lv_curtp_d.
4302    CLEAR lv_curtp_e.
4303    CLEAR lv_curtp_f.
4304    CLEAR lv_curtp_g.
4305
4306    READ TABLE lt_curtp INTO ls_curtp INDEX 1.
4307
4308    lv_curtp_k = ls_curtp-ksl.
4309    lv_curtp_o = ls_curtp-osl.
4310    lv_curtp_v = ls_curtp-vsl.
4311    lv_curtp_b = ls_curtp-bsl.
```

```
4312    lv_curtp_c = ls_curtp-csl.
4313    lv_curtp_d = ls_curtp-dsl.
4314    lv_curtp_e = ls_curtp-esl.
4315    lv_curtp_f = ls_curtp-fsl.
4316    lv_curtp_g = ls_curtp-gsl.
4317
4318    LOOP AT lt_curtp INTO ls_curtp.
4319
4320      IF lv_curtp_k <> ls_curtp-ksl.
4321        lv_different_curtp_k = abap_true.
4322      ENDIF.
4323      IF lv_curtp_o <> ls_curtp-osl.
4324        lv_different_curtp_o = abap_true.
4325      ENDIF.
4326      IF lv_curtp_v <> ls_curtp-vsl.
4327        lv_different_curtp_v = abap_true.
4328      ENDIF.
4329      IF lv_curtp_b <> ls_curtp-bsl.
4330        lv_different_curtp_b = abap_true.
4331      ENDIF.
4332      IF lv_curtp_c <> ls_curtp-csl.
4333        lv_different_curtp_c = abap_true.
4334      ENDIF.
4335      IF lv_curtp_d <> ls_curtp-dsl.
4336        lv_different_curtp_d = abap_true.
4337      ENDIF.
4338      IF lv_curtp_e <> ls_curtp-esl.
4339        lv_different_curtp_e = abap_true.
4340      ENDIF.
4341      IF lv_curtp_f <> ls_curtp-fsl.
4342        lv_different_curtp_f = abap_true.
4343      ENDIF.
4344      IF lv_curtp_g <> ls_curtp-gsl.
4345        lv_different_curtp_g = abap_true.
4346      ENDIF.
4347
4348    ENDLOOP.
4349
4350    CLEAR ls_add_curr_column_text.
4351    ls_add_curr_column_text-fieldname = 'DMBE2'.
4352    IF lv_different_curtp_k = abap_false.
4353      IF lv_curtp_k IS NOT INITIAL.
4354
4355        PERFORM read_text_for_curtype USING    lv_curtp_k
4356                                      CHANGING ls_add_curr_column_text-sel
      text_l
4357                                               ls_add_curr_column_text-sel
      text_m
4358                                               ls_add_curr_column_text-sel
      text_s.
4359      ENDIF.
4360    ELSE.
4361      ls_add_curr_column_text-seltext_l = 'Global Currency'(701).
4362      ls_add_curr_column_text-seltext_m = 'Global Currency'(701).
4363      ls_add_curr_column_text-seltext_s = 'Global Curr'(702).
4364    ENDIF.
4365    INSERT ls_add_curr_column_text INTO TABLE gt_add_curr_column_text.
4366
4367    CLEAR ls_add_curr_column_text.
4368    ls_add_curr_column_text-fieldname = 'DMBE3'.
4369    IF lv_different_curtp_o = abap_false.
```

```
4370      IF lv_curtp_o IS NOT INITIAL.
4371
4372        PERFORM read_text_for_curtype USING    lv_curtp_o
4373                                      CHANGING ls_add_curr_column_text-sel
      text_l
4374                                               ls_add_curr_column_text-sel
      text_m
4375                                               ls_add_curr_column_text-sel
      text_s.
4376      ENDIF.
4377    ELSE.
4378      ls_add_curr_column_text-seltext_l = '1st Freely Defined Currency'(60
      1).
4379      ls_add_curr_column_text-seltext_m = '1st Free Currency'(602).
4380      ls_add_curr_column_text-seltext_s = '1st Free Curr.'(603).
4381    ENDIF.
4382    INSERT ls_add_curr_column_text INTO TABLE gt_add_curr_column_text.
4383
4384    CLEAR ls_add_curr_column_text.
4385    ls_add_curr_column_text-fieldname = 'DMBE4'.
4386    IF lv_different_curtp_v = abap_false.
4387      IF lv_curtp_v IS NOT INITIAL.
4388
4389        PERFORM read_text_for_curtype USING    lv_curtp_v
4390                                      CHANGING ls_add_curr_column_text-sel
      text_l
4391                                               ls_add_curr_column_text-sel
      text_m
4392                                               ls_add_curr_column_text-sel
      text_s.
4393      ENDIF.
4394    ELSE.
4395      ls_add_curr_column_text-seltext_l = '2nd freely defined Currency'(61
      1).
4396      ls_add_curr_column_text-seltext_m = '2nd free Currency'(612).
4397      ls_add_curr_column_text-seltext_s = '2nd free Currency'(612).
4398    ENDIF.
4399    INSERT ls_add_curr_column_text INTO TABLE gt_add_curr_column_text.
4400
4401    CLEAR ls_add_curr_column_text.
4402    ls_add_curr_column_text-fieldname = 'DMBE5'.
4403    IF lv_different_curtp_b = abap_false.
4404      IF lv_curtp_b IS NOT INITIAL.
4405
4406        PERFORM read_text_for_curtype USING    lv_curtp_b
4407                                      CHANGING ls_add_curr_column_text-sel
      text_l
4408                                               ls_add_curr_column_text-sel
      text_m
4409                                               ls_add_curr_column_text-sel
      text_s.
4410      ENDIF.
4411    ELSE.
4412      ls_add_curr_column_text-seltext_l = '3rd freely defined Currency'(62
      1).
4413      ls_add_curr_column_text-seltext_m = '3rd free Currency'(622).
4414      ls_add_curr_column_text-seltext_s = '3rd free Curr.'(623).
4415    ENDIF.
4416    INSERT ls_add_curr_column_text INTO TABLE gt_add_curr_column_text.
4417
4418    CLEAR ls_add_curr_column_text.
```

```
4419    ls_add_curr_column_text-fieldname = 'DMBE6'.
4420    IF lv_different_curtp_c = abap_false.
4421      IF lv_curtp_c IS NOT INITIAL.
4422
4423        PERFORM read_text_for_curtype USING    lv_curtp_c
4424                                      CHANGING ls_add_curr_column_text-sel
      text_l
4425                                               ls_add_curr_column_text-sel
      text_m
4426                                               ls_add_curr_column_text-sel
      text_s.
4427      ENDIF.
4428    ELSE.
4429      ls_add_curr_column_text-seltext_l = '4th freely defined Currency'(63
      1).
4430      ls_add_curr_column_text-seltext_m = '4th free Currency'(632).
4431      ls_add_curr_column_text-seltext_s = '4th free Curr.'(633).
4432    ENDIF.
4433    INSERT ls_add_curr_column_text INTO TABLE gt_add_curr_column_text.
4434
4435    CLEAR ls_add_curr_column_text.
4436    ls_add_curr_column_text-fieldname = 'DMBE7'.
4437    IF lv_different_curtp_d = abap_false.
4438      IF lv_curtp_d IS NOT INITIAL.
4439
4440        PERFORM read_text_for_curtype USING    lv_curtp_d
4441                                      CHANGING ls_add_curr_column_text-sel
      text_l
4442                                               ls_add_curr_column_text-sel
      text_m
4443                                               ls_add_curr_column_text-sel
      text_s.
4444      ENDIF.
4445    ELSE.
4446      ls_add_curr_column_text-seltext_l = '5th freely defined Currency'(64
      1).
4447      ls_add_curr_column_text-seltext_m = '5th free Currency'(642).
4448      ls_add_curr_column_text-seltext_s = '5th free Curr.'(643).
4449    ENDIF.
4450    INSERT ls_add_curr_column_text INTO TABLE gt_add_curr_column_text.
4451
4452    CLEAR ls_add_curr_column_text.
4453    ls_add_curr_column_text-fieldname = 'DMBE8'.
4454    IF lv_different_curtp_e = abap_false.
4455      IF lv_curtp_e IS NOT INITIAL.
4456
4457        PERFORM read_text_for_curtype USING    lv_curtp_e
4458                                      CHANGING ls_add_curr_column_text-sel
      text_l
4459                                               ls_add_curr_column_text-sel
      text_m
4460                                               ls_add_curr_column_text-sel
      text_s.
4461      ENDIF.
4462    ELSE.
4463      ls_add_curr_column_text-seltext_l = '6th freely defined Currency'(65
      1).
4464      ls_add_curr_column_text-seltext_m = '6th free Currency'(652).
4465      ls_add_curr_column_text-seltext_s = '6th free Curr.'(653).
4466    ENDIF.
4467    INSERT ls_add_curr_column_text INTO TABLE gt_add_curr_column_text.
```

```
4468
4469    CLEAR ls_add_curr_column_text.
4470    ls_add_curr_column_text-fieldname = 'DMBE9'.
4471    IF lv_different_curtp_f = abap_false.
4472      IF lv_curtp_f IS NOT INITIAL.
4473
4474        PERFORM read_text_for_curtype USING    lv_curtp_f
4475                                      CHANGING ls_add_curr_column_text-sel
      text_l
4476                                               ls_add_curr_column_text-sel
      text_m
4477                                               ls_add_curr_column_text-sel
      text_s.
4478      ENDIF.
4479    ELSE.
4480      ls_add_curr_column_text-seltext_l = '7th freely defined Currency'(66
      1).
4481      ls_add_curr_column_text-seltext_m = '7th free Currency'(662).
4482      ls_add_curr_column_text-seltext_s = '7th free Curr.'(663).
4483    ENDIF.
4484    INSERT ls_add_curr_column_text INTO TABLE gt_add_curr_column_text.
4485
4486    CLEAR ls_add_curr_column_text.
4487    ls_add_curr_column_text-fieldname = 'DMBE10'.
4488    IF lv_different_curtp_g = abap_false.
4489      IF lv_curtp_g IS NOT INITIAL.
4490
4491        PERFORM read_text_for_curtype USING    lv_curtp_g
4492                                      CHANGING ls_add_curr_column_text-sel
      text_l
4493                                               ls_add_curr_column_text-sel
      text_m
4494                                               ls_add_curr_column_text-sel
      text_s.
4495      ENDIF.
4496    ELSE.
4497      ls_add_curr_column_text-seltext_l = '8th freely defined Currency'(67
      1).
4498      ls_add_curr_column_text-seltext_m = '8th free Currency'(672).
4499      ls_add_curr_column_text-seltext_s = '8th free Curr.'(673).
4500    ENDIF.
4501    INSERT ls_add_curr_column_text INTO TABLE gt_add_curr_column_text.
4502
4503
4504  ENDFORM.
4505  *&---------------------------------------------------------------------*
4506  *& Form READ_TEXT_FOR_CURTYPE
4507  *&---------------------------------------------------------------------*
4508  *& text
4509  *&---------------------------------------------------------------------*
4510  *      -->P_LV_CURTP_O  text
4511  *      <--P_LS_ADD_CURR_COLUMN_TEXT_SELTEX  text
4512  *      <--P_LS_ADD_CURR_COLUMN_TEXT_SELTEX  text
4513  *      <--P_LS_ADD_CURR_COLUMN_TEXT_SELTEX  text
4514  *&---------------------------------------------------------------------*
4515  FORM read_text_for_curtype  USING    p_lv_curtp TYPE curtp
4516                              CHANGING p_seltext_l TYPE scrtext_l
4517                                       p_seltext_m TYPE scrtext_m
4518                                       p_seltext_s TYPE scrtext_s.
4519
4520    DATA: lv_name TYPE fins_curtypet.
```

```
4521    DATA lv_short_name TYPE fins_curtype_stxt.
4522    DATA ls_finsc_curtypet TYPE finsc_curtypet.
4523
4524  * get names for non sap currency types
4525    IF cl_fins_acdoc_util=>is_curtype_sap( p_lv_curtp ) = abap_false.
4526      SELECT SINGLE name short_name FROM finsc_curtypet INTO CORRESPONDING
       FIELDS OF ls_finsc_curtypet WHERE curtype = p_lv_curtp
4527                                                          AND langu = sy
      -langu.
4528
4529      IF sy-subrc = 0.
4530        p_seltext_l = ls_finsc_curtypet-name.
4531        p_seltext_m = ls_finsc_curtypet-short_name.
4532        p_seltext_s = ls_finsc_curtypet-short_name.
4533      ENDIF.
4534  * fallback for missing description - write curtype currency
4535      IF ls_finsc_curtypet-name IS INITIAL OR ls_finsc_curtypet-short_name
       IS INITIAL.
4536        CONCATENATE p_lv_curtp TEXT-whr INTO p_seltext_l SEPARATED BY spac
      e.
4537        CONCATENATE p_lv_curtp TEXT-whr INTO p_seltext_m SEPARATED BY spac
      e.
4538        CONCATENATE p_lv_curtp TEXT-whr INTO p_seltext_s SEPARATED BY spac
      e.
4539      ENDIF.
4540  * get names for sap currency types
4541    ELSE.
4542      PERFORM holen_text1_zu_waehrungstyp
4543        USING p_lv_curtp
4544        CHANGING p_seltext_l
4545                 p_seltext_m
4546                 p_seltext_s.
4547    ENDIF.
4548
4549  ENDFORM.
4550  *&---------------------------------------------------------------------*
4551  *& Form HOLEN_TEXT1_ZU_WAEHRUNGSTYP
4552  *&---------------------------------------------------------------------*
4553  *& text
4554  *&---------------------------------------------------------------------*
4555  *      -->P_P_LV_CURTP  text
4556  *      <--P_P_SELTEXT_L  text
4557  *      <--P_P_SELTEXT_M  text
4558  *      <--P_P_SELTEXT_S  text
4559  *&---------------------------------------------------------------------*
4560  FORM holen_text1_zu_waehrungstyp  USING    p_lv_curtp
4561                                    CHANGING p_seltext_l
4562                                             p_seltext_m
4563                                             p_seltext_s.
4564
4565    CASE p_lv_curtp.
4566      WHEN '11'.
4567        p_seltext_l = TEXT-111.
4568        p_seltext_m = TEXT-411.
4569        p_seltext_s = TEXT-511.
4570      WHEN '12'.
4571        p_seltext_l = TEXT-211.
4572        p_seltext_m = TEXT-211.
4573        p_seltext_s = TEXT-211.
4574      WHEN '20'.                  "note 2382673
4575        p_seltext_l = TEXT-801. "note 2382673
```

```
4576        p_seltext_m = TEXT-801. "note 2382673
4577        p_seltext_s = TEXT-801. "note 2382673
4578      WHEN '30'.
4579        p_seltext_l = TEXT-012.
4580        p_seltext_m = TEXT-012.
4581        p_seltext_s = TEXT-012.
4582      WHEN '31'.
4583        p_seltext_l = TEXT-112.
4584        p_seltext_m = TEXT-112.
4585        p_seltext_s = TEXT-112.
4586      WHEN '32'.
4587        p_seltext_l = TEXT-212.
4588        p_seltext_m = TEXT-212.
4589        p_seltext_s = TEXT-212.
4590      WHEN '40'.
4591        p_seltext_l = TEXT-014.
4592        p_seltext_m = TEXT-014.
4593        p_seltext_s = TEXT-014.
4594      WHEN '50'.
4595        p_seltext_l = TEXT-015.
4596        p_seltext_m = TEXT-015.
4597        p_seltext_s = TEXT-015.
4598      WHEN '60'.
4599        p_seltext_l = TEXT-013.
4600        p_seltext_m = TEXT-013.
4601        p_seltext_s = TEXT-013.
4602      WHEN '70'.                  "note 2382673
4603        p_seltext_l = TEXT-802. "note 2382673
4604        p_seltext_m = TEXT-802. "note 2382673
4605        p_seltext_s = TEXT-802. "note 2382673
4606    ENDCASE.
4607
4608  ENDFORM.
```

```
 1  *&---------------------------------------------------------------------*
 2  *&  Include           FINS_AGING_INCLUDE
 3  *&---------------------------------------------------------------------*
 4
 5  data: gv_fins_data_temperature type date.
 6
 7  DEFINE fins_set_stack_temperature.
 8    CL_ABAP_STACK_TEMPERATURE=>SET_TEMPERATURE( IM_TEMPERATURE = CONV #( &
    1 ) ).
 9
10  END-OF-DEFINITION.
11
12  DEFINE fins_set_stack_temperaturecold.
13
14    CL_ABAP_STACK_TEMPERATURE=>SET_COLD( ).
15
16  END-OF-DEFINITION.
17
18  DEFINE fins_reset_stack_temperature.
19
20    CL_ABAP_STACK_TEMPERATURE=>RESET( ).
21
22  END-OF-DEFINITION.
23
24  DEFINE fins_set_session_temperature.
25    CL_ABAP_SESSION_TEMPERATURE=>GET_SESSION_CONTROL( )->SET_TEMPERATURE(
    IM_TEMPERATURE = CONV #( &1 ) ).
26
27  END-OF-DEFINITION.
28
29  DEFINE fins_set_session_temp_cold.
30    CL_ABAP_SESSION_TEMPERATURE=>GET_SESSION_CONTROL( )->SET_COLD( ).
31
32  END-OF-DEFINITION.
33
34  DEFINE fins_get_temperature.
35    &1 = CL_ABAP_STACK_TEMPERATURE=>GET_TEMPERATURE( ).
36
37  END-OF-DEFINITION.
```

```
*&---------------------------------------------------------------------*
*&  Include           FIN_UI_DECO_INCLUDE
*&---------------------------------------------------------------------*
* all the functions in this include are required to enable coding to be
* called in a decoupled way, i.e. from BOL/GenIL.
* In these case, statements which are related to a GUI sessions have to
be
* replaced...

* Data definition
* Messages
  DATA: gs_mess_bapiret2 TYPE bapiret2,
        gv_mess_code     TYPE string,
        gv_mess_dummy    TYPE string,
        gv_dynnr_dummy   TYPE sydynnr,
        gv_tcode_dummy   TYPE tcode.

  DATA: BEGIN OF gs_call_screen,
          dynnr   TYPE sydynnr,
          start_x TYPE numc2,
          start_y TYPE numc2,
          end_x   TYPE numc2,
          end_y   TYPE numc2,
          program TYPE programm,
        END OF gs_call_screen.

  DATA: lv_pfkey TYPE sypfkey.   " used locally in macros
  DATA: lv_progr TYPE programm.  " used locally in macros

* BAdI for Call screen replacement
  DATA: lr_call_screen_badi TYPE REF TO fin_ui_deco_call_screen.

* SCREEN
  DATA: ls_screen               LIKE screen.
  FIELD-SYMBOLS: <deco_screen> TYPE screen.

* C-Calls
  DATA: gv_ccalls_active TYPE c.

*** Messages
* The following block contains the handling of messages in the different
 modes:
* in none decoupling mode: -> sent the message directly
* in decoupling mode:      -> collect the message in the message handler

  DEFINE deco_message_0.
* raise a message without message variables
* Example: message e000(0K).                "  Parameter &1  =  E000(0K
)
*          deco_message e000(OK).

    if cl_fin_ui_deco=>mv_mode = 'X'.
      clear: gv_mess_code, gv_mess_dummy, gs_mess_bapiret2.
      gv_mess_code = '&1'.
      gs_mess_bapiret2-type = gv_mess_code(1).
      gs_mess_bapiret2-number = gv_mess_code+1(3).
      split gv_mess_code at '(' into gv_mess_dummy gs_mess_bapiret2-id.
      split gs_mess_bapiret2-id at ')' into gs_mess_bapiret2-id gv_mess_
dummy.
      call function 'FIN_UI_DECO_MESSAGE_ADD'
        exporting is_bapiret2 = gs_mess_bapiret2.
```

```
 58
 59        if gs_mess_bapiret2-type = 'E'.
 60          perform deco_initialize.
 61          message e001(fin_ui_deco) with 'MESSAGE'.
 62        elseif gs_mess_bapiret2-type = 'X'.
 63          message &1.
 64        endif.
 65      else.
 66        if sy-msgty = 'E'.
 67          perform deco_initialize.
 68        endif.
 69        message &1.
 70      endif.
 71
 72    END-OF-DEFINITION.
 73
 74    DEFINE deco_message_1.
 75  * raise a message with 1 message variable
 76  * Example: message e000(0K) WITH 'A'.
 77  *          deco_message_1 e000(0K) WITH 'A'.
 78
 79      if cl_fin_ui_deco=>mv_mode = 'X'.
 80        clear: gv_mess_code, gv_mess_dummy, gs_mess_bapiret2.
 81        gv_mess_code = '&1'.
 82        gs_mess_bapiret2-type = gv_mess_code(1).
 83        gs_mess_bapiret2-number = gv_mess_code+1(3).
 84        split gv_mess_code at '(' into gv_mess_dummy gs_mess_bapiret2-id.
 85        split gs_mess_bapiret2-id at ')' into gs_mess_bapiret2-id gv_mess_
    dummy.
 86
 87        gs_mess_bapiret2-message_v1 = &3.
 88
 89        call function 'FIN_UI_DECO_MESSAGE_ADD'
 90          exporting is_bapiret2 = gs_mess_bapiret2.
 91
 92        if gs_mess_bapiret2-type = 'E'.
 93          perform deco_initialize.
 94          message e001(fin_ui_deco) with 'MESSAGE'.
 95        elseif gs_mess_bapiret2-type = 'X'.
 96          message &1 &2 &3.
 97        endif.
 98
 99      else.
100        if sy-msgty = 'E'.
101          perform deco_initialize.
102        endif.
103        message &1 &2 &3.
104      endif.
105
106    END-OF-DEFINITION.
107
108    DEFINE deco_message_2.
109  * raise a message with 2 variables
110  * Example: message e000(0K) WITH 'A' 'B'.
111  *          deco_message_2 e000(0K) WITH 'A' #B'.
112
113      if cl_fin_ui_deco=>mv_mode = 'X'.
114        clear: gv_mess_code, gv_mess_dummy, gs_mess_bapiret2.
115        gv_mess_code = '&1'.
116        gs_mess_bapiret2-type = gv_mess_code(1).
117        gs_mess_bapiret2-number = gv_mess_code+1(3).
```

```
118         split gv_mess_code at '(' into gv_mess_dummy gs_mess_bapiret2-id.
119         split gs_mess_bapiret2-id at ')' into gs_mess_bapiret2-id gv_mess_
    dummy.
120
121         gs_mess_bapiret2-message_v1 = &3.
122         gs_mess_bapiret2-message_v2 = &4.
123
124         call function 'FIN_UI_DECO_MESSAGE_ADD'
125           exporting is_bapiret2 = gs_mess_bapiret2.
126
127         if gs_mess_bapiret2-type = 'E'.
128           perform deco_initialize.
129           message e001(fin_ui_deco) with 'MESSAGE'.
130         elseif gs_mess_bapiret2-type = 'X'.
131           message &1 &2 &3 &4.
132         endif.
133
134       else.
135         if sy-msgty = 'E'.
136           perform deco_initialize.
137         endif.
138         message &1 &2 &3 &4.
139       endif.
140
141     END-OF-DEFINITION.
142
143     DEFINE deco_message_3.
144   * raise a message with 3 variables
145   * Example: message e000(0K) WITH 'A' 'B' 'C'.
146   *          deco_message_1 e000(0K) WITH 'A' 'B' 'C'.
147
148       if cl_fin_ui_deco=>mv_mode = 'X'.
149         clear: gv_mess_code, gv_mess_dummy, gs_mess_bapiret2.
150         gv_mess_code = '&1'.
151         gs_mess_bapiret2-type = gv_mess_code(1).
152         gs_mess_bapiret2-number = gv_mess_code+1(3).
153         split gv_mess_code at '(' into gv_mess_dummy gs_mess_bapiret2-id.
154         split gs_mess_bapiret2-id at ')' into gs_mess_bapiret2-id gv_mess_
    dummy.
155
156         gs_mess_bapiret2-message_v1 = &3.
157         gs_mess_bapiret2-message_v2 = &4.
158         gs_mess_bapiret2-message_v3 = &5.
159
160         call function 'FIN_UI_DECO_MESSAGE_ADD'
161           exporting is_bapiret2 = gs_mess_bapiret2.
162
163         if gs_mess_bapiret2-type = 'E'.
164           perform deco_initialize.
165           message e001(fin_ui_deco) with 'MESSAGE'.
166         elseif gs_mess_bapiret2-type = 'X'.
167           message &1 &2 &3 &4 &5.
168         endif.
169
170       else.
171         if sy-msgty = 'E'.
172           perform deco_initialize.
173         endif.
174         message &1 &2 &3 &4 &5.
175       endif.
176
```

```
177    END-OF-DEFINITION.
178
179    DEFINE deco_message_4.
180  * raise a message with 4 variables
181  * Example: message e000(0K) WITH 'A' 'B' 'C' 'D'.
182  *          deco_message_4 e000(0K) WITH 'A' 'B' 'C' 'D'.
183
184      if cl_fin_ui_deco=>mv_mode = 'X'.
185        clear: gv_mess_code, gv_mess_dummy, gs_mess_bapiret2.
186        gv_mess_code = '&1'.
187        gs_mess_bapiret2-type = gv_mess_code(1).
188        gs_mess_bapiret2-number = gv_mess_code+1(3).
189        split gv_mess_code at '(' into gv_mess_dummy gs_mess_bapiret2-id.
190        split gs_mess_bapiret2-id at ')' into gs_mess_bapiret2-id gv_mess_
    dummy.
191
192        gs_mess_bapiret2-message_v1 = &3.
193        gs_mess_bapiret2-message_v2 = &4.
194        gs_mess_bapiret2-message_v3 = &5.
195        gs_mess_bapiret2-message_v4 = &6.
196
197        call function 'FIN_UI_DECO_MESSAGE_ADD'
198          exporting is_bapiret2 = gs_mess_bapiret2.
199
200        if gs_mess_bapiret2-type = 'E'.
201          perform deco_initialize.
202          message e001(fin_ui_deco) with 'MESSAGE'.
203        elseif gs_mess_bapiret2-type = 'X'.
204          message &1 &2 &3 &4 &5 &6.
205        endif.
206
207      else.
208        if sy-msgty = 'E'.
209          perform deco_initialize.
210        endif.
211        message &1 &2 &3 &4 &5 &6.
212      endif.
213
214    END-OF-DEFINITION.
215
216    DEFINE deco_message_sy.
217  * raise a message beased on the existing system fields (sy-msgid, sy-msg
    ty...)
218      perform deco_message_sy.
219    END-OF-DEFINITION.
220
221  *&---------------------------------------------------------------------*
222  *&      Form  deco_message_sy
223  *&---------------------------------------------------------------------*
224  *       text
225  *----------------------------------------------------------------------*
226    FORM deco_message_sy.
227
228      DATA: lv_sytabix TYPE sytabix.
229
230      IF cl_fin_ui_deco=>mv_mode = 'X' OR
231         cl_fin_ui_deco=>mv_odata_mode = 'X'.
232
233  *     Save sy-tabix, as it is changed in function module call
234        lv_sytabix = sy-tabix.
235
```

```
236         CLEAR: gv_mess_code, gv_mess_dummy, gs_mess_bapiret2.
237         gs_mess_bapiret2-id = sy-msgid.
238         gs_mess_bapiret2-type = sy-msgty.
239         gs_mess_bapiret2-number = sy-msgno.
240         gs_mess_bapiret2-message_v1 = sy-msgv1.
241         gs_mess_bapiret2-message_v2 = sy-msgv2.
242         gs_mess_bapiret2-message_v3 = sy-msgv3.
243         gs_mess_bapiret2-message_v4 = sy-msgv4.
244
245         CALL FUNCTION 'FIN_UI_DECO_MESSAGE_ADD'
246           EXPORTING
247             is_bapiret2 = gs_mess_bapiret2.
248
249         IF ( gs_mess_bapiret2-type = 'E' OR gs_mess_bapiret2-type = 'A' ).
250           PERFORM deco_initialize.
251           MESSAGE e001(fin_ui_deco) WITH 'MESSAGE'.
252         ELSEIF gs_mess_bapiret2-type = 'X'.
253           MESSAGE ID sy-msgid TYPE sy-msgty NUMBER sy-msgno
254                   WITH sy-msgv1 sy-msgv2 sy-msgv3 sy-msgv4.
255         ENDIF.
256
257         sy-tabix = lv_sytabix.
258
259       ELSE.
260
261         IF sy-msgty = 'E'.
262           PERFORM deco_initialize.
263         ENDIF.
264   * begin of note 2002772
265         IF sy-msgty = 'A' OR sy-msgty = 'X'.                "note 2562747
266           message id sy-msgid type sy-msgty number sy-msgno "note 2562747
267                   with sy-msgv1 sy-msgv2 sy-msgv3 sy-msgv4. "note 2562747
268         ELSE.                                               "note 2562747
269           CALL FUNCTION 'MESSAGE_STORE'
270             EXPORTING
271               arbgb                  = sy-msgid
272               exception_if_not_active = space
273               msgty                  = sy-msgty
274               msgv1                  = sy-msgv1
275               msgv2                  = sy-msgv2
276               msgv3                  = sy-msgv3
277               msgv4                  = sy-msgv4
278               txtnr                  = sy-msgno.
279         ENDIF.                                              "note 2562747
280   * end of note 2002772
281
282       ENDIF.
283
284     ENDFORM.                    "deco_message_sy
285
286   *** Statements
287   * The following block contains the handling of statements, to allow the
      decoupling
288
289     FORM deco_leave_program.
290   * replace the LEAVE PROGRAM statement.
291   * in the decoupling mode, the processing is stopped with an error messag
      e
292       IF cl_fin_ui_deco=>mv_mode = ' '.
293         LEAVE PROGRAM.
294       ELSE.
```

```
295         MESSAGE e001(fin_ui_deco) WITH 'LEAVE PROGRAM'.
296       ENDIF.
297
298     ENDFORM.                    "deco_leave_program
299
300   *&---------------------------------------------------------------------*
301   *&      Form  deco_leave
302   *&---------------------------------------------------------------------*
303   *       text
304   *----------------------------------------------------------------------*
305     FORM deco_leave.
306   * relace the LEAVE statement
307   * in the decoupling mode, the processing is stopped with an error messag
      e.
308       IF cl_fin_ui_deco=>mv_mode = ' '.
309         LEAVE.
310       ELSE.
311         IF sy-calld IS NOT INITIAL.
312           MESSAGE e001(fin_ui_deco) WITH 'LEAVE'.
313         ENDIF.
314       ENDIF.
315
316     ENDFORM.                    "deco_leave
317
318   *&---------------------------------------------------------------------*
319   *&      Form  deco_leave_screen
320   *&---------------------------------------------------------------------*
321   *       text
322   *----------------------------------------------------------------------*
323     FORM deco_leave_screen.
324   * replace the LEAVE SCREEN statement
325   * in the decoupling mode, the processing is stopped with an error messag
      e
326       IF cl_fin_ui_deco=>mv_mode = ' '.
327         LEAVE SCREEN.
328       ELSE.
329         MESSAGE e001(fin_ui_deco) WITH 'LEAVE SCREEN'.
330       ENDIF.
331
332     ENDFORM.                    "deco_leave_screen
333
334   *&---------------------------------------------------------------------*
335   *&      Form  deco_leave_to_screen
336   *&---------------------------------------------------------------------*
337   *       text
338   *----------------------------------------------------------------------*
339   *      -->IV_DYNNR   text
340   *----------------------------------------------------------------------*
341     FORM deco_leave_to_screen USING iv_dynnr TYPE sydynnr.
342   *  perform deco_LEAVE_TO_SCREEN using iv_dynnr.
343   * in the decoupling mode, the processing is stopped with an error messag
      e
344       IF cl_fin_ui_deco=>mv_mode = ' '.
345         LEAVE TO SCREEN iv_dynnr.
346       ELSE.
347         cl_fin_ui_deco=>mv_dynnr = iv_dynnr.
348         MESSAGE e001(fin_ui_deco) WITH 'LEAVE TO SCREEN'.
349       ENDIF.
350
351     ENDFORM.                    "deco_leave_to_screen
352
```

```
353 *&---------------------------------------------------------------------*
354 *&      Form  deco_set_screen
355 *&---------------------------------------------------------------------*
356 *       text
357 *----------------------------------------------------------------------*
358 *      -->IV_DYNNR   text
359 *----------------------------------------------------------------------*
360   FORM deco_set_screen USING iv_dynnr TYPE sydynnr.
361 * perform the SET SCREEN statement.
362 * in the decoupling mode, the screen to be called is set in the global c
    lass
363     IF cl_fin_ui_deco=>mv_mode = ' '.
364       SET SCREEN iv_dynnr.
365     ELSE.
366       cl_fin_ui_deco=>mv_dynnr = iv_dynnr.
367     ENDIF.
368
369   ENDFORM.                    "deco_set_screen
370
371 *&---------------------------------------------------------------------*
372 *&      Form  deco_leave_to_transaction
373 *&---------------------------------------------------------------------*
374 *       text
375 *----------------------------------------------------------------------*
376 *      -->IV_TCODE               text
377 *      -->IV_SKIP_FIRST_SCREEN  text
378 *----------------------------------------------------------------------*
379   FORM deco_leave_to_transaction USING iv_tcode TYPE tcode
380                                        iv_skip_first_screen TYPE boole_d
    .
381 * perform the LEAVE TO TRANSACTION statement
382 * in the decoupling mode, the processing is stopped with an error messag
    e
383     IF cl_fin_ui_deco=>mv_mode = ' '.
384       IF iv_skip_first_screen = abap_true.
385         LEAVE TO TRANSACTION iv_tcode AND SKIP FIRST SCREEN.
386       ELSE.
387         LEAVE TO TRANSACTION iv_tcode.
388       ENDIF.
389     ELSE.
390       cl_fin_ui_deco=>mv_dynnr = 0.
391       IF iv_skip_first_screen = abap_false.
392         MESSAGE e002(fin_ui_deco) WITH 'LEAVE TO TRANSACTION' iv_tcode.
393       ELSE.
394         MESSAGE e002(fin_ui_deco) WITH 'LEAVE TO TRANSACTION AND SKIP FI
    RST SCREEN' iv_tcode.
395       ENDIF.
396     ENDIF.
397
398   ENDFORM.                    "deco_leave_to_transaction
399
400 *&---------------------------------------------------------------------*
401 *&      Form  deco_leave_to_current_transaction
402 *&---------------------------------------------------------------------*
403 *       text
404 *----------------------------------------------------------------------*
405 *      -->IV_SKIP_FIRST_SCREEN  text
406 *----------------------------------------------------------------------*
407   FORM deco_leave_to_current_trans USING iv_skip_first_screen TYPE boole
    _d.
408 * perform the LEAVE TO CURRENT TRANSACTION statement
```

```
409  * in the decoupling mode, the processing is stopped with an error messag
     e
410      DATA: lv_tcode TYPE sy-tcode.
411
412      IF cl_fin_ui_deco=>mv_mode = ' '.
413        IF iv_skip_first_screen = abap_true.
414          LEAVE TO CURRENT TRANSACTION AND SKIP FIRST SCREEN.
415        ELSE.
416          LEAVE TO CURRENT TRANSACTION.
417        ENDIF.
418      ELSE.
419        lv_tcode = cl_dynpro=>get_current_transaction( ).
420        cl_fin_ui_deco=>mv_dynnr = 0.
421        IF iv_skip_first_screen = abap_false.
422          MESSAGE e002(fin_ui_deco) WITH 'LEAVE TO TRANSACTION' lv_tcode.
423        ELSE.
424          MESSAGE e002(fin_ui_deco) WITH 'LEAVE TO TRANSACTION AND SKIP FI
     RST SCREEN' lv_tcode.
425        ENDIF.
426      ENDIF.
427
428    ENDFORM.                    "deco_leave_to_transaction
429
430  *&---------------------------------------------------------------------*
431  *&      Form  deco_call_transaction
432  *&---------------------------------------------------------------------*
433  *       text
434  *----------------------------------------------------------------------*
435  *      -->IV_TCODE               text
436  *      -->IV_SKIP_FIRST_SCREEN  text
437  *----------------------------------------------------------------------*
438    FORM deco_call_transaction USING iv_tcode TYPE tcode
439                                     iv_skip_first_screen TYPE boole_d.
440  * perform the CALL TRANSACTION statement
441  * in the decoupling mode, the processing is stopped with an error messag
     e
442
443      IF cl_fin_ui_deco=>mv_mode = ' '.
444        IF iv_skip_first_screen = abap_true.
445          call transaction iv_tcode WITHOUT AUTHORITY-CHECK and skip first
      screen.
446        ELSE.
447          call transaction iv_tcode WITHOUT AUTHORITY-CHECK.
448        ENDIF.
449      ELSE.
450        IF iv_skip_first_screen = abap_true.
451          MESSAGE e002(fin_ui_deco) WITH 'CALL TRANSACTION AND SKIP FIRST
     SCREEN' iv_tcode.
452        ELSE.
453          MESSAGE e002(fin_ui_deco) WITH 'CALL TRANSACTION' iv_tcode.
454        ENDIF.
455      ENDIF.
456
457    ENDFORM.                    "deco_call_transaction
458
459  *&---------------------------------------------------------------------*
460  *&      Form  deco_call_screen
461  *&---------------------------------------------------------------------*
462  *       text
463  *----------------------------------------------------------------------*
464  *      -->IS_CALL_SCREEN  text
```

```
465  *---------------------------------------------------------------------*
466    FORM deco_call_screen USING is_call_screen STRUCTURE gs_call_screen.
467  * process the CALL SCREEN statement (with all possible parameters)
468  * in the decoupling mode, a BAdI Implemantation to handle the called scr
     een is processed
469      DATA:
470        lv_deco_dynnr   TYPE sydynnr,
471        lv_deco_sydynnr TYPE sydynnr,
472        lv_deco_sydyngr TYPE sydyngr,
473        lv_deco_program TYPE syrepid,
474        lv_deco_pfkey   TYPE sypfkey,
475        lt_screen       TYPE fin_deco_t_screen,
476        lv_deco_sysubrc TYPE sysubrc.
477
478      IF cl_fin_ui_deco=>mv_mode = ' '.
479        IF is_call_screen-start_x <> 0.
480          IF is_call_screen-end_x <> 0.
481            CALL SCREEN is_call_screen-dynnr
482                                  STARTING AT is_call_screen-start_x
483                                              is_call_screen-start_y
484                                  ENDING   AT is_call_screen-end_x
485                                              is_call_screen-end_y.
486          ELSE.
487            CALL SCREEN is_call_screen-dynnr STARTING AT is_call_screen-st
     art_x
488                                                       is_call_screen-st
     art_y.
489          ENDIF.
490        ELSE.
491          CALL SCREEN is_call_screen-dynnr.
492        ENDIF.
493      ELSE.
494
495  *-----  Buffer the screen data of the calling screen
496        lv_deco_sydynnr = cl_fin_ui_deco=>sydynnr( ).
497        lv_deco_sydyngr = cl_fin_ui_deco=>sydyngr( ).
498        lv_deco_dynnr   = cl_fin_ui_deco=>mv_dynnr.
499        lt_screen[]     = cl_fin_ui_deco=>mt_screen[].
500        lv_deco_program = cl_fin_ui_deco=>mv_pfkey_repid.
501        lv_deco_pfkey   = cl_fin_ui_deco=>mv_pfkey.
502
503  *  call the 'Call Screen' BadI
504        GET BADI lr_call_screen_badi.
505
506        CALL BADI lr_call_screen_badi->call_screen
507          EXPORTING
508            iv_dynnr  = is_call_screen-dynnr
509            iv_repid  = is_call_screen-program
510          EXCEPTIONS
511            user_exit = 1.
512
513        lv_deco_sysubrc = sy-subrc.
514
515  *-----  Reset the screen data to the values of the calling screen
516        cl_fin_ui_deco=>set_data( iv_sydynnr = lv_deco_sydynnr
517                                  iv_dynnr   = lv_deco_dynnr
518                                  iv_sydyngr = lv_deco_sydyngr
519                                  it_screen  = lt_screen[] ).
520
521        cl_fin_ui_deco=>mv_pfkey       = lv_deco_pfkey.
522        cl_fin_ui_deco=>mv_pfkey_repid = lv_deco_program.
```

```
523
524         IF lv_deco_sysubrc = 1.
525           MESSAGE ID sy-msgid TYPE sy-msgty NUMBER sy-msgno
526                   WITH sy-msgv1 sy-msgv2 sy-msgv3 sy-msgv4 RAISING user_ex
    it.
527         ENDIF.
528
529       ENDIF.
530     ENDFORM.                    "deco_call_screen
531
532   *&---------------------------------------------------------------------*
533   *&      Form  deco_call_form
534   *&---------------------------------------------------------------------*
535   *       text
536   *----------------------------------------------------------------------*
537   *  -->  p1        text
538   *  <--  p2        text
539   *----------------------------------------------------------------------*
540     FORM deco_call_form USING formname TYPE formname
541                               progname TYPE progname.
542   * process the call of an (external) form
543   * in the decoupling mode, the form is called in the program FIN_UI_DECO_
    DUPLICATES
544       IF cl_fin_ui_deco=>mv_mode = abap_false.
545         PERFORM (formname) IN PROGRAM (progname).
546       ELSE.
547         PERFORM (formname) IN PROGRAM fin_ui_deco_duplicates IF FOUND.
548       ENDIF.
549     ENDFORM.                    " deco_call_form
550
551   *&---------------------------------------------------------------------*
552   *&      Form  deco_function_not_possible
553   *&---------------------------------------------------------------------*
554   *       text
555   *----------------------------------------------------------------------*
556     FORM deco_function_not_possible USING progname TYPE progname.
557       IF cl_fin_ui_deco=>mv_mode = abap_true.
558         MESSAGE e003(fin_ui_deco) WITH progname.
559       ENDIF.
560     ENDFORM.                    "deco_function_not_possible
561   *&---------------------------------------------------------------------*
562   *&      Form  deco_call_subscreen
563   *&---------------------------------------------------------------------*
564   *       text
565   *----------------------------------------------------------------------*
566   *  -->  p1        text
567   *  <--  p2        text
568   *----------------------------------------------------------------------*
569     FORM deco_call_subscreen USING uv_function_name TYPE string.
570       DATA:
571         lv_subrc          TYPE sy-subrc,
572         lv_deco_dynnr     TYPE sydynnr,
573         lv_deco_dyngr     TYPE sydyngr,
574         lv_deco_program   TYPE syrepid,
575         lv_deco_pfkey     TYPE sypfkey,
576         lt_screen         TYPE fin_deco_t_screen,
577         ls_bapiret2       TYPE bapiret2,
578         lv_ignore_message TYPE boole_d.
579
580       CHECK cl_fin_ui_deco=>mv_mode = abap_true.
581
```

```
582  *------  Buffer the screen data of the calling screen
583      lv_deco_dyngr   = sy-dyngr.
584      lv_deco_dynnr   = cl_fin_ui_deco=>mv_dynnr.
585      lv_deco_program = cl_fin_ui_deco=>mv_pfkey_repid.
586      lv_deco_pfkey   = cl_fin_ui_deco=>mv_pfkey.
587      lt_screen[]     = cl_fin_ui_deco=>mt_screen[].
588
589  *------ Call the SUBSCREEN function module
590      CALL FUNCTION uv_function_name
591        EXCEPTIONS
592          error_message = 1.
593
594      IF sy-subrc <> 0.
595        lv_subrc = sy-subrc.
596      ENDIF.
597
598  *------  Reset the screen data to the values of the calling screen
599      sy-dynnr                          = lv_deco_dynnr.
600      sy-dyngr                          = lv_deco_dyngr.
601      cl_fin_ui_deco=>mv_dynnr          = lv_deco_dynnr.
602      cl_fin_ui_deco=>mt_screen[]       = lt_screen[].
603      cl_fin_ui_deco=>mv_pfkey          = lv_deco_pfkey.
604      cl_fin_ui_deco=>mv_pfkey_repid = lv_deco_program.
605
606      IF lv_subrc <> 0.
607        CALL FUNCTION 'BALW_BAPIRETURN_GET2'
608          EXPORTING
609            type   = sy-msgty
610            cl     = sy-msgid
611            number = sy-msgno
612            par1   = sy-msgv1
613            par2   = sy-msgv2
614            par3   = sy-msgv3
615            par4   = sy-msgv4
616          IMPORTING
617            return = ls_bapiret2.
618
619  *------ Checked whether from the new UI perspective
620  *------ the message shall be ignored
621        cl_fin_ui_deco=>check_message( EXPORTING is_bapiret2       = ls_ba
     piret2
622                                       RECEIVING rv_ignore_message = lv_ig
     nore_message ).
623
624        IF lv_ignore_message IS INITIAL.
625          MESSAGE   ID ls_bapiret2-id
626                  TYPE ls_bapiret2-type
627                NUMBER ls_bapiret2-number
628                  WITH ls_bapiret2-message_v1
629                       ls_bapiret2-message_v2
630                       ls_bapiret2-message_v3
631                       ls_bapiret2-message_v4.
632        ENDIF.
633      ENDIF.
634    ENDFORM.                      " deco_call_subscreen
635  * call screen has to be handeled individually.
636  * submit has to be handeled individually (due to lot of parameter)
637  * reject and get have to be handled individually: select / exit
638
639  *** SCREEN Table
640  * The following block contains the handling of the SCREEN related statem
```

```
ents
641
642   DEFINE deco_loop_at_screen.
643 * process the LOOP AT SCREEN statement
644
645 * in the not decoupled mode (!) the C-Calls DYNP_INIT_LOOP_AT_SCREEN and
646 * DYNP_LOOP_AT_SCREEN are used if they are available. This is checked wi
    th
647 * C-Call DYNP_GET_STATUS function 37. If they are not there,
648 * the screen table of the dynpro is copied into the screen table of
649 * the global class (CL_FIN_UI_DECO).
650 * in decoupled mode, the screen table is set external
651     perform deco_loop_at_screen.
652
653 *    loop at cl_fin_ui_deco=>mt_screen into screen.
654     do.
655       if cl_fin_ui_deco=>mv_mode = abap_true or gv_ccalls_active = 'N'.
656 *       decoupled mode or screen table copied to global class
657         read table cl_fin_ui_deco=>mt_screen into screen index sy-index.
658       else.
659 *       not decoupled mode - use C-Call to get the screen entry
660         call 'DYNP_LOOP_AT_SCREEN' id 'SCREEN' field screen.
661       endif.
662       if sy-subrc ne 0.
663         exit. "DO-ENDDO due to no more entries (4) or c-call parameter e
    rror (1)
664       endif.
665   END-OF-DEFINITION.
666
667 *&---------------------------------------------------------------------*
668 *&      Form  deco_loop_at_screen
669 *&---------------------------------------------------------------------*
670 *       text
671 *----------------------------------------------------------------------*
672   FORM deco_loop_at_screen.
673     DATA: dummy TYPE i.
674
675     CHECK cl_fin_ui_deco=>mv_mode = abap_false.
676 * this is the not-decoupled mode
677 *  - check whteher the C-Calls are available
678     IF gv_ccalls_active IS INITIAL.
679       CALL 'DYNP_GET_STATUS' ID 'FUNCTION' FIELD 37
680                              ID 'VALUE' FIELD dummy.      "#EC CI_CCALL
681       IF sy-subrc NE 0 OR dummy NE 1.
682         gv_ccalls_active = 'N'. "no, C-calls not active
683       ELSE.
684         gv_ccalls_active = 'Y'. "yes, C-calls are active
685       ENDIF.
686     ENDIF.
687 *  - initialize the LOOP AT SCREEN replacement
688     IF gv_ccalls_active = 'N'.
689       REFRESH cl_fin_ui_deco=>mt_screen.   " copy screen table from dynp
    ro
690       LOOP AT SCREEN.
691         APPEND screen TO cl_fin_ui_deco=>mt_screen.
692       ENDLOOP.
693     ELSE.
694       CALL 'DYNP_INIT_LOOP_AT_SCREEN'.                    "#EC CI_CCALL
695     ENDIF.
696
697     IF cl_fin_ui_deco=>mv_mode = abap_true.
```

```
698         cl_fin_ui_deco=>mt_screen_def = cl_fin_ui_deco=>mt_screen.
699       ENDIF.
700
701     ENDFORM.                    "deco_loop_at_screen
702
703     DEFINE deco_modify_screen.
704   * process the MODIFY SCREEN statement
705   * in case of deccoupled mode, the screen table in the global class
706   * is updated/modified.
707   * Otherwise the dynpro screen table is modified
708       perform deco_modify_screen.
709
710     END-OF-DEFINITION.
711
712   *&---------------------------------------------------------------------*
713   *&       Form  deco_modify_screen
714   *&---------------------------------------------------------------------*
715   *       text
716   *----------------------------------------------------------------------*
717     FORM deco_modify_screen.
718       DATA: ls_screen_def        TYPE screen.
719       DATA: lv_tabix             TYPE sytabix.
720       FIELD-SYMBOLS: <fs_screen> TYPE screen.
721
722       IF cl_fin_ui_deco=>mv_mode = abap_false.
723   *     not decoupled mode
724         IF gv_ccalls_active = 'N'.
725   *       no C-calls: modify dynpro from global class' screen table
726           ls_screen = screen.
727           LOOP AT SCREEN.
728             CHECK screen-name = ls_screen-name.
729             screen = ls_screen.
730             MODIFY SCREEN.
731             EXIT.
732           ENDLOOP.
733         ELSE.
734   *       C-calls active: simply use MODIFY SCREEN
735           MODIFY SCREEN.
736         ENDIF.
737       ELSE.
738
739         READ TABLE cl_fin_ui_deco=>mt_screen ASSIGNING <fs_screen> WITH KE
      Y name = screen-name.
740         IF sy-subrc = 0.
741           lv_tabix = sy-tabix.
742   * field is deactivated
743           IF screen-active = 0 AND <fs_screen>-active <> 0.
744             screen-input = 0.
745             screen-output = 0.
746             screen-invisible = 1.
747           ENDIF.
748   * field is hidden/ implicit deactivation
749           IF screen-input = 0 AND screen-output = 0 AND screen-invisible =
       1 AND <fs_screen>-active <> 1.
750             screen-active = 0.
751           ENDIF.
752           READ TABLE cl_fin_ui_deco=>mt_screen_def INTO ls_screen_def WITH
       KEY name = screen-name.
753   * field is activated
754           IF screen-active = 1 AND <fs_screen>-active = 0.
755   * attribute input is dependent on the original setting of the field in t
```

```
    he screen painter
 756            IF ls_screen_def-input = 1.
 757              screen-input = 1.
 758            ELSE.
 759              screen-input = 0.
 760            ENDIF.
 761            screen-output = 1.
 762            screen-invisible = 0.
 763          ENDIF.
 764  * a field, which is not open for input can not be mandatory
 765          IF screen-input = 0.
 766            screen-required = 0.
 767          ENDIF.
 768          MODIFY cl_fin_ui_deco=>mt_screen FROM screen INDEX lv_tabix.
 769        ELSE.
 770          ASSERT 0 = 1.
 771        ENDIF.
 772      ENDIF.
 773    ENDFORM.                    "deco_modify_screen
 774
 775    DEFINE deco_endloop_screen.
 776  * process the ENDLOOP of the LOOP At SCREEN statement
 777  * we have to close the open DO loop and to clear the secure-flag
 778      enddo. "closes the DO of macro deco_loop_at_screen
 779
 780      perform deco_endloop_screen.
 781
 782    END-OF-DEFINITION.
 783
 784  *&---------------------------------------------------------------------*
 785  *&      Form  deco_endloop_screen
 786  *&---------------------------------------------------------------------*
 787  *       text
 788  *----------------------------------------------------------------------*
 789    FORM deco_endloop_screen.
 790    ENDFORM.                    "deco_endloop_screen
 791
 792    DEFINE deco_exit_from_step_loop.
 793  * process the EXIT FROM STEPLOOP statement
 794  * in the decoupled mode, this statement is not necessary
 795      if cl_fin_ui_deco=>mv_mode = abap_false.
 796        exit from step-loop.
 797      endif.
 798
 799    END-OF-DEFINITION.
 800
 801  *&---------------------------------------------------------------------*
 802  *&      Form  deco_initialize
 803  *&---------------------------------------------------------------------*
 804  *       text
 805  *----------------------------------------------------------------------*
 806    FORM deco_initialize.
 807  * initialize the global data
 808    ENDFORM.                    "deco_initialize
 809
 810  *&---------------------------------------------------------------------*
 811  *&      Form  DECO_CHECK_SIM_MODE
 812  *&---------------------------------------------------------------------*
 813  *       text
 814  *----------------------------------------------------------------------*
 815    FORM deco_check_sim_mode CHANGING cv_deco_sim_mode.
```

```
816       DATA:
817         lv_deco_mode TYPE abap_bool,
818         lv_bvorg     TYPE bvorg,
819         ls_verifdoc  TYPE fud_verifdoc_key,
820         lv_bvtst     TYPE abap_bool.
821
822   *------ Get the status of UI decoupling mode
823   *------ (export in the function module FUD_REVERSE_FI_DOCUMENT)
824       IMPORT lv_deco_mode lv_bvorg fud_verifdoc_key = ls_verifdoc
825                                      FROM MEMORY ID 'VERIFDOC_KEY'.
826       IF lv_deco_mode IS NOT INITIAL.
827
828   *------ Get the status of reversal simulation
829         IMPORT bvtst = lv_bvtst FROM MEMORY ID 'BVORG'.
830       ENDIF.
831
832       IF lv_bvtst                               IS NOT INITIAL
833       OR cl_fin_ui_deco=>mv_simulation_mode IS NOT INITIAL.
834         cv_deco_sim_mode = abap_true.
835       ENDIF.
836     ENDFORM.                    "DECO_CHECK_SIM_MODE
837
838   *** PF-Status handling
839     DEFINE deco_set_pf_status.
840   * SET PF-STATUS without excluding
841   * Example: set pf-status '0100'.
842   *          deco_set_pf_status '0100'.
843
844       if cl_fin_ui_deco=>mv_mode = 'X'.
845         lv_pfkey = &1.
846         call method cl_fin_ui_deco=>set_pfkey
847           exporting
848             iv_pfkey = lv_pfkey
849             iv_repid = sy-repid.
850       else.
851         set pf-status &1.
852       endif.
853
854     END-OF-DEFINITION.
855
856     DEFINE deco_set_pf_status_excluding.
857   * SET PF-STATUS with excluding
858   * Example: set pf-status '0100'  excluding excl_tab.
859   *          deco_set_pf_status '0100' excluding excl_tab.
860
861       if cl_fin_ui_deco=>mv_mode = 'X'.
862         lv_pfkey = &1.
863         call method cl_fin_ui_deco=>set_pfkey
864           exporting
865             iv_pfkey = lv_pfkey
866             iv_repid = sy-repid
867             it_excl_tab = &3[].
868
869       else.
870         set pf-status &1 &2 &3.
871       endif.
872     END-OF-DEFINITION.
873
874     DEFINE deco_set_pf_status_of_program.
875   * SET PF-STATUS of a program
876   * Example: SET PF-STATUS GUI_STATUS OF PROGRAM GUI_PROGRAM.
```

```
877  *          deco_set_pf_status_of program GUI_STATUS OF PROGRAM GUI_PROGR
     AM.
878
879      if cl_fin_ui_deco=>mv_mode = 'X'.
880        lv_pfkey = &1.
881        lv_progr = &4.
882        call method cl_fin_ui_deco=>set_pfkey
883          exporting
884            iv_pfkey = lv_pfkey
885            iv_repid = lv_progr.
886
887      else.
888        set pf-status &1 &2 &3 &4.
889      endif.
890    END-OF-DEFINITION.
```

```
*&---------------------------------------------------------------------*
*&  Include           RFDBEXPC                                         *
*&---------------------------------------------------------------------*
*&  Expiring currencies: conversion tools                              *
*&---------------------------------------------------------------------*

*&---------------------------------------------------------------------*
*&      Form  ITEM_CURR_CONVERT_BSID
*&---------------------------------------------------------------------*
*&      input ID_HWAER is optional!
*&---------------------------------------------------------------------*
FORM ITEM_CURR_CONVERT_BSID USING    ID_PROC   TYPE TPRCPROC
                                     VALUE(ID_HWAER) TYPE WAERS
                                     ID_DATE   TYPE SYDATUM
                            CHANGING CS_BSID   TYPE BSID.

  STATICS: ST_FLDLIST_BSID  TYPE DDFLDNAM OCCURS 5 WITH HEADER LINE.
  DATA: LD_LINES         TYPE I,
        LD_CURR_NEW      TYPE WAERS.

* document currency expired?
  CALL FUNCTION 'CURRENCY_GET_SUBSEQUENT'
       EXPORTING
            CURRENCY     = CS_BSID-WAERS
            PROCESS      = ID_PROC
            DATE         = ID_DATE
            BUKRS        = CS_BSID-BUKRS
       IMPORTING
            CURRENCY_NEW = LD_CURR_NEW.

  IF LD_CURR_NEW NE CS_BSID-WAERS.
*   fill list of BSID amount fields referring to document currency:
    DESCRIBE TABLE ST_FLDLIST_BSID LINES LD_LINES.
    IF LD_LINES = 0.
      PERFORM FILL_WAERS_FIELDLIST TABLES ST_FLDLIST_BSID
                                   USING  'BSID'.
    ENDIF.
*   get local currency:
    IF ID_HWAER IS INITIAL.
      SELECT SINGLE WAERS FROM T001 INTO ID_HWAER
                          WHERE BUKRS = CS_BSID-BUKRS.
    ENDIF.
*   convert amounts:
    CALL FUNCTION 'CURRENCY_DOCUMENT_CONVERT'
         EXPORTING
              FROM_CURRENCY       = CS_BSID-WAERS
              TO_CURRENCY         = LD_CURR_NEW
              LOCAL_CURRENCY      = ID_HWAER
              DATE                = ID_DATE
              CONVERSION_MODE     = 'O'
         TABLES
              FIELDLIST           = ST_FLDLIST_BSID
         CHANGING
              LINE                = CS_BSID
         EXCEPTIONS
              FIELD_UNKNOWN       = 1
              FIELD_NOT_AMOUNT    = 2
              ERROR_IN_CONVERSION = 3
              ILLEGAL_PARAMETERS  = 4
              OTHERS              = 5.
*   update currency field:
```

```
  62      IF SY-SUBRC = 0.
  63        CS_BSID-WAERS = LD_CURR_NEW.
  64      ENDIF.
  65    ENDIF.
  66
  67  * payment currency expired?
  68    CHECK NOT CS_BSID-PYCUR IS INITIAL.
  69    CALL FUNCTION 'CURRENCY_GET_SUBSEQUENT'
  70         EXPORTING
  71              CURRENCY     = CS_BSID-PYCUR
  72              PROCESS      = ID_PROC
  73              DATE         = ID_DATE
  74              BUKRS        = CS_BSID-BUKRS
  75         IMPORTING
  76              CURRENCY_NEW = LD_CURR_NEW.
  77
  78  * convert payment amount:
  79    IF LD_CURR_NEW NE CS_BSID-PYCUR.
  80  *   get local currency:
  81      IF ID_HWAER IS INITIAL.
  82        SELECT SINGLE WAERS FROM T001 INTO ID_HWAER
  83                              WHERE BUKRS = CS_BSID-BUKRS.
  84      ENDIF.
  85      CALL FUNCTION 'CONVERT_FOREIGN_TO_FOREIGN_CUR'
  86           EXPORTING
  87                DATE              = ID_DATE
  88                FROM_AMOUNT       = CS_BSID-PYAMT
  89                FROM_CURRENCY     = CS_BSID-PYCUR
  90                TO_CURRENCY       = LD_CURR_NEW
  91                LOCAL_CURRENCY    = ID_HWAER
  92                CONVERSION_MODE   = 'X'
  93           IMPORTING
  94                TO_AMOUNT         = CS_BSID-PYAMT
  95           EXCEPTIONS
  96                NO_RATE_FOUND     = 1
  97                OVERFLOW          = 2
  98                NO_FACTORS_FOUND  = 3
  99                NO_SPREAD_FOUND   = 4
 100                DERIVED_2_TIMES   = 5
 101                OTHERS            = 6.
 102  *   update currency field:
 103      IF SY-SUBRC = 0.
 104        CS_BSID-PYCUR = LD_CURR_NEW.
 105      ENDIF.
 106    ENDIF.
 107
 108  ENDFORM.                    " ITEM_CURR_CONVERT_BSID
 109
 110  *&---------------------------------------------------------------------*
 111  *&      Form  ITEM_CURR_CONVERT_BSIK
 112  *&---------------------------------------------------------------------*
 113  *&      input ID_HWAER is optional!
 114  *&---------------------------------------------------------------------*
 115  FORM ITEM_CURR_CONVERT_BSIK USING    ID_PROC   TYPE TPRCPROC
 116                                       VALUE(ID_HWAER) TYPE WAERS
 117                                       ID_DATE   TYPE SYDATUM
 118                              CHANGING CS_BSIK   TYPE BSIK.
 119
 120    STATICS: ST_FLDLIST_BSIK  TYPE DDFLDNAM OCCURS 5 WITH HEADER LINE.
 121    DATA: LD_LINES         TYPE I,
 122          LD_CURR_NEW      TYPE WAERS.
```

```
123
124   * document currency expired?
125     CALL FUNCTION 'CURRENCY_GET_SUBSEQUENT'
126          EXPORTING
127               CURRENCY     = CS_BSIK-WAERS
128               PROCESS      = ID_PROC
129               DATE         = ID_DATE
130               BUKRS        = CS_BSIK-BUKRS
131          IMPORTING
132               CURRENCY_NEW = LD_CURR_NEW.
133
134     IF LD_CURR_NEW NE CS_BSIK-WAERS.
135   *   fill list of BSIK amount fields referring to document currency:
136       DESCRIBE TABLE ST_FLDLIST_BSIK LINES LD_LINES.
137       IF LD_LINES = 0.
138         PERFORM FILL_WAERS_FIELDLIST TABLES ST_FLDLIST_BSIK
139                                      USING  'BSIK'.
140       ENDIF.
141   *   get local currency:
142       IF ID_HWAER IS INITIAL.
143         SELECT SINGLE WAERS FROM T001 INTO ID_HWAER
144                               WHERE BUKRS = CS_BSIK-BUKRS.
145       ENDIF.
146   *   convert amounts:
147       CALL FUNCTION 'CURRENCY_DOCUMENT_CONVERT'
148            EXPORTING
149                 FROM_CURRENCY       = CS_BSIK-WAERS
150                 TO_CURRENCY         = LD_CURR_NEW
151                 LOCAL_CURRENCY      = ID_HWAER
152                 DATE                = ID_DATE
153                 CONVERSION_MODE     = 'O'
154            TABLES
155                 FIELDLIST           = ST_FLDLIST_BSIK
156            CHANGING
157                 LINE                = CS_BSIK
158            EXCEPTIONS
159                 FIELD_UNKNOWN       = 1
160                 FIELD_NOT_AMOUNT    = 2
161                 ERROR_IN_CONVERSION = 3
162                 ILLEGAL_PARAMETERS  = 4
163                 OTHERS              = 5.
164   *   update currency field:
165       IF SY-SUBRC = 0.
166         CS_BSIK-WAERS = LD_CURR_NEW.
167       ENDIF.
168     ENDIF.
169
170   * payment currency expired?
171     CHECK NOT CS_BSIK-PYCUR IS INITIAL.
172     CALL FUNCTION 'CURRENCY_GET_SUBSEQUENT'
173          EXPORTING
174               CURRENCY     = CS_BSIK-PYCUR
175               PROCESS      = ID_PROC
176               DATE         = ID_DATE
177               BUKRS        = CS_BSIK-BUKRS
178          IMPORTING
179               CURRENCY_NEW = LD_CURR_NEW.
180
181   * convert payment amount:
182     IF LD_CURR_NEW NE CS_BSIK-PYCUR.
183   *   get local currency:
```

```
184       IF ID_HWAER IS INITIAL.
185         SELECT SINGLE WAERS FROM T001 INTO ID_HWAER
186                               WHERE BUKRS = CS_BSIK-BUKRS.
187       ENDIF.
188       CALL FUNCTION 'CONVERT_FOREIGN_TO_FOREIGN_CUR'
189           EXPORTING
190                DATE              = ID_DATE
191                FROM_AMOUNT       = CS_BSIK-PYAMT
192                FROM_CURRENCY     = CS_BSIK-PYCUR
193                TO_CURRENCY       = LD_CURR_NEW
194                LOCAL_CURRENCY    = ID_HWAER
195                CONVERSION_MODE   = 'X'
196           IMPORTING
197                TO_AMOUNT         = CS_BSIK-PYAMT
198           EXCEPTIONS
199                NO_RATE_FOUND     = 1
200                OVERFLOW          = 2
201                NO_FACTORS_FOUND  = 3
202                NO_SPREAD_FOUND   = 4
203                DERIVED_2_TIMES   = 5
204                OTHERS            = 6.
205   *   update currency field:
206       IF SY-SUBRC = 0.
207         CS_BSIK-PYCUR = LD_CURR_NEW.
208       ENDIF.
209     ENDIF.
210
211   ENDFORM.                    " ITEM_CURR_CONVERT_BSIK
212
213   *&---------------------------------------------------------------------*
214   *&      Form  ITEM_CURR_CONVERT_BSIS
215   *&---------------------------------------------------------------------*
216   *&      input ID_HWAER is optional!
217   *&---------------------------------------------------------------------*
218   FORM ITEM_CURR_CONVERT_BSIS USING    ID_PROC   TYPE TPRCPROC
219                                        VALUE(ID_HWAER) TYPE WAERS
220                                        ID_DATE   TYPE SYDATUM
221                               CHANGING CS_BSIS   TYPE BSIS.
222
223     STATICS: ST_FLDLIST_BSIS  TYPE DDFLDNAM OCCURS 5 WITH HEADER LINE.
224     DATA: LD_LINES         TYPE I,
225           LD_CURR_NEW      TYPE WAERS.
226
227   * document currency expired?
228     CALL FUNCTION 'CURRENCY_GET_SUBSEQUENT'
229          EXPORTING
230               CURRENCY     = CS_BSIS-WAERS
231               PROCESS      = ID_PROC
232               DATE         = ID_DATE
233               BUKRS        = CS_BSIS-BUKRS
234          IMPORTING
235               CURRENCY_NEW = LD_CURR_NEW.
236
237     IF LD_CURR_NEW NE CS_BSIS-WAERS.
238   *   fill list of BSIK amount fields referring to document currency:
239       DESCRIBE TABLE ST_FLDLIST_BSIS LINES LD_LINES.
240       IF LD_LINES = 0.
241         PERFORM FILL_WAERS_FIELDLIST TABLES ST_FLDLIST_BSIS
242                                      USING  'BSIS'.
243       ENDIF.
244   *   get local currency:
```

```
245       IF ID_HWAER IS INITIAL.
246         SELECT SINGLE WAERS FROM T001 INTO ID_HWAER
247                               WHERE BUKRS = CS_BSIS-BUKRS.
248       ENDIF.
249   *   convert amounts:
250       CALL FUNCTION 'CURRENCY_DOCUMENT_CONVERT'
251            EXPORTING
252                 FROM_CURRENCY       = CS_BSIS-WAERS
253                 TO_CURRENCY         = LD_CURR_NEW
254                 LOCAL_CURRENCY      = ID_HWAER
255                 DATE                = ID_DATE
256                 CONVERSION_MODE     = 'O'
257            TABLES
258                 FIELDLIST           = ST_FLDLIST_BSIS
259            CHANGING
260                 LINE                = CS_BSIS
261            EXCEPTIONS
262                 FIELD_UNKNOWN       = 1
263                 FIELD_NOT_AMOUNT    = 2
264                 ERROR_IN_CONVERSION = 3
265                 ILLEGAL_PARAMETERS  = 4
266                 OTHERS              = 5.
267   *   update currency field:
268       IF SY-SUBRC = 0.
269         CS_BSIS-WAERS = LD_CURR_NEW.
270       ENDIF.
271     ENDIF.
272
273   ENDFORM.                    " ITEM_CURR_CONVERT_BSIS
274
275   *&---------------------------------------------------------------------*
276   *&      Form  ITEM_CURR_CONVERT_BSEGA
277   *&---------------------------------------------------------------------*
278   *&      input ID_HWAER is optional!
279   *&---------------------------------------------------------------------*
280   FORM ITEM_CURR_CONVERT_BSEGA USING    ID_PROC   TYPE TPRCPROC
281                                         ID_BUKRS  TYPE BUKRS
282                                         VALUE(ID_HWAER) TYPE WAERS
283                                         ID_WAERS  TYPE WAERS
284                                         ID_DATE   TYPE SYDATUM
285                                CHANGING CS_BSEGA  TYPE BSEGA.
286
287     STATICS: ST_FLDLIST_BSEGA TYPE DDFLDNAM OCCURS 5 WITH HEADER LINE.
288     DATA: LD_LINES          TYPE I,
289           LD_CURR_NEW       TYPE WAERS.
290
291   * document currency expired?
292     CALL FUNCTION 'CURRENCY_GET_SUBSEQUENT'
293          EXPORTING
294               CURRENCY     = ID_WAERS
295               PROCESS      = ID_PROC
296               DATE         = ID_DATE
297               BUKRS        = ID_BUKRS
298          IMPORTING
299               CURRENCY_NEW = LD_CURR_NEW.
300
301     IF LD_CURR_NEW NE ID_WAERS.
302   *   fill list of BSEGA amount fields referring to document currency:
303       DESCRIBE TABLE ST_FLDLIST_BSEGA LINES LD_LINES.
304       IF LD_LINES = 0.
305         ST_FLDLIST_BSEGA-NAME = 'WRSOL'.
```

```
306         APPEND ST_FLDLIST_BSEGA.
307         ST_FLDLIST_BSEGA-NAME = 'WRHAB'.
308         APPEND ST_FLDLIST_BSEGA.
309         ST_FLDLIST_BSEGA-NAME = 'WRSHB'.
310         APPEND ST_FLDLIST_BSEGA.
311         ST_FLDLIST_BSEGA-NAME = 'MWSHB'.
312         APPEND ST_FLDLIST_BSEGA.
313       ENDIF.
314   *   get local currency:
315       IF ID_HWAER IS INITIAL.
316         SELECT SINGLE WAERS FROM T001 INTO ID_HWAER
317                               WHERE BUKRS = ID_BUKRS.
318       ENDIF.
319   *   convert amounts:
320       CALL FUNCTION 'CURRENCY_DOCUMENT_CONVERT'
321            EXPORTING
322                 FROM_CURRENCY       = ID_WAERS
323                 TO_CURRENCY         = LD_CURR_NEW
324                 LOCAL_CURRENCY      = ID_HWAER
325                 DATE                = ID_DATE
326                 CONVERSION_MODE     = 'O'
327            TABLES
328                 FIELDLIST           = ST_FLDLIST_BSEGA
329            CHANGING
330                 LINE                = CS_BSEGA
331            EXCEPTIONS
332                 FIELD_UNKNOWN       = 1
333                 FIELD_NOT_AMOUNT    = 2
334                 ERROR_IN_CONVERSION = 3
335                 ILLEGAL_PARAMETERS  = 4
336                 OTHERS              = 5.
337     ENDIF.
338
339   ENDFORM.                    " ITEM_CURR_CONVERT_BSEGA
340
341   *&---------------------------------------------------------------------*
342   *&      Form  ITEM_CURR_CONVERT_BKPF
343   *&---------------------------------------------------------------------*
344   FORM ITEM_CURR_CONVERT_BKPF USING    ID_PROC   TYPE TPRCPROC
345                                        ID_DATE   TYPE SYDATUM
346                               CHANGING CS_BKPF   TYPE BKPF.
347
348     DATA: LD_CURR_NEW      TYPE WAERS.
349
350   * document currency expired?
351     CALL FUNCTION 'CURRENCY_GET_SUBSEQUENT'
352          EXPORTING
353               CURRENCY     = CS_BKPF-WAERS
354               PROCESS      = ID_PROC
355               DATE         = ID_DATE
356               BUKRS        = CS_BKPF-BUKRS
357          IMPORTING
358               CURRENCY_NEW = LD_CURR_NEW.
359
360   * convert currency key:
361     IF LD_CURR_NEW NE CS_BKPF-WAERS.
362       CS_BKPF-WAERS = LD_CURR_NEW.
363     ENDIF.
364
365   ENDFORM.                    " ITEM_CURR_CONVERT_BKPF
366
```

```
367  *&---------------------------------------------------------------------*
368  *&      Form  ITEM_CURR_CONVERT_BSEG
369  *&---------------------------------------------------------------------*
370  *&      input ID_HWAER is optional!
371  *&---------------------------------------------------------------------*
372  FORM ITEM_CURR_CONVERT_BSEG USING    ID_PROC   TYPE TPRCPROC
373                                       VALUE(ID_HWAER) TYPE WAERS
374                                       ID_WAERS  TYPE WAERS
375                                       ID_DATE   TYPE SYDATUM
376                              CHANGING CS_BSEG   TYPE BSEG.
377
378    STATICS: ST_FLDLIST_BSEG  TYPE DDFLDNAM OCCURS 5 WITH HEADER LINE.
379    DATA: LD_LINES         TYPE I,
380          LD_CURR_NEW      TYPE WAERS.
381
382  * document currency expired?
383    CALL FUNCTION 'CURRENCY_GET_SUBSEQUENT'
384         EXPORTING
385              CURRENCY     = ID_WAERS
386              PROCESS      = ID_PROC
387              DATE         = ID_DATE
388              BUKRS        = CS_BSEG-BUKRS
389         IMPORTING
390              CURRENCY_NEW = LD_CURR_NEW.
391
392    IF LD_CURR_NEW NE ID_WAERS.
393  *   fill list of BSEG amount fields referring to document currency:
394      DESCRIBE TABLE ST_FLDLIST_BSEG LINES LD_LINES.
395      IF LD_LINES = 0.
396        PERFORM FILL_WAERS_FIELDLIST TABLES ST_FLDLIST_BSEG
397                                     USING  'BSEG'.
398      ENDIF.
399  *   get local currency:
400      IF ID_HWAER IS INITIAL.
401        SELECT SINGLE WAERS FROM T001 INTO ID_HWAER
402                            WHERE BUKRS = CS_BSEG-BUKRS.
403      ENDIF.
404  *   convert amounts:
405      CALL FUNCTION 'CURRENCY_DOCUMENT_CONVERT'
406           EXPORTING
407                FROM_CURRENCY       = ID_WAERS
408                TO_CURRENCY         = LD_CURR_NEW
409                LOCAL_CURRENCY      = ID_HWAER
410                DATE                = ID_DATE
411                CONVERSION_MODE     = 'O'
412           TABLES
413                FIELDLIST           = ST_FLDLIST_BSEG
414           CHANGING
415                LINE                = CS_BSEG
416           EXCEPTIONS
417                FIELD_UNKNOWN       = 1
418                FIELD_NOT_AMOUNT    = 2
419                ERROR_IN_CONVERSION = 3
420                ILLEGAL_PARAMETERS  = 4
421                OTHERS              = 5.
422    ENDIF.
423
424  * payment currency expired?
425    CHECK NOT CS_BSEG-PYCUR IS INITIAL.
426    CALL FUNCTION 'CURRENCY_GET_SUBSEQUENT'
427         EXPORTING
```

```
428              CURRENCY     = CS_BSEG-PYCUR
429              PROCESS      = ID_PROC
430              DATE         = ID_DATE
431              BUKRS        = CS_BSEG-BUKRS
432            IMPORTING
433              CURRENCY_NEW = LD_CURR_NEW.
434
435    * convert payment amount:
436      IF LD_CURR_NEW NE CS_BSEG-PYCUR.
437    *   get local currency:
438        IF ID_HWAER IS INITIAL.
439          SELECT SINGLE WAERS FROM T001 INTO ID_HWAER
440                                WHERE BUKRS = CS_BSEG-BUKRS.
441        ENDIF.
442        CALL FUNCTION 'CONVERT_FOREIGN_TO_FOREIGN_CUR'
443            EXPORTING
444                 DATE              = ID_DATE
445                 FROM_AMOUNT       = CS_BSEG-PYAMT
446                 FROM_CURRENCY     = CS_BSEG-PYCUR
447                 TO_CURRENCY       = LD_CURR_NEW
448                 LOCAL_CURRENCY    = ID_HWAER
449                 CONVERSION_MODE   = 'X'
450            IMPORTING
451                 TO_AMOUNT         = CS_BSEG-PYAMT
452            EXCEPTIONS
453                 NO_RATE_FOUND     = 1
454                 OVERFLOW          = 2
455                 NO_FACTORS_FOUND  = 3
456                 NO_SPREAD_FOUND   = 4
457                 DERIVED_2_TIMES   = 5
458                 OTHERS            = 6.
459    *   update currency field:
460        IF SY-SUBRC = 0.
461          CS_BSEG-PYCUR = LD_CURR_NEW.
462        ENDIF.
463      ENDIF.
464
465    ENDFORM.                    " ITEM_CURR_CONVERT_BSEG
466
467    *&---------------------------------------------------------------------*
468    *&      Form  ITEM_CURR_CONVERT_WITH_ITEM
469    *&---------------------------------------------------------------------*
470    *&      input ID_HWAER is optional!
471    *&---------------------------------------------------------------------*
472    FORM ITEM_CURR_CONVERT_WITH_ITEM USING    ID_PROC      TYPE TPRCPROC
473                                              VALUE(ID_HWAER) TYPE WAERS
474                                              ID_WAERS     TYPE WAERS
475                                              ID_DATE      TYPE SYDATUM
476                                    CHANGING CS_WITH_ITEM TYPE WITH_ITEM.
477
478      STATICS: ST_FLDLIST_WITH  TYPE DDFLDNAM OCCURS 5 WITH HEADER LINE.
479      DATA: LD_LINES         TYPE I,
480            LD_CURR_NEW      TYPE WAERS.
481
482    * document currency expired?
483      CALL FUNCTION 'CURRENCY_GET_SUBSEQUENT'
484           EXPORTING
485                CURRENCY     = ID_WAERS
486                PROCESS      = ID_PROC
487                DATE         = ID_DATE
488                BUKRS        = CS_WITH_ITEM-BUKRS
```

```
489         IMPORTING
490              CURRENCY_NEW = LD_CURR_NEW.
491
492    IF LD_CURR_NEW NE ID_WAERS.
493    *   fill list of WITH_ITEM amount fields referring to document currency:
494      DESCRIBE TABLE ST_FLDLIST_WITH LINES LD_LINES.
495      IF LD_LINES = 0.
496        PERFORM FILL_WAERS_FIELDLIST TABLES ST_FLDLIST_WITH
497                                     USING  'WITH_ITEM'.
498      ENDIF.
499    *   get local currency:
500      IF ID_HWAER IS INITIAL.
501        SELECT SINGLE WAERS FROM T001 INTO ID_HWAER
502                            WHERE BUKRS = CS_WITH_ITEM-BUKRS.
503      ENDIF.
504    *   convert amounts:
505      CALL FUNCTION 'CURRENCY_DOCUMENT_CONVERT'
506           EXPORTING
507                FROM_CURRENCY       = ID_WAERS
508                TO_CURRENCY         = LD_CURR_NEW
509                LOCAL_CURRENCY      = ID_HWAER
510                DATE                = ID_DATE
511                CONVERSION_MODE     = 'O'
512           TABLES
513                FIELDLIST           = ST_FLDLIST_WITH
514           CHANGING
515                LINE                = CS_WITH_ITEM
516           EXCEPTIONS
517                FIELD_UNKNOWN       = 1
518                FIELD_NOT_AMOUNT    = 2
519                ERROR_IN_CONVERSION = 3
520                ILLEGAL_PARAMETERS  = 4
521                OTHERS              = 5.
522    ENDIF.
523
524  ENDFORM.                    " ITEM_CURR_CONVERT_WITH_ITEM
525
526  *&---------------------------------------------------------------------*
527  *&      Form  ITEM_CURR_CONVERT_GSEG
528  *&---------------------------------------------------------------------*
529  *&      input ID_HWAER is optional!
530  *&---------------------------------------------------------------------*
531  FORM ITEM_CURR_CONVERT_GSEG USING    ID_PROC   TYPE TPRCPROC
532                                       VALUE(ID_HWAER) TYPE WAERS
533                                       ID_WAERS  TYPE WAERS
534                                       ID_DATE   TYPE SYDATUM
535                              CHANGING CS_GSEG   TYPE GSEG.
536
537    STATICS: ST_FLDLIST_GSEG  TYPE DDFLDNAM OCCURS 5 WITH HEADER LINE.
538    DATA: LD_LINES         TYPE I,
539          LD_CURR_NEW      TYPE WAERS.
540
541  * document currency expired?
542    CALL FUNCTION 'CURRENCY_GET_SUBSEQUENT'
543         EXPORTING
544              CURRENCY     = ID_WAERS
545              PROCESS      = ID_PROC
546              DATE         = ID_DATE
547              BUKRS        = CS_GSEG-BUKRS
548         IMPORTING
549              CURRENCY_NEW = LD_CURR_NEW.
```

```
550
551     IF LD_CURR_NEW NE ID_WAERS.
552   *   fill list of BSEG amount fields referring to document currency:
553       DESCRIBE TABLE ST_FLDLIST_GSEG LINES LD_LINES.
554       IF LD_LINES = 0.
555         PERFORM FILL_WAERS_FIELDLIST TABLES ST_FLDLIST_GSEG
556                                      USING  'GSEG'.
557       ENDIF.
558   *   get local currency:
559       IF ID_HWAER IS INITIAL.
560         SELECT SINGLE WAERS FROM T001 INTO ID_HWAER
561                             WHERE BUKRS = CS_GSEG-BUKRS.
562       ENDIF.
563   *   convert amounts:
564       CALL FUNCTION 'CURRENCY_DOCUMENT_CONVERT'
565            EXPORTING
566                 FROM_CURRENCY       = ID_WAERS
567                 TO_CURRENCY         = LD_CURR_NEW
568                 LOCAL_CURRENCY      = ID_HWAER
569                 DATE                = ID_DATE
570                 CONVERSION_MODE     = 'O'
571            TABLES
572                 FIELDLIST           = ST_FLDLIST_GSEG
573            CHANGING
574                 LINE                = CS_GSEG
575            EXCEPTIONS
576                 FIELD_UNKNOWN       = 1
577                 FIELD_NOT_AMOUNT    = 2
578                 ERROR_IN_CONVERSION = 3
579                 ILLEGAL_PARAMETERS  = 4
580                 OTHERS              = 5.
581     ENDIF.
582
583   * payment currency expired?
584     CHECK NOT CS_GSEG-PYCUR IS INITIAL.
585     CALL FUNCTION 'CURRENCY_GET_SUBSEQUENT'
586          EXPORTING
587               CURRENCY     = CS_GSEG-PYCUR
588               PROCESS      = ID_PROC
589               DATE         = ID_DATE
590               BUKRS        = CS_GSEG-BUKRS
591          IMPORTING
592               CURRENCY_NEW = LD_CURR_NEW.
593
594   * convert payment amount:
595     IF LD_CURR_NEW NE CS_GSEG-PYCUR.
596   *   get local currency:
597       IF ID_HWAER IS INITIAL.
598         SELECT SINGLE WAERS FROM T001 INTO ID_HWAER
599                             WHERE BUKRS = CS_GSEG-BUKRS.
600       ENDIF.
601       CALL FUNCTION 'CONVERT_FOREIGN_TO_FOREIGN_CUR'
602            EXPORTING
603                 DATE              = ID_DATE
604                 FROM_AMOUNT       = CS_GSEG-PYAMT
605                 FROM_CURRENCY     = CS_GSEG-PYCUR
606                 TO_CURRENCY       = LD_CURR_NEW
607                 LOCAL_CURRENCY    = ID_HWAER
608                 CONVERSION_MODE   = 'X'
609            IMPORTING
610                 TO_AMOUNT         = CS_GSEG-PYAMT
```

```
611           EXCEPTIONS
612                NO_RATE_FOUND    = 1
613                OVERFLOW         = 2
614                NO_FACTORS_FOUND = 3
615                NO_SPREAD_FOUND  = 4
616                DERIVED_2_TIMES  = 5
617                OTHERS           = 6.
618   *   update currency field:
619       IF SY-SUBRC = 0.
620         CS_GSEG-PYCUR = LD_CURR_NEW.
621       ENDIF.
622     ENDIF.
623
624   ENDFORM.                    " ITEM_CURR_CONVERT_GSEG
625
626   *&---------------------------------------------------------------------*
627   *&      Form  ITEM_CURR_CONVERT_BSET
628   *&---------------------------------------------------------------------*
629   *&      input ID_HWAER is optional!
630   *&---------------------------------------------------------------------*
631   FORM ITEM_CURR_CONVERT_BSET USING    ID_PROC   TYPE TPRCPROC
632                                        VALUE(ID_HWAER) TYPE WAERS
633                                        ID_WAERS  TYPE WAERS
634                                        ID_DATE   TYPE SYDATUM
635                               CHANGING CS_BSET   TYPE BSET.
636
637     STATICS: ST_FLDLIST_BSET  TYPE DDFLDNAM OCCURS 5 WITH HEADER LINE.
638     DATA: LD_LINES         TYPE I,
639           LD_CURR_NEW      TYPE WAERS.
640
641   * document currency expired?
642     CALL FUNCTION 'CURRENCY_GET_SUBSEQUENT'
643          EXPORTING
644               CURRENCY     = ID_WAERS
645               PROCESS      = ID_PROC
646               DATE         = ID_DATE
647               BUKRS        = CS_BSET-BUKRS
648          IMPORTING
649               CURRENCY_NEW = LD_CURR_NEW.
650
651     IF LD_CURR_NEW NE ID_WAERS.
652   *   fill list of BSET amount fields referring to document currency:
653       DESCRIBE TABLE ST_FLDLIST_BSET LINES LD_LINES.
654       IF LD_LINES = 0.
655         PERFORM FILL_WAERS_FIELDLIST TABLES ST_FLDLIST_BSET
656                                      USING  'BSET'.
657       ENDIF.
658   *   get local currency:
659       IF ID_HWAER IS INITIAL.
660         SELECT SINGLE WAERS FROM T001 INTO ID_HWAER
661                             WHERE BUKRS = CS_BSET-BUKRS.
662       ENDIF.
663   *   convert amounts:
664       CALL FUNCTION 'CURRENCY_DOCUMENT_CONVERT'
665            EXPORTING
666                 FROM_CURRENCY       = ID_WAERS
667                 TO_CURRENCY         = LD_CURR_NEW
668                 LOCAL_CURRENCY      = ID_HWAER
669                 DATE                = ID_DATE
670                 CONVERSION_MODE     = 'O'
671            TABLES
```

```
672               FIELDLIST           = ST_FLDLIST_BSET
673            CHANGING
674               LINE                = CS_BSET
675            EXCEPTIONS
676               FIELD_UNKNOWN       = 1
677               FIELD_NOT_AMOUNT    = 2
678               ERROR_IN_CONVERSION = 3
679               ILLEGAL_PARAMETERS  = 4
680               OTHERS              = 5.
681     ENDIF.
682
683   ENDFORM.                    " ITEM_CURR_CONVERT_BSET
684
685   *&---------------------------------------------------------------------*
686   *&      Form  FILL_WAERS_FIELDLIST
687   *&---------------------------------------------------------------------*
688   FORM FILL_WAERS_FIELDLIST TABLES CT_FLDLIST STRUCTURE DDFLDNAM
689                             USING  ID_TABNAME TYPE DDOBJNAME.
690
691     DATA: LT_DFIES    TYPE DFIES OCCURS 0 WITH HEADER LINE,
692           LS_DDFLDNAM TYPE DDFLDNAM.
693
694     REFRESH CT_FLDLIST.
695     CALL FUNCTION 'DDIF_NAMETAB_GET'
696          EXPORTING
697               TABNAME   = ID_TABNAME
698          TABLES
699               DFIES_TAB = LT_DFIES
700          EXCEPTIONS
701               NOT_FOUND = 1
702               OTHERS    = 2.
703     IF SY-SUBRC NE 0.
704       EXIT.
705     ENDIF.
706     LOOP AT LT_DFIES.
707       IF  LT_DFIES-DATATYPE = 'CURR'
708       AND ( LT_DFIES-REFTABLE = ID_TABNAME OR LT_DFIES-REFTABLE = 'BKPF' )
709       AND LT_DFIES-REFFIELD = 'WAERS'.
710         CT_FLDLIST-NAME = LT_DFIES-FIELDNAME.
711         APPEND CT_FLDLIST.
712       ENDIF.
713     ENDLOOP.
714
715   ENDFORM.                    " FILL_WAERS_FIELDLIST
716   ***INCLUDE RFDBEXPC .
```

```
***INCLUDE RFDBRMAC.

* Aufruf: COPY itab1 TO itab2.
* Initialisiert itab2 und kopiert dann itab1 nach itab2
DEFINE COPY.
  IF '&2' = 'TO'. ENDIF. "Dummyverwendung des zweiten Arguments
  REFRESH &3.
  LOOP AT &1 INTO &3.
    APPEND &3.
  ENDLOOP.
  CLEAR &3.
END-OF-DEFINITION.

DEFINE GET_FRAME_TITLE.
  PERFORM GET_FRAME_TITLE(SAPDBBRF) USING &1 _TEXT_&1.
END-OF-DEFINITION.

DEFINE BEGIN_OF_BLOCK.
  SELECTION-SCREEN BEGIN OF BLOCK &1 WITH FRAME TITLE _TEXT_&1.
END-OF-DEFINITION.

DEFINE END_OF_BLOCK.
  SELECTION-SCREEN END   OF BLOCK &1.
END-OF-DEFINITION.

* Makros für Tabstrips
DEFINE BEGIN_OF_SCREEN.                                          " 4.6A
  SELECTION-SCREEN BEGIN OF SCREEN &1 AS SUBSCREEN.              " 4.6A
END-OF-DEFINITION.                                               " 4.6A

DEFINE END_OF_SCREEN.                                            " 4.6A
  SELECTION-SCREEN END   OF SCREEN &1.                           " 4.6A
END-OF-DEFINITION.                                               " 4.6A

DEFINE GET_TABSTRIP_TITLE.                                       " 4.6A
  DATA: TS_TEXT&1 LIKE RFPDO-ALLGLINE.                           " 4.6A
  PERFORM GET_FRAME_TITLE(SAPDBBRF) USING &1 TS_TEXT&1.          " 4.6A
  TAB&1 = TS_TEXT&1.                                             " 4.6A
END-OF-DEFINITION.                                               " 4.6A

DEFINE TABSTRIP.                                                 " 4.6A
  SELECTION-SCREEN TAB &2 TAB&1                                  " 4.6A
                          USER-COMMAND UCOM&1                    " 4.6A
                          DEFAULT SCREEN &1.                     " 4.6A
END-OF-DEFINITION.

DEFINE BEGIN_OF_TABBED_BLOCK.                                    " 4.6A
  SELECTION-SCREEN BEGIN OF TABBED BLOCK TABBL&1                 " 4.6A
                                     FOR &2 LINES.               " 4.6A
END-OF-DEFINITION.                                               " 4.6A

* Makros für Pufferung
*_____________________
DEFINE BUFFER.
* Felder zur Cursorverwaltung
  DATA: CURSOR-&1 LIKE SY-INDEX.
* Interne Tabellen zur Zwischenpufferung
  DATA: BEGIN OF X&1 OCCURS 0.
          INCLUDE STRUCTURE &1.
  DATA: END OF X&1.
END-OF-DEFINITION.
```

```
  62
  63  * Abarbeiten des Puffers fuer Kundennummer
  64  * Beispiel 1: ACCESS_BUFFER_1 KNAS KUNNR
  65  * Beispiel 2: ACCESS_BUFFER_1 LFAS LIFNR
  66  DEFINE ACCESS_BUFFER_1.
  67    IF CURSOR-&1 = 0.
  68      PERFORM UPDATE_X&1.
  69    ENDIF.
  70    DO.
  71      READ TABLE X&1 INDEX CURSOR-&1.
  72      IF SY-SUBRC NE 0.
  73        EXIT.
  74      ENDIF.
  75      IF SAVE_&2 < X&1-&2. "Eintrag nicht enthalten
  76        EXIT.
  77      ELSEIF SAVE_&2 > X&1-&2. "Weiterlesen
  78        CURSOR-&1 = CURSOR-&1 + 1.
  79      ELSE. "Treffer
  80        &1 = X&1.
  81        PUT &1.
  82        CURSOR-&1 = CURSOR-&1 + 1.
  83      ENDIF. "KUNNR
  84    ENDDO.
  85  END-OF-DEFINITION.
  86
  87  * Abarbeiten des Puffers fuer Kundennummer und Buchungskreis
  88  * Beispiel 1: ACCESS_BUFFER_2 KNC1 KUNNR BUKRS
  89  * Beispiel 2: ACCESS_BUFFER_2 LFC1 LIFNR BUKRS
  90  DEFINE ACCESS_BUFFER_2.
  91    IF CURSOR-&1 = 0.
  92      PERFORM UPDATE_X&1.
  93    ENDIF.
  94    DO.
  95      READ TABLE X&1 INDEX CURSOR-&1.
  96      IF SY-SUBRC NE 0.
  97        EXIT.
  98      ENDIF.
  99      IF SAVE_&2 < X&1-&2. "Eintrag nicht enthalten
 100        EXIT.
 101      ELSEIF SAVE_&2 > X&1-&2. "Weiterlesen
 102        CURSOR-&1 = CURSOR-&1 + 1.
 103      ELSE. "Treffer
 104        IF SAVE_&3 < X&1-&3.
 105          EXIT.
 106        ELSEIF SAVE_&3 > X&1-&3.
 107          CURSOR-&1 = CURSOR-&1 + 1.
 108        ELSE.
 109          &1 = X&1.
 110          PUT &1.
 111          CURSOR-&1 = CURSOR-&1 + 1.
 112        ENDIF. "BUKRS
 113      ENDIF. "KUNNR
 114    ENDDO.
 115  END-OF-DEFINITION.
 116
 117  DEFINE INIT_WHERE.
 118    MOVE '&1' TO L_DS_CLAUSES-TABLENAME.
 119    READ TABLE DYN_SEL-CLAUSES WITH KEY L_DS_CLAUSES-TABLENAME
 120                                INTO L_DS_CLAUSES.
 121    IF SY-SUBRC = 0.
 122      &1_WHERE[] =  L_DS_CLAUSES-WHERE_TAB[].
```

```
123    ENDIF.
124  END-OF-DEFINITION.
125
126  DEFINE INIT_FIELDS.
127    MOVE '&1' TO L_TAB_FIELDS-TABLENAME.
128    READ TABLE SELECT_FIELDS WITH KEY L_TAB_FIELDS-TABLENAME
129                               INTO L_TAB_FIELDS.
130    IF SY-SUBRC = 0.
131      &1_FIELDS[] = L_TAB_FIELDS-FIELDS[].
132    ENDIF.
133  END-OF-DEFINITION.
134
135  DEFINE CHECK_WIA_AND_MODIFY_SCREEN.
136    CALL FUNCTION 'CHECK_PLANTS_ABROAD_ACTIVE'
137         EXPORTING
138              I_CUST        = 'X'
139         IMPORTING
140              E_FI_ISACTIVE = &1.
141    IF &1 = SPACE.
142      LOOP AT SCREEN.
143        IF SCREEN-GROUP1 = 'WIA'.
144          SCREEN-INPUT = '0'.
145          SCREEN-INVISIBLE = '1'.
146          MODIFY SCREEN.
147        ENDIF.
148      ENDLOOP.
149      REFRESH SEL_LSTM.
150      CLEAR SEL_LSTM.
151    ENDIF.
152  END-OF-DEFINITION.
153
154  * Problems with unicode in headerline
155  DEFINE UC_DATA.
156    DATA: L_LINE    TYPE SYULINE,
157          L_OFFSET  TYPE I,
158          L_OFF_TAB TYPE ABAP_OFFSET_TAB.
159  END-OF-DEFINITION.
160
161  DEFINE UC_INITIALIZE.
162    CLEAR L_OFF_TAB.
163  END-OF-DEFINITION.
164
165  DEFINE UC_VLINE_OFFSET.
166    APPEND &1 TO L_OFF_TAB.
167  END-OF-DEFINITION.
168
169  DEFINE UC_HEADERLINE.
170    L_LINE = &1.
171    CALL METHOD CL_ABAP_LIST_UTILITIES=>DISPLAY_TO_MEMORY
172      EXPORTING
173        DISPLAY_DATA = L_LINE
174        OFFSET_TAB   = L_OFF_TAB
175      IMPORTING
176        MEMORY_DATA  = L_LINE.
177
178    LOOP AT L_OFF_TAB INTO L_OFFSET.
179      IF L_LINE+L_OFFSET(1) = SPACE.
180        WRITE SY-VLINE TO L_LINE+L_OFFSET(1).
181      ENDIF.
182    ENDLOOP.
183
```

```
184    CALL METHOD CL_ABAP_LIST_UTILITIES=>MEMORY_TO_DISPLAY
185      EXPORTING
186        MEMORY_DATA  = L_LINE
187        OFFSET_TAB   = L_OFF_TAB
188      IMPORTING
189        DISPLAY_DATA = L_LINE.
190  END-OF-DEFINITION.
```

```
1  *---------------------------------------------------------------------*
2  *   INCLUDE RFITEM_SEL                                                *
3  *---------------------------------------------------------------------*
4
5  *... dark select options for derived RFPOSX fields
6  *    (which are not accessible by free selections):
7  select-options: so_bwwrt for it_pos-bwwrt no-display,
8                  so_bwwr2 for it_pos-bwwr2 no-display,
9                  so_bwwr3 for it_pos-bwwr3 no-display,
10                 so_dmshb for it_pos-dmshb no-display,
11                 so_hwaer for it_pos-hwaer no-display,
12                 so_dmbe2 for it_pos-dmbe2 no-display,
13                 so_hwae2 for it_pos-hwae2 no-display,
14                 so_dmbe3 for it_pos-dmbe3 no-display,
15                 so_hwae3 for it_pos-hwae3 no-display,
16                 so_jamon for it_pos-jamon no-display,
17                 so_koart for it_pos-koart no-display.
18
19 select-options: so_qbshb for it_pos-qbshb no-display,
20                 so_qsfbt for it_pos-qsfbt no-display,
21                 so_qsshb for it_pos-qsshb no-display,
22                 so_skfbt for it_pos-skfbt no-display,
23                 so_wskto for it_pos-wskto no-display,
24                 so_sknto for it_pos-sknto no-display,
25                 so_verz1 for it_pos-verz1 no-display,
26                 so_verzn for it_pos-verzn no-display,
27                 so_wrshb for it_pos-wrshb no-display,
28                 so_waers for it_pos-waers no-display,
29                 so_zaldt for it_pos-zaldt no-display,
30                 so_zinsz for it_pos-zinsz no-display.
31
32 select-options: so_askto for it_pos-askto no-display,
33                 so_kurse for it_pos-kurse no-display,
34                 so_kkbwr for it_pos-kkbwr no-display.
35
36
37 select-options: so_gsber for it_pos-gsber no-display.
38
39 *... check debit/credit flag according to negative posting (F*10N):
40 select-options: so_shkzg for it_pos-shkzg no-display.
41
42 PARAMETERS fs_dyns       TYPE rsds_type NO-DISPLAY.         "n2199162
```

Title:
G/L Account Line Item Display

Text elements:

001 List Output
002 Status
003 Open Items
004 Cleared Items
005 All Items
006 Type
007 Line Item Selection
008 Items. Selection continuing.....
009 List being generated.....
012 Group Currency
013 Global Company Currency
014 Hard Currency
015 Index-Based Currency
101 Normal Items
102 Entry View
111 Company Code Currency/Group Valuation
112 Group Currency/Group Valuation
120 With Balance Adjustments
211 Company Code Currency/Profit Center Valuation
212 Group Currency/Profit Center Valuation
270 Custom Selections
271 Active
272 Choose Ledger
273 Continue
274 Save as User Parameters
275 Set Ledger
276 Free Selection
277 G/L View
278 Entry View
300 AcctType
320 1st Freely Defined Currency
321 2nd Freely Defined Currency
322 2nd Free Currency
323 2nd Free Curr.
324 3rd Freely Defined Currency
325 3rd Free Currency
326 3rd Free Curr.
327 Curr. 1st Free. Def. Curr.
328 1st Curr.
329 FDCur1
330 Curr. 2nd Free. Def. Curr.
331 Curr. 2nd
332 FDCur2
333 Curr. 3rd Free. Def. Curr.
334 Curr. 3rd
335 FDCur3
411 Loc Crcy/Grp Valuat.
511 Loc Grp
601 1st Freely Defined Currency
602 1st Free Currency
603 1st Free Currency
611 2nd Freely Defined Currency
612 2nd Free Currency
613 2nd Free Currency
621 3rd Freely Defined Currency

622 3rd Free Currency
623 3rd Free Currency
631 4th Freely Defined Currency
632 4th Free Currency
633 4th Free Currency
641 5th Freely Defined Currency
642 5th Free Currency
643 5th Free Currency
651 6th Freely Defined Currency
652 6th Free Currency
653 6th Free Currency
661 7th Freely Defined Currency
662 7th Free Currency
663 7th Free Currency
671 8th Freely Defined Currency
672 8th Free Currency
673 8th Free Currency
701 Group Currency
702 Global Currency
801 Controlling Area Currency
802 CO Object Currency
W01 Activate Worklist
W02 G/L Account
W03 Company Code
W04 Deactivate Worklist
WAR Currency
WHR Currency

Selection texts:

PA_NMAX
Maximum Number of Items
PA_STID2
Open at Key Date
PA_STIDA
Open at Key Date
PA_VARI
Layout
PA_WLBUK
Company Code Worklist
PA_WLSAK
G/L Account Worklist
RLDNR
Ledger
SO_AUGDT
Clearing Date
SO_BUDAT
Posting Date
SO_WLBUK
Or Values
SO_WLSAK
Or Values
X_AISEL
All Items
X_CLSEL
Cleared Items
X_MERK
Noted Items
X_NORM

```
        Normal Items
X_OPSEL
        Open Items
X_PARK
        Parked Items
X_GLYEC
D       .
```

Owner..... SAP
Short text..... Einzelposten/Salden/Konfiguration
Edit Language DE
Status 20.12.21

F4430 No line item display possible for account & &

Diagnosis

System Response

Procedure

Procedure for System Administration

Owner..... SAP
Short text..... TR-FM Funds Management
Edit Language DE
Status 21.11.24

FI001 System error when deleting FI/CO object number &
FI002 Master record for commitment item does not exist
FI003 FM area & does not exist

Diagnosis

System Response

Procedure

Procedure for System Administration

Owner..... SAP
Short text..... Messages FI-Reporting
Edit Language DE
Status 04.06.26

FR812 No authorization for & &
FR866 New general ledger is not implemented; use transaction &

Owner..... SAP
Short text..... FI-SL-Hilfsmittel
Edit Language DE
Status 25.02.25

GU167 Data cannot be posted to the combination ledger & compan code &

Diagnosis
When trying to update data for ledger and company code , it was found that data cannot be posted to this combination.
System Response
The data is not updated in the central system.
Procedure
Check the ledger - company code assignment in configuration.

Owner..... SAP
Short text..... Nachrichten Transaktionsvarianten, Batchinput Aufzeichnung
Edit Language DE
Status 28.04.23

MS030 Function only supported with cross-client variants

Diagnosis
You choose the "Transport" function for a cross-client transaction variant.
Cross-client transaction variants are automatically included in a transport request any time they are altered, manual transport is not necessary.
System Response

Procedure

Procedure for System Administration

MS040 Variant group &1 is not used in transactions

Owner..... SAP
Short text..... Einzelpostenanzeige mit ALV, RFITEM**
Edit Language DE
Status 06.04.21

MSITEM011 Internal error in REUSE_ALV_FIELDCATALOG_MERGE
Diagnosis

System Response

Procedure

Procedure for System Administration

MSITEM012 Internal error in ANALYZE_ACT_FIELDCAT
Diagnosis

System Response

Procedure

Procedure for System Administration

MSITEM019 Caution! Long selection times possible!
Diagnosis

System Response

Procedure

Procedure for System Administration

MSITEM020 Select at least one item type
MSITEM022 Internal error in ITEM_DERIVE_FIELDS
Diagnosis

System Response

Procedure

Procedure for System Administration

MSITEM023 Maximum number of &1 items reached
Diagnosis

System Response

Procedure

Procedure for System Administration

MSITEM024 & items displayed
MSITEM033 No items selected (see long text)
Diagnosis

System Response

Procedure

Procedure for System Administration

MSITEM100 Entry view because New General Ledger Accounting not act in CoCode &1
MSITEM101 Custom selections deleted due to Entry View
Diagnosis

System Response

Procedure

Procedure for System Administration

MSITEM102 Custom selections deleted due to change in ledger
Diagnosis

System Response

Procedure

Procedure for System Administration

MSITEM103 Custom selections deleted because fields not active in ledger
Diagnosis

System Response

Procedure

Procedure for System Administration

MSITEM104 Internal error in custom selections
Diagnosis

System Response

Procedure

Procedure for System Administration

MSITEM105 Entry View set because of migration
Diagnosis

System Response

Procedure

Procedure for System Administration

MSITEM106 Due to a migration, not all data may be selected
Diagnosis

System Response

Procedure

Procedure for System Administration

MSITEM107 Company code &1 was not assigned to a controlling area
Diagnosis

System Response

Procedure

Procedure for System Administration

MSITEM108 Ledger &1 is not a ledger in New General Ledger Accounti
Diagnosis

System Response

Procedure

Procedure for System Administration

MSITEM111 Account &, company code & is clearing account
Diagnosis

System Response

Procedure

Procedure for System Administration

MSITEM112 & archived line item(s) selected
MSITEM113 & line item(s) selected with archiving indicator
Diagnosis

System Response

Procedure

Procedure for System Administration

MSITEM114 No authorization for transfer prices
MSITEM115 No authorization for transfer prices, second local curre
MSITEM116 No authorization for transfer prices, third local curren
MSITEM123 Large number of custom selections could lead to problems during selection
Diagnosis

System Response

Procedure

Procedure for System Administration

MSITEM125 General ledger view is set without taking migration into account
Diagnosis

System Response

Procedure

Procedure for System Administration

Subroutines:		Line	Page
ALV_VARIANT_F4.....	FAGL_ACCOUNT_ITEMS_GL	944	17
	FAGL_ACCOUNT_ITEMS_INC	4	131
ANALYZE_ACT_FIELDCAT.....	FAGL_ACCOUNT_ITEMS_INC	68	132
	FAGL_ACCOUNT_ITEMS_INC	171	133
	FAGL_ACCOUNT_ITEMS_INC	412	137
ANALYZE_ACT_FIELDCAT_BOE.....	FAGL_ACCOUNT_ITEMS_INC	561	140
	FAGL_ACCOUNT_ITEMS_INC	3249	184
ANALYZE_ACT_FIELDCAT_LDGRP....	FAGL_ACCOUNT_ITEMS_INC	565	140
	FAGL_ACCOUNT_ITEMS_INC	4134	199
ANALYZE_CURT.....	FAGL_ACCOUNT_ITEMS_GL	1121	20
	FAGL_ACCOUNT_ITEMS_INC	4219	200
AUTHORITY_TCODE.....	FAGL_ACCOUNT_ITEMS_GL	1301	23
	FAGL_ACCOUNT_ITEMS_INC	2990	180
CB_ITEM_ALV_USERCOMM.....	FAGL_ACCOUNT_ITEMS_INC	35	131
CHANGE_STATUS.....	FAGL_ACCOUNT_ITEMS_GL	284	6
	FAGL_ACCOUNT_ITEMS_GL	430	9
	FAGL_ACCOUNT_ITEMS_GL	603	12
	FAGL_ACCOUNT_ITEMS_INC	2501	172
CHECK_AUTH_LEDGER.....	FAGL_ACCOUNT_ITEMS_GL	869	16
	FAGL_ACCOUNT_ITEMS_GL	5590	96
CHECK_BADI_ACTIVE.....	FAGL_ACCOUNT_ITEMS_GL	341	7
	FAGL_ACCOUNT_ITEMS_INC	4020	197
CHECK_DATE.....	FAGL_ACCOUNT_ITEMS_GL	827	15
	FAGL_ACCOUNT_ITEMS_INC	2923	179
CHECK_GLYEC_ACCOUNT.....	FAGL_ACCOUNT_ITEMS_GL	1210	22
	FAGL_ACCOUNT_ITEMS_GL	2869	49
	FAGL_ACCOUNT_ITEMS_GL	6709	114
CHECK_ITEM_OK.....	FAGL_ACCOUNT_ITEMS_GL	1645	29
	FAGL_ACCOUNT_ITEMS_INC	1488	155
CHECK_MSG_EXIST.....	FAGL_ACCOUNT_ITEMS_GL	1281	23
	FAGL_ACCOUNT_ITEMS_INC	3919	195
CHECK_REPORTING_FIELDS.....	FAGL_ACCOUNT_ITEMS_GL	782	14
	FAGL_ACCOUNT_ITEMS_GL	3522	60
	FAGL_ACCOUNT_ITEMS_GL	4466	75
CHECK_SELECTIONS.....	FAGL_ACCOUNT_ITEMS_GL	1068	19
	FAGL_ACCOUNT_ITEMS_GL	6844	116
CHECK_SLT_ACCOUNT.....	FAGL_ACCOUNT_ITEMS_GL	1203	22
	FAGL_ACCOUNT_ITEMS_GL	2836	49
	FAGL_ACCOUNT_ITEMS_INC	3850	194
CHECK_TP_AUTH.....	FAGL_ACCOUNT_ITEMS_INC	2255	168
	FAGL_ACCOUNT_ITEMS_INC	2728	175
CHECK_TP_T021S.....	FAGL_ACCOUNT_ITEMS_INC	2659	174
	FAGL_ACCOUNT_ITEMS_INC	2662	174
	FAGL_ACCOUNT_ITEMS_INC	2663	174
	FAGL_ACCOUNT_ITEMS_INC	2664	174
	FAGL_ACCOUNT_ITEMS_INC	2670	174
	FAGL_ACCOUNT_ITEMS_INC	2673	175
	FAGL_ACCOUNT_ITEMS_INC	2676	175
	FAGL_ACCOUNT_ITEMS_INC	2679	175
	FAGL_ACCOUNT_ITEMS_INC	2682	175
	FAGL_ACCOUNT_ITEMS_INC	2685	175
	FAGL_ACCOUNT_ITEMS_INC	2688	175
	FAGL_ACCOUNT_ITEMS_INC	2698	175
CHECK_TP_T021S_2.....	FAGL_ACCOUNT_ITEMS_INC	2660	174
	FAGL_ACCOUNT_ITEMS_INC	2666	174
	FAGL_ACCOUNT_ITEMS_INC	2667	174
	FAGL_ACCOUNT_ITEMS_INC	2668	174
	FAGL_ACCOUNT_ITEMS_INC	2671	174
	FAGL_ACCOUNT_ITEMS_INC	2674	175

	FAGL_ACCOUNT_ITEMS_INC	2677	175
	FAGL_ACCOUNT_ITEMS_INC	2680	175
	FAGL_ACCOUNT_ITEMS_INC	2683	175
	FAGL_ACCOUNT_ITEMS_INC	2686	175
	FAGL_ACCOUNT_ITEMS_INC	2689	175
	FAGL_ACCOUNT_ITEMS_INC	2713	175
CHECK_TRTM_ACCOUNT.....	FAGL_ACCOUNT_ITEMS_GL	1209	22
	FAGL_ACCOUNT_ITEMS_GL	2868	49
	FAGL_ACCOUNT_ITEMS_GL	6576	112
CHOOSE_LEDGER.....	FAGL_ACCOUNT_ITEMS_GL	757	14
	FAGL_ACCOUNT_ITEMS_GL	3216	55
COMPLETE_BKPF.....	FAGL_ACCOUNT_ITEMS_INC	595	140
	FAGL_ACCOUNT_ITEMS_INC	627	141
COMPLETE_BSBV.....	FAGL_ACCOUNT_ITEMS_INC	612	141
	FAGL_ACCOUNT_ITEMS_INC	1426	154
COMPLETE_BSEC.....	FAGL_ACCOUNT_ITEMS_INC	608	141
	FAGL_ACCOUNT_ITEMS_INC	1278	152
COMPLETE_BSED.....	FAGL_ACCOUNT_ITEMS_INC	610	141
	FAGL_ACCOUNT_ITEMS_INC	1352	153
COMPLETE_BSEG.....	FAGL_ACCOUNT_ITEMS_INC	600	140
	FAGL_ACCOUNT_ITEMS_INC	792	144
COMPLETE_BSEGC.....	FAGL_ACCOUNT_ITEMS_INC	605	140
	FAGL_ACCOUNT_ITEMS_INC	1252	151
COMPLETE_DOCUMENT_TEXT.....	FAGL_ACCOUNT_ITEMS_INC	616	141
	FAGL_ACCOUNT_ITEMS_INC	2585	173
COMPLETE_PAYR.....	FAGL_ACCOUNT_ITEMS_INC	614	141
	FAGL_ACCOUNT_ITEMS_INC	1379	153
CREATE_LOG.....	FAGL_ACCOUNT_ITEMS_GL	1128	20
	FAGL_ACCOUNT_ITEMS_INC	3821	193
CREATE_PERFORMANCE_TABLES.....	FAGL_ACCOUNT_ITEMS_INC	590	140
	FAGL_ACCOUNT_ITEMS_INC	3323	185
CURR_CONVERT_ITEM.....	FAGL_ACCOUNT_ITEMS_GL	1720	30
	FAGL_ACCOUNT_ITEMS_INC	3589	190
DD_GET_FLAG.....	FAGL_ACCOUNT_ITEMS_INC	2560	173
DD_MODIFY_SCREEN.....	FAGL_ACCOUNT_ITEMS_INC	2570	173
DECO_AT_END_OF_SELECTION.....	FAGL_ACCOUNT_ITEMS_DECO	185	121
	FAGL_ACCOUNT_ITEMS_GL	1003	18
	FAGL_ACCOUNT_ITEMS_GL	1224	22
DECO_AT_INITIALIZATION.....	FAGL_ACCOUNT_ITEMS_DECO	25	118
	FAGL_ACCOUNT_ITEMS_GL	233	5
	FAGL_ACCOUNT_ITEMS_GL	245	6
DECO_AT_OUTPUT_ALV.....	FAGL_ACCOUNT_ITEMS_GL	1004	18
	FAGL_ACCOUNT_ITEMS_GL	1315	23
DECO_AT_SELECTION_GET.....	FAGL_ACCOUNT_ITEMS_DECO	212	121
	FAGL_ACCOUNT_ITEMS_GL	1000	18
	FAGL_ACCOUNT_ITEMS_GL	1137	20
DECO_AT_SELECTION_START.....	FAGL_ACCOUNT_ITEMS_DECO	134	120
	FAGL_ACCOUNT_ITEMS_GL	981	18
	FAGL_ACCOUNT_ITEMS_GL	1012	18
DECO_AT_SEL_SCREEN.....	FAGL_ACCOUNT_ITEMS_DECO	130	120
	FAGL_ACCOUNT_ITEMS_GL	552	11
	FAGL_ACCOUNT_ITEMS_GL	559	11
DECO_AT_SEL_SCREEN_COMPANY....	FAGL_ACCOUNT_ITEMS_DECO	129	120
	FAGL_ACCOUNT_ITEMS_GL	518	10
	FAGL_ACCOUNT_ITEMS_GL	525	10
DECO_AT_SEL_SCREEN_GLACCOUNT..	FAGL_ACCOUNT_ITEMS_DECO	128	120
	FAGL_ACCOUNT_ITEMS_GL	484	10
	FAGL_ACCOUNT_ITEMS_GL	491	10
DECO_AT_SEL_SCREEN_OUTPUT.....	FAGL_ACCOUNT_ITEMS_DECO	26	118
	FAGL_ACCOUNT_ITEMS_GL	358	7
	FAGL_ACCOUNT_ITEMS_GL	365	8

DECO_AT_SEL_SCREEN_TYPE.....	FAGL_ACCOUNT_ITEMS_DECO	131	120
	FAGL_ACCOUNT_ITEMS_GL	927	17
	FAGL_ACCOUNT_ITEMS_GL	934	17
DECO_CALL_FORM.....	FIN_UI_DECO_INCLUDE	540	218
DECO_CALL_SCREEN.....	FIN_UI_DECO_INCLUDE	466	217
DECO_CALL_SUBSCREEN.....	FIN_UI_DECO_INCLUDE	569	218
DECO_CALL_TRANSACTION.....	FIN_UI_DECO_INCLUDE	438	216
DECO_CHECK_SIM_MODE.....	FIN_UI_DECO_INCLUDE	815	222
DECO_CLEAR_GLOBAL_VARIABLES...	FAGL_ACCOUNT_ITEMS_DECO	220	121
DECO_ENDLOOP_SCREEN.....	FIN_UI_DECO_INCLUDE	789	222
DECO_FUNCTION_NOT_POSSIBLE....	FIN_UI_DECO_INCLUDE	556	218
DECO_GET_RESULT.....	FAGL_ACCOUNT_ITEMS_DECO	196	121
DECO_INIT.....	FAGL_ACCOUNT_ITEMS_DECO	12	118
DECO_INITIALIZE.....	DECO_MESSAGE_0	0	
	DECO_MESSAGE_0	0	
	DECO_MESSAGE_0	0	

	DECO_MESSAGE_1	0	
	DECO_MESSAGE_1	0	
	DECO_MESSAGE_1	0	
	DECO_MESSAGE_1	0	
	DECO_MESSAGE_1	0	
	DECO_MESSAGE_1	0	
	DECO_MESSAGE_1	0	
	DECO_MESSAGE_1	0	
	DECO_MESSAGE_1	0	
	DECO_MESSAGE_1	0	
	DECO_MESSAGE_1	0	
	DECO_MESSAGE_1	0	
	DECO_MESSAGE_1	0	
	DECO_MESSAGE_1	0	
	DECO_MESSAGE_2	0	
	DECO_MESSAGE_2	0	
	DECO_MESSAGE_2	0	
	DECO_MESSAGE_2	0	
	DECO_MESSAGE_2	0	
	DECO_MESSAGE_2	0	
	FIN_UI_DECO_INCLUDE	250	213
	FIN_UI_DECO_INCLUDE	262	213
	FIN_UI_DECO_INCLUDE	806	222
DECO_LDB_CALLBACK.....	FAGL_ACCOUNT_ITEMS_DECO	206	121
DECO_LEAVE.....	FIN_UI_DECO_INCLUDE	305	214
DECO_LEAVE_PROGRAM.....	FAGL_ACCOUNT_ITEMS_GL	1296	23
	FAGL_ACCOUNT_ITEMS_INC	106	132
	FAGL_ACCOUNT_ITEMS_INC	150	133
	FIN_UI_DECO_INCLUDE	289	213
DECO_LEAVE_SCREEN.....	FIN_UI_DECO_INCLUDE	323	214
DECO_LEAVE_TO_CURRENT_TRANS...	FIN_UI_DECO_INCLUDE	407	215
DECO_LEAVE_TO_SCREEN.....	FIN_UI_DECO_INCLUDE	341	214
DECO_LEAVE_TO_TRANSACTION.....	FIN_UI_DECO_INCLUDE	379	215
DECO_LOOP_AT_SCREEN.....	FIN_UI_DECO_INCLUDE	672	220
DECO_MESSAGE_SY.....	DECO_MESSAGE_SY	0	
	DECO_MESSAGE_SY	0	
	DECO_MESSAGE_SY	0	
	DECO_MESSAGE_SY	0	
	DECO_MESSAGE_SY	0	
	DECO_MESSAGE_SY	0	
	DECO_MESSAGE_SY	0	
	DECO_MESSAGE_SY	0	
	DECO_MESSAGE_SY	0	
	DECO_MESSAGE_SY	0	
	DECO_MESSAGE_SY	0	
	FIN_UI_DECO_INCLUDE	226	212
DECO_MODIFY_SCREEN.....	FIN_UI_DECO_INCLUDE	717	221
DECO_SELECTION.....	FAGL_ACCOUNT_ITEMS_DECO	120	119
DECO_SET_SCREEN.....	FIN_UI_DECO_INCLUDE	360	215
DECO_SUPPLY_SELECTION.....	FAGL_ACCOUNT_ITEMS_DECO	33	118
DEL_FREE_SELECTIONS_SI_TAB....	FAGL_ACCOUNT_ITEMS_GL	777	14
	FAGL_ACCOUNT_ITEMS_GL	792	15
	FAGL_ACCOUNT_ITEMS_GL	3622	62
DISPLAY_GRID_OR_CLASSIC.....	FAGL_ACCOUNT_ITEMS_GL	1237	22
	FAGL_ACCOUNT_ITEMS_INC	1984	163
DYNP_GET_STATUS.....	FAGL_ACCOUNT_ITEMS_GL	372	8
	FAGL_ACCOUNT_ITEMS_INC	2350	169
ENFORCE_OUTER_JOIN.....	FAGL_ACCOUNT_ITEMS_GL	1015	18
	FAGL_ACCOUNT_ITEMS_GL	6865	117
ERFS_MODIFY_SCREEN.....	FAGL_ACCOUNT_ITEMS_GL	415	8
	FAGL_ACCOUNT_ITEMS_INC	3739	192

ERFS_ON_OFF.....	FAGL_ACCOUNT_ITEMS_GL	788	15
	FAGL_ACCOUNT_ITEMS_INC	3792	193
EXCLUDE_PL_ACCOUNTS.....	FAGL_ACCOUNT_ITEMS_GL	1125	20
	FAGL_ACCOUNT_ITEMS_GL	2684	46
EXPORT_FILITEXTS_DATA.....	FAGL_ACCOUNT_ITEMS_GL	1271	23
	FAGL_ACCOUNT_ITEMS_INC	2367	169
EXTND_FIELDNAME.....	FAGL_ACCOUNT_ITEMS_INC	321	136
	FAGL_ACCOUNT_ITEMS_INC	327	136
	FAGL_ACCOUNT_ITEMS_INC	355	136
	FAGL_ACCOUNT_ITEMS_INC	398	137
FILL_ACCT_TABLE.....	FAGL_ACCOUNT_ITEMS_GL	1875	33
	FAGL_ACCOUNT_ITEMS_GL	2878	49
	FAGL_ACCOUNT_ITEMS_INC	1558	156
FILL_COMP_TABLE.....	FAGL_ACCOUNT_ITEMS_GL	1751	31
	FAGL_ACCOUNT_ITEMS_INC	1579	157
FILL_TP_DMBEX.....	FAGL_ACCOUNT_ITEMS_FORMS	12	130
FILL_WAERS_FIELDLIST.....	RFDBEXPC	35	225
	RFDBEXPC	138	227
	RFDBEXPC	241	228
	RFDBEXPC	396	231
	RFDBEXPC	496	233
	RFDBEXPC	555	234
	RFDBEXPC	655	235
	RFDBEXPC	688	236
FIND_RELEVANT_LEDGER.....	FAGL_ACCOUNT_ITEMS_GL	293	6
	FAGL_ACCOUNT_ITEMS_GL	3306	56
FIND_SI_TABLE.....	FAGL_ACCOUNT_ITEMS_GL	298	6
	FAGL_ACCOUNT_ITEMS_GL	424	9
	FAGL_ACCOUNT_ITEMS_GL	772	14
	FAGL_ACCOUNT_ITEMS_GL	3599	61
	FAGL_ACCOUNT_ITEMS_GL	6306	108
FREE_SELECTIONS.....	FAGL_ACCOUNT_ITEMS_GL	739	14
	FAGL_ACCOUNT_ITEMS_GL	3365	57
FS_SET_SSCRTEXTS_DYNSEL.....	FAGL_ACCOUNT_ITEMS_GL	394	8
	FAGL_ACCOUNT_ITEMS_GL	3153	54
	FAGL_ACCOUNT_ITEMS_INC	2395	170
FS_SET_SSCRTEXTS_DYNSEL3.....	FAGL_ACCOUNT_ITEMS_GL	3175	54
	FAGL_ACCOUNT_ITEMS_INC	2409	170
FS_SET_SSCRTEXTS_DYNSEL4.....	FAGL_ACCOUNT_ITEMS_GL	411	8
	FAGL_ACCOUNT_ITEMS_GL	3194	55
	FAGL_ACCOUNT_ITEMS_INC	2418	170
GET_EXISTING_VARIANT.....	FAGL_ACCOUNT_ITEMS_INC	166	133
	FAGL_ACCOUNT_ITEMS_INC	2856	178
GET_FLEXGL_FIELDS.....	FAGL_ACCOUNT_ITEMS_INC	2190	167
	FAGL_ACCOUNT_ITEMS_INC	3418	187
GET_FREE_SELECTIONS.....	FAGL_ACCOUNT_ITEMS_GL	6565	112
GET_GENERAL_PARAM.....	FAGL_ACCOUNT_ITEMS_GL	328	7
	FAGL_ACCOUNT_ITEMS_INC	1697	158
GET_INITIAL_FIELDCAT.....	FAGL_ACCOUNT_ITEMS_INC	168	133
	FAGL_ACCOUNT_ITEMS_INC	180	133
GET_LEADING_LEDGER.....	FAGL_ACCOUNT_ITEMS_GL	301	6
	FAGL_ACCOUNT_ITEMS_GL	4444	75
GLYEC_MODIFY_SCREEN.....	FAGL_ACCOUNT_ITEMS_GL	416	8
	FAGL_ACCOUNT_ITEMS_INC	3761	192
HOLEN_TEXT1_ZU_WAEHRUNGSTYP...	FAGL_ACCOUNT_ITEMS_INC	4542	206
	FAGL_ACCOUNT_ITEMS_INC	4560	206
IMPORT_ARCH_FROM_ITAB.....	FAGL_ACCOUNT_ITEMS_GL	4751	80
	FAGL_ACCOUNT_ITEMS_GL	5020	86
	FAGL_ACCOUNT_ITEMS_GL	5128	87
	FAGL_ACCOUNT_ITEMS_GL	5303	91
	FAGL_ACCOUNT_ITEMS_GL	5469	93

	FAGL_ACCOUNT_ITEMS_GL	5567	95
	FAGL_ACCOUNT_ITEMS_INC	4070	198
INCLUDE_PL_ACCOUNTS.....	FAGL_ACCOUNT_ITEMS_GL	1233	22
	FAGL_ACCOUNT_ITEMS_GL	2779	48
INIT_ADMIN_TABLES.....	FAGL_ACCOUNT_ITEMS_GL	322	7
	FAGL_ACCOUNT_ITEMS_INC	234	134
INIT_ADMIN_TABLES_BOE.....	FAGL_ACCOUNT_ITEMS_INC	274	135
	FAGL_ACCOUNT_ITEMS_INC	3223	184
INIT_EXPCUR.....	FAGL_ACCOUNT_ITEMS_GL	281	6
	FAGL_ACCOUNT_ITEMS_INC	2627	174
INIT_RRI_DATA.....	FAGL_ACCOUNT_ITEMS_GL	317	7
	FAGL_ACCOUNT_ITEMS_GL	5617	96
INSERT_TP_FIELDS.....	FAGL_ACCOUNT_ITEMS_GL	339	7
	FAGL_ACCOUNT_ITEMS_INC	2648	174
ITEMS_CHECK_AUTH_C.....	FAGL_ACCOUNT_ITEMS_GL	1245	22
	FAGL_ACCOUNT_ITEMS_INC	4051	197
ITEM_CHECK_APPEND.....	FAGL_ACCOUNT_ITEMS_GL	1725	30
	FAGL_ACCOUNT_ITEMS_INC	1605	157
ITEM_CHECK_AUTH.....	FAGL_ACCOUNT_ITEMS_GL	1713	30
	FAGL_ACCOUNT_ITEMS_INC	3456	187
ITEM_CURR_CONVERT_BKPF.....	FAGL_ACCOUNT_ITEMS_INC	663	141
	FAGL_ACCOUNT_ITEMS_INC	707	142
	FAGL_ACCOUNT_ITEMS_INC	754	143
	RFDBEXPC	344	230
ITEM_CURR_CONVERT_BSEG.....	FAGL_ACCOUNT_ITEMS_INC	906	145
	FAGL_ACCOUNT_ITEMS_INC	984	147
	FAGL_ACCOUNT_ITEMS_INC	1117	149
	FAGL_ACCOUNT_ITEMS_INC	1165	150
	FAGL_ACCOUNT_ITEMS_INC	1229	151
	RFDBEXPC	372	231
ITEM_CURR_CONVERT_BSEGA.....	RFDBEXPC	280	229
ITEM_CURR_CONVERT_BSET.....	RFDBEXPC	631	235
ITEM_CURR_CONVERT_BSID.....	RFDBEXPC	12	225
ITEM_CURR_CONVERT_BSIK.....	RFDBEXPC	115	226
ITEM_CURR_CONVERT_BSIS.....	RFDBEXPC	218	228
ITEM_CURR_CONVERT_GSEG.....	RFDBEXPC	531	233
ITEM_CURR_CONVERT_WITH_ITEM...	RFDBEXPC	472	232
IT_POS_SORT_BACK.....	FAGL_ACCOUNT_ITEMS_INC	784	143
	FAGL_ACCOUNT_ITEMS_INC	1244	151
	FAGL_ACCOUNT_ITEMS_INC	3024	180
MAKE_FIELDCATALOG.....	FAGL_ACCOUNT_ITEMS_GL	1071	19
	FAGL_ACCOUNT_ITEMS_INC	2027	164
MAKE_FIELDCATALOG2.....	FAGL_ACCOUNT_ITEMS_GL	1111	20
	FAGL_ACCOUNT_ITEMS_INC	2243	167
MAP_SEL_TO_ITEMS.....	FAGL_ACCOUNT_ITEMS_GL	1065	19
	FAGL_ACCOUNT_ITEMS_GL	1956	34
	FAGL_ACCOUNT_ITEMS_GL	2797	48
MAP_SEL_TO_ITEMS_MAST_DAT.....	FAGL_ACCOUNT_ITEMS_GL	1183	21
	FAGL_ACCOUNT_ITEMS_GL	2860	49
	FAGL_ACCOUNT_ITEMS_GL	4592	77
MOVE_SPECIAL_FIELDS.....	FAGL_ACCOUNT_ITEMS_INC	673	142
	FAGL_ACCOUNT_ITEMS_INC	680	142
	FAGL_ACCOUNT_ITEMS_INC	717	142
	FAGL_ACCOUNT_ITEMS_INC	724	142
	FAGL_ACCOUNT_ITEMS_INC	765	143
	FAGL_ACCOUNT_ITEMS_INC	773	143
	FAGL_ACCOUNT_ITEMS_INC	912	146
	FAGL_ACCOUNT_ITEMS_INC	990	147
	FAGL_ACCOUNT_ITEMS_INC	1022	147
	FAGL_ACCOUNT_ITEMS_INC	1063	148
	FAGL_ACCOUNT_ITEMS_INC	1123	149

	FAGL_ACCOUNT_ITEMS_INC	1171	150
	FAGL_ACCOUNT_ITEMS_INC	1235	151
	FAGL_ACCOUNT_ITEMS_INC	1268	151
	FAGL_ACCOUNT_ITEMS_INC	1318	152
	FAGL_ACCOUNT_ITEMS_INC	1341	153
	FAGL_ACCOUNT_ITEMS_INC	1367	153
	FAGL_ACCOUNT_ITEMS_INC	1417	154
	FAGL_ACCOUNT_ITEMS_INC	1447	154
	FAGL_ACCOUNT_ITEMS_INC	1456	154
MOVE_SPECIAL_FIELDS_BOE.....	FAGL_ACCOUNT_ITEMS_INC	1369	153
	FAGL_ACCOUNT_ITEMS_INC	3290	185
MSG_ADD_TO_LOG.....	FAGL_ACCOUNT_ITEMS_GL	1252	22
	FAGL_ACCOUNT_ITEMS_GL	1255	22
	FAGL_ACCOUNT_ITEMS_GL	1289	23
	FAGL_ACCOUNT_ITEMS_INC	3872	194
	FAGL_ACCOUNT_ITEMS_INC	3883	194
POS_TABLE_FILL.....	FAGL_ACCOUNT_ITEMS_GL	1635	29
	FAGL_ACCOUNT_ITEMS_GL	4909	84
	FAGL_ACCOUNT_ITEMS_GL	4968	85
	FAGL_ACCOUNT_ITEMS_GL	5015	85
	FAGL_ACCOUNT_ITEMS_GL	5079	87
	FAGL_ACCOUNT_ITEMS_GL	5123	87
	FAGL_ACCOUNT_ITEMS_GL	5183	88
	FAGL_ACCOUNT_ITEMS_GL	5244	90
	FAGL_ACCOUNT_ITEMS_GL	5298	90
	FAGL_ACCOUNT_ITEMS_GL	5343	91
	FAGL_ACCOUNT_ITEMS_GL	5411	92
	FAGL_ACCOUNT_ITEMS_GL	5464	93
	FAGL_ACCOUNT_ITEMS_GL	5517	94
	FAGL_ACCOUNT_ITEMS_GL	5562	95
PRINT_APPL_LOG.....	FAGL_ACCOUNT_ITEMS_GL	1291	23
	FAGL_ACCOUNT_ITEMS_INC	3952	196
READ_SPECIAL_TABLES.....	FAGL_ACCOUNT_ITEMS_INC	70	132
	FAGL_ACCOUNT_ITEMS_INC	173	133
	FAGL_ACCOUNT_ITEMS_INC	580	140
READ_TEXT_FOR_CURTYPE.....	FAGL_ACCOUNT_ITEMS_INC	4355	202
	FAGL_ACCOUNT_ITEMS_INC	4372	203
	FAGL_ACCOUNT_ITEMS_INC	4389	203
	FAGL_ACCOUNT_ITEMS_INC	4406	203
	FAGL_ACCOUNT_ITEMS_INC	4423	204
	FAGL_ACCOUNT_ITEMS_INC	4440	204
	FAGL_ACCOUNT_ITEMS_INC	4457	204
	FAGL_ACCOUNT_ITEMS_INC	4474	205
	FAGL_ACCOUNT_ITEMS_INC	4491	205
	FAGL_ACCOUNT_ITEMS_INC	4515	205
RESOLVE_WORKLIST.....	FAGL_ACCOUNT_ITEMS_GL	498	10
	FAGL_ACCOUNT_ITEMS_GL	532	10
	FAGL_ACCOUNT_ITEMS_INC	2469	171
SAVE_ALL_SELECTIONS.....	FAGL_ACCOUNT_ITEMS_GL	1235	22
	FAGL_ACCOUNT_ITEMS_INC	1901	162
SDF_SELSCREEN.....	FAGL_ACCOUNT_ITEMS_GL	2903	50
SELECT_ITEMS.....	FAGL_ACCOUNT_ITEMS_GL	1211	22
	FAGL_ACCOUNT_ITEMS_GL	2870	49
	FAGL_ACCOUNT_ITEMS_GL	4711	79
SELECT_ITEMS_OLD.....	FAGL_ACCOUNT_ITEMS_GL	4780	81
	FAGL_ACCOUNT_ITEMS_GL	4820	82
SEL_ACCOUNT_CHECK.....	FAGL_ACCOUNT_ITEMS_GL	825	15
	FAGL_ACCOUNT_ITEMS_GL	1522	27
SEL_TYPE_CHECK.....	FAGL_ACCOUNT_ITEMS_GL	938	17
	FAGL_ACCOUNT_ITEMS_GL	1626	29
SETLMNT_ACCOUNTS.....	FAGL_ACCOUNT_ITEMS_GL	1117	20

	FAGL_ACCOUNT_ITEMS_GL	4395	74
SET_ACCT_AND_CCODE.....	FAGL_ACCOUNT_ITEMS_GL	288	6
	FAGL_ACCOUNT_ITEMS_GL	744	14
	FAGL_ACCOUNT_ITEMS_GL	2615	45
SET_DARK_SELECT_OPTIONS.....	FAGL_ACCOUNT_ITEMS_GL	4741	80
	FAGL_ACCOUNT_ITEMS_GL	4789	81
SET_DYN_SEL_LDB.....	FAGL_ACCOUNT_ITEMS_GL	1062	19
	FAGL_ACCOUNT_ITEMS_GL	3535	60
SET_GENERAL_PARAM.....	FAGL_ACCOUNT_ITEMS_GL	2565	44
	FAGL_ACCOUNT_ITEMS_INC	1767	160
SET_PARAMETERS.....	FAGL_ACCOUNT_ITEMS_GL	1055	19
	FAGL_ACCOUNT_ITEMS_GL	2563	44
SET_PA_CENT.....	FAGL_ACCOUNT_ITEMS_INC	1904	162
	FAGL_ACCOUNT_ITEMS_INC	1946	163
SET_PRINT_LIST_OPTIONS.....	FAGL_ACCOUNT_ITEMS_INC	3982	196
	FAGL_ACCOUNT_ITEMS_INC	4006	196
SKAT_INFO_FILL.....	FAGL_ACCOUNT_ITEMS_GL	1175	21
	FAGL_ACCOUNT_ITEMS_GL	1777	31
	FAGL_ACCOUNT_ITEMS_GL	2875	49
SKB1_INFO_FILL.....	FAGL_ACCOUNT_ITEMS_GL	1177	21
	FAGL_ACCOUNT_ITEMS_GL	1839	32
SPECIAL_FIELDS_INIT.....	FAGL_ACCOUNT_ITEMS_GL	1240	22
	FAGL_ACCOUNT_ITEMS_INC	160	133
SPECIAL_FIELDS_MODIFY.....	FAGL_ACCOUNT_ITEMS_INC	739	143
	FAGL_ACCOUNT_ITEMS_INC	1040	148
	FAGL_ACCOUNT_ITEMS_INC	1081	148
	FAGL_ACCOUNT_ITEMS_INC	1129	149
	FAGL_ACCOUNT_ITEMS_INC	3190	183
SPECIAL_FIELDS_USED.....	FAGL_ACCOUNT_ITEMS_INC	593	140
	FAGL_ACCOUNT_ITEMS_INC	598	140
	FAGL_ACCOUNT_ITEMS_INC	603	140
	FAGL_ACCOUNT_ITEMS_INC	3138	182
START_AGAIN.....	FAGL_ACCOUNT_ITEMS_INC	2012	164
T001_INFO_FILL.....	FAGL_ACCOUNT_ITEMS_GL	1173	21
	FAGL_ACCOUNT_ITEMS_GL	1739	31
	FAGL_ACCOUNT_ITEMS_GL	2802	48
UPD_SPECIAL_STATUS.....	FAGL_ACCOUNT_ITEMS_INC	620	141
	FAGL_ACCOUNT_ITEMS_INC	1470	155
WL_FLAG_AND_BUTTON.....	FAGL_ACCOUNT_ITEMS_GL	286	6
	FAGL_ACCOUNT_ITEMS_INC	2392	170
	FAGL_ACCOUNT_ITEMS_INC	2439	171
	FAGL_ACCOUNT_ITEMS_INC	3810	193
WL_MEMORY_SET_GET.....	FAGL_ACCOUNT_ITEMS_INC	2377	170
	FAGL_ACCOUNT_ITEMS_INC	2399	170
	FAGL_ACCOUNT_ITEMS_INC	2432	171
	FAGL_ACCOUNT_ITEMS_INC	2436	171
WL_MODIFY_SCREEN.....	FAGL_ACCOUNT_ITEMS_GL	375	8
	FAGL_ACCOUNT_ITEMS_INC	2447	171
WORKLIST_ON_OFF.....	FAGL_ACCOUNT_ITEMS_GL	743	14
	FAGL_ACCOUNT_ITEMS_INC	2427	170

Screens:		Line	Page
?.....	FIN_UI_DECO_INCLUDE	481	217
?.....	FIN_UI_DECO_INCLUDE	487	217
?.....	FIN_UI_DECO_INCLUDE	491	217
IV_DYNNR.....	FIN_UI_DECO_INCLUDE	364	215

Messages:

		Line	Page
?.....	FAGL_ACCOUNT_ITEMS_GL	913	17
?.....	FAGL_ACCOUNT_ITEMS_INC	4286	201
?.....	FIN_UI_DECO_INCLUDE	253	213
?.....	FIN_UI_DECO_INCLUDE	266	213
?.....	FIN_UI_DECO_INCLUDE	525	218
?.....	FIN_UI_DECO_INCLUDE	625	219
F4430.....	DECO_MESSAGE_2	0	
	DECO_MESSAGE_2	0	
	DECO_MESSAGE_2	0	
	DECO_MESSAGE_2	0	
FIN_UI_DECO001.....	DECO_MESSAGE_0	0	
	DECO_MESSAGE_0	0	
	DECO_MESSAGE_0	0	
	DECO_MESSAGE_0	0	
	DECO_MESSAGE_0	0	
	DECO_MESSAGE_0	0	
	DECO_MESSAGE_0	0	
	DECO_MESSAGE_0	0	
	DECO_MESSAGE_0	0	
	DECO_MESSAGE_0	0	
	DECO_MESSAGE_0	0	
	DECO_MESSAGE_0	0	
	DECO_MESSAGE_0	0	
	DECO_MESSAGE_0	0	
	DECO_MESSAGE_0	0	
	DECO_MESSAGE_0	0	
	DECO_MESSAGE_0	0	
	DECO_MESSAGE_0	0	
	DECO_MESSAGE_0	0	
	DECO_MESSAGE_0	0	
	DECO_MESSAGE_0	0	
	DECO_MESSAGE_0	0	
	DECO_MESSAGE_0	0	
	DECO_MESSAGE_0	0	
	DECO_MESSAGE_0	0	
	DECO_MESSAGE_0	0	
	DECO_MESSAGE_0	0	
	DECO_MESSAGE_0	0	
	DECO_MESSAGE_0	0	
	DECO_MESSAGE_0	0	
	DECO_MESSAGE_1	0	
	DECO_MESSAGE_1	0	
	DECO_MESSAGE_1	0	
	DECO_MESSAGE_1	0	
	DECO_MESSAGE_1	0	
	DECO_MESSAGE_1	0	
	DECO_MESSAGE_1	0	
	DECO_MESSAGE_2	0	
	DECO_MESSAGE_2	0	
	DECO_MESSAGE_2	0	
	FIN_UI_DECO_INCLUDE	251	213
	FIN_UI_DECO_INCLUDE	295	214
	FIN_UI_DECO_INCLUDE	312	214
	FIN_UI_DECO_INCLUDE	329	214
	FIN_UI_DECO_INCLUDE	348	214
FIN_UI_DECO002.....	FIN_UI_DECO_INCLUDE	392	215
	FIN_UI_DECO_INCLUDE	394	215
	FIN_UI_DECO_INCLUDE	422	216
	FIN_UI_DECO_INCLUDE	424	216
	FIN_UI_DECO_INCLUDE	451	216
	FIN_UI_DECO_INCLUDE	453	216
FIN_UI_DECO003.....	FIN_UI_DECO_INCLUDE	558	218
FR812.....	DECO_MESSAGE_2	0	

	DECO_MESSAGE_2	0	
FR866.....	DECO_MESSAGE_1	0	
	DECO_MESSAGE_1	0	
	FAGL_ACCOUNT_ITEMS_GL	274	6
GU167.....	FAGL_ACCOUNT_ITEMS_GL	887	16
MSITEM011.....	DECO_MESSAGE_0	0	
	DECO_MESSAGE_0	0	
MSITEM012.....	DECO_MESSAGE_0	0	
	DECO_MESSAGE_0	0	
	DECO_MESSAGE_0	0	
	DECO_MESSAGE_0	0	
	DECO_MESSAGE_0	0	
	DECO_MESSAGE_0	0	
	DECO_MESSAGE_0	0	
	DECO_MESSAGE_0	0	
MSITEM019.....	DECO_MESSAGE_0	0	
	DECO_MESSAGE_0	0	
MSITEM020.....	DECO_MESSAGE_0	0	
	DECO_MESSAGE_0	0	
MSITEM022.....	DECO_MESSAGE_0	0	
	DECO_MESSAGE_0	0	
MSITEM023.....	DECO_MESSAGE_1	0	
	DECO_MESSAGE_1	0	
MSITEM024.....	DECO_MESSAGE_1	0	
	DECO_MESSAGE_1	0	
MSITEM030.....	DECO_MESSAGE_0	0	
	DECO_MESSAGE_0	0	
	DECO_MESSAGE_0	0	
	DECO_MESSAGE_0	0	
MSITEM033.....	DECO_MESSAGE_0	0	
	DECO_MESSAGE_0	0	
	DECO_MESSAGE_0	0	
	DECO_MESSAGE_0	0	
	DECO_MESSAGE_0	0	
	DECO_MESSAGE_0	0	
	FAGL_ACCOUNT_ITEMS_GL	1288	23
MSITEM040.....	DECO_MESSAGE_1	0	
	DECO_MESSAGE_1	0	
MSITEM100.....	DECO_MESSAGE_1	0	
	DECO_MESSAGE_1	0	
MSITEM101.....	DECO_MESSAGE_0	0	
	DECO_MESSAGE_0	0	
MSITEM102.....	DECO_MESSAGE_0	0	
	DECO_MESSAGE_0	0	
MSITEM103.....	DECO_MESSAGE_0	0	
	DECO_MESSAGE_0	0	
MSITEM104.....	DECO_MESSAGE_0	0	
	DECO_MESSAGE_0	0	
	DECO_MESSAGE_0	0	
	DECO_MESSAGE_0	0	
MSITEM105.....	DECO_MESSAGE_0	0	
	DECO_MESSAGE_0	0	
MSITEM106.....	DECO_MESSAGE_0	0	
	DECO_MESSAGE_0	0	
MSITEM107.....	DECO_MESSAGE_1	0	
	DECO_MESSAGE_1	0	
MSITEM108.....	DECO_MESSAGE_1	0	
	DECO_MESSAGE_1	0	
MSITEM111.....	FAGL_ACCOUNT_ITEMS_INC	3871	194
MSITEM112.....	FAGL_ACCOUNT_ITEMS_GL	1251	22
MSITEM113.....	FAGL_ACCOUNT_ITEMS_GL	1254	22

MSITEM114.....	DECO_MESSAGE_0	0	
	DECO_MESSAGE_0	0	
MSITEM115.....	DECO_MESSAGE_0	0	
	DECO_MESSAGE_0	0	
MSITEM116.....	DECO_MESSAGE_0	0	
	DECO_MESSAGE_0	0	
MSITEM123.....	FAGL_ACCOUNT_ITEMS_GL	6859	117
MSITEM125.....	DECO_MESSAGE_0	0	
	DECO_MESSAGE_0	0	

External calls

Function Modules

		Line	Page
?.....	FIN_UI_DECO_INCLUDE	590	219
AUTHORITY_CHECK_TCODE.....	FAGL_ACCOUNT_ITEMS_INC	3005	180
BALW_BAPIRETURN_GET2.....	FIN_UI_DECO_INCLUDE	607	219
BAL_DSP_LOG_DISPLAY.....	FAGL_ACCOUNT_ITEMS_INC	3993	196
BAL_DSP_LOG_PRINT.....	FAGL_ACCOUNT_ITEMS_INC	3984	196
BAL_DSP_PROFILE_NO_TREE_GET...	FAGL_ACCOUNT_ITEMS_INC	3956	196
BAL_GLB_SEARCH_MSG.....	FAGL_ACCOUNT_ITEMS_INC	3926	195
BAL_LOG_CREATE.....	FAGL_ACCOUNT_ITEMS_INC	3830	194
BAL_LOG_MSG_ADD.....	FAGL_ACCOUNT_ITEMS_INC	3894	195
BUKRS_AUTHORITY_CHECK.....	FAGL_ACCOUNT_ITEMS_GL	814	15
	FAGL_ACCOUNT_ITEMS_GL	819	15
CONVERT_FOREIGN_TO_FOREIGN_CUR	FAGL_ACCOUNT_ITEMS_INC	3666	191
	FAGL_ACCOUNT_ITEMS_INC	3703	191
	RFDBEXPC	85	226
	RFDBEXPC	188	228
	RFDBEXPC	442	232
	RFDBEXPC	601	234
CURRENCY_CHECK_FOR_PROCESS....	FAGL_ACCOUNT_ITEMS_INC	2632	174
CURRENCY_DOCUMENT_CONVERT.....	FAGL_ACCOUNT_ITEMS_INC	3628	190
	RFDBEXPC	44	225
	RFDBEXPC	147	227
	RFDBEXPC	250	229
	RFDBEXPC	320	230
	RFDBEXPC	405	231
	RFDBEXPC	505	233
	RFDBEXPC	564	234
	RFDBEXPC	664	235
CURRENCY_GET_SUBSEQUENT.....	FAGL_ACCOUNT_ITEMS_INC	3619	190
	FAGL_ACCOUNT_ITEMS_INC	3657	191
	FAGL_ACCOUNT_ITEMS_INC	3694	191
	RFDBEXPC	22	225
	RFDBEXPC	69	226
	RFDBEXPC	125	227
	RFDBEXPC	172	227
	RFDBEXPC	228	228
	RFDBEXPC	292	229
	RFDBEXPC	351	230
	RFDBEXPC	383	231
	RFDBEXPC	426	231
	RFDBEXPC	483	232
	RFDBEXPC	542	233
	RFDBEXPC	585	234
	RFDBEXPC	642	235
CUSTOMIZED_MESSAGE.....	FAGL_ACCOUNT_ITEMS_GL	1262	22
DDIF_FIELDINFO_GET.....	FAGL_ACCOUNT_ITEMS_GL	382	8
	FAGL_ACCOUNT_ITEMS_INC	3597	190
DDIF_NAMETAB_GET.....	FAGL_ACCOUNT_ITEMS_GL	6315	108
	FAGL_ACCOUNT_ITEMS_GL	6358	108
	FAGL_ACCOUNT_ITEMS_INC	296	135
	FAGL_ACCOUNT_ITEMS_INC	2127	166
	FAGL_ACCOUNT_ITEMS_INC	2177	166
	FAGL_ACCOUNT_ITEMS_INC	3198	183
	RFDBEXPC	695	236
FAGL_AUTHORITY_LEDGER.....	FAGL_ACCOUNT_ITEMS_GL	5602	96
FAGL_AUTHORITY_SEGMENT.....	FAGL_ACCOUNT_ITEMS_INC	3502	188
FAGL_BUKRS_ACTIVE_IN_CLIENT...	FAGL_ACCOUNT_ITEMS_GL	264	6

FAGL_CHECK_GLFLEX_ACTIVE.....	FAGL_ACCOUNT_ITEMS_GL	257	6
	FAGL_ACCOUNT_ITEMS_GL	575	11
FAGL_CHECK_IF_LEDGER_IS_GLFLEX	FAGL_ACCOUNT_ITEMS_GL	848	16
FAGL_DETERMINE_TEMPERATURE....	FAGL_ACCOUNT_ITEMS_GL	985	18
	FAGL_ACCOUNT_ITEMS_GL	2842	49
FAGL_GET_ACTIVE_MGPLN.....	FAGL_ACCOUNT_ITEMS_GL	624	12
FAGL_GET_ARCH_ITEMS_BSEG.....	FAGL_ACCOUNT_ITEMS_GL	5098	87
FAGL_GET_ARCH_ITEMS_BSIS.....	FAGL_ACCOUNT_ITEMS_GL	4989	85
	FAGL_ACCOUNT_ITEMS_GL	5431	93
FAGL_GET_ARCH_ITEMS_FAGLBSAS..	FAGL_ACCOUNT_ITEMS_GL	5266	90
	FAGL_ACCOUNT_ITEMS_GL	5537	95
FAGL_GET_FAGL_SETTINGS.....	FAGL_ACCOUNT_ITEMS_GL	4676	79
FAGL_GET_ITEMS_BSAS.....	FAGL_ACCOUNT_ITEMS_GL	4934	84
	FAGL_ACCOUNT_ITEMS_GL	5375	92
FAGL_GET_ITEMS_BSEG.....	FAGL_ACCOUNT_ITEMS_GL	5048	86
FAGL_GET_ITEMS_BSIS.....	FAGL_ACCOUNT_ITEMS_GL	4872	83
FAGL_GET_ITEMS_FAGLBSAS.....	FAGL_ACCOUNT_ITEMS_GL	5208	89
	FAGL_ACCOUNT_ITEMS_GL	5484	94
FAGL_GET_ITEMS_FAGLBSIS.....	FAGL_ACCOUNT_ITEMS_GL	5149	88
FAGL_GET_ITEMS_GLOBALS_GET....	FAGL_ACCOUNT_ITEMS_GL	6880	117
FAGL_GET_ITEMS_GLOBALS_SET....	FAGL_ACCOUNT_ITEMS_GL	6876	117
FAGL_GET_ITEMS_VBSEGS.....	FAGL_ACCOUNT_ITEMS_GL	5319	91
FAGL_GET_LEADING_LEDGER.....	FAGL_ACCOUNT_ITEMS_GL	3343	57
	FAGL_ACCOUNT_ITEMS_GL	4446	75
FAGL_GET_REPORTING_FIELDS.....	FAGL_ACCOUNT_ITEMS_GL	4501	76
FAGL_GET_TABLENAMES.....	FAGL_ACCOUNT_ITEMS_GL	3604	61
FAGL_GET_TABLE_FIELDS.....	FAGL_ACCOUNT_ITEMS_INC	3428	187
FAGL_ITEMS_CONVERT_FIELDSNAMES	FAGL_ACCOUNT_ITEMS_INC	3433	187
FAGL_ITEMS_DISPLAY.....	FAGL_ACCOUNT_ITEMS_GL	1490	27
FAGL_ITEM_DERIVE_FIELDS.....	FAGL_ACCOUNT_ITEMS_GL	1694	30
FAGL_MGPLN_IN_CLIENT.....	FAGL_ACCOUNT_ITEMS_GL	606	12
FIN_UI_DECO_MESSAGE_ADD.....	DECO_MESSAGE_0	0	
	DECO_MESSAGE_0	0	
	DECO_MESSAGE_0	0	

	DECO_MESSAGE_1	0	
	DECO_MESSAGE_2	0	
	DECO_MESSAGE_2	0	
	DECO_MESSAGE_2	0	
	FIN_UI_DECO_INCLUDE	245	213
FIN_UI_DECO_MESSAGE_SET_ID....	FAGL_ACCOUNT_ITEMS_DECO	21	118
FI_ACCT_DET_CLEARING.....	FAGL_ACCOUNT_ITEMS_GL	4409	74
FI_ADD_AUTHORITY_CHECK.....	FAGL_ACCOUNT_ITEMS_GL	305	7
FI_BLART_AUTH_CHECK.....	FAGL_ACCOUNT_ITEMS_INC	3463	187
FI_CHECK_DATE.....	FAGL_ACCOUNT_ITEMS_INC	2959	179
FI_CURRENCY_INFORMATION.....	FAGL_ACCOUNT_ITEMS_GL	1758	31
FI_GSBER_AUTH_CHECK.....	FAGL_ACCOUNT_ITEMS_INC	3479	188
FI_PERIOD_DETERMINE.....	FAGL_ACCOUNT_ITEMS_GL	702	13
	FAGL_ACCOUNT_ITEMS_INC	1584	157
FI_TEXTS_DOCUMENT.....	FAGL_ACCOUNT_ITEMS_INC	2590	173
FREE_SELECTIONS_DIALOG.....	FAGL_ACCOUNT_ITEMS_GL	3483	59
FREE_SELECTIONS_INIT.....	FAGL_ACCOUNT_ITEMS_GL	3428	58
FREE_SELECTIONS_RANGE_2_EX....	FAGL_ACCOUNT_ITEMS_DECO	97	119
	FAGL_ACCOUNT_ITEMS_GL	471	9
	FAGL_ACCOUNT_ITEMS_GL	4554	77
	FAGL_ACCOUNT_ITEMS_GL	6535	111
	FAGL_ACCOUNT_ITEMS_INC	1935	162
FREE_SELECTIONS_RANGE_2_WHERE.	FAGL_ACCOUNT_ITEMS_DECO	103	119
	FAGL_ACCOUNT_ITEMS_GL	466	9
	FAGL_ACCOUNT_ITEMS_GL	4560	77
	FAGL_ACCOUNT_ITEMS_GL	6541	111
GET_CHECK_INFORMATION.....	FAGL_ACCOUNT_ITEMS_INC	1396	153
GL_USER_DATA_GET.....	FAGL_ACCOUNT_ITEMS_GL	3321	57
G_SET_GET_ALL_VALUES.....	FAGL_ACCOUNT_ITEMS_GL	5897	101
	FAGL_ACCOUNT_ITEMS_GL	6251	107
IDCN_GLACC_TEXT_GET.....	FAGL_ACCOUNT_ITEMS_GL	1859	33
KOKRS_GET_FROM_BUKRS.....	FAGL_ACCOUNT_ITEMS_INC	3517	188
K_PCA_RESP_AUTHORITY_CHECK....	FAGL_ACCOUNT_ITEMS_INC	3533	189
K_REPO_CCA_AUTHORITY_CHECK....	FAGL_ACCOUNT_ITEMS_INC	3552	189
LDB_PROCESS.....	FAGL_ACCOUNT_ITEMS_DECO	156	120
MESSAGE_STORE.....	FIN_UI_DECO_INCLUDE	269	213
OWN_LOGICAL_SYSTEM_GET.....	FAGL_ACCOUNT_ITEMS_INC	642	141
POPUP_GET_VALUES_USER_BUTTONS.	FAGL_ACCOUNT_ITEMS_GL	3246	55
REUSE_ALV_FIELDCATALOG_MERGE..	FAGL_ACCOUNT_ITEMS_INC	191	134
REUSE_ALV_GRID_LAYOUT_INFO_GET	FAGL_ACCOUNT_ITEMS_INC	57	131
	FAGL_ACCOUNT_ITEMS_INC	82	132
	FAGL_ACCOUNT_ITEMS_INC	3072	181
REUSE_ALV_LIST_LAYOUT_INFO_GET	FAGL_ACCOUNT_ITEMS_INC	49	131
	FAGL_ACCOUNT_ITEMS_INC	74	132
	FAGL_ACCOUNT_ITEMS_INC	3064	181
REUSE_ALV_VARIANT_DEFAULT_GET.	FAGL_ACCOUNT_ITEMS_INC	2864	178
REUSE_ALV_VARIANT_EXISTENCE...	FAGL_ACCOUNT_ITEMS_INC	2877	178
	FAGL_ACCOUNT_ITEMS_INC	2900	178
REUSE_ALV_VARIANT_F4.....	FAGL_ACCOUNT_ITEMS_INC	19	131
REUSE_ALV_VARIANT_SELECT.....	FAGL_ACCOUNT_ITEMS_INC	206	134
RF_WORKLISTS_VALUES_GET.....	FAGL_ACCOUNT_ITEMS_INC	2476	171
RGRE_ERGSL_TO_RACCT_CONVERT...	FAGL_ACCOUNT_ITEMS_GL	5939	102
RSTI_APPL_STACK_INITIALIZE....	FAGL_ACCOUNT_ITEMS_GL	5720	98
	FAGL_ACCOUNT_ITEMS_GL	6549	112
RSTI_APPL_STACK_POP.....	FAGL_ACCOUNT_ITEMS_GL	5677	97
RSTI_REPORT_FIELDS_FIND.....	FAGL_ACCOUNT_ITEMS_GL	6381	109
RSTI_SELECTION_IMPORT.....	FAGL_ACCOUNT_ITEMS_GL	5712	98
RS_REFRESH_FROM_DYNAMICAL_SEL.	FAGL_ACCOUNT_ITEMS_INC	1922	162
RS_REFRESH_FROM_SELECTOPTIONS.	FAGL_ACCOUNT_ITEMS_INC	1907	162
RS_SET_SELSCREEN_STATUS.....	FAGL_ACCOUNT_ITEMS_INC	2547	172
RS_SUBMIT_INFO.....	FAGL_ACCOUNT_ITEMS_INC	2510	172

SAPGUI_PROGRESS_INDICATOR.....	FAGL_ACCOUNT_ITEMS_GL	1486	26
	FAGL_ACCOUNT_ITEMS_INC	1687	158
TP_VALUATION_AUTHORITY.....	FAGL_ACCOUNT_ITEMS_INC	2750	176
	FAGL_ACCOUNT_ITEMS_INC	2779	176
	FAGL_ACCOUNT_ITEMS_INC	2828	177
VALUATION_FETCH_BSBV.....	FAGL_ACCOUNT_ITEMS_INC	1430	154

C functions:

		Line	Page
DYNP_GET_STATUS.....	FAGL_ACCOUNT_ITEMS_INC	2355	169
	FIN_UI_DECO_INCLUDE	679	220
DYNP_INIT_LOOP_AT_SCREEN.....	FIN_UI_DECO_INCLUDE	694	220

SET/GET Parameters

		Line	Page
AVB.....	FAGL_ACCOUNT_ITEMS_GL	2581	44
	FAGL_ACCOUNT_ITEMS_GL	2585	45
	FAGL_ACCOUNT_ITEMS_GL	2655	46
AVS.....	FAGL_ACCOUNT_ITEMS_GL	2580	44
	FAGL_ACCOUNT_ITEMS_GL	2584	45
	FAGL_ACCOUNT_ITEMS_GL	2654	46
BUK.....	FAGL_ACCOUNT_ITEMS_GL	2578	44
	FAGL_ACCOUNT_ITEMS_GL	2594	45
	FAGL_ACCOUNT_ITEMS_GL	2637	45
	FAGL_ACCOUNT_ITEMS_GL	2665	46
FAGLL03_DEL_MAX_SEL.....	FAGL_ACCOUNT_ITEMS_GL	6852	117
FAGLL03_TEST.....	FAGL_ACCOUNT_ITEMS_GL	4769	81
	FAGL_ACCOUNT_ITEMS_GL	4782	81
	FAGL_ACCOUNT_ITEMS_GL	4841	82
	FAGL_ACCOUNT_ITEMS_GL	6829	116
FAGLL03_TEST_SPECIAL.....	FAGL_ACCOUNT_ITEMS_GL	6831	116
FAGL_MIG_FAGLL03.....	FAGL_ACCOUNT_ITEMS_GL	271	6
FIT_ALVC.....	FAGL_ACCOUNT_ITEMS_INC	1994	163
FIT_ALV_FAGL_AP.....	FAGL_ACCOUNT_ITEMS_INC	1725	159
	FAGL_ACCOUNT_ITEMS_INC	1847	161
	FAGL_ACCOUNT_ITEMS_INC	1850	161
FIT_ALV_FAGL_AR.....	FAGL_ACCOUNT_ITEMS_INC	1721	159
	FAGL_ACCOUNT_ITEMS_INC	1836	161
	FAGL_ACCOUNT_ITEMS_INC	1839	161
FIT_ALV_FAGL_GL.....	FAGL_ACCOUNT_ITEMS_INC	1729	159
	FAGL_ACCOUNT_ITEMS_INC	1858	161
	FAGL_ACCOUNT_ITEMS_INC	1861	161
FIT_DUE_DATE_SEL.....	FAGL_ACCOUNT_ITEMS_INC	2562	173
FIT_WL.....	FAGL_ACCOUNT_ITEMS_INC	2381	170
	FAGL_ACCOUNT_ITEMS_INC	2383	170
	FAGL_ACCOUNT_ITEMS_INC	2532	172
FIT_X_FAGL_AP.....	FAGL_ACCOUNT_ITEMS_INC	1724	159
	FAGL_ACCOUNT_ITEMS_INC	1843	161
FIT_X_FAGL_AR.....	FAGL_ACCOUNT_ITEMS_INC	1720	159
	FAGL_ACCOUNT_ITEMS_INC	1832	161
FIT_X_FAGL_GL.....	FAGL_ACCOUNT_ITEMS_INC	1728	159
	FAGL_ACCOUNT_ITEMS_INC	1854	161
FIT_X_NMAX.....	FAGL_ACCOUNT_ITEMS_INC	1757	159
	FAGL_ACCOUNT_ITEMS_INC	1894	162
FI_STIDA.....	FAGL_ACCOUNT_ITEMS_GL	2085	36
	FAGL_ACCOUNT_ITEMS_GL	2088	36
FI_WORKLISTS_FLAG.....	FAGL_ACCOUNT_ITEMS_INC	2527	172

FOP.....	FAGL_ACCOUNT_ITEMS_INC	1746	159
	FAGL_ACCOUNT_ITEMS_INC	1834	161
	FAGL_ACCOUNT_ITEMS_INC	1845	161
	FAGL_ACCOUNT_ITEMS_INC	1856	161
	FAGL_ACCOUNT_ITEMS_INC	1880	161
SAK.....	FAGL_ACCOUNT_ITEMS_GL	2573	44
	FAGL_ACCOUNT_ITEMS_GL	2589	45
	FAGL_ACCOUNT_ITEMS_GL	2622	45
	FAGL_ACCOUNT_ITEMS_GL	2657	46

Reports:

		Line	Page
?.....	FAGL_ACCOUNT_ITEMS_INC	124	133
?.....	FAGL_ACCOUNT_ITEMS_INC	141	133
?.....	FAGL_ACCOUNT_ITEMS_INC	2015	164

Transactions:

		Line	Page
?.....	FIN_UI_DECO_INCLUDE	445	216
?.....	FIN_UI_DECO_INCLUDE	447	216

External subroutines

		Line	Page
(FORMNAME).....	FIN_UI_DECO_INCLUDE	545	218
	FIN_UI_DECO_INCLUDE	547	218
MODIF_DYN_SEL.....	FAGL_ACCOUNT_ITEMS_GL	3588	61

DATABASE OPERATIONS

SELECT:

		Line	Page
ACDOCA	FAGL_ACCOUNT_ITEMS_INC	1041	148
BKPF	FAGL_ACCOUNT_ITEMS_INC	695	142
	FAGL_ACCOUNT_ITEMS_INC	980	147
BSEC	FAGL_ACCOUNT_ITEMS_INC	1299	152
BSED	FAGL_ACCOUNT_ITEMS_INC	1356	153
BSEG	FAGL_ACCOUNT_ITEMS_INC	939	146
BSEGC	FAGL_ACCOUNT_ITEMS_INC	1256	151
BSEG_ADD	FAGL_ACCOUNT_ITEMS_INC	999	147
FINSC_CURTYPET	FAGL_ACCOUNT_ITEMS_INC	4526	206
FINSC_LD_CMP	FAGL_ACCOUNT_ITEMS_GL	881	16
FINSV_T001A	FAGL_ACCOUNT_ITEMS_GL	1074	19
SKA1	FAGL_ACCOUNT_ITEMS_GL	5852	100
	FAGL_ACCOUNT_ITEMS_GL	5856	100
	FAGL_ACCOUNT_ITEMS_GL	5862	100
	FAGL_ACCOUNT_ITEMS_GL	5865	100
	FAGL_ACCOUNT_ITEMS_GL	5881	101
	FAGL_ACCOUNT_ITEMS_GL	5958	102
	FAGL_ACCOUNT_ITEMS_GL	5983	102
SKAT	FAGL_ACCOUNT_ITEMS_GL	1811	32
	FAGL_ACCOUNT_ITEMS_GL	1819	32
	FAGL_ACCOUNT_ITEMS_GL	1827	32
SKB1	FAGL_ACCOUNT_ITEMS_GL	1545	27
	FAGL_ACCOUNT_ITEMS_GL	1555	28
	FAGL_ACCOUNT_ITEMS_GL	1564	28
	FAGL_ACCOUNT_ITEMS_GL	1577	28
	FAGL_ACCOUNT_ITEMS_GL	2809	48
T001	FAGL_ACCOUNT_ITEMS_GL	572	11
	FAGL_ACCOUNT_ITEMS_GL	878	16
	FAGL_ACCOUNT_ITEMS_GL	897	16
	FAGL_ACCOUNT_ITEMS_GL	1089	20
	FAGL_ACCOUNT_ITEMS_GL	1746	31
	FAGL_ACCOUNT_ITEMS_GL	2711	47
	FAGL_ACCOUNT_ITEMS_GL	2718	47
	FAGL_ACCOUNT_ITEMS_GL	4406	74
	FAGL_ACCOUNT_ITEMS_GL	5594	96
	FAGL_ACCOUNT_ITEMS_GL	5598	96
	FAGL_ACCOUNT_ITEMS_INC	2955	179
	FAGL_ACCOUNT_ITEMS_INC	4258	201
	FAGL_ACCOUNT_ITEMS_INC	4265	201
	RFDBEXPC	40	225
	RFDBEXPC	82	226
	RFDBEXPC	143	227
	RFDBEXPC	185	228
	RFDBEXPC	246	229
	RFDBEXPC	316	230
	RFDBEXPC	401	231
	RFDBEXPC	439	232
	RFDBEXPC	501	233
	RFDBEXPC	560	234
	RFDBEXPC	598	234
	RFDBEXPC	660	235
T021S	FAGL_ACCOUNT_ITEMS_INC	247	135
	FAGL_ACCOUNT_ITEMS_INC	1295	152
USR05	FAGL_ACCOUNT_ITEMS_INC	1748	159
VBKPF	FAGL_ACCOUNT_ITEMS_INC	741	143
	FAGL_ACCOUNT_ITEMS_INC	1112	149

	FAGL_ACCOUNT_ITEMS_INC	1160	150
VBSEC.....	FAGL_ACCOUNT_ITEMS_INC	1328	152
VBSEGD.....	FAGL_ACCOUNT_ITEMS_INC	1082	148
VBSEGK.....	FAGL_ACCOUNT_ITEMS_INC	1130	149
VBSEGS.....	FAGL_ACCOUNT_ITEMS_INC	1191	150

MODIFY:

		Line	Page
CL_FIN_UI_DECO=>MT_SCREEN.....	FIN_UI_DECO_INCLUDE	768	222
FS_DYNS-TRANGE.....	FAGL_ACCOUNT_ITEMS_GL	4548	77
LS_DYNS-CLAUSES.....	FAGL_ACCOUNT_ITEMS_GL	3547	60
	FAGL_ACCOUNT_ITEMS_GL	3550	60
LS_DYNS-TEXPR.....	FAGL_ACCOUNT_ITEMS_GL	3562	61
	FAGL_ACCOUNT_ITEMS_GL	3565	61
LS_DYNS-TRANGE.....	FAGL_ACCOUNT_ITEMS_GL	3577	61
	FAGL_ACCOUNT_ITEMS_GL	3580	61
USR05.....	FAGL_ACCOUNT_ITEMS_INC	1888	162

DELETE:

		Line	Page
USR05.....	FAGL_ACCOUNT_ITEMS_INC	1885	162

INTERNAL TABLE OPERATIONS

READ TABLE:

		Line	Page
CL_FIN_UI_DECO=>MT_SCREEN.....	FIN_UI_DECO_INCLUDE	739	221
CL_FIN_UI_DECO=>MT_SCREEN_DEF.	FIN_UI_DECO_INCLUDE	752	221
GT_FAGL_MIG_001.....	FAGL_ACCOUNT_ITEMS_GL	640	12
GT_FIELDCAT.....	FAGL_ACCOUNT_ITEMS_GL	1324	24
	FAGL_ACCOUNT_ITEMS_GL	1387	25
GT_REPORTING_FIELDS.....	FAGL_ACCOUNT_ITEMS_GL	4524	76
GT_YRPER.....	FAGL_ACCOUNT_ITEMS_GL	2210	38
	FAGL_ACCOUNT_ITEMS_GL	2963	51
IT_CENTRAL.....	FAGL_ACCOUNT_ITEMS_INC	1953	163
IT_H_T001.....	FAGL_ACCOUNT_ITEMS_GL	1741	31
IT_POS.....	FAGL_ACCOUNT_ITEMS_INC	584	140
	FAGL_ACCOUNT_ITEMS_INC	668	142
	FAGL_ACCOUNT_ITEMS_INC	712	142
	FAGL_ACCOUNT_ITEMS_INC	759	143
	FAGL_ACCOUNT_ITEMS_INC	890	145
	FAGL_ACCOUNT_ITEMS_INC	969	146
	FAGL_ACCOUNT_ITEMS_INC	1014	147
	FAGL_ACCOUNT_ITEMS_INC	1054	148
	FAGL_ACCOUNT_ITEMS_INC	1102	149
	FAGL_ACCOUNT_ITEMS_INC	1150	149
	FAGL_ACCOUNT_ITEMS_INC	1217	151
	FAGL_ACCOUNT_ITEMS_INC	1335	152
	FAGL_ACCOUNT_ITEMS_INC	1361	153
IT_SPECFLD.....	FAGL_ACCOUNT_ITEMS_INC	510	139
	FAGL_ACCOUNT_ITEMS_INC	3257	184
	FAGL_ACCOUNT_ITEMS_INC	4161	199
IT_SPECTAB.....	FAGL_ACCOUNT_ITEMS_INC	497	139
	FAGL_ACCOUNT_ITEMS_INC	525	139
	FAGL_ACCOUNT_ITEMS_INC	3269	184
	FAGL_ACCOUNT_ITEMS_INC	4177	199
IT_T021S.....	FAGL_ACCOUNT_ITEMS_INC	2123	165
	FAGL_ACCOUNT_ITEMS_INC	2700	175
	FAGL_ACCOUNT_ITEMS_INC	2715	175
	FAGL_ACCOUNT_ITEMS_INC	4166	199
IT_TP_FIELD2.....	FAGL_ACCOUNT_ITEMS_INC	472	138
IT_TP_FIELDS.....	FAGL_ACCOUNT_ITEMS_INC	438	138
LT_ACCOUNTS.....	FAGL_ACCOUNT_ITEMS_GL	5957	102
LT_BSEC.....	FAGL_ACCOUNT_ITEMS_INC	1311	152
LT_BUKRS_EXT.....	FAGL_ACCOUNT_ITEMS_GL	885	16
LT_CURTP.....	FAGL_ACCOUNT_ITEMS_INC	4306	201
LT_DFIES_BSIX.....	FAGL_ACCOUNT_ITEMS_INC	2219	167
LT_DFIES_FAGLPOSE.....	FAGL_ACCOUNT_ITEMS_INC	2149	166
LT_DUPLICATES.....	FAGL_ACCOUNT_ITEMS_INC	2157	166
LT_FIELDS.....	FAGL_ACCOUNT_ITEMS_GL	6277	107
	FAGL_ACCOUNT_ITEMS_GL	6281	107
	FAGL_ACCOUNT_ITEMS_GL	6285	107
	FAGL_ACCOUNT_ITEMS_GL	6418	109
	FAGL_ACCOUNT_ITEMS_INC	264	135
	FAGL_ACCOUNT_ITEMS_INC	2200	167
LT_SELSCREEN.....	FAGL_ACCOUNT_ITEMS_INC	114	132
LT_TTXID.....	FAGL_ACCOUNT_ITEMS_INC	2606	173
PERIOD.....	FAGL_ACCOUNT_ITEMS_GL	2277	40
	FAGL_ACCOUNT_ITEMS_GL	2284	40
	FAGL_ACCOUNT_ITEMS_GL	2289	40
SD_BUKRS.....	FAGL_ACCOUNT_ITEMS_GL	1531	27
	FAGL_ACCOUNT_ITEMS_GL	1611	28

	FAGL_ACCOUNT_ITEMS_GL	2575	44
SD_SAKNR.....	FAGL_ACCOUNT_ITEMS_GL	1529	27
	FAGL_ACCOUNT_ITEMS_GL	1601	28
	FAGL_ACCOUNT_ITEMS_GL	2570	44
SO_WLBUK.....	FAGL_ACCOUNT_ITEMS_GL	2591	45
SO_WLSAK.....	FAGL_ACCOUNT_ITEMS_GL	2586	45
S_TAB.....	FAGL_ACCOUNT_ITEMS_INC	2998	180
XBKPF.....	FAGL_ACCOUNT_ITEMS_INC	653	141
	FAGL_ACCOUNT_ITEMS_INC	901	145
	FAGL_ACCOUNT_ITEMS_INC	4081	198
XBSEG.....	FAGL_ACCOUNT_ITEMS_INC	858	145
	FAGL_ACCOUNT_ITEMS_INC	4095	198
YEAR.....	FAGL_ACCOUNT_ITEMS_GL	2296	40
	FAGL_ACCOUNT_ITEMS_GL	2303	40
	FAGL_ACCOUNT_ITEMS_GL	2308	40

MODIFY:

		Line	Page
GT_YRPER.....	FAGL_ACCOUNT_ITEMS_GL	2213	38
	FAGL_ACCOUNT_ITEMS_GL	2966	51
IT_POS.....	FAGL_ACCOUNT_ITEMS_INC	674	142
	FAGL_ACCOUNT_ITEMS_INC	681	142
	FAGL_ACCOUNT_ITEMS_INC	718	142
	FAGL_ACCOUNT_ITEMS_INC	725	142
	FAGL_ACCOUNT_ITEMS_INC	766	143
	FAGL_ACCOUNT_ITEMS_INC	774	143
	FAGL_ACCOUNT_ITEMS_INC	913	146
	FAGL_ACCOUNT_ITEMS_INC	991	147
	FAGL_ACCOUNT_ITEMS_INC	1023	147
	FAGL_ACCOUNT_ITEMS_INC	1064	148
	FAGL_ACCOUNT_ITEMS_INC	1124	149
	FAGL_ACCOUNT_ITEMS_INC	1172	150
	FAGL_ACCOUNT_ITEMS_INC	1236	151
	FAGL_ACCOUNT_ITEMS_INC	1269	151
	FAGL_ACCOUNT_ITEMS_INC	1319	152
	FAGL_ACCOUNT_ITEMS_INC	1342	153
	FAGL_ACCOUNT_ITEMS_INC	1370	153
	FAGL_ACCOUNT_ITEMS_INC	1418	154
	FAGL_ACCOUNT_ITEMS_INC	1448	154
	FAGL_ACCOUNT_ITEMS_INC	2611	173
IT_SPECFLD.....	FAGL_ACCOUNT_ITEMS_INC	521	139
	FAGL_ACCOUNT_ITEMS_INC	1479	155
	FAGL_ACCOUNT_ITEMS_INC	3267	184
IT_SPECTAB.....	FAGL_ACCOUNT_ITEMS_INC	500	139
	FAGL_ACCOUNT_ITEMS_INC	534	139
	FAGL_ACCOUNT_ITEMS_INC	536	139
	FAGL_ACCOUNT_ITEMS_INC	1475	155
	FAGL_ACCOUNT_ITEMS_INC	3277	184
	FAGL_ACCOUNT_ITEMS_INC	4180	199
IT_TP_AUTH.....	FAGL_ACCOUNT_ITEMS_INC	2817	177
LT_SELSCREEN.....	FAGL_ACCOUNT_ITEMS_INC	117	132
RT_VAR_FIELDCAT.....	FAGL_ACCOUNT_ITEMS_INC	458	138
	FAGL_ACCOUNT_ITEMS_INC	464	138
	FAGL_ACCOUNT_ITEMS_INC	479	138
	FAGL_ACCOUNT_ITEMS_INC	485	138
	FAGL_ACCOUNT_ITEMS_INC	554	140

DELETE:		Line	Page
DS_TRANGE_TAB_LINE-FRANGE_T...	FAGL_ACCOUNT_ITEMS_GL	4543	77
FS_DYNS-CLAUSES.....	FAGL_ACCOUNT_ITEMS_GL	3642	62
FS_DYNS-TEXPR.....	FAGL_ACCOUNT_ITEMS_GL	3649	62
FS_DYNS-TRANGE.....	FAGL_ACCOUNT_ITEMS_GL	459	9
	FAGL_ACCOUNT_ITEMS_GL	3656	62
FS_FIELD.....	FAGL_ACCOUNT_ITEMS_GL	448	9
	FAGL_ACCOUNT_ITEMS_GL	3663	62
GT_ACCOCO.....	FAGL_ACCOUNT_ITEMS_GL	2738	47
GT_ACCOUNT_SLT.....	FAGL_ACCOUNT_ITEMS_GL	2767	48
GT_FAGL_MIG_001.....	FAGL_ACCOUNT_ITEMS_GL	637	12
	FAGL_ACCOUNT_ITEMS_GL	638	12
	FAGL_ACCOUNT_ITEMS_GL	639	12
GT_FIELDCAT.....	FAGL_ACCOUNT_ITEMS_INC	2248	168
	FAGL_ACCOUNT_ITEMS_INC	2251	168
GT_SPEC_FIELDS.....	FAGL_ACCOUNT_ITEMS_INC	3206	183
IT_POS.....	FAGL_ACCOUNT_ITEMS_GL	1480	26
	FAGL_ACCOUNT_ITEMS_INC	4061	197
IT_SPECTAB.....	FAGL_ACCOUNT_ITEMS_INC	391	137
IT_T021S.....	FAGL_ACCOUNT_ITEMS_INC	276	135
IT_TP_AUTH.....	FAGL_ACCOUNT_ITEMS_GL	1079	19
LR_TRANGE->FRANGE_T.....	FAGL_ACCOUNT_ITEMS_GL	455	9
LS_DYNS-CLAUSES.....	FAGL_ACCOUNT_ITEMS_GL	3553	60
LS_DYNS-TEXPR.....	FAGL_ACCOUNT_ITEMS_GL	3568	61
LS_DYNS-TRANGE.....	FAGL_ACCOUNT_ITEMS_GL	3583	61
LT_SEL.....	FAGL_ACCOUNT_ITEMS_GL	5732	98
	FAGL_ACCOUNT_ITEMS_GL	5740	98
	FAGL_ACCOUNT_ITEMS_GL	5772	99
	FAGL_ACCOUNT_ITEMS_GL	5773	99
	FAGL_ACCOUNT_ITEMS_GL	5780	99
	FAGL_ACCOUNT_ITEMS_GL	5797	99
	FAGL_ACCOUNT_ITEMS_GL	5870	100
	FAGL_ACCOUNT_ITEMS_GL	5889	101
	FAGL_ACCOUNT_ITEMS_GL	5975	102
	FAGL_ACCOUNT_ITEMS_GL	5992	102
	FAGL_ACCOUNT_ITEMS_GL	6004	103
	FAGL_ACCOUNT_ITEMS_GL	6028	103
	FAGL_ACCOUNT_ITEMS_GL	6033	103
	FAGL_ACCOUNT_ITEMS_GL	6065	104
	FAGL_ACCOUNT_ITEMS_GL	6080	104
	FAGL_ACCOUNT_ITEMS_GL	6085	104
	FAGL_ACCOUNT_ITEMS_GL	6135	105
	FAGL_ACCOUNT_ITEMS_GL	6144	105
	FAGL_ACCOUNT_ITEMS_GL	6178	106
	FAGL_ACCOUNT_ITEMS_GL	6184	106
	FAGL_ACCOUNT_ITEMS_GL	6206	106
	FAGL_ACCOUNT_ITEMS_GL	6217	106
	FAGL_ACCOUNT_ITEMS_GL	6228	106
	FAGL_ACCOUNT_ITEMS_GL	6238	106
	FAGL_ACCOUNT_ITEMS_GL	6257	107
	FAGL_ACCOUNT_ITEMS_GL	6299	107
	FAGL_ACCOUNT_ITEMS_GL	6333	108
	FAGL_ACCOUNT_ITEMS_GL	6347	108
	FAGL_ACCOUNT_ITEMS_GL	6377	109
PERIOD.....	FAGL_ACCOUNT_ITEMS_GL	2272	39
SD_BUKRS.....	FAGL_ACCOUNT_ITEMS_GL	2754	47
SD_SAKNR.....	FAGL_ACCOUNT_ITEMS_GL	2751	47
YEAR.....	FAGL_ACCOUNT_ITEMS_GL	2273	39